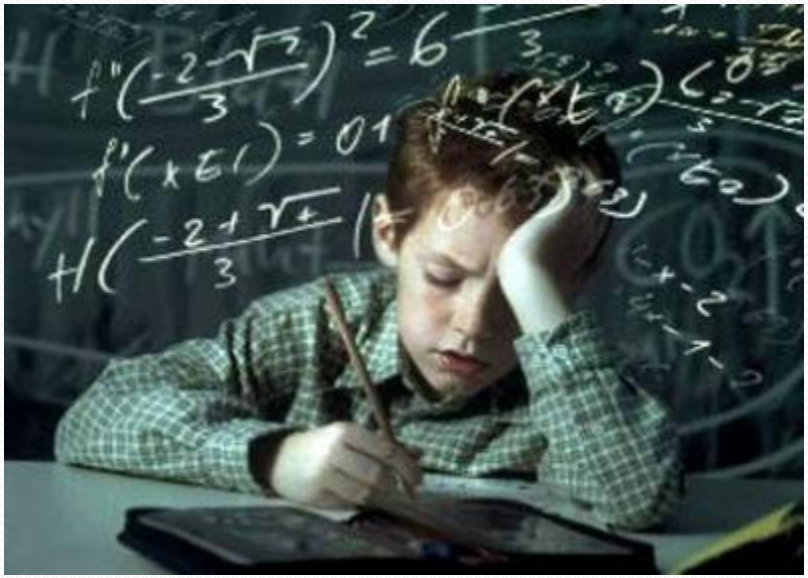


UML

Unified Modeling Language

- **Introducción**
- **Diagramas de Comportamiento**
  - Diagrama de casos de uso
  - Diagrama de máquina de estados
  - Diagrama de actividad
  - Diagramas de Interacción
    - *Diagrama de secuencia*
    - *Diagrama de comunicación*
    - *Diagrama de tiempos*
    - *Diagrama global de interacción*
- **Diagramas de estructura**
  - Diagrama de clases
  - Diagrama de objetos
  - Diagrama de estructura compuesta
  - Diagrama de componentes
  - Diagrama de paquetes
  - Diagrama de despliegue
  - Diagrama de perfil



¿Qué es un modelo?

- **Un modelo es una simplificación de la realidad. El modelo nos proporciona los planos de un sistema**, desde los más generales, que proporcionan una visión general del sistema, hasta los más detallados.
- En un modelo se han de incluir los elementos que tengan más relevancia y omitir los que no son interesantes para el nivel de abstracción que se ha elegido.
- A través del modelado conseguimos cuatro objetivos:
  - Los modelos nos ayudan a **visualizar cómo es o queremos que sea un sistema**.
  - Los modelos nos **permiten especificar la estructura o el comportamiento de un sistema**.
  - Los modelos nos **proporcionan plantillas que nos guían en la construcción de un sistema**.
  - Los modelos **documentan las decisiones que hemos adoptado**.

Modelado

- **El modelado es la espina dorsal del desarrollo software de calidad**.
- Se construyen modelos:
  - para poder **comunicarnos** con otros
  - para explicar el **comportamiento** del sistema a desarrollar
  - para **comprender**, nosotros mismos, mejor ese sistema
  - para **controlar el riesgo** y en definitiva para poder atacar problemas que sin el modelado su resolución seria imposible.

- Un **lenguaje de modelado** permite expresar los distintos modelos que se producen en el proceso de desarrollo.
- Un **diagrama** es una representación de (parte de) un modelo de diseño.
- Un modelo se representa por uno o más diagramas

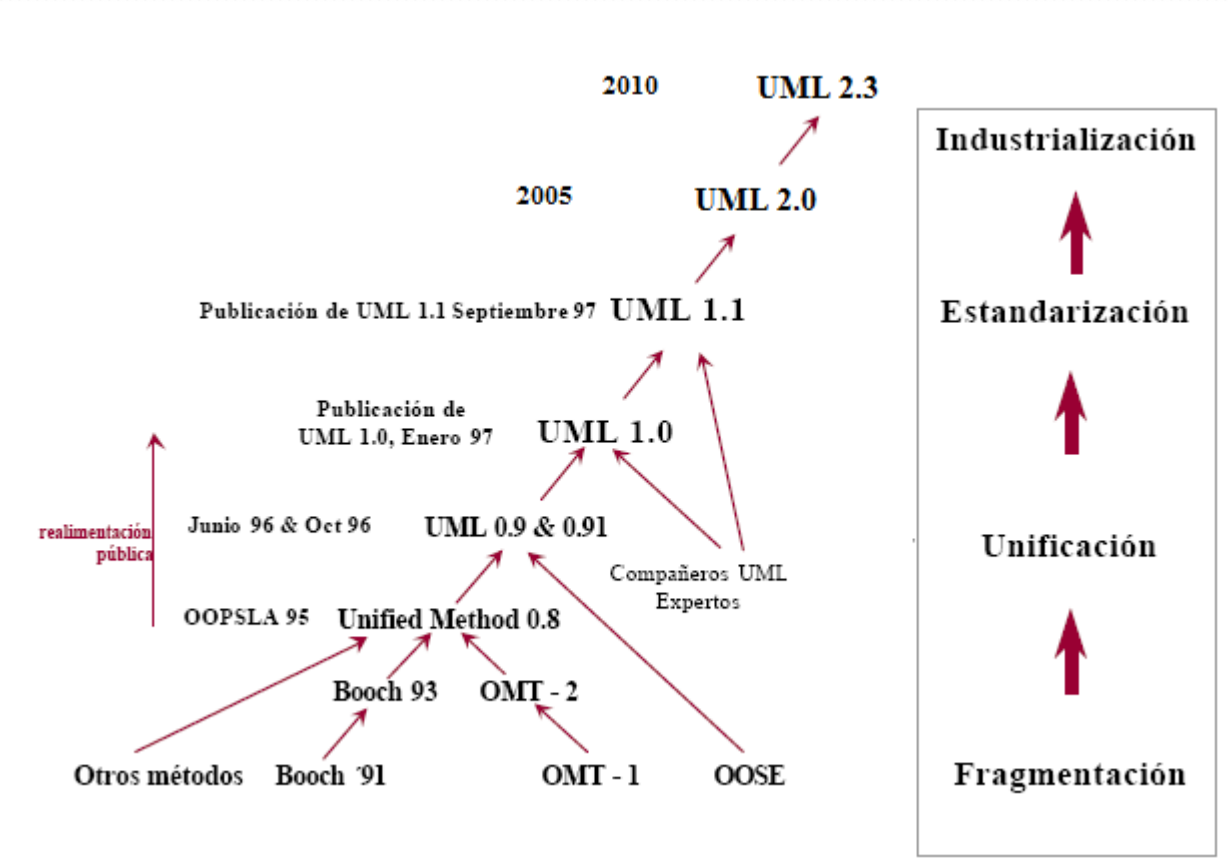
¿Qué es el UML?

- Unified **M**odeling **L**anguaje - **L**enguaje **U**nificado de **M**odelado.
  - Es un **lenguaje estándar** que sirve para escribir los planos del software.
  - Es un **lenguaje gráfico** para visualizar, especificar, construir y documentar un sistema.
  - Es **extensible**. Es posible personalizar el lenguaje para un dominio concreto.
  - Modela sistemas, desde los requisitos hasta los artefactos ejecutables, **utilizando técnicas de Orientación a Objetos**.
  - Es **escalable**: puede manejar sistemas grandes y pequeños.
  - Es **exhaustivo**: describe todos los aspectos importantes del sistema.
  - **Construido sobre la experiencia**: es la culminación de las mejores prácticas en la comunidad de OO de los últimos 15 años.
  - **Permite ver el sistema desde distintas perspectivas (vistas).** UML abarca todas las fases del ciclo de vida de un proyecto, soporta diferentes maneras de visualización dependiendo de quién tenga que interpretar *los planos y en que fase del proyecto se encuentre*.
  - **No es una metodología**. Se puede usar en distintos tipos de metodología (cascada, iterativo,...).

Cómo nació el UML

- A mediados de los noventa existían muchas técnicas de modelado OO.
- UML comenzó a gestarse en octubre de 1994, cuando **Rumbaugh** se unió a la compañía **Rational** fundada por **Booch** (dos reputados investigadores en el área de metodología del software).
- El **objetivo de ambos** era **unificar dos métodos** que habían desarrollado: **el método Booch y el OMT (Object Modelling Tool)**.
- El primer borrador apareció en octubre de 1995. En esa misma época otro reputado investigador, **Jacobson**, **se unió a Rational y se incluyeron ideas suyas**.
- Estas tres personas son conocidas como los "**tres amigos**".
- Además, este lenguaje se abrió a la colaboración de otras empresas para que aportaran sus ideas. Todas estas colaboraciones condujeron a la definición de la primera versión de UML.
- En 1997 la **versión UML 1.1 fue aprobada por la OMG convirtiéndose en la notación estándar de facto para el análisis y el diseño orientado a objetos**.
- **OMG (Object Management Group** - *Grupo de Gestión de Objetos*): consorcio dedicado al cuidado y el establecimiento de diversos estándares de tecnologías orientadas a objetos, tales como **UML, XMI, CORBA**. Es una organización sin ánimo de lucro que promueve el uso de tecnología orientada a objetos mediante guías y especificaciones para las mismas. El grupo está formado por compañías y organizaciones de software como:
  - **Hewlett-Packard**
  - **Oracle**
  - **Microsoft**
  - **etc.**

Historia del UML



Versiones de UML

OMG Formally Released Versions Of UML®

Version	Release Date	URL
2.4.1	August 2011	http://www.omg.org/spec/UML/2.4.1
2.4	March 2011	http://www.omg.org/spec/UML/2.4
2.3	May 2010	http://www.omg.org/spec/UML/2.3
2.2	February 2009	http://www.omg.org/spec/UML/2.2
2.1.2	November 2007	http://www.omg.org/spec/UML/2.1.2
2.1.1	August 2007	http://www.omg.org/spec/UML/2.1.1
Please note that version 2.1 was never released as a formal specification.		
2.0	July 2005	http://www.omg.org/spec/UML/2.0
1.5 combines v1.4 and action semantics	March 2003	http://www.omg.org/spec/UML/1.5
1.4	September 2001	http://www.omg.org/spec/UML/1.4
1.3	March 2000	http://www.omg.org/spec/UML/1.3

"In Process" Version Of UML®

Version	Release Date	URL
2.5	October 2012	http://www.omg.org/spec/UML/2.5

Version	Date	Description
1.1	11-1997	UML 1.1 proposal is adopted by the OMG.
1.3	03-2000	Contains a number of changes to the UML metamodel, semantics, and notation, but should be considered a minor upgrade to the original proposal.
1.4	09-2001	Mostly "tuning" release but not completely upward compatible with the UML 1.3. Addition of **profiles** as UML extensions grouped together. Updated **visibility** of features. Stick arrowhead in **interaction diagrams** now denotes **asynchronous call**. Model element may now have multiple **stereotypes**. Clarified collaborations. Refined definitions of components and related concepts. **Artifact** was added to represent physical representations of components.
1.5	03-2003	Added **actions** (see Part 5 of spec) - executable actions and procedures, including their run-time semantics, defined the concept of a data flow to carry data between actions, etc.
1.4.2	01-2005	This version was accepted as ISO specification (standard) ISO/IEC 19501. UML 1.5 was released 2 years before.
2.0	08-2005	New diagrams: object diagrams, **package diagrams**, **composite structure diagrams**, interaction overview diagrams, timing diagrams, **profile diagrams**. **Collaboration diagrams** were renamed to **communication diagrams**. **Activity diagrams** and **sequence diagrams** were enhanced. Activities were redesigned to use a Petri-like semantics. Edges can now be contained in partitions. Partitions can be hierarchical and multidimensional. Explicitly modeled **object flows** are new. Classes have been extended with internal structures and **ports** (composite structures). Information flows were added. A collaboration now is a kind of classifier, and can have any kind of behavioral descriptions associated. Interactions are now contained within classifiers and not only within collaborations. It is now possible for **use cases** to be owned by **classifiers** in general and not just packages. New notation for concurrency and branching using combined fragments. Notation and/or semantics were updated for components, realization, deployments of artifacts. Components can no longer be directly deployed to **nodes**. **Artifacts** should be deployed instead. Implementation has been replaced by «**manifest**». Artifacts can now manifest any packageable element (not just components, as before). It is now possible to deploy to nodes with an internal structure. New metaclasses were added: connector, collaboration use, connector end, **device**, deployment specification, **execution environment**, accept event action, send object action, structural feature action, value pin, activity final, central buffer node, data stores, flow final, interruptible regions, loop nodes, parameter, **port**, behavior, behaviored classifier, duration, interval, time constraint, combined fragment, creation event, destruction event, execution event, interaction fragment, interaction use, receive signal event, send signal event, extension, etc. Many stereotypes were eliminated from the Standard UML Profile, e.g. «destroy», «facade», «friend», «profile», «requirement», «table», «thread». Integration between structural and behavioral models was improved with better support for executable models.
2.1	04-2006	Minor revision to UML 2.0 - corrections and consistency improvements.
2.1.1	02-2007	Minor revision to the UML 2.1
2.1.2	11-2007	Minor revision to the UML 2.1.1
2.2	02-2009	Fixed numerous minor consistency problems and added clarifications to UML 2.1.2
2.3	05-2010	Minor revision to the UML 2.2, clarified **associations** and association classes, added **final classifier**, updated **component diagrams**, composite structures, actions, etc.
2.4.1	08-2011	**Current version of UML**, a minor revision to the UML 2.3, with few fixes and updates to classes, packages - added **URI package attribute**; updated actions; removed creation event, execution event, send and receive operation events, send and receive signal events, renamed destruction event to **destruction occurrence specification**; **profiles** - changed stereotypes and applied stereotypes to have upper-case first letter - «**Metaclass**» and **stereotype application**.

Versiones de UML

Versión actual -> UML 2.4.1

- Documentos de la especificación:
  - UML Infrastructure.
    Define las construcciones de lenguaje fundacionales requeridas por UML. Es un núcleo de metalenguaje que puede ser reusado para definir otros metamodelos y mecanismos de extensión para UML
  - UML Superstructure.
    Complementa a UML Infrastructure. Define construcciones a nivel de usuario requeridas por UML.
  - Documentos XMI de Especificación de intercambio de diagramas. XMI (XML Metadata Interchange). Sirven para poder compartir modelos UML entre diferentes herramientas de modelado. Usa un esquema xml que permite construir una representación SVG (Scalable Vector Graphics). Típicamente esta especificación es utilizada solamente por quienes desarrollan herramientas de modelado UML.

Versión en desarrollo -> UML 2.5

2.5 FTF – Beta 1	11-2012	Work in progress, in its finalization phase. From the UML language perspective, it will be a minor revision to the UML 2.4.1, while they spent a lot of efforts reorganizing specification document. It seems that this time professors took over researchers and industry practitioners, and instead of fixing actual UML issues they re-written UML specification *to make it easier to read*. For example, they tried *to reduce forward references as much as possible* which is an obvious requirement for textbooks but not for specifications. There will be no separate UML 2.5 Infrastructure document, so that the UML 2.5 specification will be a single document. **Package merge** will no longer be used within the specification itself. Four UML compliance levels (L0, L1, L2, and L3) are to be eliminated, as they were not useful in practice. UML 2.5 tools will have to support complete UML specification. **Information flows**, **models**, and **templates** will no longer be auxiliary UML constructs. At the same time, **use cases**, **deployments**, and the **information flows** to become UML supplementary concepts! Default for **generalization sets** changed from **{incomplete, disjoint}** to **{incomplete, overlapping}**.

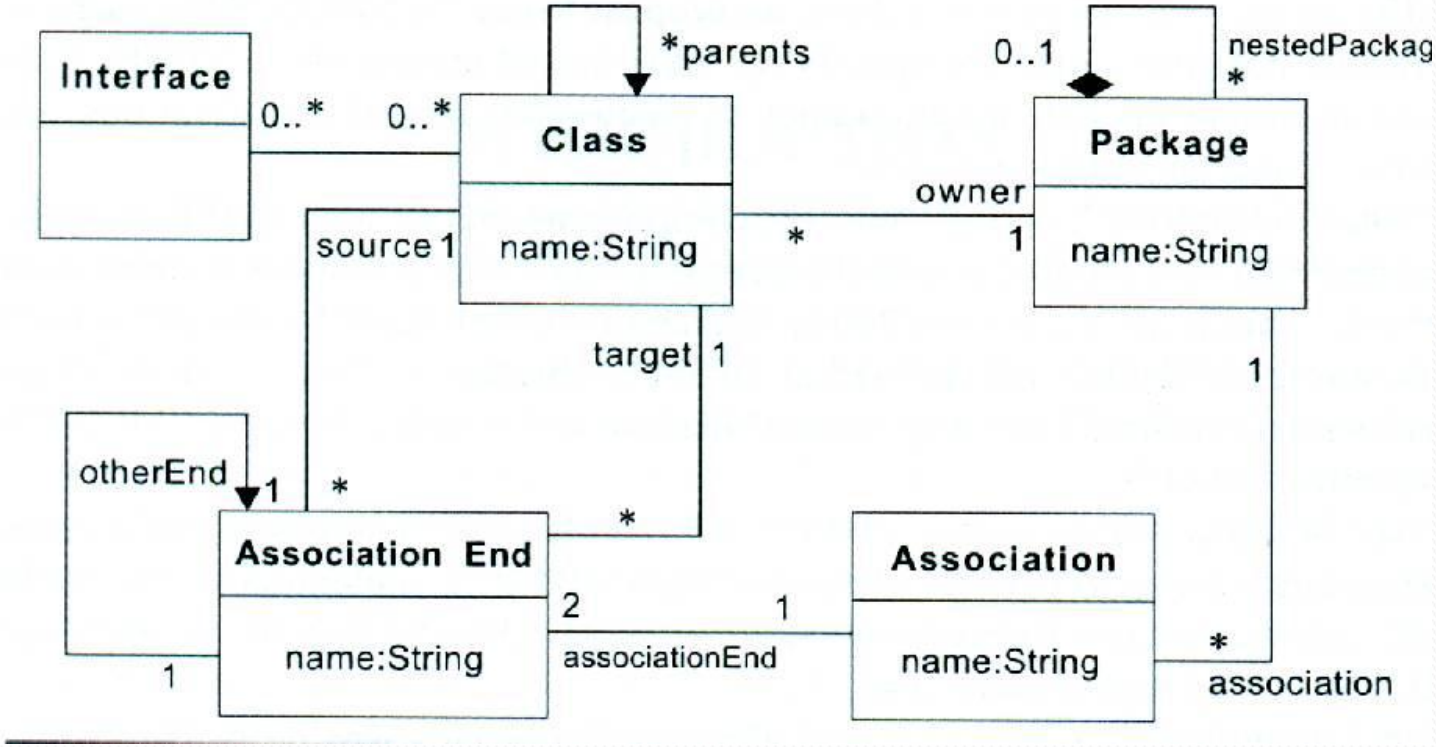
Metamodelo

- Un modelo se expresa en un lenguaje.
- Un lenguaje se define por un metamodelo.

- Un metamodelo:
  - es un modelo que define el lenguaje para expresar un modelo.
  - describe todos los conceptos que pueden usarse en el modelo.

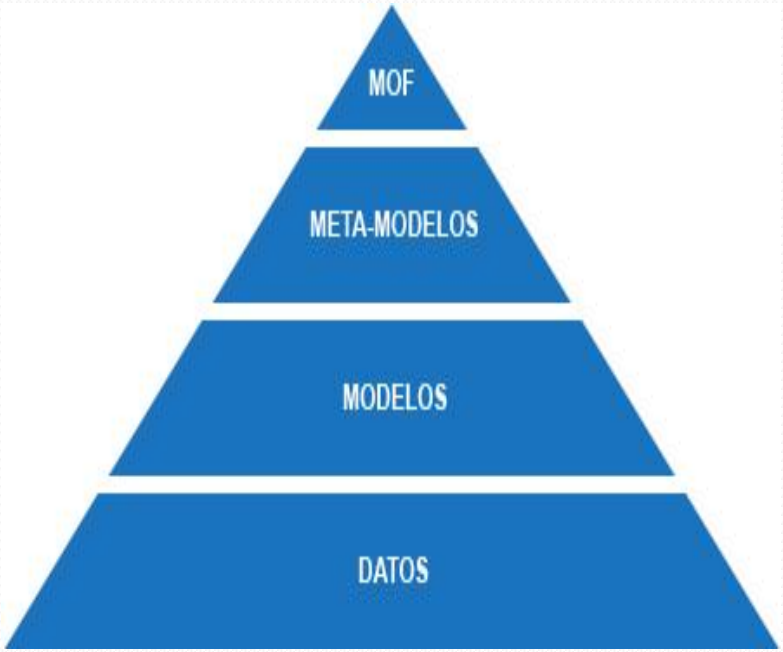
- Cada elemento de un modelo es una instancia de una metaclase en el metamodelo.
- Una **clase** define a sus **objetos** y una **metaclase** define a los **elementos del modelo.**
- Los lenguajes de modelamiento como UML tienen, en general, una sintaxis gráfica que requiere otro tipo de descripción, descripción que se da mediante un **metamodelo**.

Un ejemplo de metamodelo UML



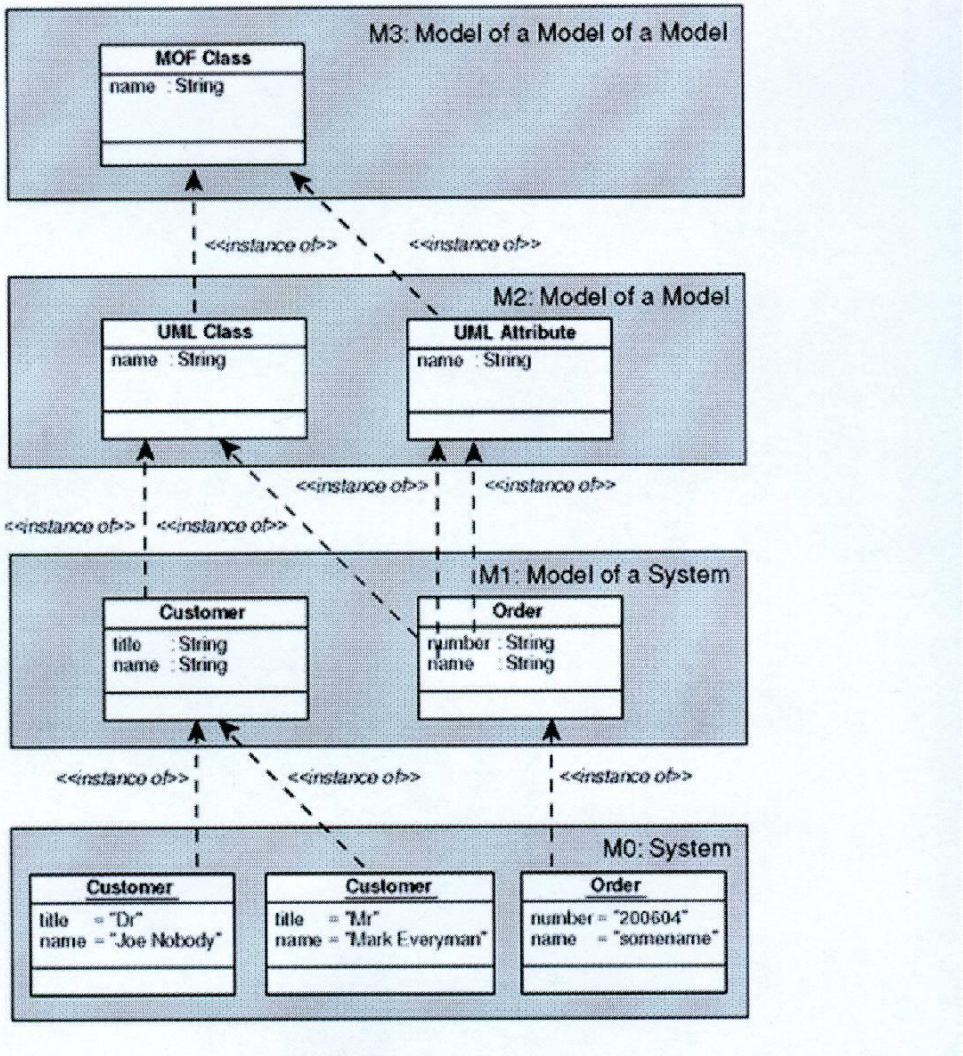
Arquitectura de 4 niveles

- **OMG define una arquitectura basada en cuatro niveles de abstracción** que van a permitir distinguir entre los distintos niveles conceptuales que intervienen en el modelado de un sistema. A esos niveles se les denomina comúnmente con las iniciales M0, M1, M2 y M3.
- **MOF (Meta Object Facility)** es un **meta-meta-modelo** que permite definir meta-modelos (como UML).
- **UML** es un **meta-modelo** que permite definir modelos. Es una instancia de MOF



M3 – MOF . Permite definir meta-modelos. Define los elementos que constituyen los distintos lenguajes de modelado. De esta forma, el concepto de "clase" definido en UML (que pertenece al nivel M2) puede verse como una instancia del correspondiente elemento del nivel M3, en donde se define de forma precisa ese concepto, así como sus características y las relaciones con otros elementos.
M2 – Meta-modelos. Permiten definir modelos.
Los elementos de este nivel son los lenguajes de modelado.
En el caso UML los conceptos propios del nivel M2 son "Clase", "Atributo", o "Asociación"
M1 – Modelos de los sistemas concretos. En este nivel es donde se definen, por ejemplo, los conceptos como "Cliente", "Compra" y "Libro"
M0 – Datos o instancias. Modela el sistema real, y sus elementos son las instancias que componen dicho sistema. Ejemplos de dichos elementos son el cliente "Juan" que vive en "Paseo de la Castellana, Madrid" y ha comprado el ejemplar número "123" del libro "Ulises".

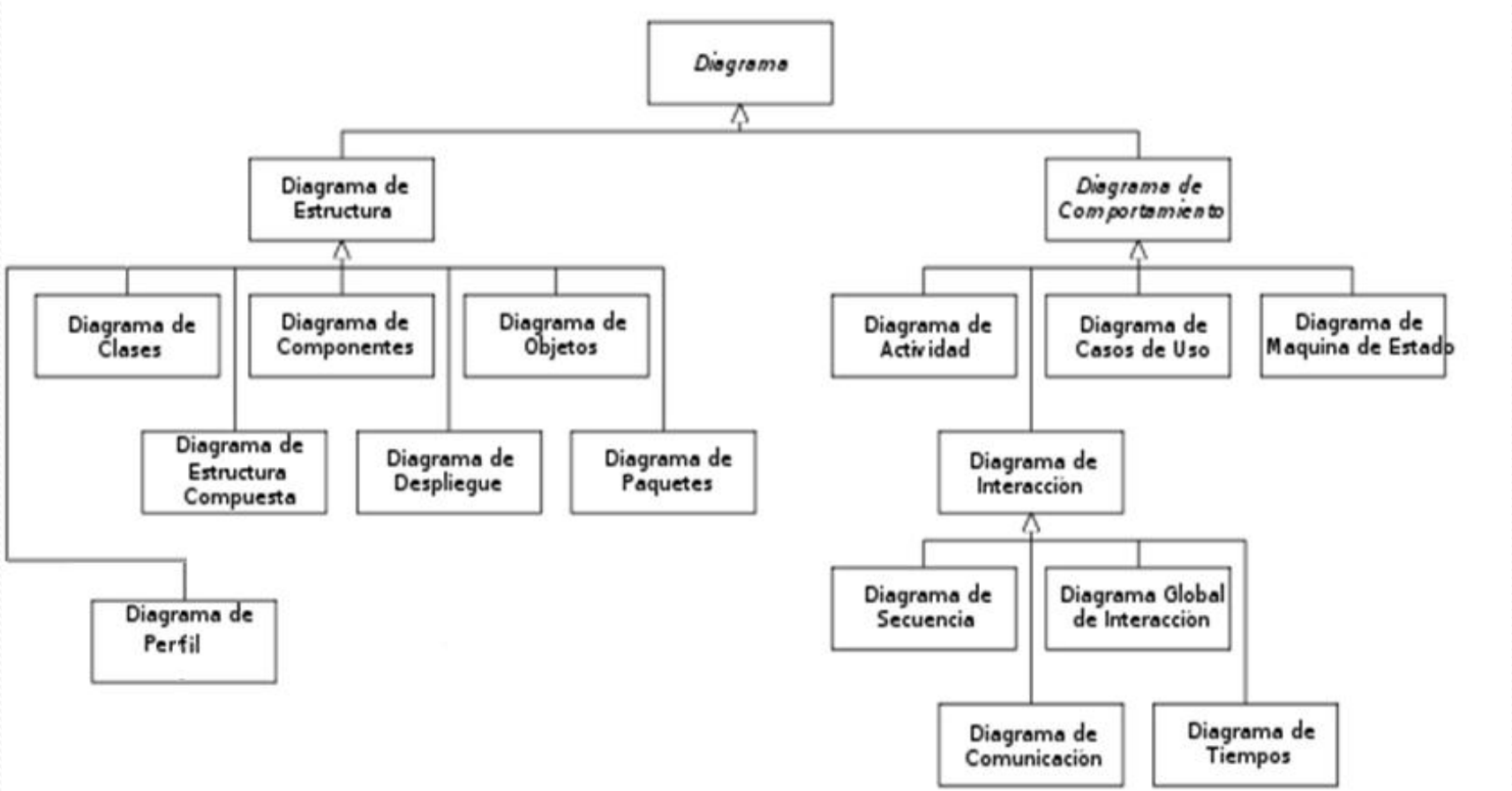
Arquitectura de 4 niveles



Diagramas UML

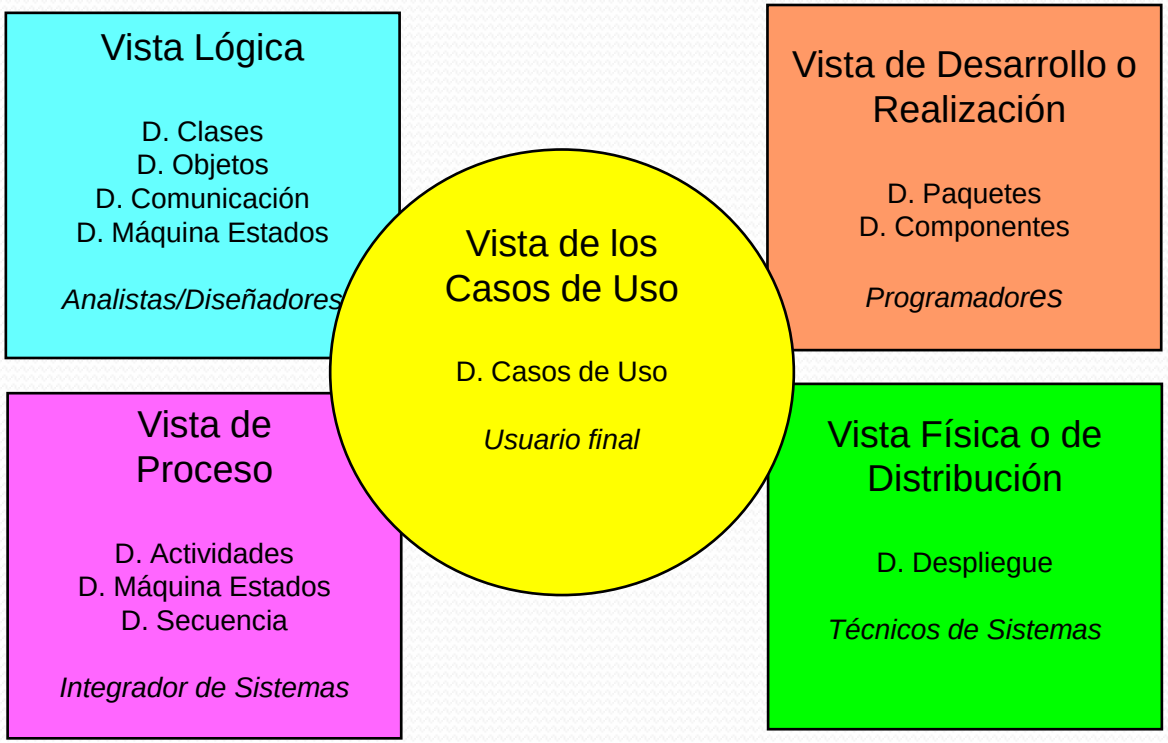
- Presentan diferentes perspectivas de un sistema.
- Cada diagrama tiene fines distintos dentro del proceso de desarrollo.
- Son una combinación de diferentes elementos gráficos que se combinan siguiendo unas reglas.
- Tipos de diagramas UML:
  - **Diagramas de estructura**: proporcionan una vista estática del sistema.
  - **Diagramas de comportamiento**: proporcionan una vista dinámica del sistema (muestran un cambio progresivo).

Tipos de diagramas UML 2.x



Perspectivas: modelo de vistas "4+1" de Philippe Kruchten

- **Vista de Casos de Uso**: comprende la descripción del comportamiento del sistema tal y como es percibido por los usuarios finales, analistas y encargados de las pruebas.
- **Vista Lógica**: comprende las clases, interfaces y colaboraciones que forman el vocabulario del problema y de la solución.
- **Vista de Proceso**: describe los procesos del sistema.
- **Vista de Desarrollo**: describe cómo está organizado el sistema en módulos y componentes.
- **Vista Física**: describe la arquitectura física del sistema.

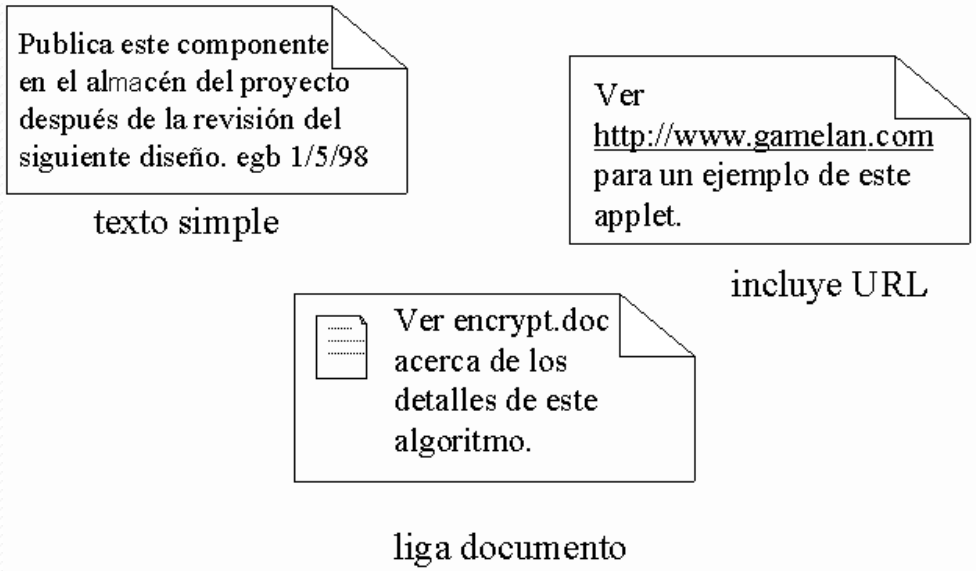


Qué modela cada diagrama

- **Casos de uso**: interacciones entre el sistema y usuarios o sistemas externos. Requerimientos funcionales.
- **Actividad**: actividades secuenciales o paralelas del sistema. Flujos de trabajo.
- **Secuencia**: interacciones entre objetos en las que el orden de las interacciones es importante.
- **Comunicación**: es semánticamente equivalente al diagrama de secuencia. Llamado diagrama de colaboración en UML (1.x). Mientras que el diagrama de secuencia se organiza de acuerdo al tiempo, el diagrama de colaboración se organiza de acuerdo al espacio.
- **Tiempos**: muestra el cambio en el estado o valor de uno o más elementos en el tiempo.
- **Global de interacción**: agrupación de los diagramas de secuencia, comunicación y tiempo.
- **Clases**: clases, interfaces y relaciones entre ellos.
- **Objetos**: instancias de las clases del diagrama de clases que son importantes para el sistema.
- **Estructura compuesta**: muestra la estructura interna de una clase.
- **Componentes**: componentes importantes del sistema y los interfaces para interactuar con ellos.
- **Paquetes**: organización jerárquica de clases y componentes.
- **Máquina de estados**: estados de un objeto a lo largo de su ciclo de vida y los eventos que pueden cambiar ese estado.
- **Despliegue**: cómo se despliega el sistema en el mundo real. Ubicación del SW en el HW. También se le conoce como diagrama de distribución.
- **Perfil**: se utiliza para proveer un mecanismo de extensión al UML estándar. Está basado en la definición de estereotipos, restricciones y valores etiquetados personalizados.

Notas en UML

- **Notas**: son el equivalente gráfico de un papel adhesivo. Se utilizan para aclarar algún punto dentro de un diagrama.



Estereotipos

- Son el mecanismo de extensibilidad incorporado más utilizado dentro de UML.
- Representa una **distinción de uso**, encapsula los comportamientos.
- Un **estereotipo** es una subclase de un elemento existente con los mismos atributos y relaciones que ese elemento pero con una intención distinta y, posiblemente, restricciones adicionales, es decir para especializar un elemento.
- Y se representa con **<<estereotipo>>**
- Puede ser aplicado a cualquier elemento de modelado, incluyendo clases, paquetes, relaciones de herencia, etc.

Estereotipos

Name	Applies to	Description
«Auxiliary»	Class	A class that supports another more central or fundamental class, typically by implementing secondary logic or control flow. The class that the auxiliary supports may be defined explicitly using a Focus class or implicitly by a dependency relationship. Auxiliary classes are typically used together with Focus classes, and are particularly useful for specifying the secondary business logic or control flow of components during design. See also: «Focus».
«BuildComponent»	Component	A collection of elements defined for the purpose of system level development activities, such as compilation and versioning.
«Call»	Usage	A usage dependency whose source is an operation and whose target is an operation. The relationship may also be subsumed to the class containing an operation, with the meaning that there exists an operation in the class to which the dependency applies. A call dependency specifies that the source operation or an operation in the source class invokes the target operation or an operation in the target class. A call dependency may connect a source operation to any target operation that is within scope including, but not limited to, operations of the enclosing classifier and operations of other visible classifiers.
«Create»	Usage	A usage dependency denoting that the client classifier creates instances of the supplier classifier.
«Create»	BehavioralFeature	Specifies that the designated feature creates an instance of the classifier to which the feature is attached. May be promoted to the Classifier containing the feature.
«Derive»	Abstraction	Specifies a derivation relationship among model elements that are usually, but not necessarily, of the same type. A derived dependency specifies that the client may be computed from the supplier. The mapping specifies the computation. The client may be implemented for design reasons, such as efficiency, even though it is logically redundant.
«Destroy»	BehavioralFeature	Specifies that the designated feature destroys an instance of the classifier to which the feature is attached. May be promoted to the classifier containing the feature.
«Document»	Artifact	A generic file that is not a «Source» file or «Executable». Subclass of «File».
«Entity»	Component	A persistent information component representing a business concept.
«Executable»	Artifact	A program file that can be executed on a computer system. Subclass of «File».
«File»	Artifact	A physical file in the context of the system developed.
«Focus»	Class	A class that defines the core logic or control flow for one or more auxiliary classes that support it. Support classes may be defined explicitly using Auxiliary classes or implicitly by dependency relationships. Focus classes are typically used together with one or more Auxiliary classes, and are particularly useful for specifying the core business logic or control flow of components during design. See also: «Auxiliary».
«Framework»	Package	A package that contains model elements that specify a reusable architecture for all or part of a system. Frameworks typically include classes, patterns, or templates. When frameworks are specialized for an application domain they are sometimes referred to as application frameworks.

Estereotipos

«Implement»	Component	A component definition that is not intended to have a specification itself. Rather, it is an implementation for a separate «Specification» to which it has a Dependency.
«Implementation Class»	Class	The implementation of a class in some programming language (e.g., C++, Smalltalk, Java) in which an instance may not have more than one class. This is in contrast to Class, for which an instance may have multiple classes at one time and may gain or lose classes over time, and an object (a child of instance) may dynamically have multiple classes. An Implementation class is said to realize a Classifier if it provides all of the operations defined for the Classifier with the same behavior as specified for the Classifier's operations. An Implementation Class may realize a number of different Types. The physical attributes and associations of the Implementation class do not have to be the same as those of any Classifier it realizes and the Implementation Class may provide methods for its operations in terms of its physical attributes and associations. See also: «Type».
«Instantiate»	Usage	A usage dependency among classifiers indicating that operations on the client create instances of the supplier.
«Library»	Artifact	A static or dynamic library file. Subclass of «File».
«Metaclass»	Class	A class whose instances are also classes.
«Metamodel»	Model	A model that specifies the modeling concepts of some modeling language (e.g., a MOF model). See «Metaclass».
«ModelLibrary»	Package	A package that contains model elements that are intended to be reused by other packages. Model libraries are frequently used in conjunction with applied profiles. This is expressed by defining a dependency between a profile and a model library package, or by defining a model library as contained in a profile package. The classes in a model library are not stereotypes and tagged definitions extending the metamodel. A model library is analogous to a class library in some programming languages. When a model library is defined as a part of a profile, it is imported or deleted with the application or removal of the profile. The profile is implicitly applied to its model library. In the other case, when the model library is defined as an external package imported by a profile, the profile requires that the model library be there in the model at the stage of the profile application. The application or the removal of the profile does not affect the presence of the model library elements.
«Process»	Component	A transaction based component.
«Realization»	Classifier	A classifier that specifies a domain of objects and that also defines the physical implementation of those objects. For example, a Component stereotyped by «Realization» will only have realizing Classifiers that implement behavior specified by a separate «Specification» Component. See «Specification». This differs from «Implementation class» because an «Implementation class» is a realization of a Class that can have features such as attributes and methods that are useful to system designers.
«Refine»	Abstraction	Specifies a refinement relationship between model elements at different semantic levels, such as analysis and design. The mapping specifies the relationship between the two elements or sets of elements. The mapping may or may not be computable, and it may be unidirectional or bidirectional. Refinement can be used to model transformations from analysis to design and other such changes.
«Responsibility»	Usage	A contract or an obligation of an element in its relationship to other elements.
«Script»	Artifact	A script file that can be interpreted by a computer system. Subclass of «File».
«Send»	Usage	A usage dependency whose source is an operation and whose target is a signal, specifying that the source sends the target signal.
«Service»	Component	A stateless, functional component (computes a value).

«Source»	Artifact	A source file that can be compiled into an executable file. Subclass of «File».
«Specification»	Classifier	A classifier that specifies a domain of objects without defining the physical implementation of those objects. For example, a Component stereotyped by «Specification» will only have provided and required interfaces, and is not intended to have any realizingClassifiers as part of its definition. This differs from «Type» because a «Type» can have features such as attributes and methods that are useful to analysts modeling systems. Also see: «Realization»
«Subsystem»	Component	A unit of hierarchical decomposition for large systems. A subsystem is commonly instantiated indirectly. Definitions of subsystems vary widely among domains and methods, and it is expected that domain and method profiles will specialize this construct. A subsystem may be defined to have specification and realization elements. See also: «Specification» and «Realization».
«SystemModel»	Model	A SystemModel is a stereotyped model that contains a collection of models of the same system. A SystemModel also contains all relationships and constraints between model elements contained in different models.
«Trace»	Abstraction	Specifies a trace relationship between model elements or sets of model elements that represent the same concept in different models. Traces are mainly used for tracking requirements and changes across models. As model changes can occur in both directions, the directionality of the dependency can often be ignored. The mapping specifies the relationship between the two, but it is rarely computable and is usually informal.
«Type»	Class	A class that specifies a domain of objects together with the operations applicable to the objects, without defining the physical implementation of those objects. However, it may have attributes and associations. Behavioral specifications for type operations may be expressed using, for example, activity diagrams. An object may have at most one implementation class, however it may conform to multiple different types. See also: «ImplementationClass».
«Utility»	Class	A class that has no instances, but rather denotes a named collection of non-member attributes and operations, all of which are class-scoped.

UNIFIED MODELING LANGUAGE

Tipos primitivos de datos en UML

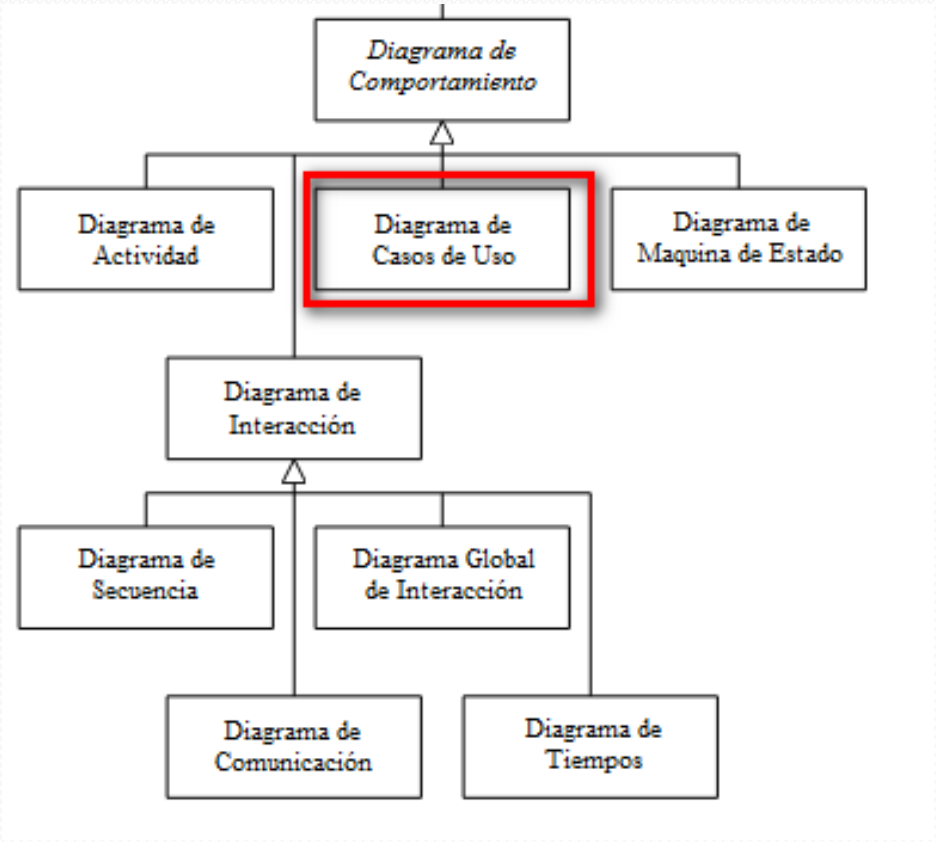
Integer	An instance of Integer is a value in the (infinite) set of integers (…-2, -1, 0, 1, 2…).
Boolean	An instance of Boolean is one of the predefined values *true* and *false*.
String	An instance of String defines a sequence of characters. Character sets may include non-Roman alphabets. The semantics of the string itself depends on its purpose; it can be a comment, computational language expression, OCL expression, etc.
UnlimitedNatural	An instance of UnlimitedNatural is a value in the (infinite) set of natural numbers (0, 1, 2…) plus *unbounded*. The value of *unbounded* is shown using an asterisk ('*'). UnlimitedNatural values are typically used to denote the upper bound of a range, such as a multiplicity; *unbounded* is used whenever the range is specified to have no upper bound.
Real	An instance of Real is a value in the (infinite) set of real numbers. Typically an implementation will represent Real numbers using a floating point standard such as IEEE 754.

¿Cuál de los siguientes NO es un tipo primitivo de datos definido en el Core de UML según la especificación de UML 2.3? (TIC A2 AGE 2011 Promo Interna)

- a) Integer
- b) Boolean
- c) UnlimitedNatural
- d) Char ----------

- **En UML, NO es un diagrama dinámico (examen TIC A1 2011):**
  - Diagrama de secuencia.
  - Diagrama de estado.
  - Diagrama de despliegue. ----------
  - Diagrama de colaboración.
- **Indique cual NO es la opción correcta para ser considerada como uno de los tipos de diagramas que se usan en UML (A1 Xunta Galicia 2011):**
  - Diagrama de colaboración
  - Diagrama de estados
  - Diagrama de atributos ----------
  - Diagrama de distribución
- **En UML 2.0 hay 13 tipos diferentes de diagramas. Estos se pueden dividir en: Diagramas de estructura y de Comportamiento. ¿Cuál de los siguientes es un diagrama de comportamiento? (TIC A2 AGE 2011 Promo Interna)**
  - Diagrama de clases.
  - Diagrama de casos de uso. ----------
  - Diagrama de componentes.
  - Diagrama de paquetes.
- **¿Cuál de los siguientes diagramas no aparece recogido en UML? (TIC A2 AGE 2010):**
  - Diagrama de secuencia
  - Diagrama de actividades
  - Diagrama de estados
  - Diagrama de perspectiva ----------

Casos de uso


Diagrama de Comportamiento
Diagrama de Actividad
Diagrama de Casos de Uso
Diagrama de Maquina de Estado
Diagrama de Interacción
Diagrama de Secuencia
Diagrama Global de Interacción
Diagrama de Comunicación
Diagrama de Tiempos

Casos de uso

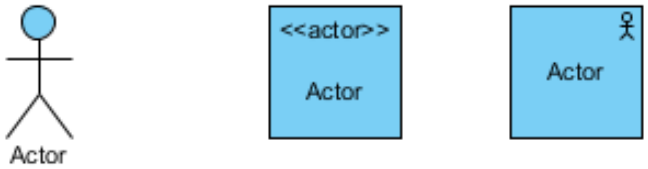
- ¿Qué son los casos de uso?
  - Casos de Uso es una técnica para capturar información de cómo un sistema o negocio trabaja actualmente, o de cómo se desea que trabaje.
  - Se usan para la captura de requisitos del sistema.
  - Representan los **requisitos funcionales** del sistema.
- **Actores**: Entidades externas que inician las secuencias de eventos.
- Los actores interactúan con los casos de uso

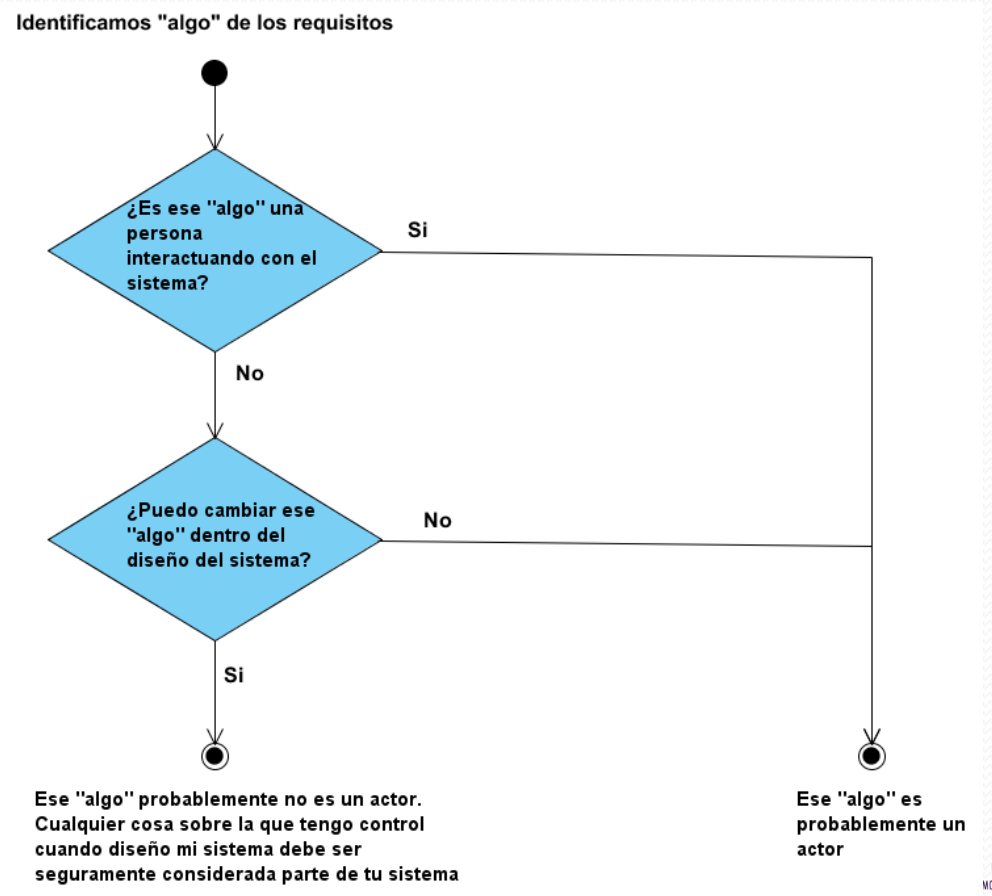
Diagrama de casos de uso

- Definición.
  - Un diagrama de Casos de Uso es un grafo con:
    - Actores
    - Casos de Uso
    - Comunicaciones (participación) entre Actores y Casos de Uso
    - Relaciones entre Casos de Uso.
- Elementos participantes
  - Actores
  - Casos de uso

Actores

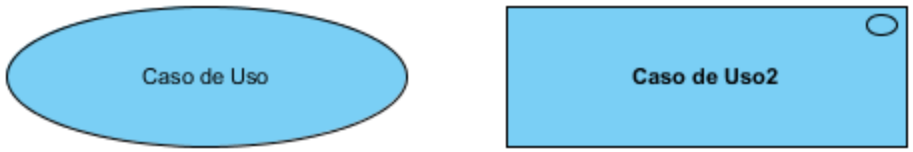
- Un actor representa un conjunto coherente de **roles** que juegan los usuarios de los casos de uso al interaccionar con el sistema.
- Un actor es cualquier persona, organización o sistema que interacciona con la aplicación, siempre y cuando el actor sea **externo** a la misma.
- Un actor necesita el caso de uso y/o participa en él.





Casos de uso

- Los casos de uso modelan un diálogo entre un actor y el sistema describiendo la funcionalidad que ofrece el sistema al actor.
- El conjunto de casos de uso del sistema constituyen todas la formas de uso definidas en el sistema.
- Se representa mediante un óvalo con la descripción del caso de uso en su interior o en forma de recuadro con un ovalo dentro



Relaciones

- Caso de Uso - Actor
  - **Asociación** ``comunica''. Representa una comunicación entre un actor y un caso de uso.
    - Puede ser en general navegable en ambas direcciones (de actor a caso de uso y de caso de uso a actor), o en una sola dirección (de actor a caso de uso o de caso de uso a actor). La dirección en que es navegable la asociación refleja quién inicia la comunicación.
    - La flecha indica el flujo de información. En UML, el sentido de la flecha **no es necesario**.



- Ejemplo. Los estudiantes se matriculan en un curso y por lo tanto participan en el caso de uso “Matriculación en Curso”. El Secretario realiza la matriculación.

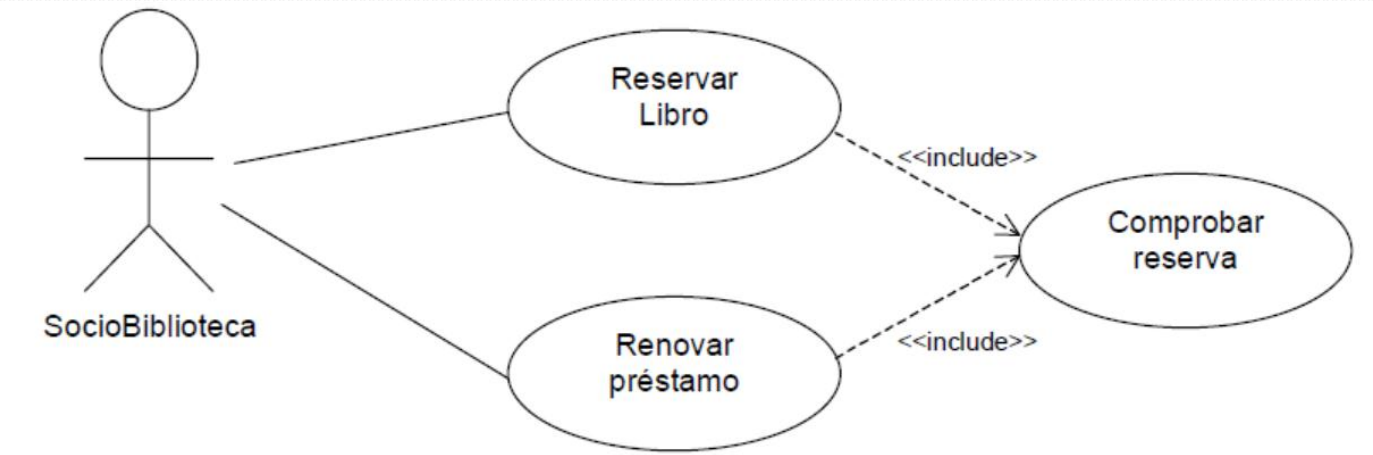
Diagrama de casos de uso: Relaciones

- **Caso de Uso - Caso de Uso**
  - Existen tres tipos de relación entre casos de uso
    - Inclusión (<<include>> o <<use>>)
    - Extensión (<<Extend>>)
    - Generalización

- **Cual de las siguientes afirmaciones es FALSA respecto de UML (Unified Modeling Language)?**
  - Los paquetes pueden contener clases, objetos, relaciones, componentes, nodos y diagramas asociados
  - Los diagramas de clases expresan la estructura estática de un sistema, con las clases, los atributos y las operaciones.
  - Las relaciones entre los casos de uso son de subordinación, dependencia, igualdad e superioridad
    ---
  - Los nodos del diagrama de despliegue se representan con un cubo, y las líneas que los unen simbolizan los soportes de las comunicaciones

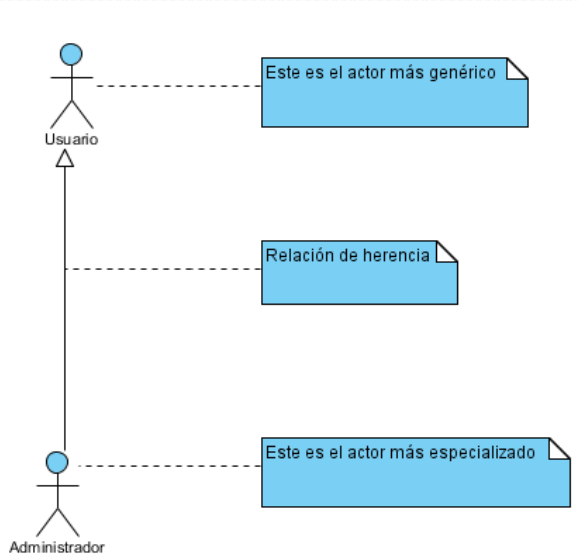
Inclusión

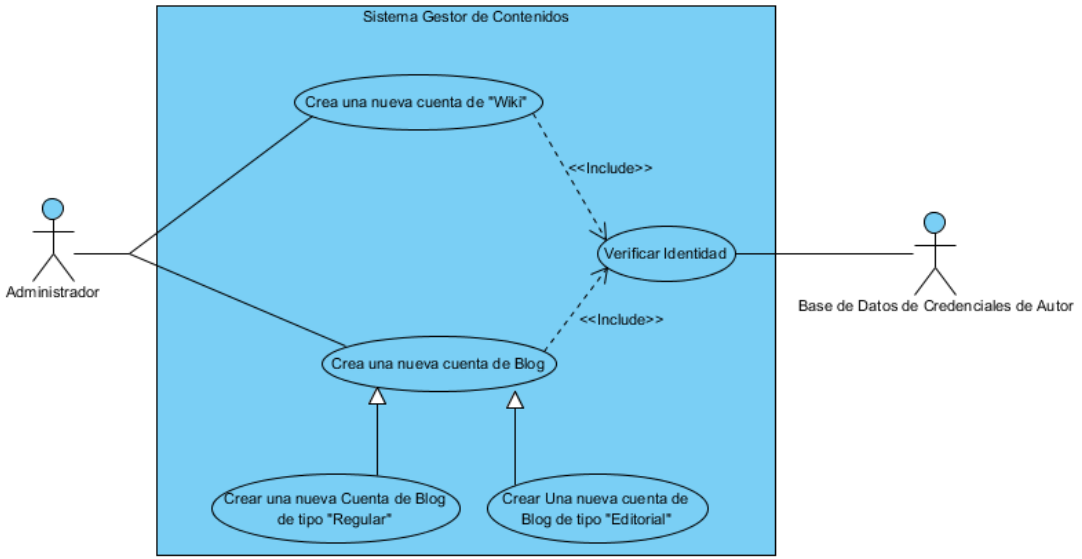
- Relación de dependencia entre dos casos de uso que denota la inclusión del comportamiento de un escenario en otro.
- Si X incluye Y, Y siempre se realiza cuando se realiza X.
- Se estereotipa mediante <<uses>> o <<include>>



Generalización

- La Generalización es la actividad de identificar elementos en común entre conceptos y definir las relaciones de una superclase (concepto general) y subclase (concepto especializado).
- Este tipo de relaciones se pueden dar tanto entre casos de uso como entre actores.
- En la generalización un caso de uso **hereda el comportamiento y significado de otro**.
- El caso de uso origen hereda la especificación del caso de uso destino y posiblemente la modifica y/o extiende.
- Cada paso del caso de uso padre es heredado y debe ser realizado por el hijo, así como todas sus relaciones (por tanto se heredan los "includes" y relaciones con actores que tenga)



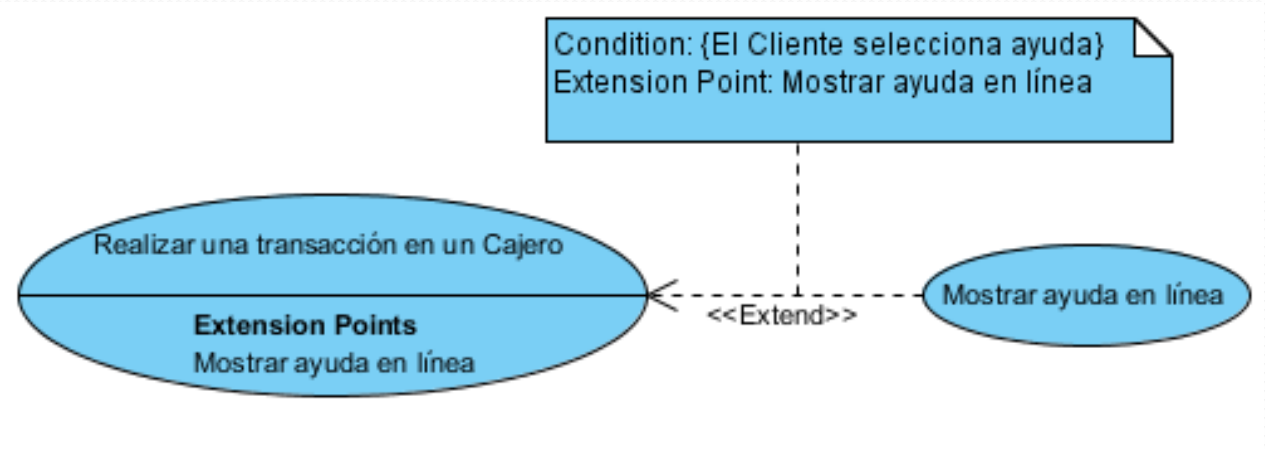


Extensión

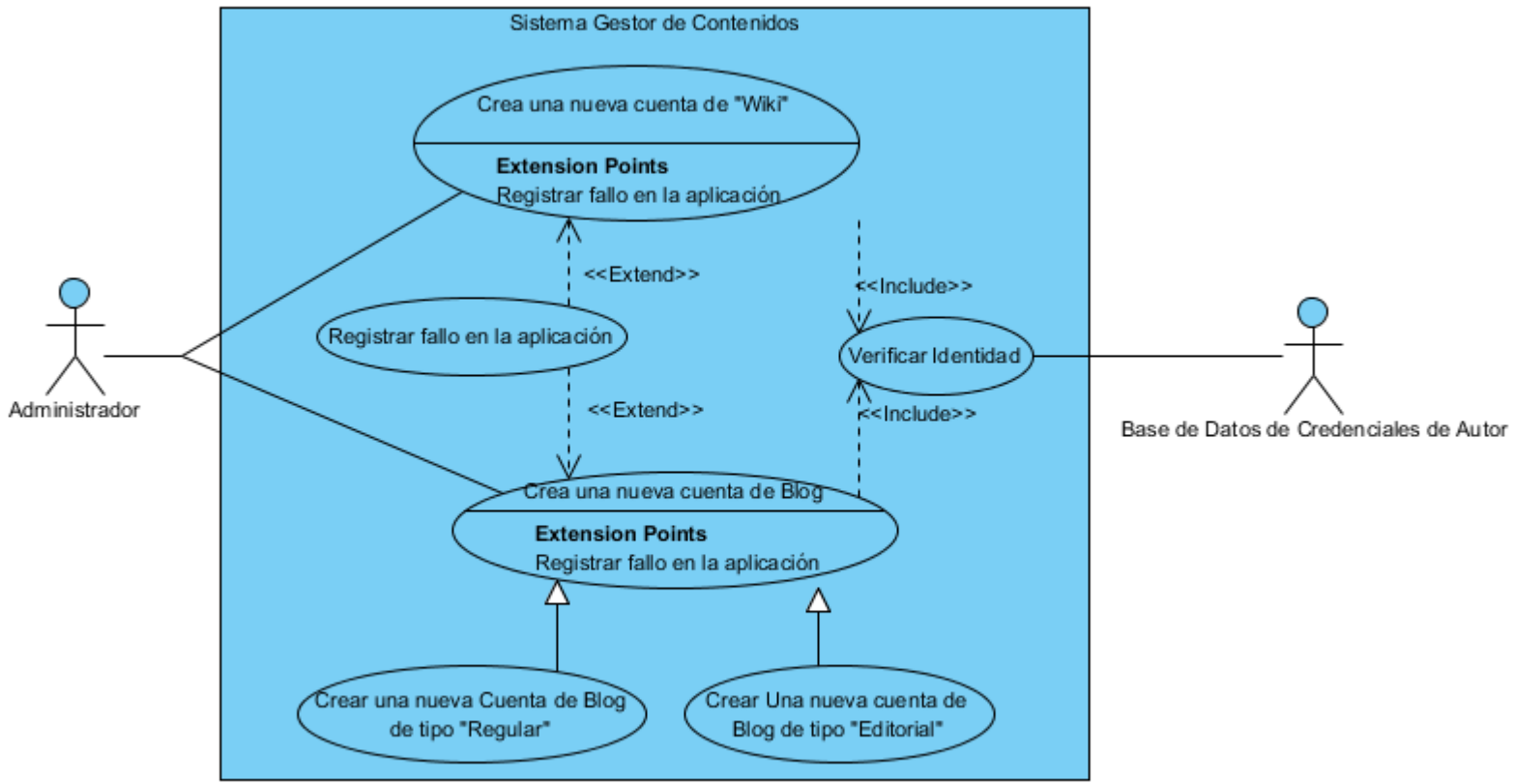
- El caso de uso origen extiende el comportamiento del caso de uso destino.
- Si Y extiende de X, bajo ciertas condiciones de ejecución de X también se ejecuta Y.
- Sirve para modelar
  - la parte **opcional** del sistema
  - un subflujo que sólo se ejecuta bajo ciertas condiciones
  - varios flujos que se pueden insertar en un punto
- Se utiliza para reutilizar el comportamiento de otro caso de uso bajo ciertas condiciones.
- Cada extensión se muestra de manera opcional en el caso de uso mediante un **punto de extensión**
- Se muestra como una línea discontinua que va desde el caso de uso que extiende al extendido.
- Se etiqueta con la palabra clave <<extend>>
- La condición de la relación y el punto de extensión al que hace referencia se puede de manera opcional mostrar en una nota adherida a la relación.

Notación

- Ejemplo:
  - El punto de extensión es “Mostrar ayuda en línea”
  - El caso de uso de ayuda será ejecutado si el usuario selecciona la ayuda.

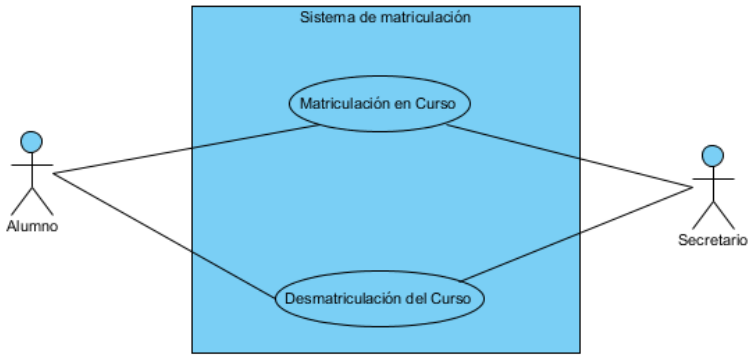


Ejemplo relaciones entre casos de uso

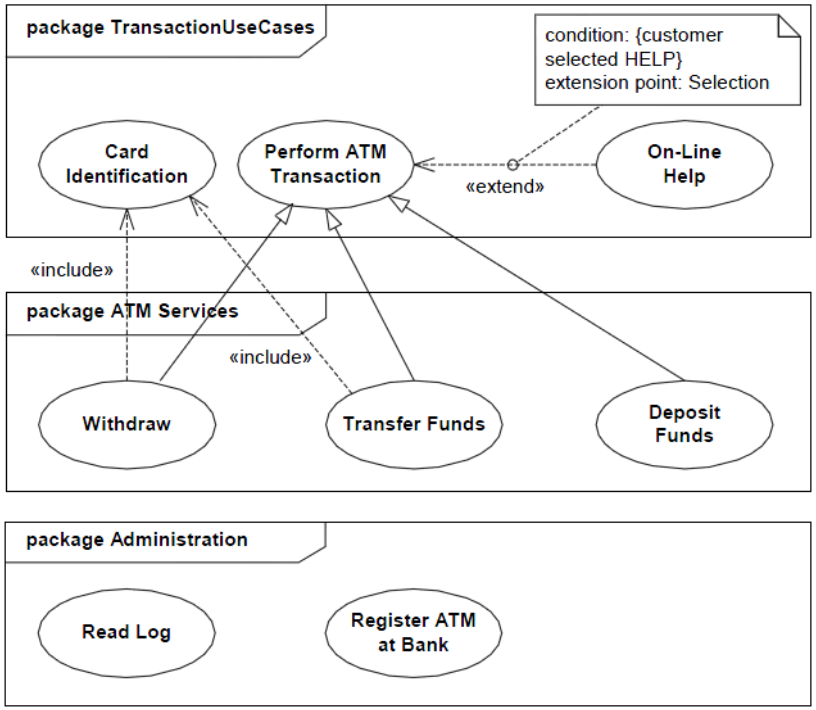


Límites del sistema

- Se especifica mediante una caja
- Es opcional



- Los casos de uso incluso se pueden clasificar en distintos paquetes del sistema



Tipos de Casos de Uso

- **Primario**: El caso de uso representa una funcionalidad básica del sistema.
- **Secundario**: El caso de uso es un proceso menor o de rara ejecución.
- **Opcional**. El caso de uso puede no ser considerado.
- **Esencial**: Un caso de uso cuya ejecución depende muy poco del estado de la tecnología o de los detalles de implementación.
  - Por lo general todos los caso de uso de alto nivel serán esenciales.
- **Real.** Describe un proceso dependiente de la implementación (habrá que poner mas cuidado al detallarlo)

Especificación de los casos de uso

- Los casos de uso se especificarán siguiendo una plantilla.
- Recoge, en un primer momento, una **descripción general**. Esta descripción reflejará posiblemente uno o varios requisitos funcionales del sistema o formará parte de algún requisito.
- Se puede completar la descripción definiendo cuáles son las **precondiciones** y **postcondiciones** del sistema, es decir, qué condiciones deben cumplirse para que se realice un caso de uso y cuáles son aquellas condiciones que se cumplen posteriormente al caso de uso.
- También se pueden enumerar los diferentes **escenarios** del caso de uso si los tuviese y dar una breve descripción de ellos.
  - Los escenarios son los distintos caminos por los que puede evolucionar un caso de uso, dependiendo de las condiciones que se van dando en su realización.
  - Los escenarios se describen dentro de la plantilla.

Ejemplo plantilla para un caso de uso

CU-XXX	Nombre del caso de uso	
Actores		
Propósito		
Descripción		
Tipo	Primario, Esencial.	
Referencias	FRQ-XXXX	
Precondiciones		
Escenario	Acción del actor	Respuesta del sistema
Escenario alternativo ...	Acción del actor	Respuesta del sistema
Excepciones		
Postcondiciones		
Interfaz usuario	UI-XXXX	

Diagrama de Máquina de Estados

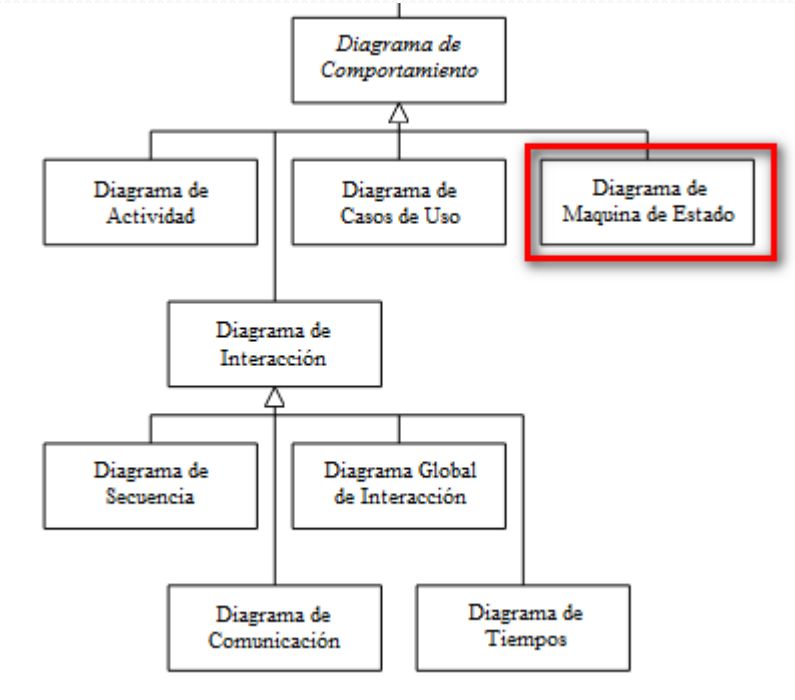


Diagrama de Máquina de Estados

- Los Diagramas de Máquina de Estados representan **estados** y **transiciones** de los componentes de un sistema.
- Modelan el comportamiento de los **objetos significativos**:
  - Cada objeto está en un estado en cierto instante
  - El estado en el que se encuentra un objeto determina su comportamiento
  - El estado está caracterizado parcialmente por los valores de los atributos del objeto
  - Cada objeto sigue el comportamiento descrito en el Diagrama de Máquina de Estados asociado a su clase
- Los objetos no significativos se puede considerar que tienen un único estado.
- En UML 1.x se llamaba Diagrama de transición de Estados

Un diagrama de transición de estados (TIC A1 AGE 200):

- Describe el comportamiento dinámico del sistema de información mostrando cómo interactúan los objetos entre sí.
- Es un tipo de diagrama de interacción cuyo objetivo es describir el comportamiento dinámico del sistema de información haciendo énfasis en la secuencia de los mensajes intercambiados por los objetos.
- Consiste en la descomposición sucesiva de los procesos, desde un nivel general, hasta llegar al nivel de detalle necesario para reflejar toda la semántica que debe soportar el sistema en estudio.
- Representa los estados que puede tomar un componente o un sistema y muestra los eventos que implican el cambio de un estado a otro. ----------

Estados

- Representa algún comportamiento que es observable externamente.
- Perdura durante un periodo de tiempo finito.
- Viene dado por el valor de uno o varios atributos que lo caracterizan en un momento dado.
- Elementos de un estado
  - Nombre
  - Acciones entrada/salida
  - Transiciones internas
  - Subestados
  - Eventos diferidos



Estado Final

- Tipo de estado especial que indica que la región donde aparece ha sido completada.
- Si el resto de regiones también se han completado, entonces significa que la máquina de estados entera ha sido completada.
- Estado especial que:
  - No tiene transiciones de salida.
  - No puede tener definidas regiones dentro.
  - No puede referenciar a otras submáquinas de estado.
  - No tiene comportamiento interno ni transiciones internas.
- Se muestra como un círculo rodeando a otro pequeño círculo sólido interno.

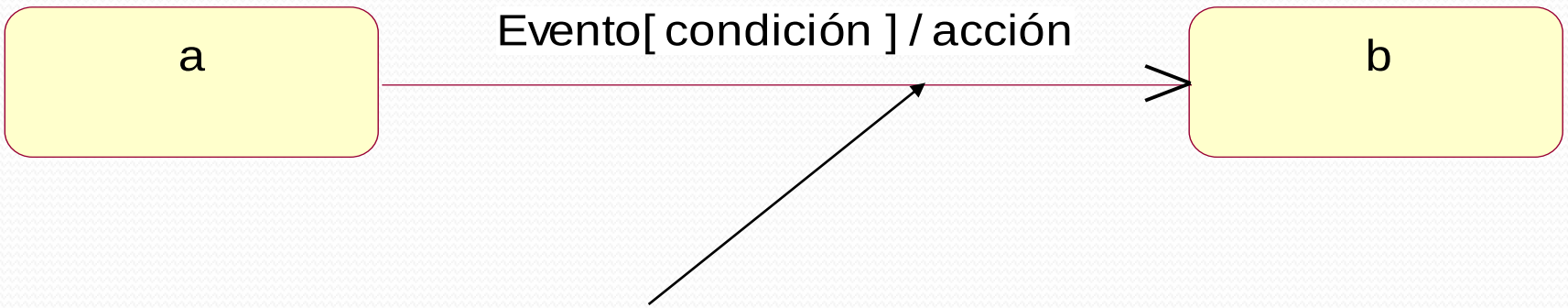


Transiciones

- Una transición se representa con una flecha continua que une dos estados y que se dirige al estado al que cambia el componente.
- Una transición de un estado A a un estado B, se produce cuando se origina el evento asociado y se satisface la condición especificada, en cuyo caso se ejecuta la acción de salida de A, la acción de entrada a B y la acción asociada a la transición.
  - nombre-evento (parámetros) [condición] /acción
- Elementos de una transición:
  - Estados origen y destino
  - Evento de disparo
  - Condición de guarda
  - Acción

Transiciones: Acción

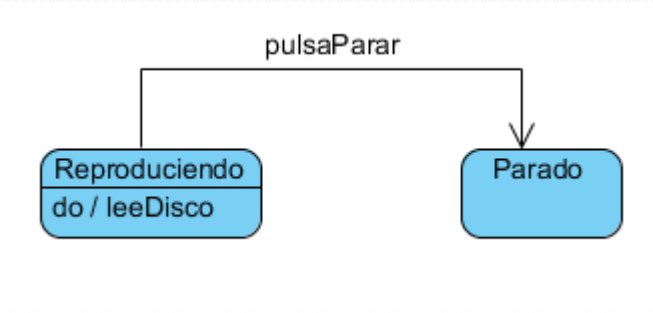
- La **acción** será el comportamiento que se obtiene cuando ocurre la transición.
- Se considera que ocurren de forma rápida y no interrumpible



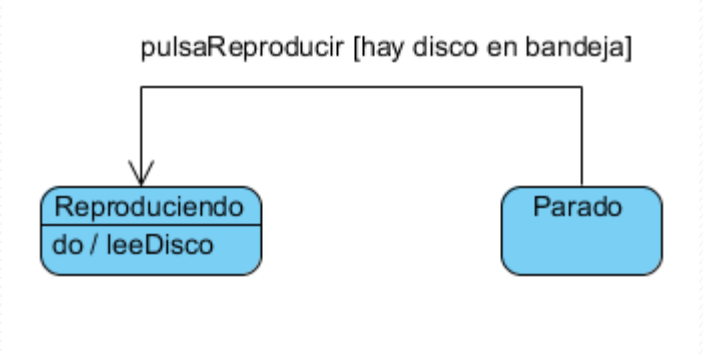
Dicha acción se considera instantánea

Transiciones

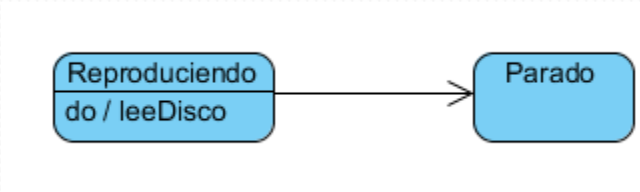
- Ejemplos



La transición se produce al pulsar parar



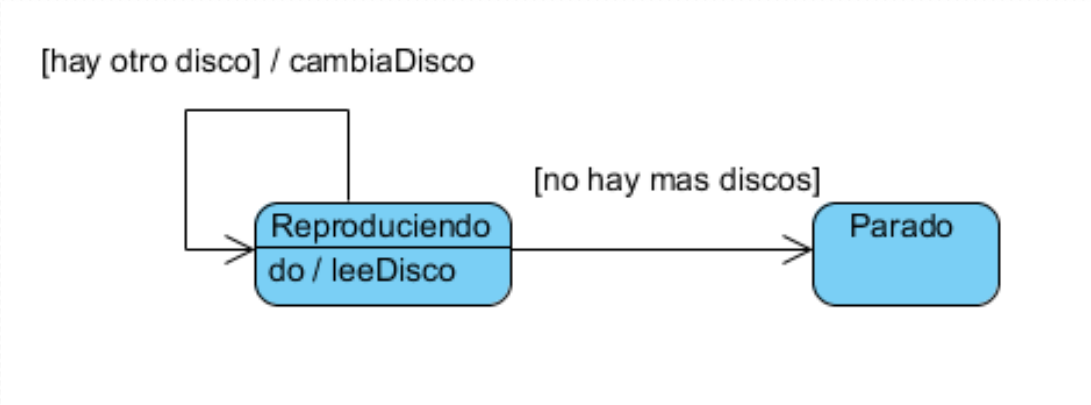
La transición se produce al pulsar reproducir y si se cumple la condición de que hay un disco en la bandeja



La transición se produce cuando ha terminado de leer el disco

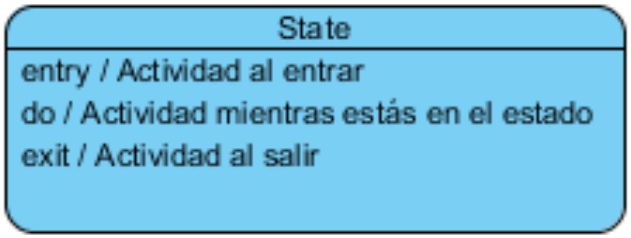
Transiciones

- Las condiciones de guarda pueden utilizarse para elegir flujos de transición



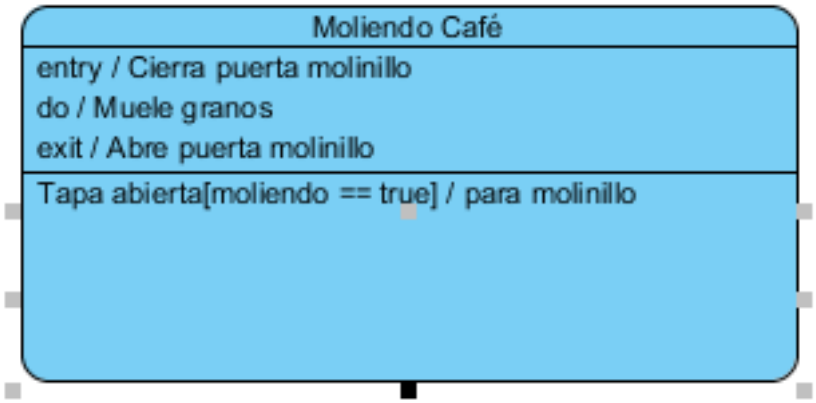
Comportamiento interno

- Es el comportamiento del estado cuando está activo
- Su notación es:
  - Etiqueta/comportamiento
- Existen tres tipos de comportamiento interno:
  - Comportamiento interno de entrada “Entry” -> **Entry/Comportamiento**
  - Comportamiento interno “Doing” -> **Do/Comportamiento**
  - Comportamiento interno de salida “Exit” -> **Exit/Comportamiento**



Transiciones internas

- Es una transición que **permanece en el mismo estado**, en vez de involucrar dos estados distintos.
- Representa un evento que **no causa cambio de estado**.
- Se diferencia de las transiciones que el estado se hace a si mismo en que las transiciones internas no lanzan las actividades de entrada y salida
- Se denota como una cadena adicional en el compartimiento de acciones del estado.

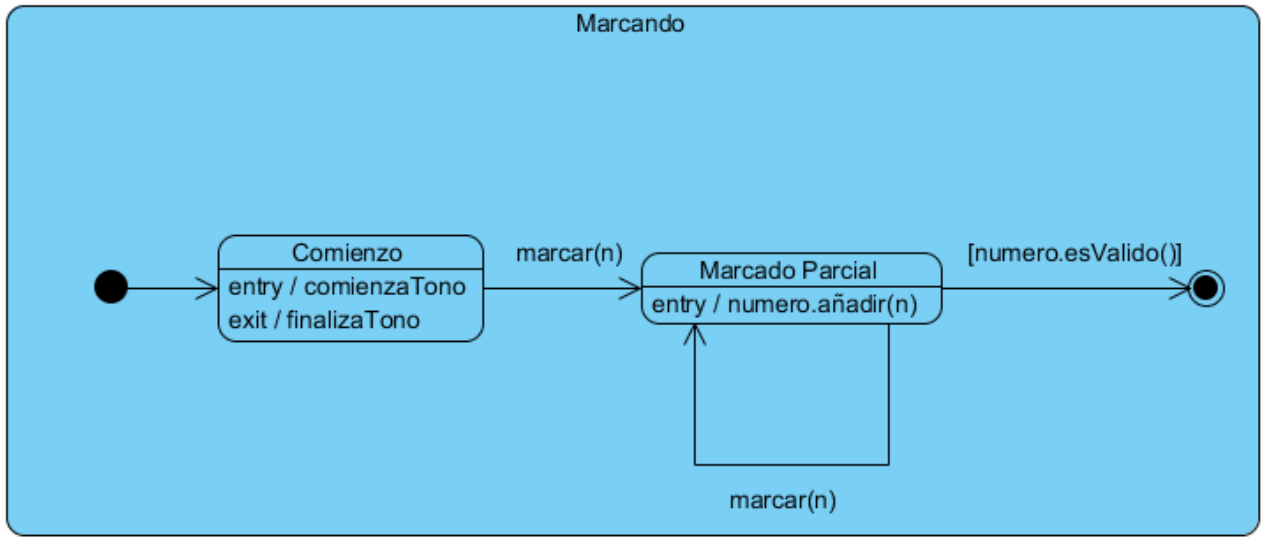


Estado compuesto

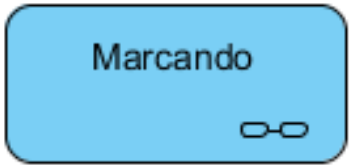
- Es un estado que contiene uno o mas diagramas de estado
- Cada diagrama pertenece a una región.
- Las regiones están separadas dentro del estado por líneas punteadas.
- Los estados dentro de las regiones son subestados del estado compuesto.
- Los **subestados** tienen transiciones entre ellos y conexiones al nivel superior (**superestado**).
- Las conexiones se ven al nivel inferior como estados de inicio o fin, los cuales se suponen conectados a las entradas y salidas del nivel inmediatamente superior. Estos estados iniciales y finales se consideran **pseudoestados**, pues son más un artificio notacional que un elemento plenamente significativo.
- Los subestados heredan las variables de estado y las transiciones externas.
- Cuando se entra en un estado compuesto, cada uno de los diagramas de estados que componen el estado compuesto se vuelven activos y comienzan su ejecución.
- Cada región del estado compuesto se ejecuta en paralelo y es perfectamente aceptable que unas regiones terminen antes que otras.
- Si se produce un evento que hace salir del subestado compuesto, todos los subestados del mismo serán concluidos.

Estado Compuesto

- Ejemplo:



- Notación equivalente



Pseudo-Estados

- Los **Pseudo-estados** son una abstracción del UML 2.x para varios tipos de extremos usados en los Diagramas de Máquinas de Estados. Los Pseudo-estados se usan para expresar caminos de transición complejos:
  - Inicial
  - Terminar
  - Puntos de Entrada
  - Puntos de Salida
  - Decisión
  - Unión
  - Bifurcación
  - Reunión
  - Historia

Pseudo-Estado: Inicial

- Es el punto de comienzo de la máquina de estados.
- El pseudoestado *inicial* es utilizado en los diagramas de Actividades y de Estados.
- Solo puede haber un estado inicial por región definida.
- La transición asociada de salida puede tener un comportamiento, pero no un evento ni una condición de guarda.
- Con las Máquinas de Estados, el elemento inicial es un pseudoestado usado para denotar el estado predeterminado de un estado compuesto
- Puede haber un estado inicial en cada región del estado compuesto.
- Se muestra como un círculo sólido relleno.



Pseudo.Estado: Terminar

- Implica la terminación de la ejecución de la máquina de estados.
- Es equivalente a invocar una destrucción de un objeto.
- Se representa con un aspa



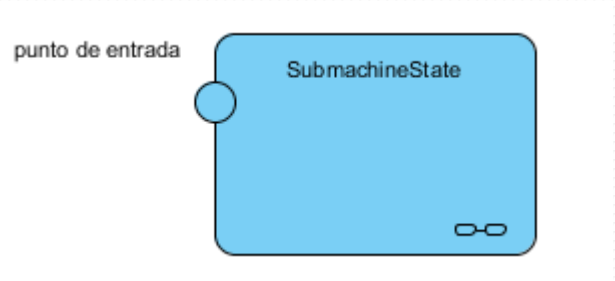
Pseudo-Estado: Puntos de entrada

- Los *Puntos de entrada* se usan para definir el comienzo de una submáquina de estados o de un estado compuesto.
- Un punto de entrada se muestra como un pequeño círculo con el nombre asociado en él.



Punto de entrada

- Pueden ser situados en los bordes de un estado compuesto.



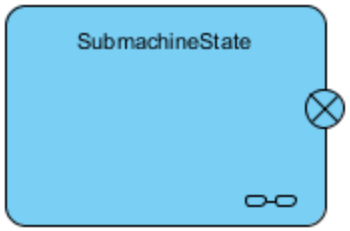
Pseudo-Estado: Puntos de salida

- Los *Puntos de salida* se usan para definir la salida de una submáquina de estados o de un estado compuesto.
- Entrar en un punto de salida dentro de una región de un estado compuesto o máquina de estados referenciada por una submáquina de estados significa salir del estado compuesto o de la submáquina de estados y disparar la posible transición que tenga definida el punto de salida.
- Un punto de salida se muestra como un pequeño círculo con una cruz y con el nombre asociado en él.

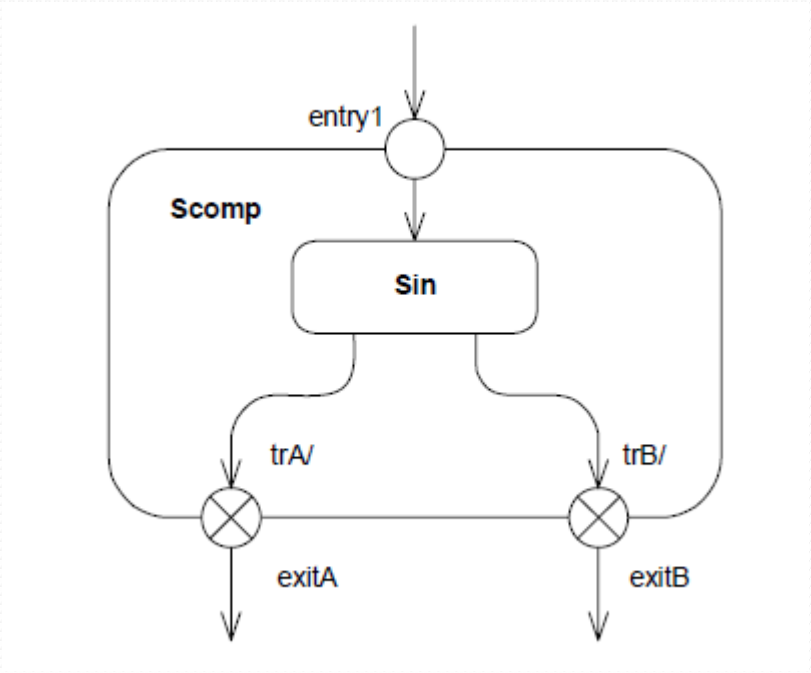


Punto de salida

- Pueden ser situados tanto en los bordes de un estado compuesto.

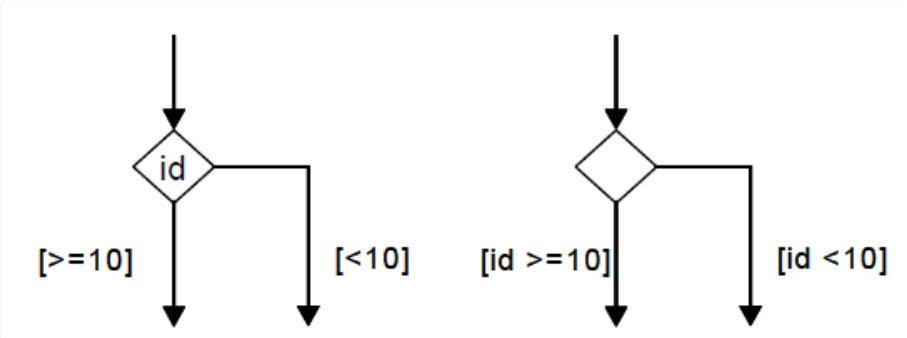


Ejemplo. Puntos de entrada y salida



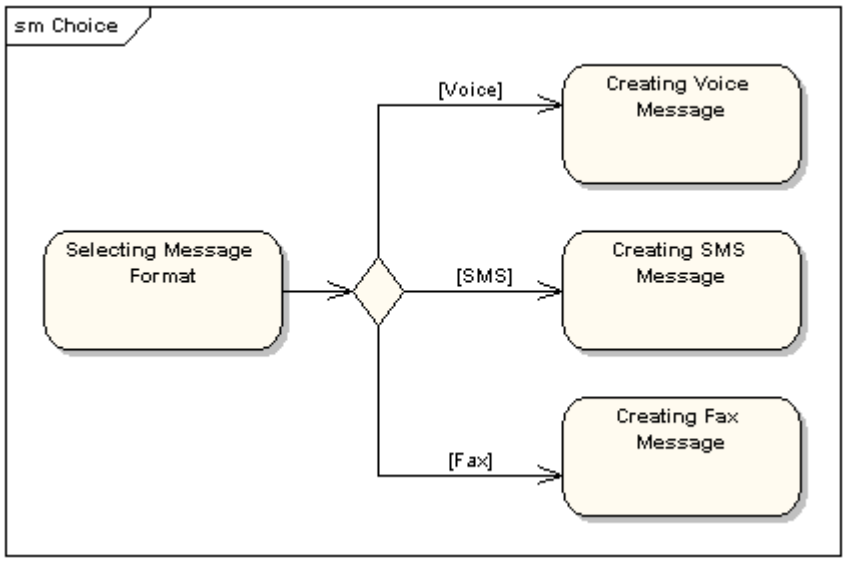
Pseudo-Estado: Decisión.

- Es una rama condicional **dinámica**. Decide los caminos de transición en tiempo de ejecución.
- Hace una evaluación dinámica de las condiciones de guarda de las transiciones de salida.
- *Permite dividir una transición de entrada en múltiples caminos de salida de tal manera que el camino elegido* ***será función de los resultados de las acciones previas*** *llevadas a cabo en el mismo paso de ejecución.*
- ***Siempre debe existir una transición de entrada y dos o mas transiciones de salida que implementarán la decisión dinámica basándose en el evento de la transición de entrada.***
- No confundir con las ramas de decisión estáticas, definidas por los pseudoestados "junction" (se verá mas adelante).
- Si mas de una condición de guarda se cumple, se elige un camino arbitrariamente.
- Si ninguna condición de guarda se cumple entonces el modelo es incorrecto. Hay que evitar esto definiendo siempre una condición de tipo "else" por defecto en caso de que ninguna condición de guarda se cumpliese.



Pseudo-Estado: Decisión.

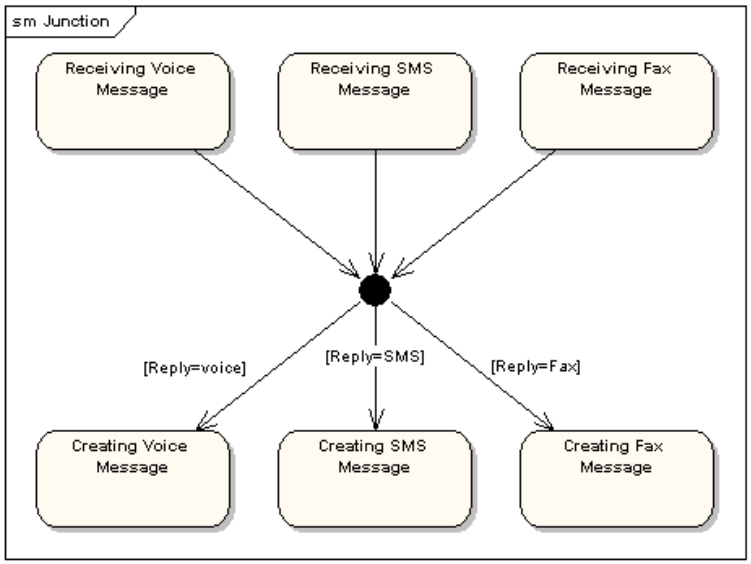
- Ejemplo:
  - El siguiente diagrama muestra que cualquier estado al que se llega después del pseudo estado elección **depende del formato del mensaje** seleccionado durante la ejecución del estado anterior.



Pseudo-Estado: Unión

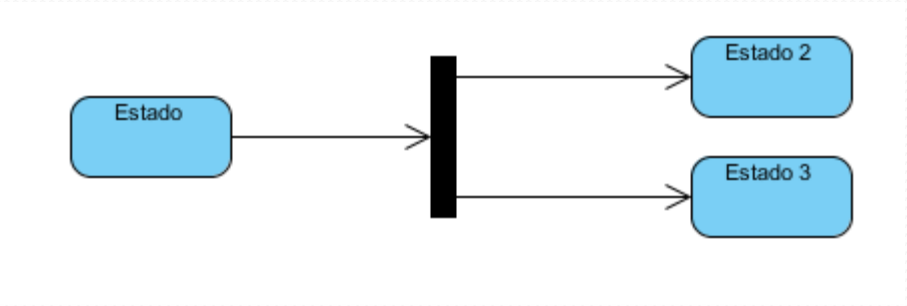
- *Los vértices "**juction**" o unión se utilizan como punto de **confluencia de varias transiciones.***
- *Pueden ser utilizados para:*
  - **Combinar** o mezclar múltiples rutas dentro de una ruta de transición compartida.
  - ***Rama condicional estática**. Dividir la transiciones de entrada en múltiples transiciones de salida cada una con sus condiciones de guarda. Las transiciones cuyas condiciones de guarda no se cumplan son desactivadas, y siempre debe haber una guarda de tipo "else" para el caso de que ninguna condición de guarda se cumpliese.*
- **Las uniones son libres de semántica**; una unión que divide una transición de entrada en transiciones de salida múltiples realiza una rama **condicional estática**, opuesto a un pseudo estado elección que realiza una rama condicional dinámica.
- *Se representan como un pequeño círculo negro donde llegan y parten las transiciones.*

El siguiente diagrama muestra que cualquier estado al que se llega después del pseudo estado elección **depende del formato del mensaje** seleccionado durante la ejecución del estado anterior.



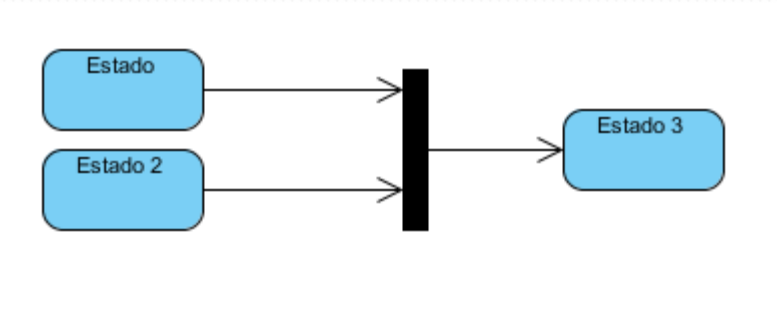
Pseudo-Estado: Bifurcación

- En el nodo bifurcación ("**fork node**") su transición entrante provendrá de un único estado y tendrá múltiples transiciones de salida.
- Las transiciones de salida se realizan de manera simultanea (concurrentemente).
- A diferencia del pseudo-estado decisión, las bifurcaciones no pueden tener disparadores o guardas.

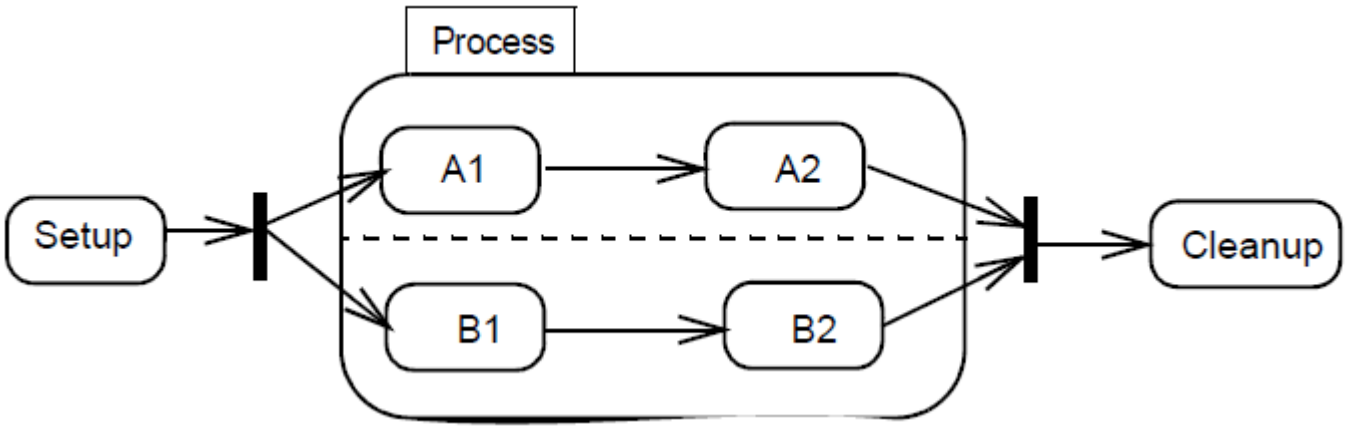


Pseudo-Estado: Reunión

- En el nodo Reunión ("**join node**") muchos estados realizan una transición concurrentemente hacia la unión y dentro de un único estado.
- Todos los estados concurrentes deben terminar para que se continúe la ejecución.
- No puede tener disparadores o guardas.



Ejemplo: Join y Fork

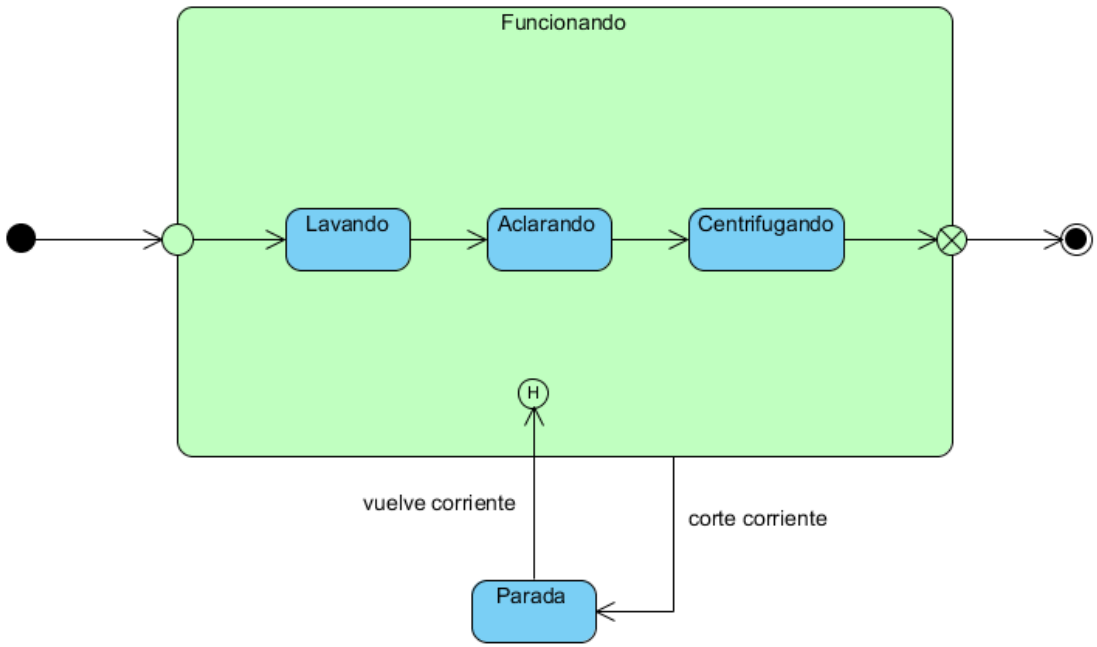


Pseudo-Estado:Historia

- Dos tipos de pseudoestados *históricos* definidos en UML:
  - **Históricos superficial:**
    - Representar el más reciente subestado activo de un estado compuesto.
    - No es recursivo dentro de la configuración del subestado activo.
    - Se representan mediante una H.
  - **Histórico profundo**:
    - Refleja la mas reciente configuración activa del estado compuesto.
    - Incluye subestados activos de todas las regiones.
    - Es recursivo dentro de aquellos subestados activos del subestado (si es que existen).
    - Se representan mediante una H*

Pseudo-Estado: Historia

- Ejemplo:
  - En esta maquina de estado, cuando un lavarropas comience, su proceso será desde el lavado y enjuague hasta el secado. Si hay un corte de luz, el lavarropas se detendrá por lo que pasará al estado Power off (apagado). Luego, cuando la energía retorne, el estado Running (ejecutar) ingresa al símbolo de estado Historial, lo que significa que debería seguir con el proceso donde quedó cuando se corto la energía.



Señales

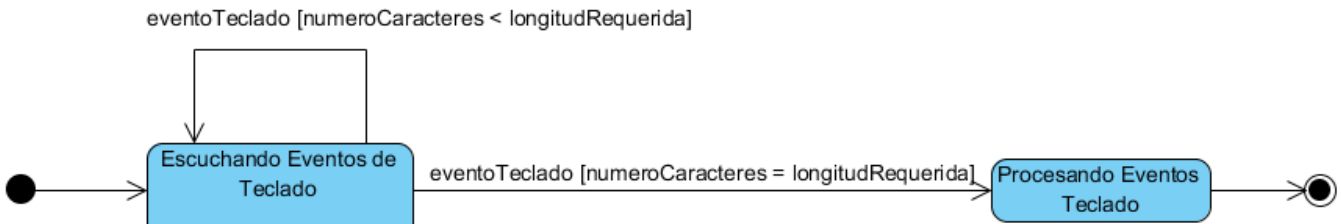
- Se pueden utilizar iconos especiales para resaltar las señales de envío y recepción en las transiciones.
- Representación mediante triángulos con el nombre dentro



Señal de entrada



Señal de salida



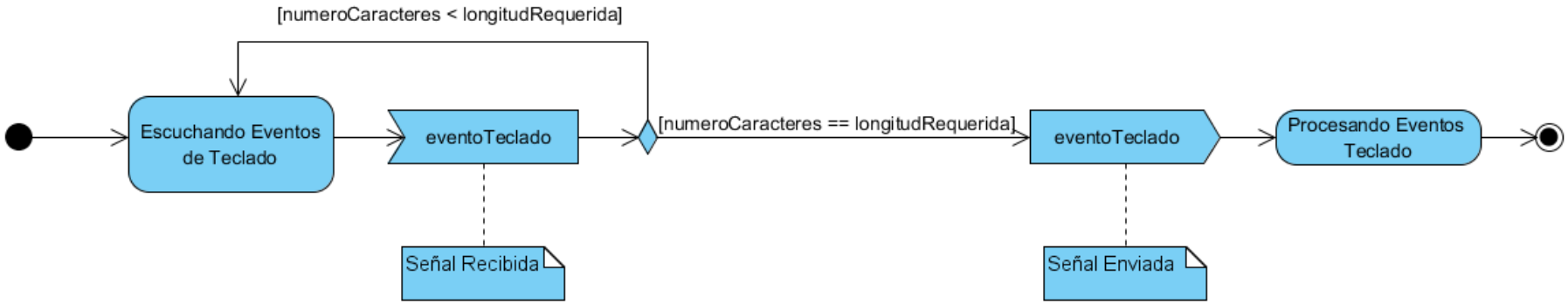


Diagrama de Actividad

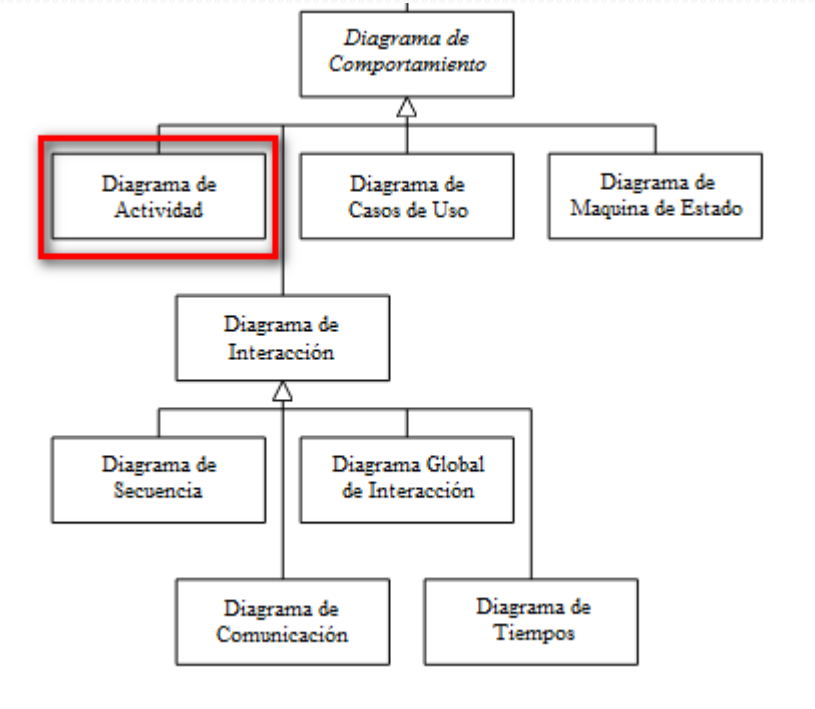


Diagrama de Actividad

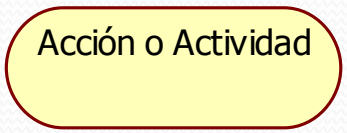
- El Diagrama de Actividades es una variante de los Diagramas de Estados, organizado con respecto a las **acciones.**
- Muestran un flujo de actividades.
- Se utilizan:
  - En el modelado de los procesos del negocio.
    - Permiten especificar y evaluar el flujo de trabajo de los procesos de negocio.
  - En el análisis de un caso de uso.
    - Permiten comprender qué acciones deben ocurrir y cuáles son las dependencias de comportamiento.
  - En la comprensión del flujo de trabajo, a través de varios casos de uso.
    - Permiten representar con claridad las relaciones de flujo de trabajo (workflow) entre varios casos de uso.
  - Cuando se trata de expresar aplicaciones multihilos.
- Generalmente se suelen utilizar para modelar los pasos de un algoritmo.
- Una actividad puede considerarse un estereotipo de estado.

Qué diagrama UML muestra el flujo de acciones en curso, no necesariamente atómicas, en relación a la estructura de estados del sistema? (examen TIC A1 2011):

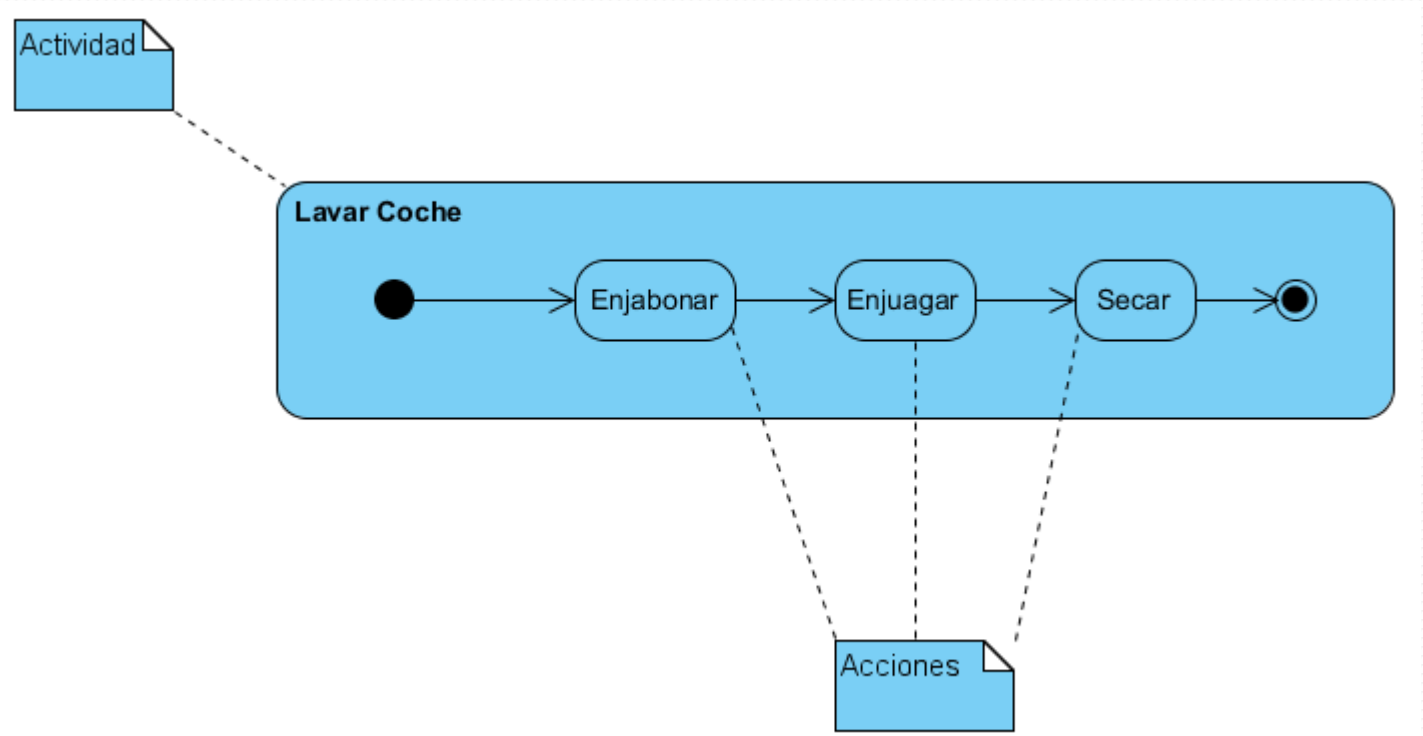
Diagrama de secuencia
Diagrama de estados.
Diagrama de actividad. ----------
Diagrama de colaboración.

Acciones y Actividades

- Una **acción.**
  - Ejecución de una acción atómica.
  - No pueden descomponerse y la aparición de eventos no puede interrumpir su ejecución.
  - Generalmente se considera que su ejecución conlleva un tiempo insignificante.
- Una **actividad**.
  - No es atómica, se compone de otras acciones y actividades.
  - Tienen duración y se ejecutan dentro de un estado del objeto.
  - Las actividades se enlazan por transiciones automáticas.
  - Cuando una actividad termina se desencadena el paso a la siguiente actividad.
  - No poseen transiciones internas ni transiciones desencadenadas por eventos
- Acciones y actividades tienen la misma notación:

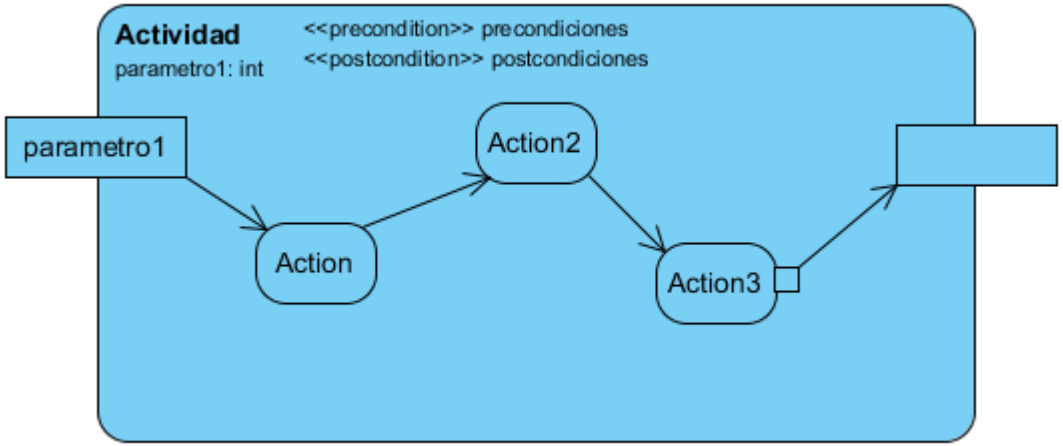


Acciones y Actividades



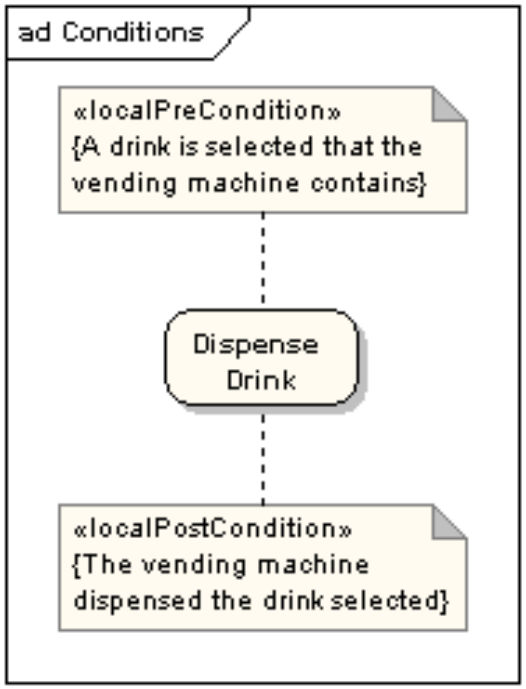
Actividad

- Se pueden especificar:
  - Nombre de la actividad
  - Parámetros
  - Precondiciones y postcondiciones



Restricciones

- Se pueden especificar de manera opcional mediante etiquetas.

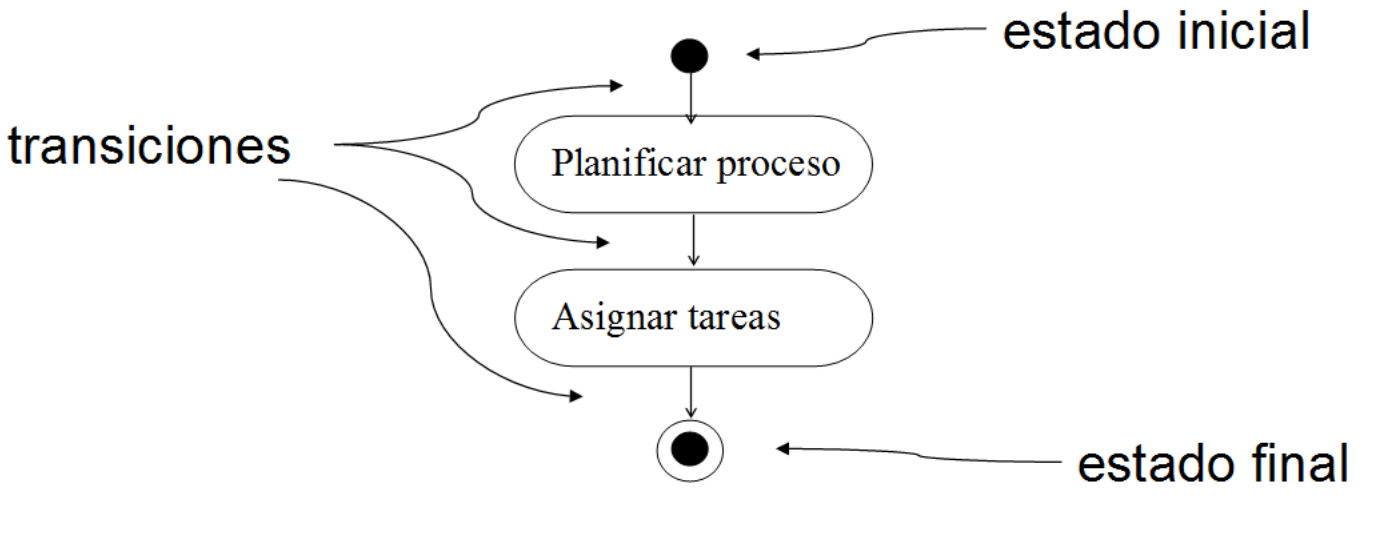


Transiciones

- Las flechas dirigidas entre estados de acción representan transiciones con evento implícito.
- Podemos encontrar diferentes tipos de transacciones:
  - Secuencial o sin disparadores.
  - Decisiones (“decision node”) y fusiones (“merge node”) .
  - Bifurcaciones (“fork node”) y uniones (“join node”) .

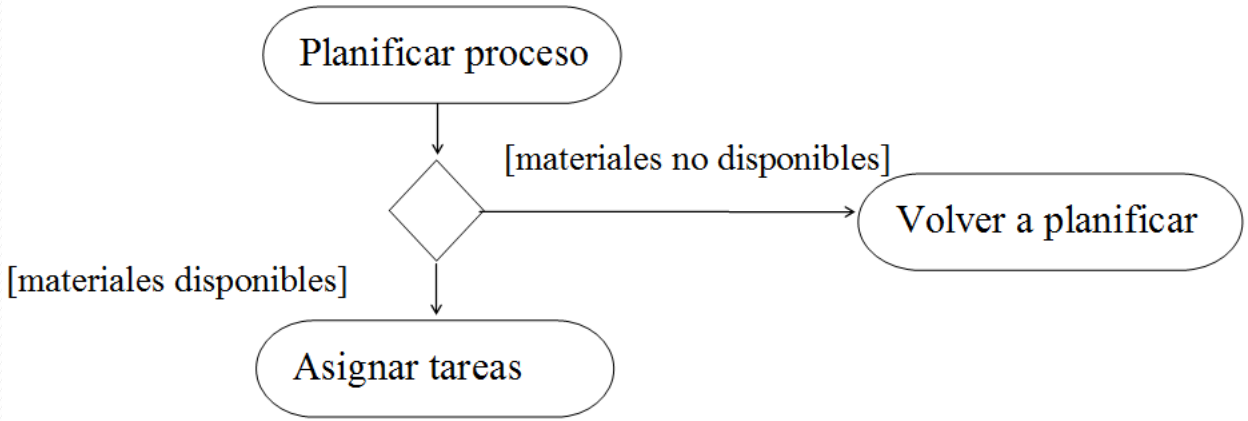
Secuencial o Sin Disparadores

- Al completar la acción del estado origen se ejecuta la acción de salida y, sin ningún retraso, el control sigue por la transición y pasa al siguiente estado.



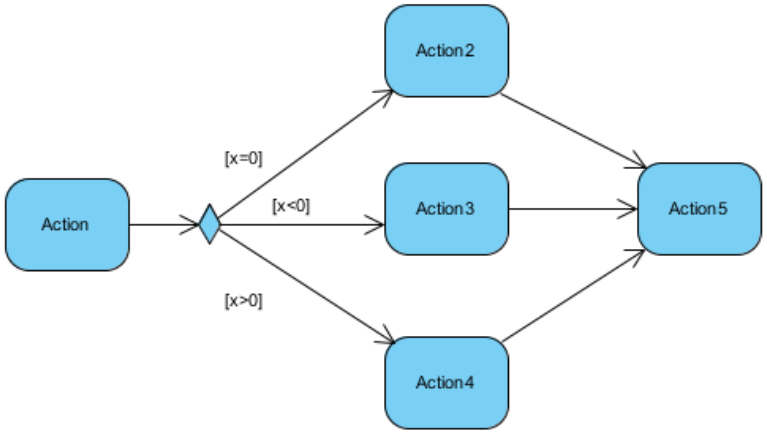
Decisiones (decision node)

- Las decisiones se representan mediante una transición múltiple que sale de un estado y donde cada camino tiene una etiqueta distinta.
- Tienen una condición o guarda asociada.
- Se deben cubrir todas las posibilidad con las condiciones de guarda pues si no el modelo quedaría en una situación inconsistente.
- Se representa mediante un rombo al cual llega la transición del estado origen y del cual salen las múltiples transiciones de los estados destino.

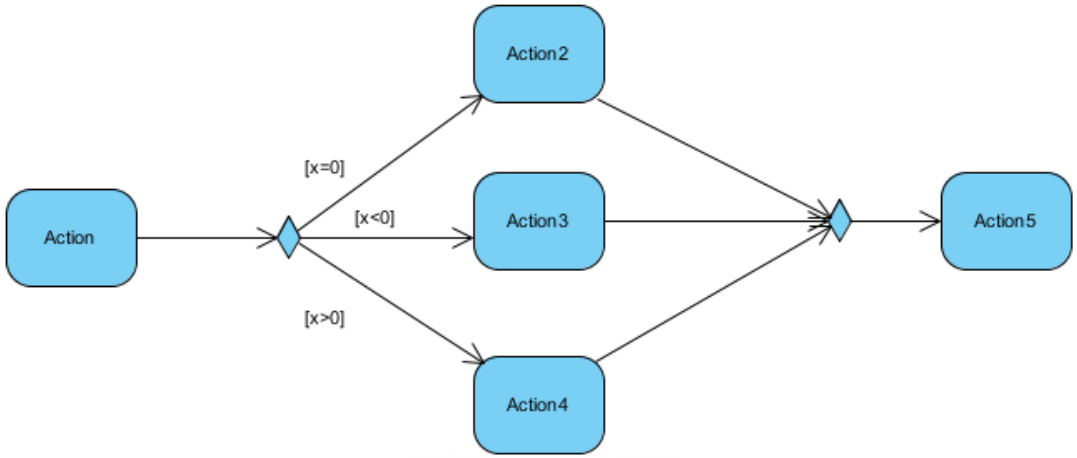


Fusión (merge node)

- Junta varios flujos de ejecución (merge).
- Se representa mediante un rombo como punto de llegada de varios flujos.
- ¡Ojo!, en UML 1.x es común ver varios flujos que salen de un nodo decisión llegar directamente a la actividad, el merge está implícito. En UML 2.x hay que especificar el nodo unión porque **varios flujos llegando a una actividad significan que la misma no comenzará hasta que todos los flujos entrantes se hayan completado**, cosa que no tiene sentido si vienen de un nodo decisión.



MAL EN UML 2.X



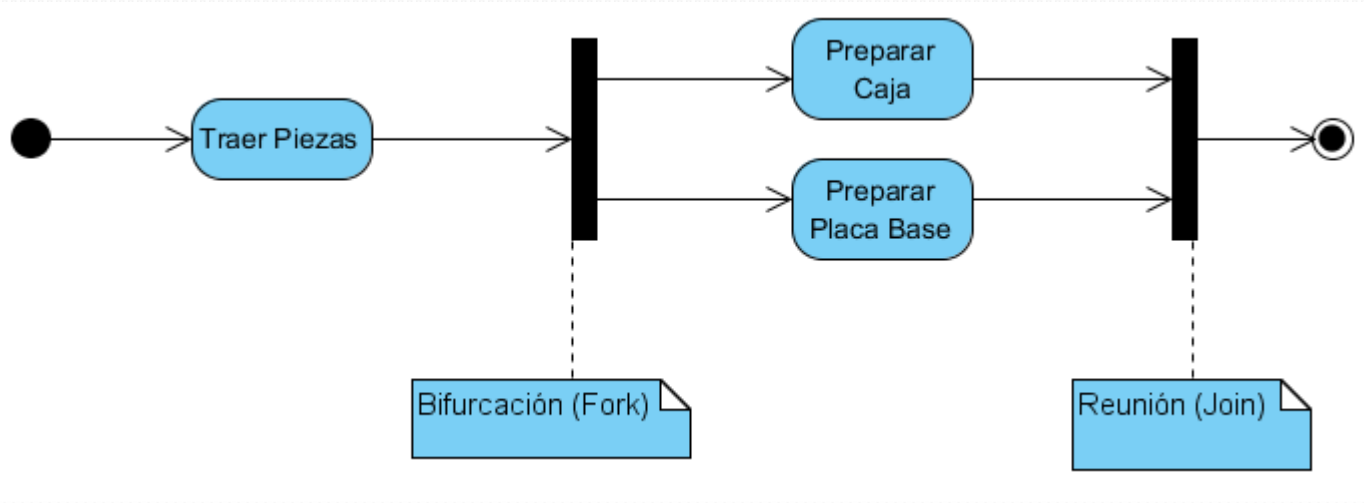
BIEN EN UML 2.X

Bifurcación y unión(reunión)

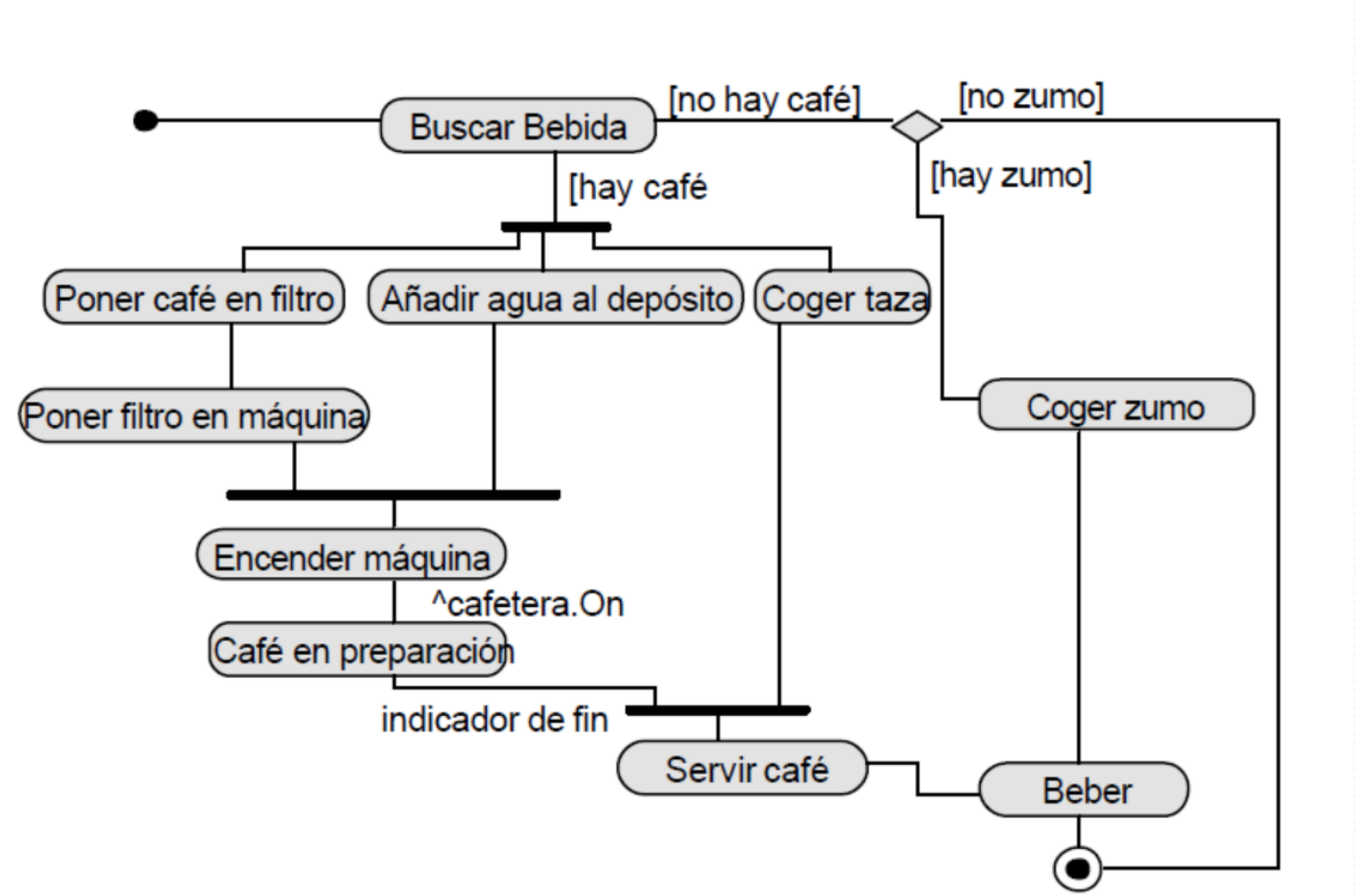
- La bifurcación (fork) permiten expresar la sincronización paralela de actividades. Las actividades invocadas después de una bifurcación (fork) son concurrentes.
- La unión (join) expresa la confluencia concurrente de actividades.
- Por definición, en la unión (join) los flujos entrantes se sincronizan, es decir, cada uno espera hasta que todos los flujos de entrada han alcanzado la unión.
- Se simbolizan mediante líneas paralelas donde entran o salen los flujos de las actividades.

- **¿Qué tipo de nodo de control se emplea en un diagrama de actividades de UML 2.0 para identificar la sincronización de varios flujos concurrentes en uno sólo? (TIC A2 AGE 2010)**
  - División (fork node)
  - Bifurcación (decision node)
  - Fusion (merge node)
  - Unión (join node) ----------

Bifurcación y Reunión

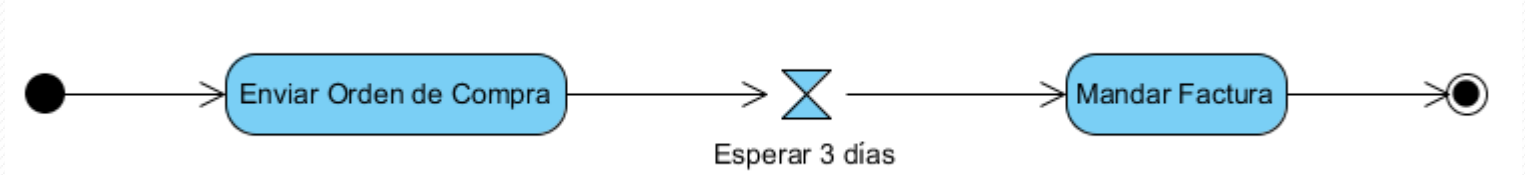


Ejemplo

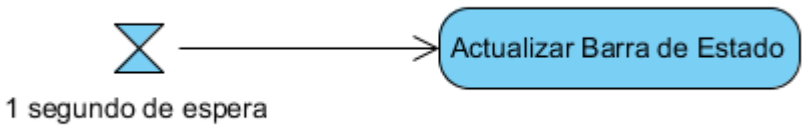

Buscar Bebida
[no hay café]
[no zumo]
[hay café
[hay zumo]
Poner café en filtro
Añadir agua al depósito
Coger taza
Poner filtro en máquina
Coger zumo
Encender máquina
^cafetera.On
Café en preparación
indicador de fin
Servir café
Beber

Eventos de tiempo

- Se utilizan para modelar periodos de tiempo entre actividades.
- Se representan como dos triángulos enfrentados, con la descripción del evento de tiempo en una etiqueta.

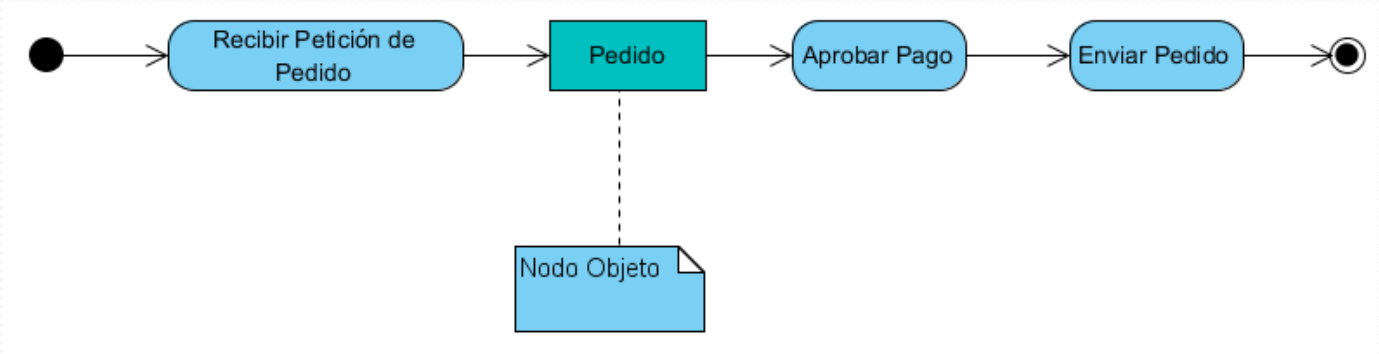


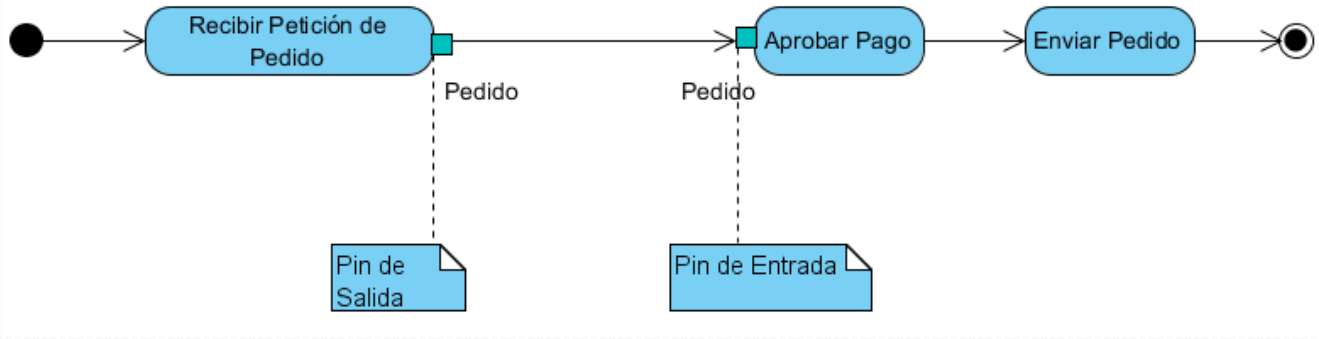
- Si un evento de tiempo no tiene flujo de entrada entonces expresa un evento de tiempo recursivo que se repetirá el tiempo indicado en su descripción.
- Los eventos de tiempo recursivos son puntos de comienzo de los diagramas de actividad.



Objetos

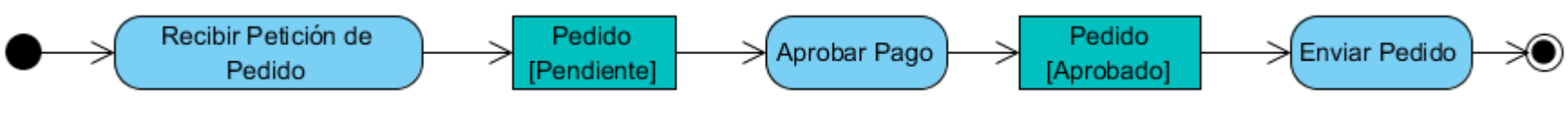
- Los objetos pueden ser modelados dentro de los diagramas de actividad.
- Se usan nodos objetos para representar el flujo de datos a través de una actividad.
- Los objetos pueden ser creados, usados o modificados por las distintas acciones de la actividad.
- Los nodos objetos se representan como una caja rectangular.
- También se puede representar el flujo de un objeto mediante “pins” de entrada y de salida en las actividades. Enfatizan el requerimiento del objeto en la entrada y en la salida .



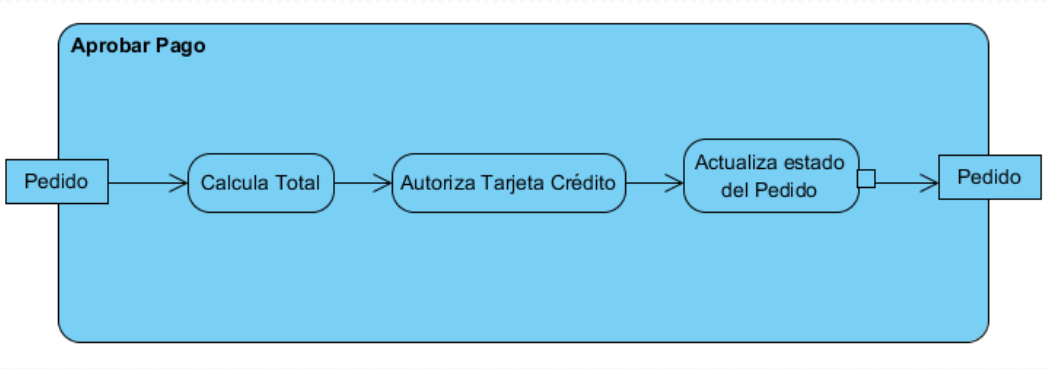


Objetos

- Se puede especificar el cambio de estado de un objeto en la actividad mostrándolo entre []
- En el ejemplo se ve como cambia el objeto petición de estado "pendiente" a "aprobado"



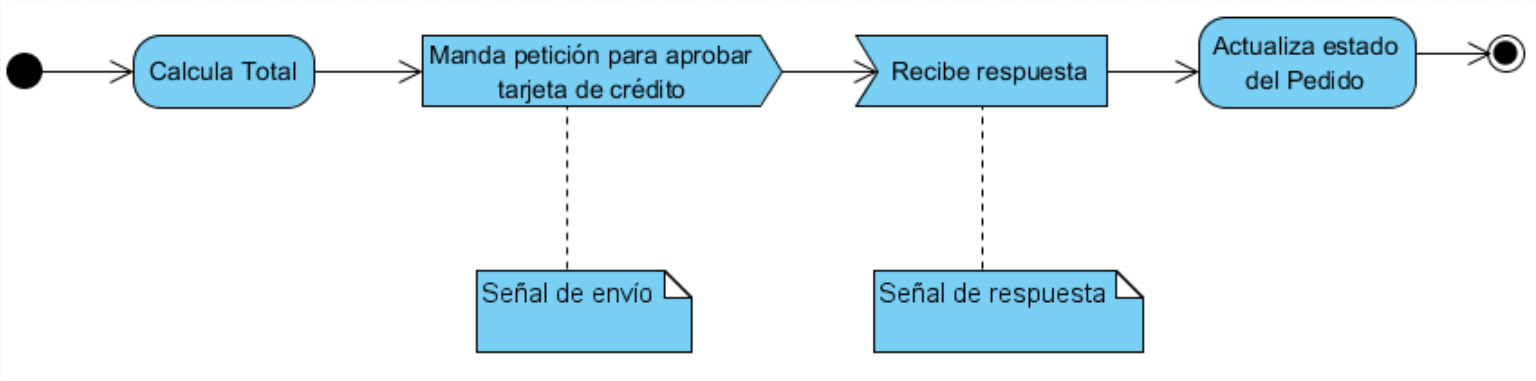
- Los nodos objetos pueden ser utilizados como parámetros de entrada y salida de una actividad.
- Se dibujarían en los bordes del marco de la actividad.



Señales

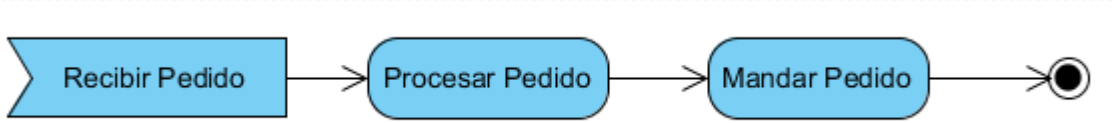
- Representan interacciones con participantes externos.
- Una señal recibida tiene el efecto de lanzar una acción dentro del diagrama de actividad. Se sabe como reaccionar a la señal pero no se sabe cuando se recibirá.
- También se pueden enviar señales a participantes externos. La respuesta a estas señales por parte del participante externo no se modelará en nuestro sistema. El envío de señales es asíncrono, no esperándose respuesta.

- Ejemplo.
  - Nótese como el envío y recepción de una señal se utiliza como llamada síncrona.



Señales

- Un nodo de recepción de señal sin flujo de entrada significa que el nodo está esperando la señal de entrada, siempre y cuando la actividad donde esté incluido esté activa.
- Este tipo de nodos pueden hacer las veces de un nodo inicial de una actividad.
- Difiere de los nodos de recepción de señal con flujos de entrada en que estos último necesitan que las actividades previas hayan terminado para poder responder a la señal.

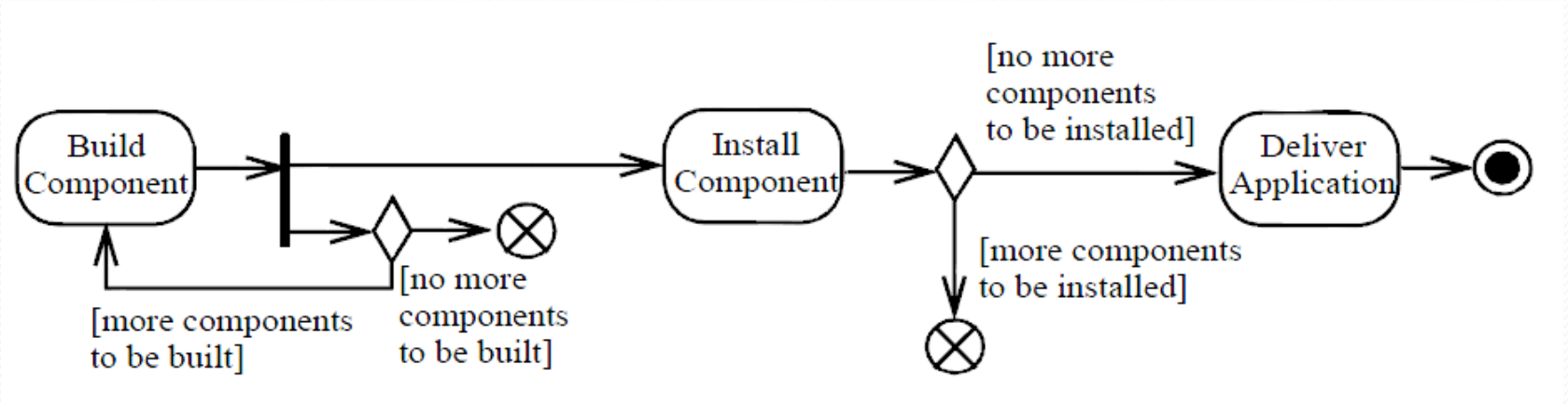


Nodos de comienzo y finalización

- Nodo Inicial. 
  - La manera mas sencilla y común para comenzar una actividad es mediante un nodo “inicial”.
- Nodo Finalizador de actividad. 
  - Cuando se alcanza un nodo finalizador de actividad, todas los flujos de esa actividad son finalizados.
  - Si hay varios nodos finalizadores de actividad, el primero que se alcance parará todas las acciones de la actividad.
- Finalización de flujo. 
  - Se usa cuando no se quieren finalizar todas las acciones y flujos de una actividad, sino solo el flujo afectado.
  - Afecta solo al flujo que llega, y no al resto de flujos.

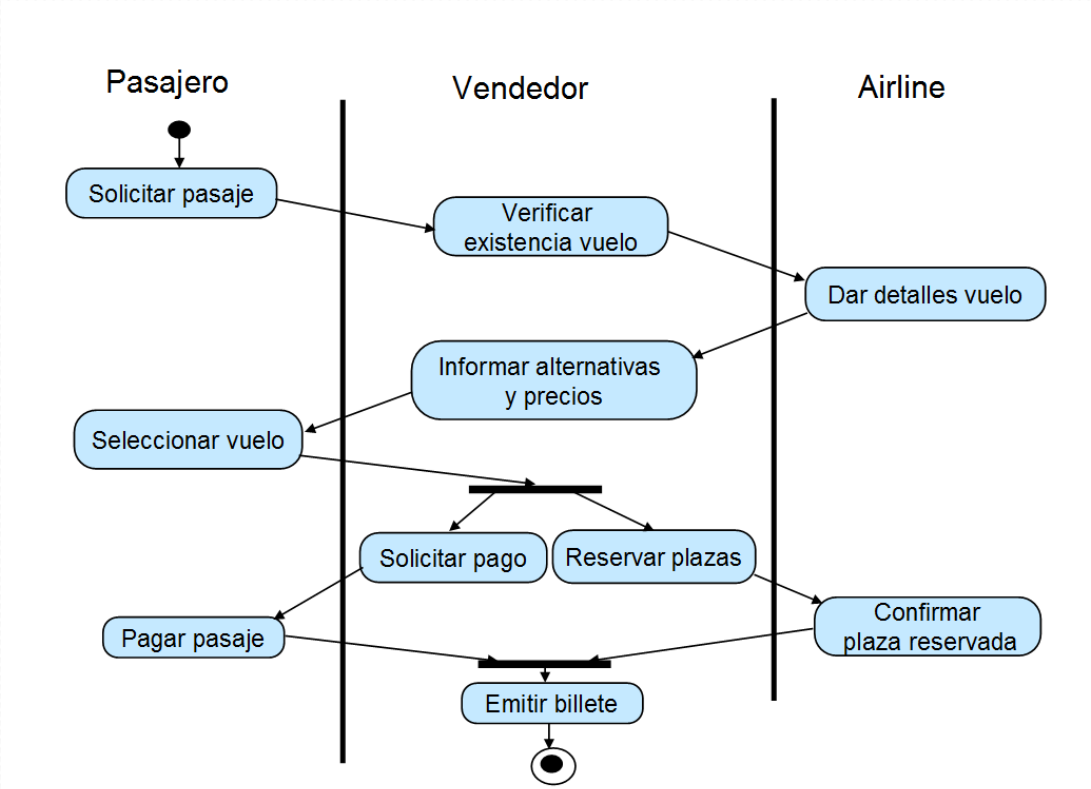
Nodos de Finalización

- Ejemplo con nodo final de actividad y nodos final de flujo.



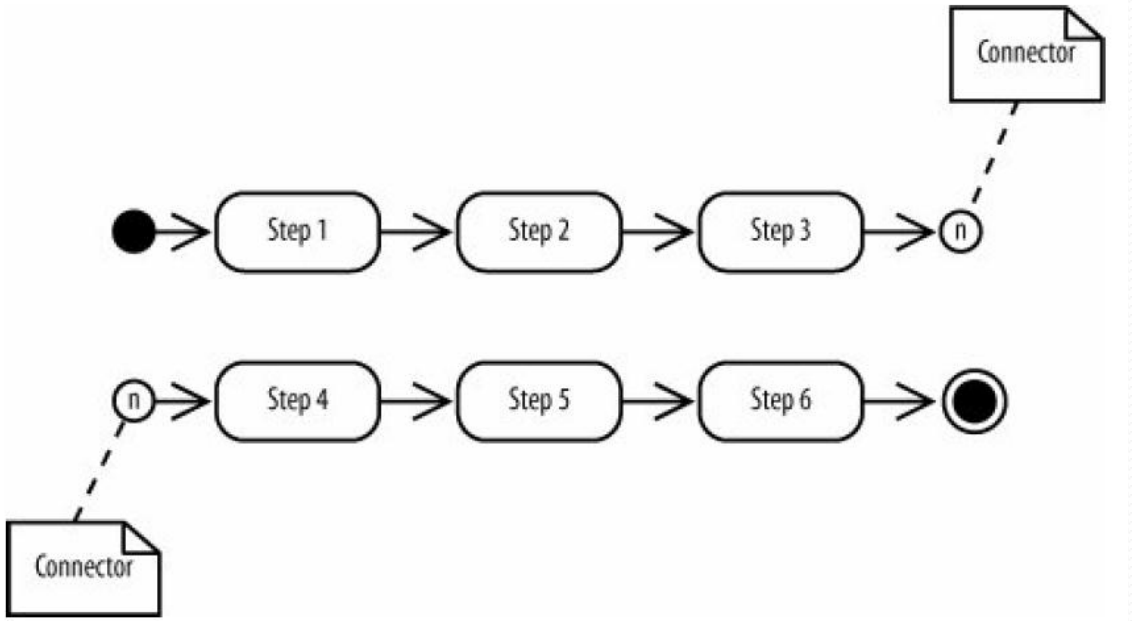
Calles (Swimlanes)

- Permiten ver **QUIENES** son los responsables de realizar las distintas actividades.
- Especificar qué parte de la organización es responsable de una actividad.
- Cada calle tiene un nombre único dentro del diagrama.



Conectores

- Permiten interconectar unas partes del diagrama con otras.
- Se utilizan para simplificar la realización y comprensión del diagrama de actividad.
- Se representan mediante un pequeño círculo con el nombre del conector dentro.
- El nombre del conector suele ser una letra.
- Un conector siempre viene a pares, representando un lado el flujo de entrada y el otro el flujo de salida.

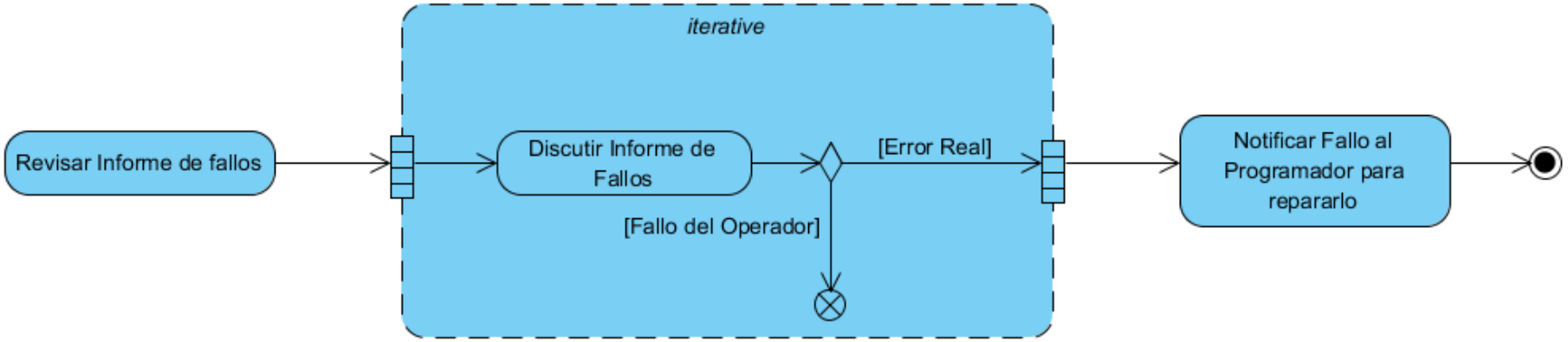


Regiones de Expansión.

- Se utilizan para representar una acción sobre un conjunto de elementos de una colección.
- Se representan con un rectángulo punteado con cuatro pequeñas cajas alineadas a cada lado (ojo, esto **no** implica que la colección sea de cuatro elementos).
- Dentro de una región de expansión se puede especificar mediante una palabra clave como será la ejecución en la región:
  - <<Parallel>> La ejecución es concurrente
  - <<Iterative>> La ejecución es secuencial
  - <<stream>> La ejecución es continua. Puede producir datos de salida mientras se está procesando la entrada.

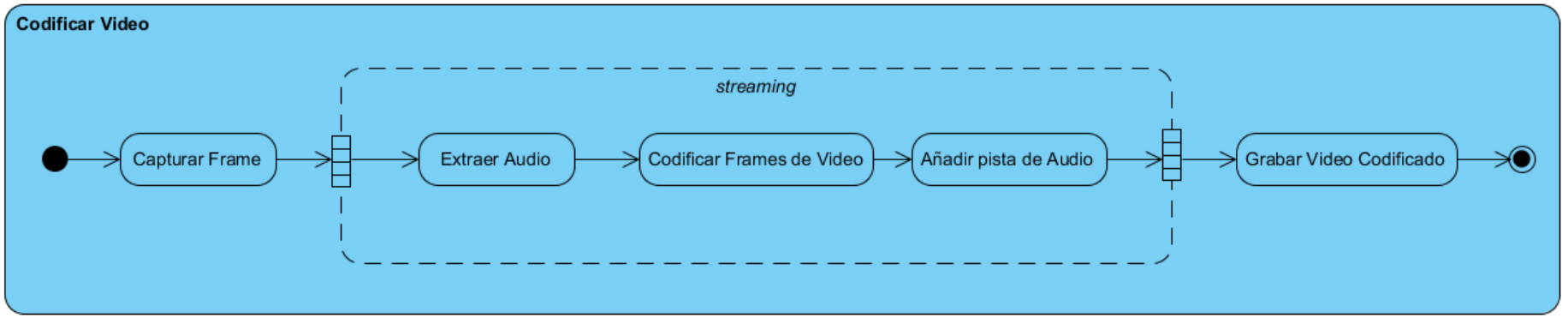
Regiones de expansión

- Ejemplo.
  - Cada informe de fallo en la colección se discute para ver si es real o si es un fallo del operador



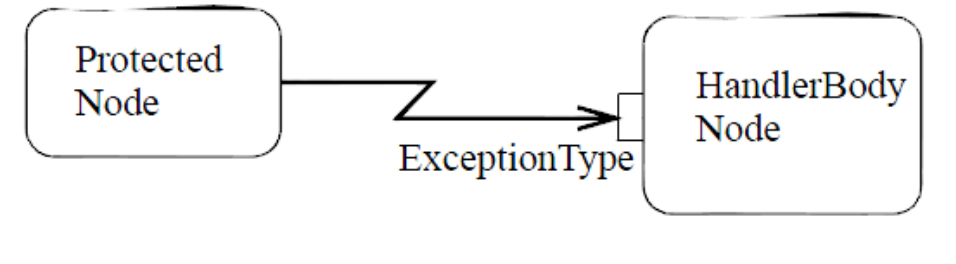
Regiones de expansión

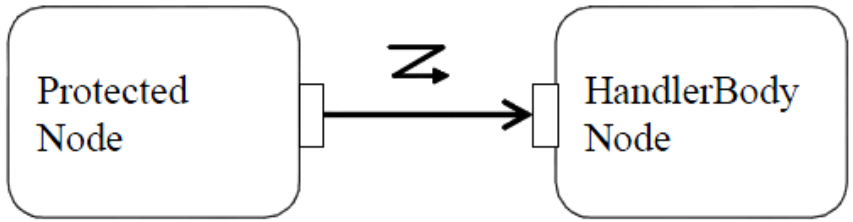
- La región de tipo “stream” procesa los datos de entrada según están disponibles.



Excepciones

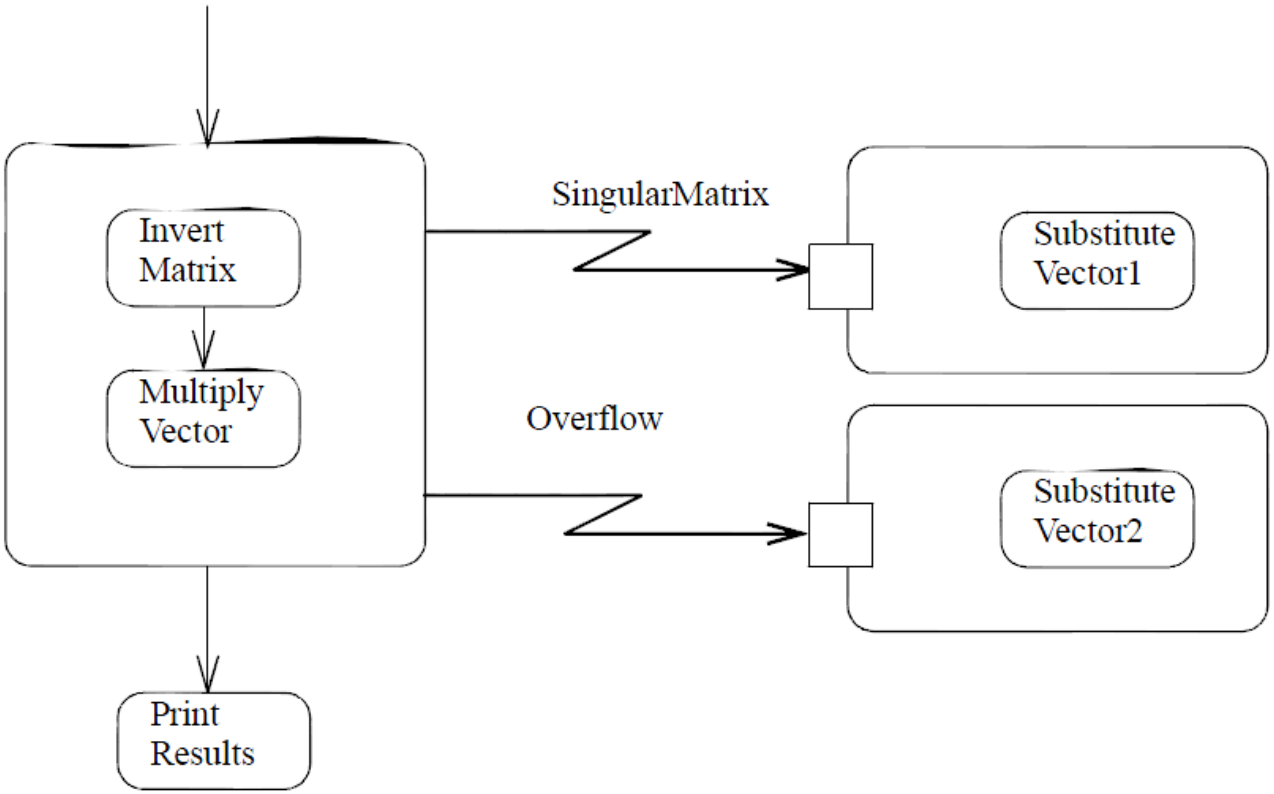
- Un manejador de excepciones es un elemento que especifica el cuerpo a ejecutar en caso de que una excepción específica ocurra durante la ejecución del nodo protegido.
- El sucesor del nodo manejador de excepción será el mismo que el del nodo protegido, no siendo necesario dibujar la transición.





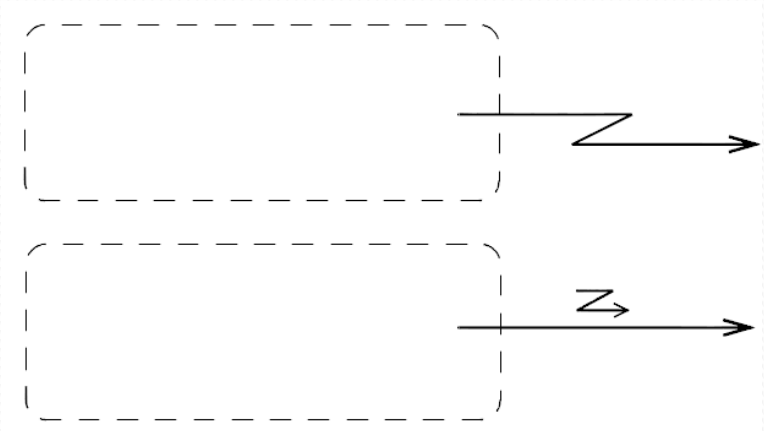
Excepciones

- Ejemplo.
  - Cálculo de una matriz. Si la matriz es singular no puede invertirse y se lanza una excepción. También se produce una excepción en matrices muy grandes por desbordamiento
  - En todo caso se imprimirán los resultados.

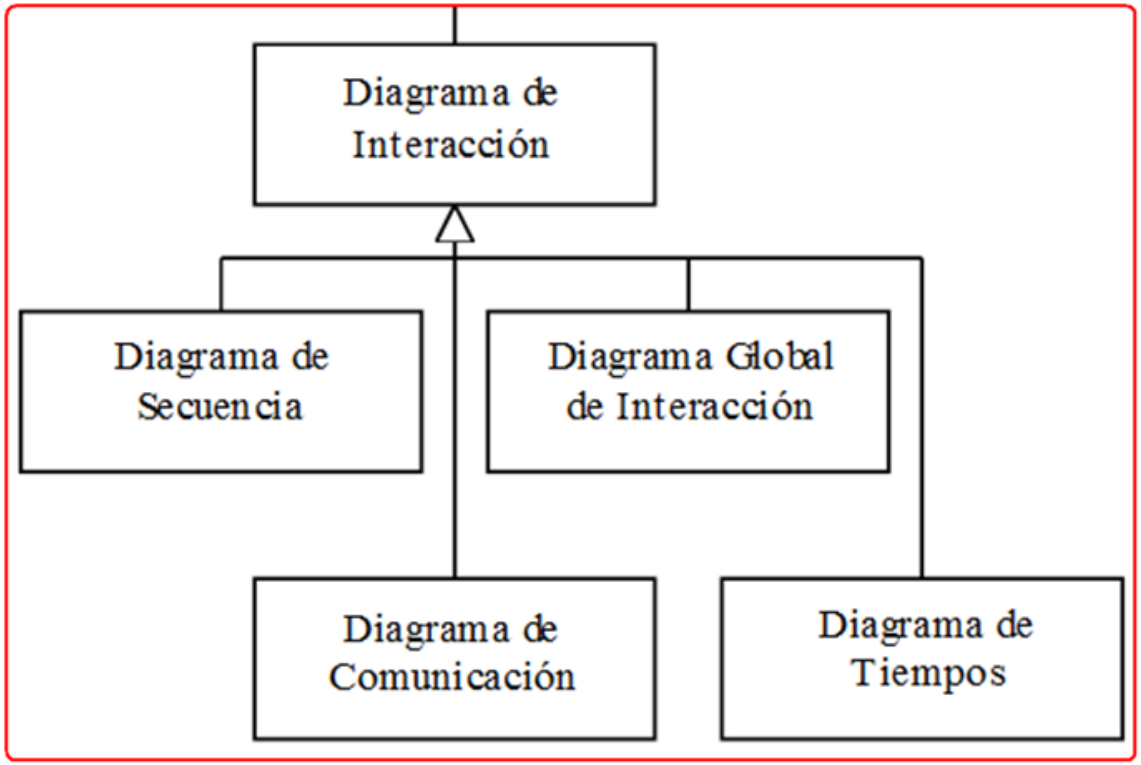


Región Interrumpible.

- Se pueden definir regiones en las cuales el grupo de actividades o de acciones puede interrumpirse en su ejecución.
- Todos los flujos de ejecución dentro de la región de interrupción terminarán si se produce la interrupción.
- Se representa mediante un rectángulo punteado y una línea zigzagueante.



Diagramas de Interacción



Diagramas de secuencia

- Describen el **comportamiento dinámico** del sistema de información mediante el paso de mensajes entre los objetos del mismo.
- Muestran de forma explícita la **secuencia de los mensajes** intercambiados por los objetos.
- Adecuados para observar la **perspectiva cronológica** de las interacciones.
- Puede obtenerse a partir del diagrama de Comunicación (son isomorfos)
- Representan un medio para verificar la **coherencia del sistema** mediante la validación con el modelo de clases.
- Describen en **detalle** un determinado escenario de un caso de uso.

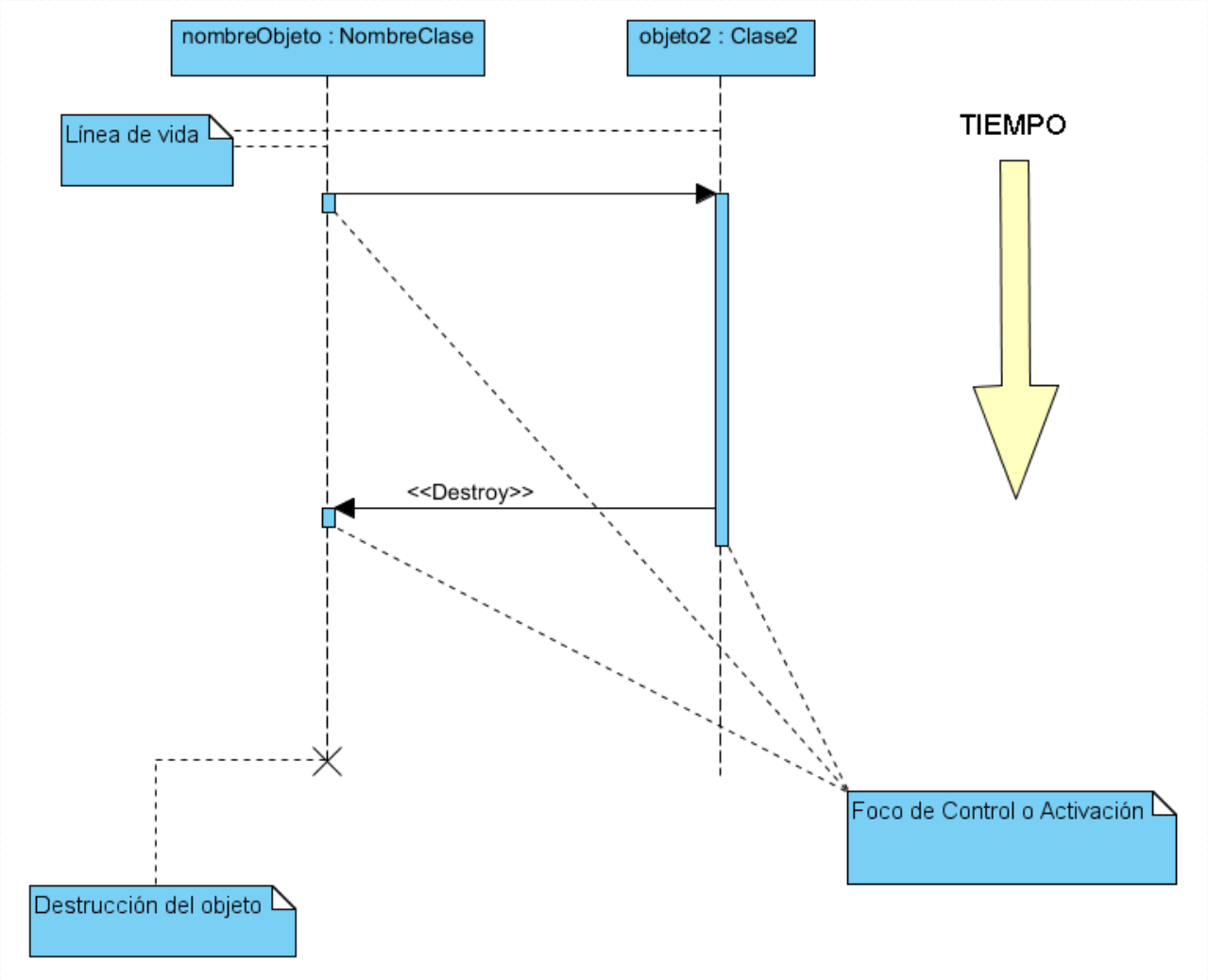
Respecto al diagrama de secuencia, ¿Cuál de las siguientes es la correcta? (TIC A2 2009)

- Muestra los objetos participantes en la interacción y los mensajes que intercambian ordenados según su secuencia en el tiempo. ----------
- El eje horizontal representa el tiempo y en el eje vertical se colocan los objetos y actores participantes en la interacción, sin un orden prefijado.
- Cada objeto o actor tiene una línea horizontal, y los mensajes se representan mediante etiquetas entre los distintos objetos
- El tiempo fluye de abajo a arriba y de izquierda a derecha.

Diagrama de Secuencia

- Dos dimensiones:
  - El eje vertical representa el tiempo.
  - El eje horizontal los diferentes objetos.
- El tiempo avanza desde la parte superior del diagrama hacia la inferior.
- El **orden** de los objetos es **irrelevante**, aunque su colocación debería poseer la mayor **claridad** posible.
- Cada objeto tiene asociados:
  - **Línea de vida**: Intervalo de tiempo de existencia del objeto. Un objeto se representa como una línea vertical punteada con un rectángulo de encabezado y con rectángulos a través de la línea principal que denotan la ejecución de métodos (activación). El rectángulo de encabezado contiene el nombre del objeto y el de su clase, en un formato nombreObjeto : nombreClase.
  - **Focos de control o activación**: periodo de tiempo en el cual el objeto se encuentra ejecutando alguna operación. Se denota como un rectángulo delgado sobre la línea de vida del objeto.
  - **Destrucción de un objeto:** Se representa como una X al final de la línea de ejecución del objeto.

Diagrama de Secuencia



Mensajes

- Se representa como una flecha horizontal entre las líneas de vida de los objetos que intercambian el mensaje.
- La flecha va desde el objeto que envía el mensaje al que lo recibe.
- Además, un objeto puede mandarse un mensaje a sí mismo, en este caso la flecha comienza y termina en su línea de vida.
- También pueden ser etiquetados los mensajes con un número de secuencia, sin embargo, este número **no es necesario** porque la localización física de las flechas que representan a los mensajes ya indica el orden de los mismos.
- Formato del mensaje:
  - **atributo = nombre_mensaje(argumentos) : retorno**
  - Ejemplos:
    - nombreMensaje()
    - nombreMensaje(n1:Number,n2:Number)
    - nombreMensaje(): ClaseRetorno
    - miVariable = nombreMensaje(): ClaseRetorno

Tipos de mensajes



- **Síncrono o llamada**: El objeto que envía el mensaje queda bloqueado hasta que termina la llamada y recibe respuesta.



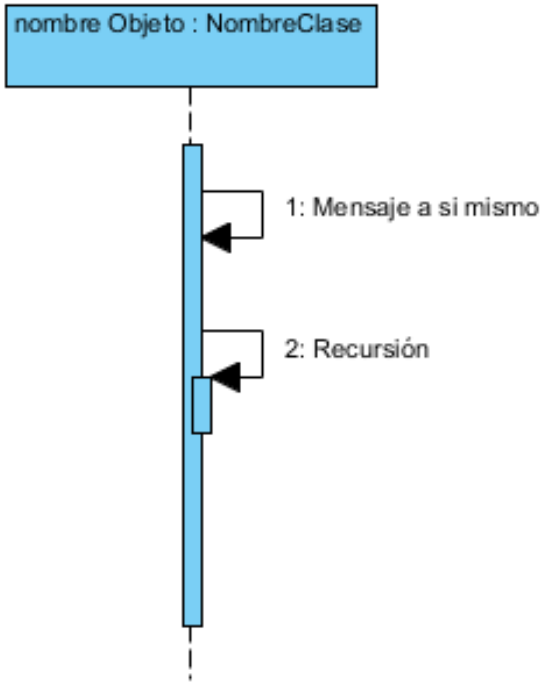
- **Retorno**. Respuesta a un mensaje previo.



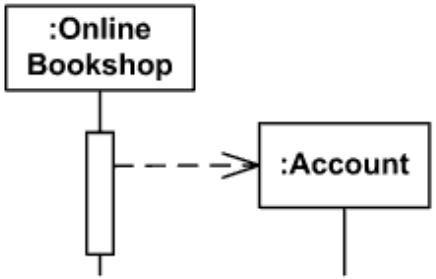
- **Asíncrono**. El originario no espera respuesta y permanece activo pudiendo enviar mensajes.

Mensajes

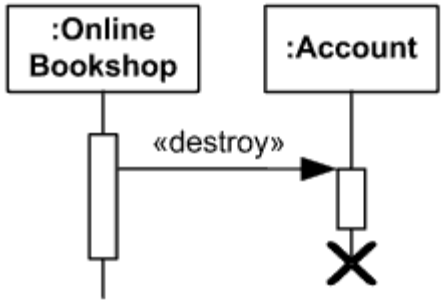
- Mensaje a si mismo y recursión



- Creación y destrucción de un objeto



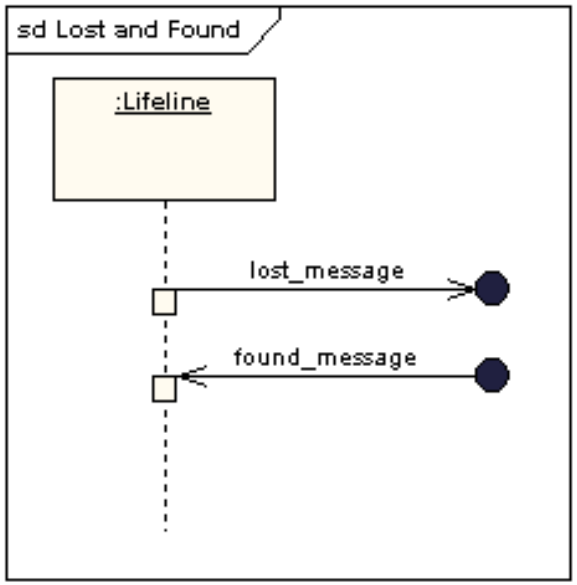
Online Bookshop creates Account.



Online Bookshop terminates Account.

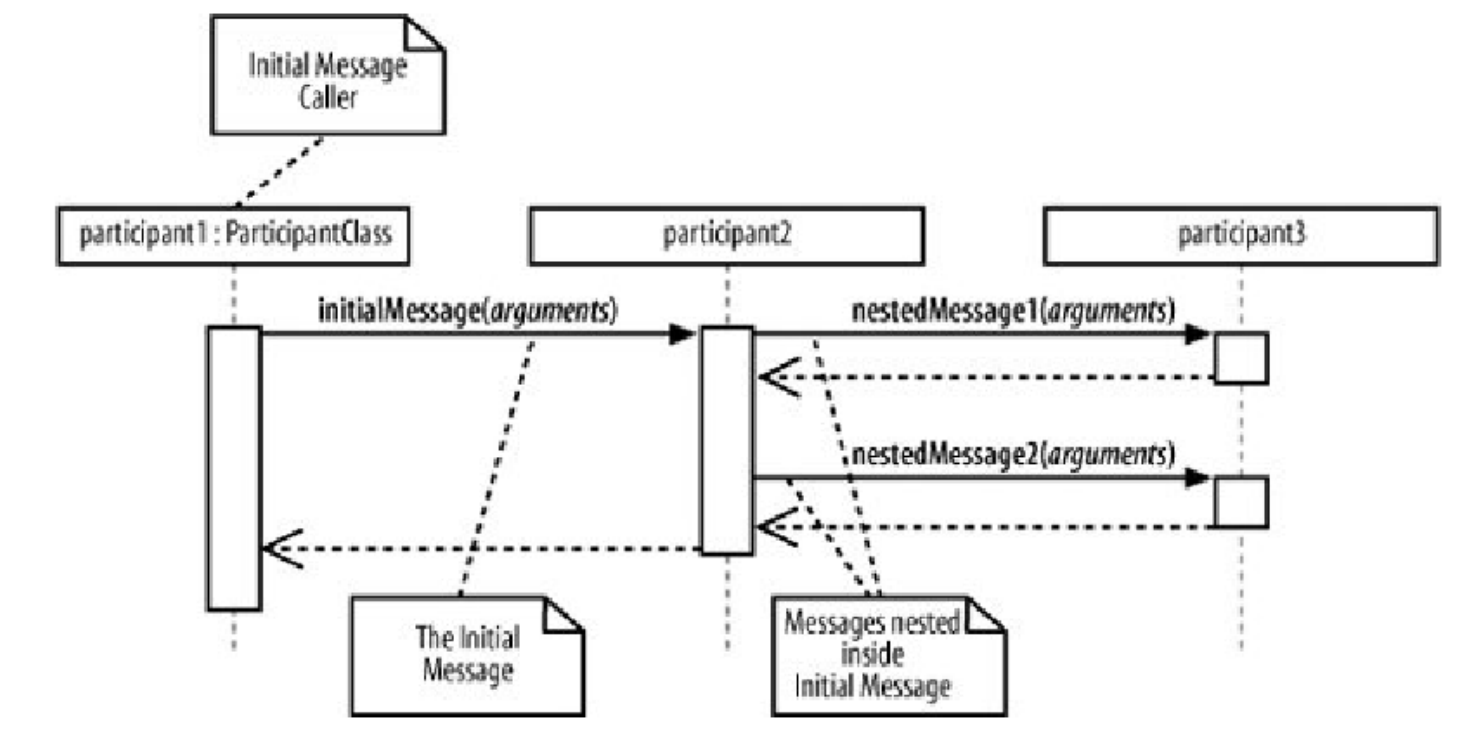
Mensajes

- **Los mensajes perdidos:**
  - Son aquellos que han sido enviados pero que no han llegado al destino esperado, o que han llegado a un destino que no se muestra en el diagrama actual.
- **Los mensajes encontrados:**
  - Son aquellos que llegan de un remitente no conocido, o de un remitente no conocido en el diagrama actual.
  - Se denotan yendo o llegando desde un elemento de punto final.



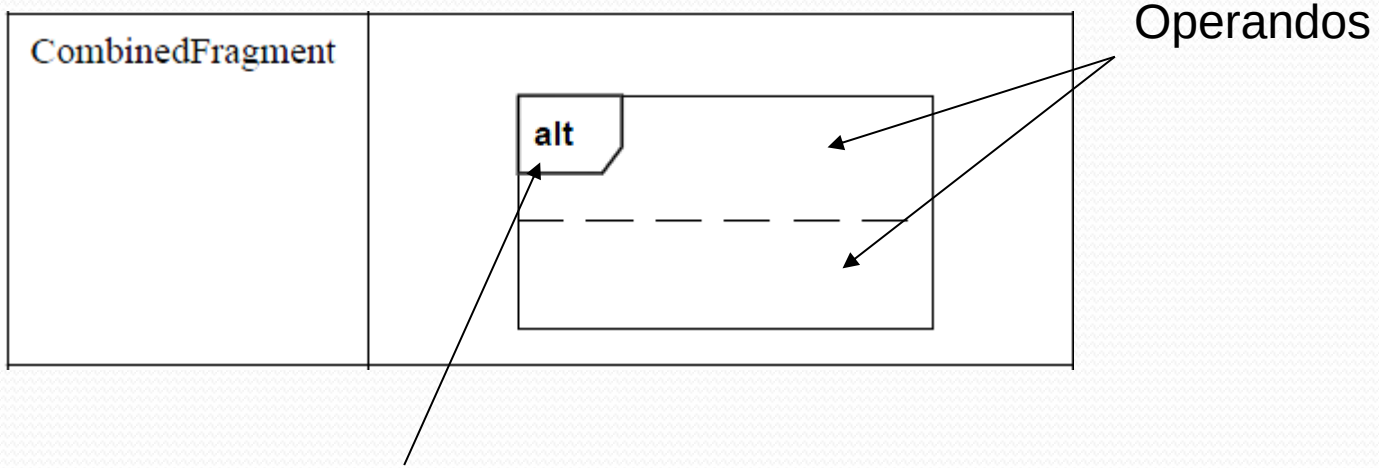
Anidamiento de mensajes

- Los mensajes se pueden anidar unos dentro de otros.



Fragmentos Combinados

- **Fragmentos combinados**
  - Permiten agregar un grado de lógicas de procedimientos a los diagramas.
  - Es una o más secuencias de procesos incluidas en un marco y ejecutadas bajo circunstancias nombradas específicas.

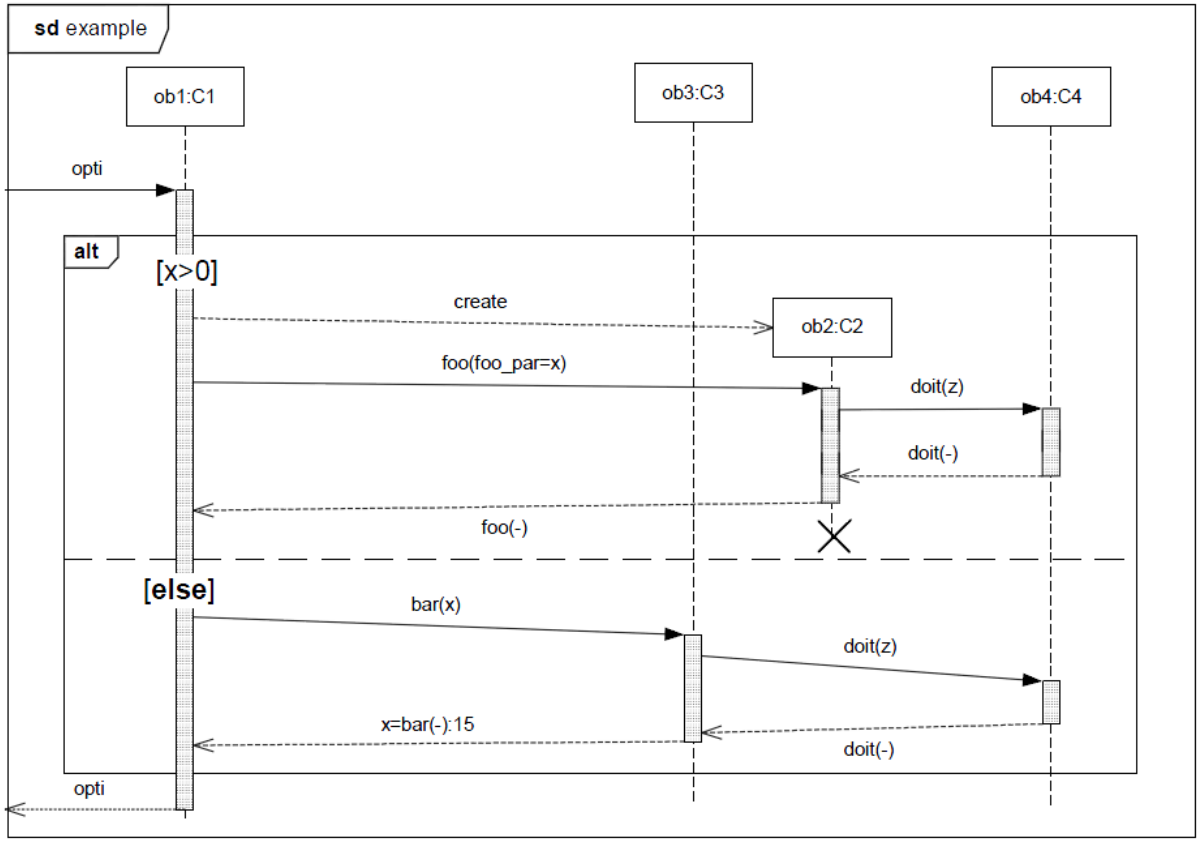


Fragmentos combinados

- Los fragmentos disponibles son:
  - El fragmento **Alternative** (denotedo “alt”) modela estructuras if…then…else.
  - El fragmento **Option** (denotado “opt”) estructuras condicional.
  - El fragmento **Break** modela una secuencia alternativa de eventos que se procesa en lugar de todo del resto del diagrama.
  - El fragmento **Parallel** (denotado “par”) modela procesos concurrentes.
  - El fragmento de secuenciado **Weak** (denotado “seq”) incluye un número de secuencias para las cuales todos los mensajes se deben procesar en un segmento anterior, antes de que el siguiente segmento pueda comenzar, pero que no impone ningún secuenciado en los mensajes que no comparten una línea de vida.
  - El fragmento de secuenciado **Strict** (denotado “strict”) incluye una serie de mensajes que se deben procesar en el orden proporcionado.
  - El fragmento **Negative** (denotado “neg”) incluye una serie de mensajes inválidos.
  - El fragmento **Critical** incluye una sección crítica. Los mensajes dentro de una sección crítica se ejecutan de manera atómica.
  - El fragmento **Ignore** declara un mensaje o mensajes que no son de ningún interés si este aparece en el contexto actual.
  - El fragmento **Consider** es el opuesto del fragmento Ignore: cualquier mensaje que no se incluya en el fragmento Consider se debería ignorar.
  - El fragmento **Assertion** (denotado “assert”) designa que cualquier secuencia que no se muestra como un operando de la aserción es inválida.
  - El fragmento **Loop**. Modela bucles.

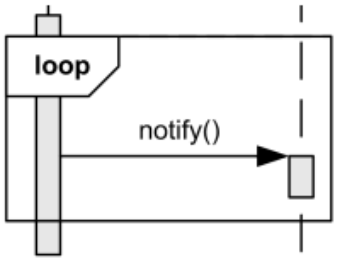
Fragmentos combinados

- El fragmento **Alternative** (denotedo “alt”) modela estructuras if…then…else

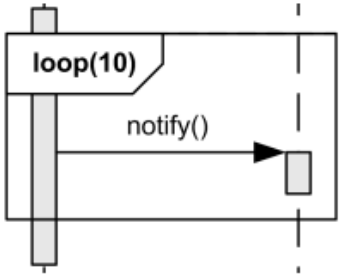


Fragmentos combinados

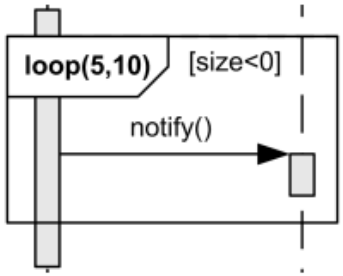
- El fragmento **Loop** incluye una serie de mensajes que están repetidos.



Potentially infinite loop.



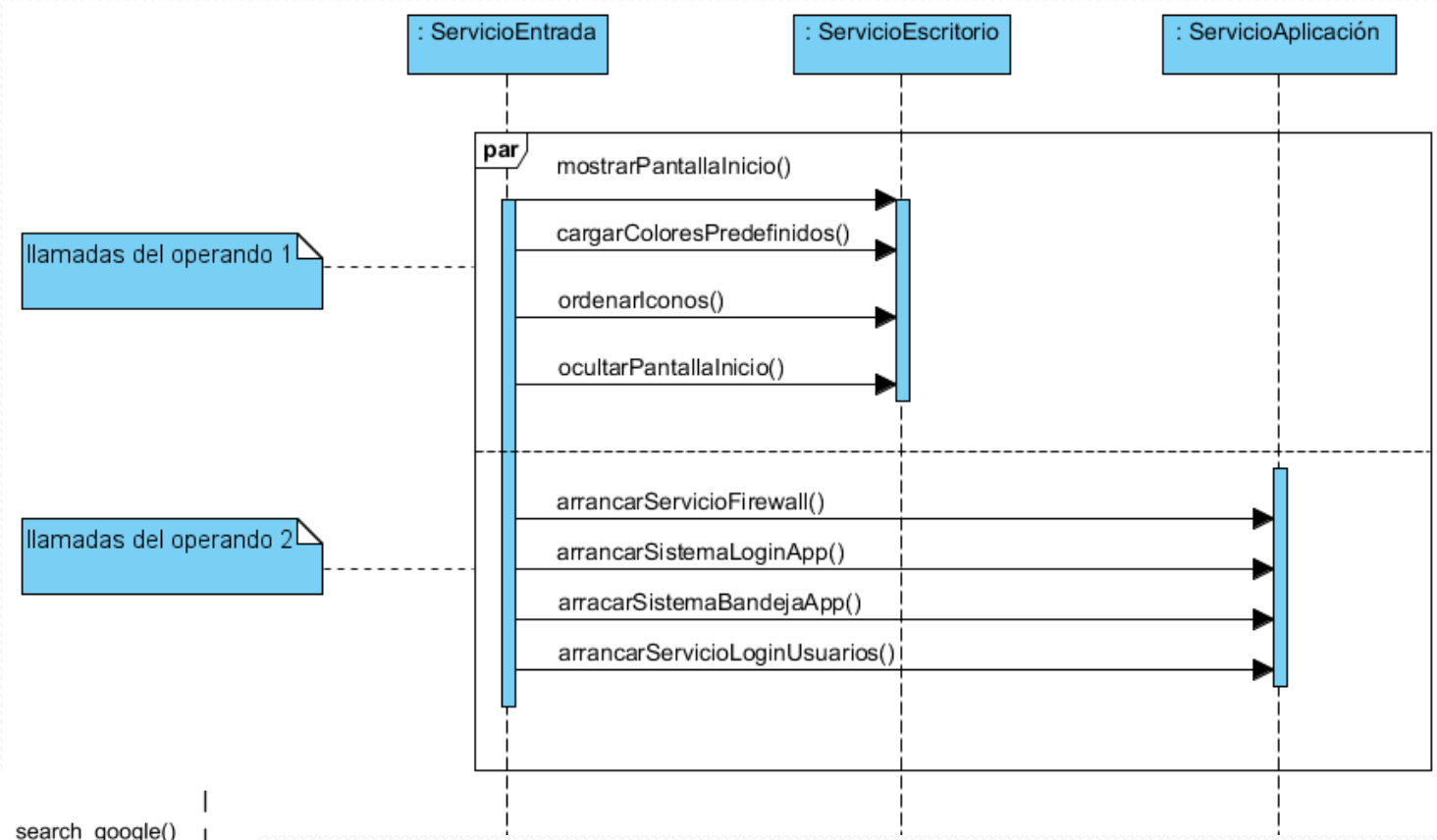
Loop to execute exactly 10 times.

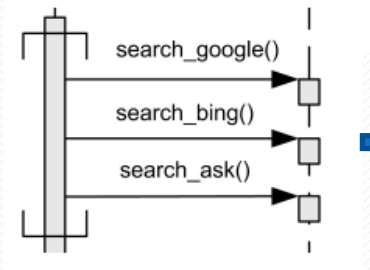


We may guess that as per UML 2.3, the loop is expected to execute minimum 5 times and no more than 10 times.
If guard condition [size<0] becomes false loop terminates regardless of the minimum number of iterations specified.
(Then why do we need that min number specified?!)

Fragmentos combinados

- El fragmento **Parallel** (denotado "par") modela procesos concurrentes.
  - Las llamadas de los operandos 1 y 2 se ejecutarán de manera concurrente

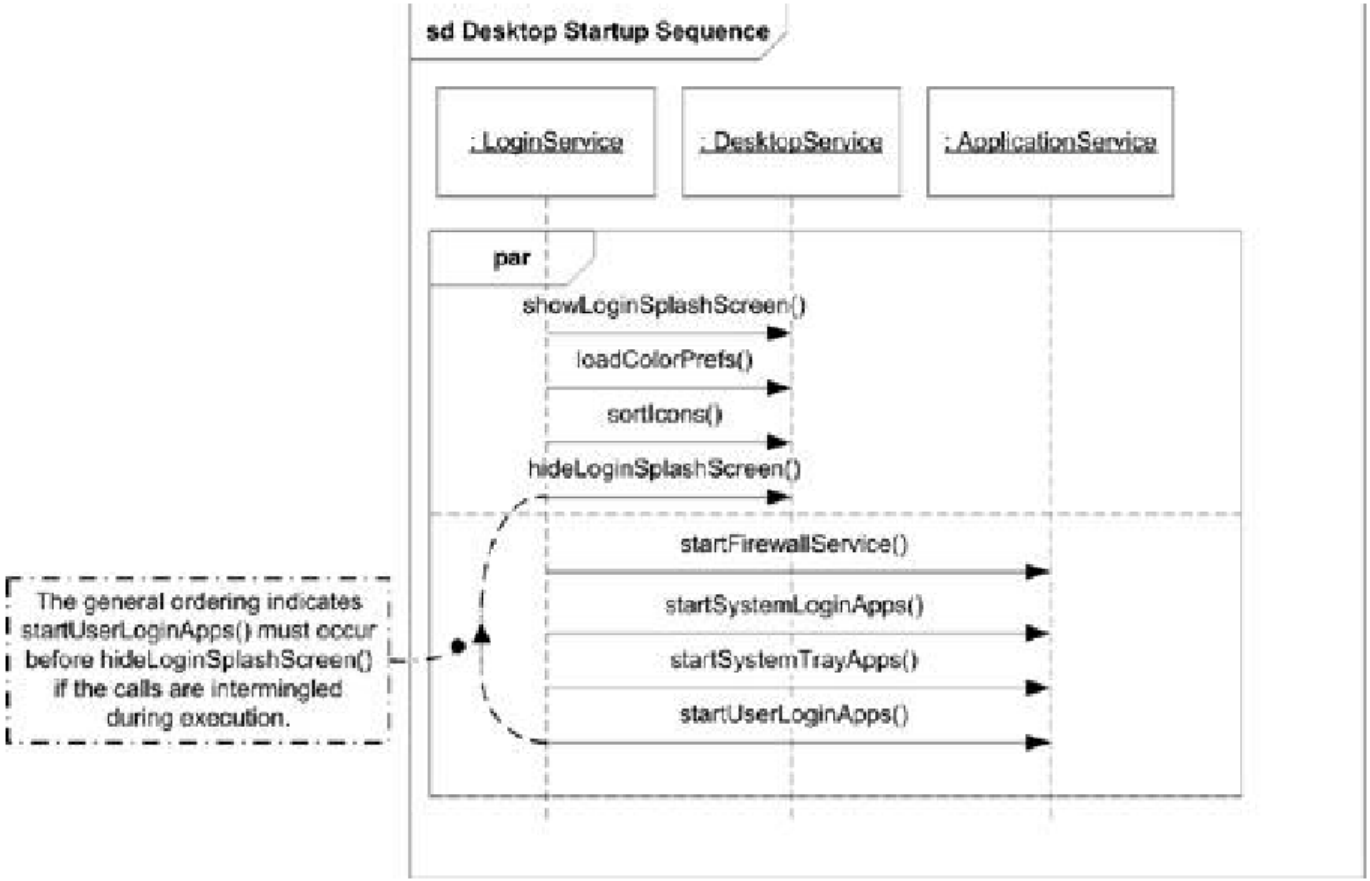




Otra notación equivalente. Equivale a tener distintos operandos concurrentes

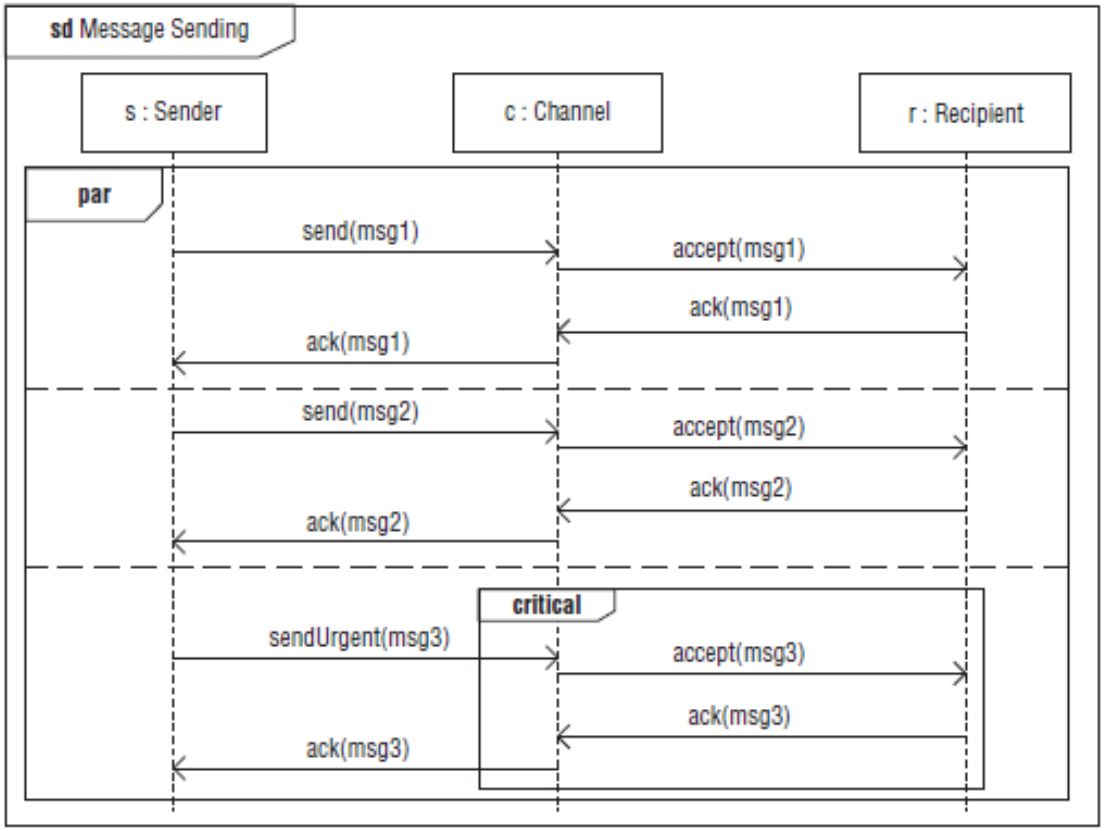
Fragmentos combinados

- Nota: **GeneralOrdering**. Indica ordenamiento en los mensajes. Se representa mediante una línea discontinua con una flecha en el centro uniendo los dos mensajes. Ej.hideLoginSplashScreen() no se ejecutaría hasta que no se hubiese terminado de ejecutar startUserLoginApps()



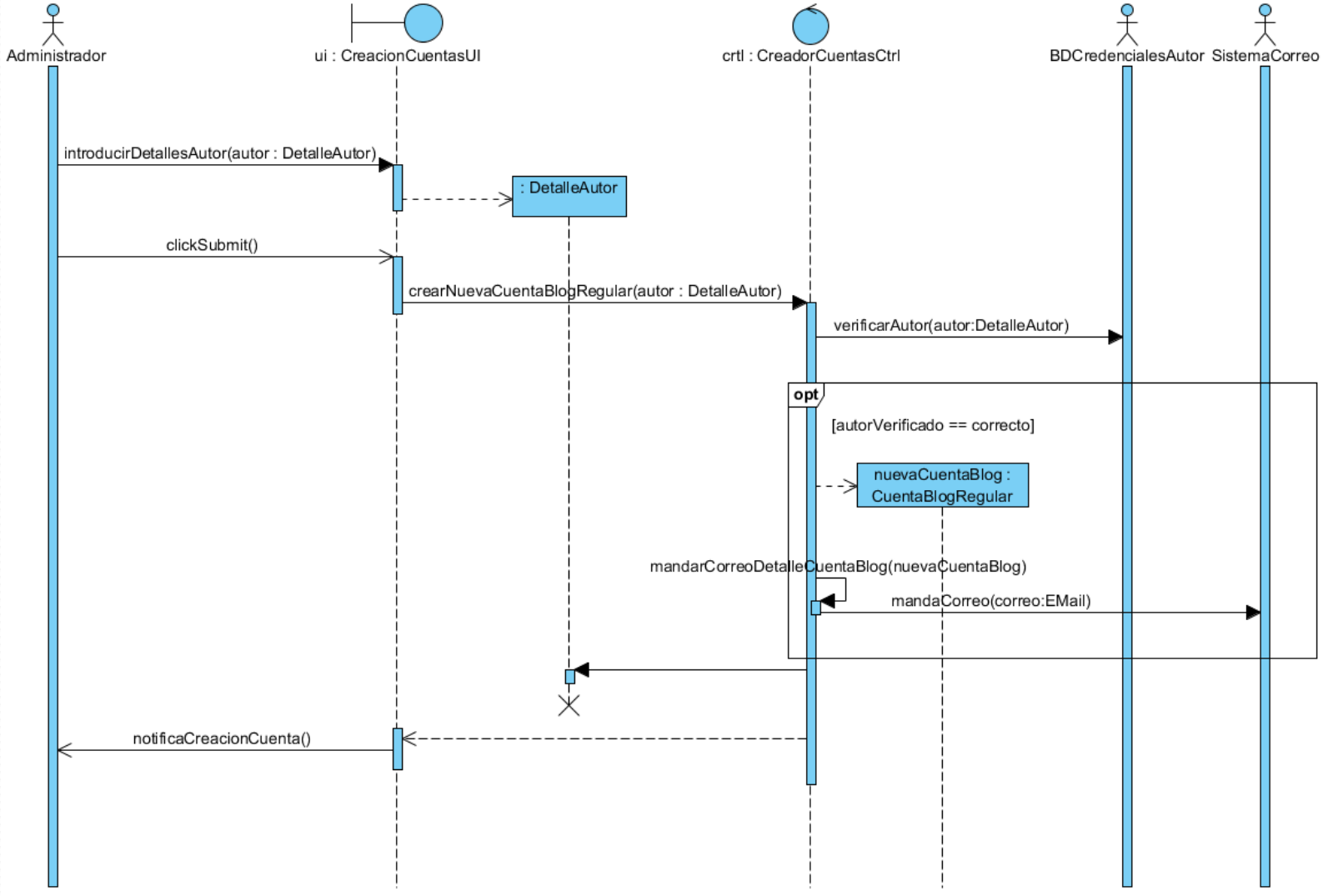
Fragmentos combinados

- El fragmento **Critical** incluye una sección crítica. Las trazas dentro de la región crítica no pueden solaparse con las trazas de otros fragmentos. Los mensajes dentro de una sección crítica se ejecutan de manera atómica.



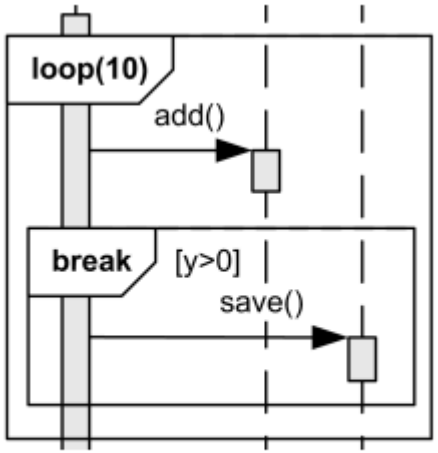
Fragmentos combinados

- El fragmento **Option** (denotado "opt"). Modela un comportamiento condicional.



Fragmentos combinados

- El fragmento **Break** modela una secuencia alternativa de eventos que se procesa en lugar de todo del resto del fragmento.



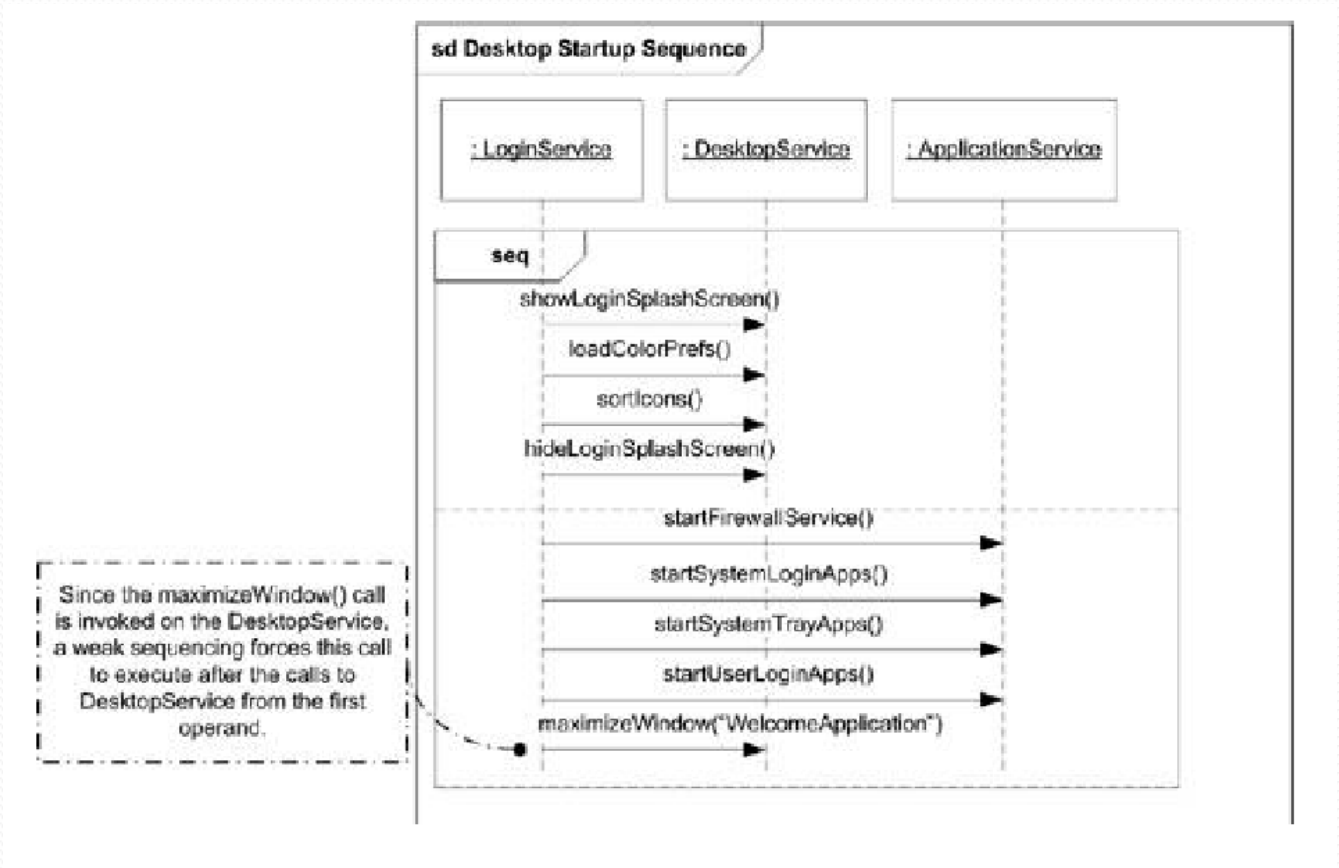
Break enclosing loop if y>0.

Fragmentos combinados

- El fragmento de combinado **Weak** (denotado “seq”) incluye un número de secuencias para las cuales todos los mensajes se deben procesar en un segmento anterior, antes de que el siguiente segmento pueda comenzar, pero que **no impone** ningún secuenciado en los mensajes que no comparten una línea de vida.
- *Sigue las siguientes reglas:*
  - El orden de los eventos en cada operando se mantiene.
  - Si los eventos de los dos operandos ocurren en líneas de vida diferentes podrían secuenciarse en cualquier orden.
  - Si los eventos de los dos operandos ocurren en misma línea de vida solo podrían secuenciarse de modo que los eventos del primer operando se ejecutasen antes que los del segundo.
- Un fragmento combinado Weak se reduce a uno paralelo cuando todos sus operando están en lineas de vida diferentes.
- Un fragmento combinado Weak se reduce a uno de secuenciamiento estricto cuando sus operandos trabajan en la misma linea de vida.

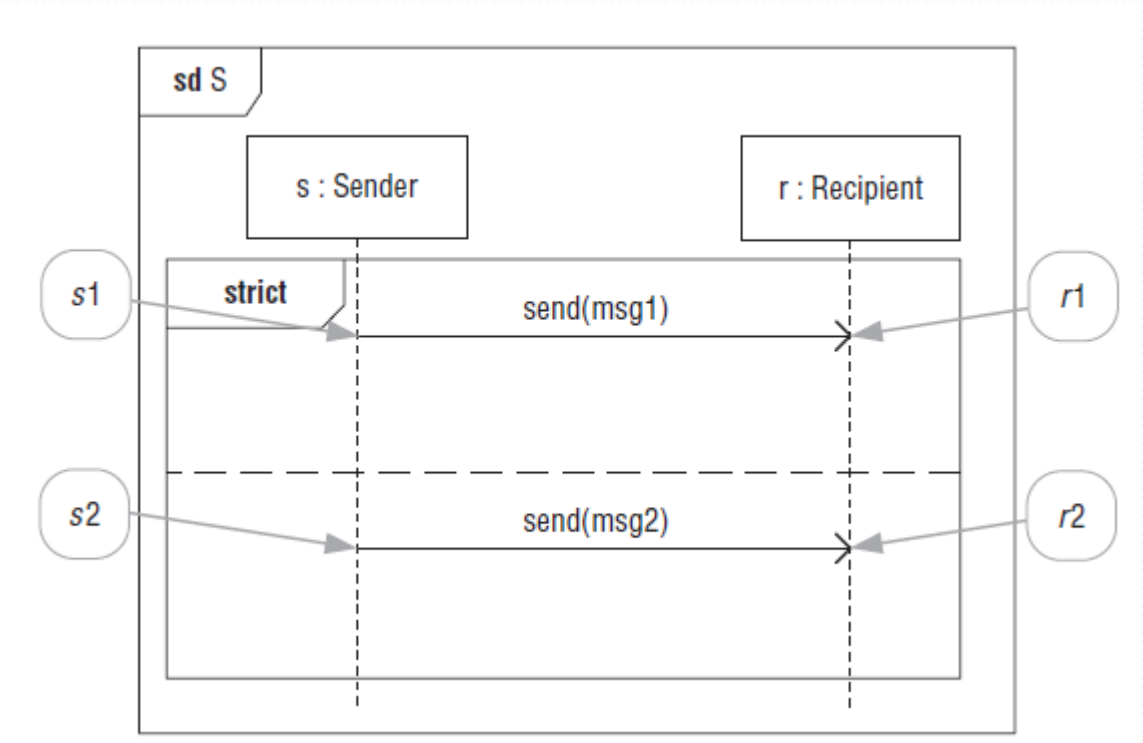
Secuenciamiento Débil

- Ejemplo de “seq”



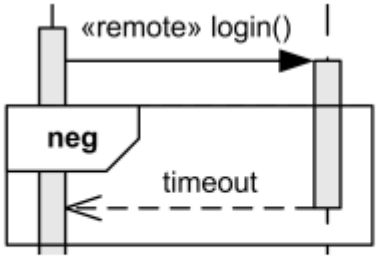
Fragmentos combinados

- El fragmento de secuenciado **Strict** (denotado “strict”) incluye una serie de mensajes que se deben procesar en el orden proporcionado.
- Los operandos deben ser ejecutados en orden estricto, de arriba abajo sin importar que estén en diferentes líneas de vida



Fragmentos combinados

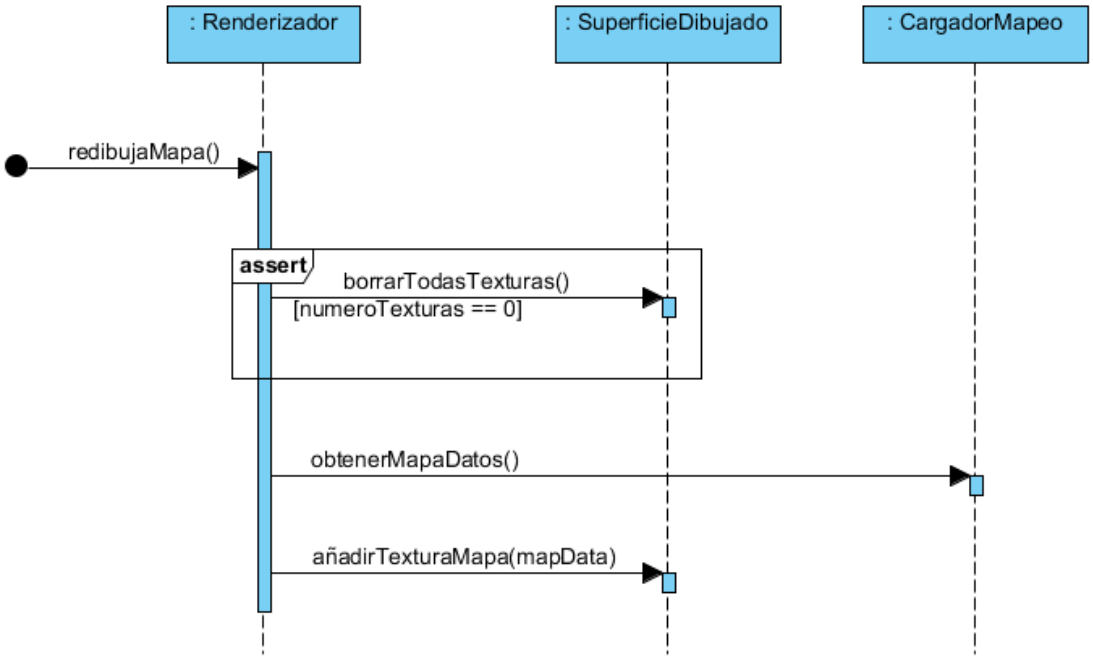
- El fragmento **Negative** (denotado “neg”) incluye una serie de mensajes inválidos.
- Raramente se usa y si se utiliza es para remarcar que una secuencia de mensajes es inválida.



Should we receive back timeout message, it means the system has failed.

Fragmentos combinados

- El fragmento **Assertion** (denotado “assert”) designa que cualquier secuencia que no se muestra como un operando de la aserción es inválida. La secuencia especificada por el operador es la única válida.
- Las interacciones contenidas dentro del fragmento assert deben ocurrir tal y como vienen especificadas, sino se consideran inválidas.
  - Ej. La aserción nos garantiza que todas las texturas han sido eliminadas.

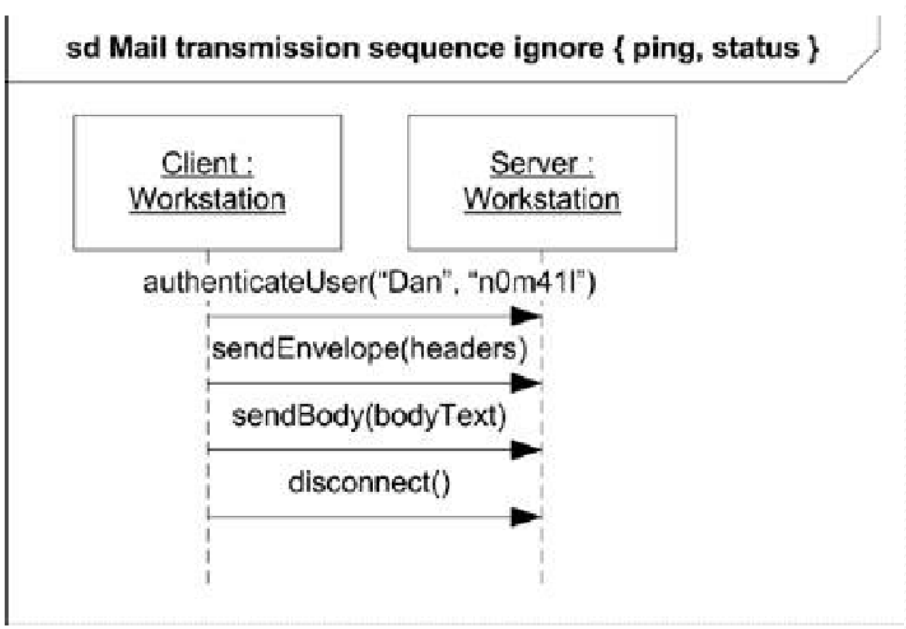


Fragmentos combinados

- El fragmento **Ignore** declara un mensaje o mensajes que no son de ningún interés si este aparece en el contexto actual.
- Sintáxis:

Ignore { nombreMensaje, nombreMensaje,.....}

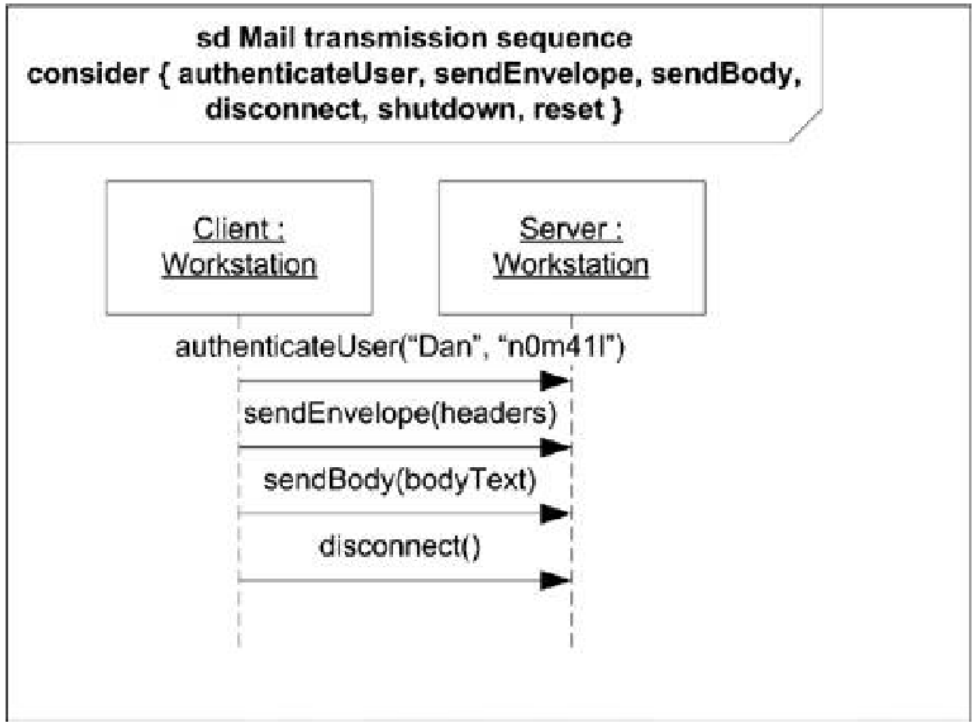
- Ejemplo:
  - Los mensajes ping e status son ignorados. Podrían ser tratados por el resto del sistema pero en el contexto actual se ignoran.



Fragmentos combinados

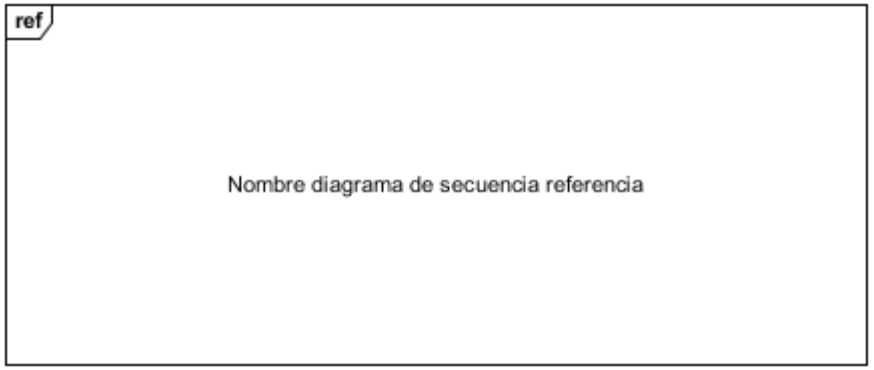
- El fragmento **Consider** es el opuesto del fragmento Ignore, cualquier mensaje que no se incluya en el fragmento Consider se debería ignorar.
- Sintáxis:

consider { nombreMensaje, nombreMensaje,.....}

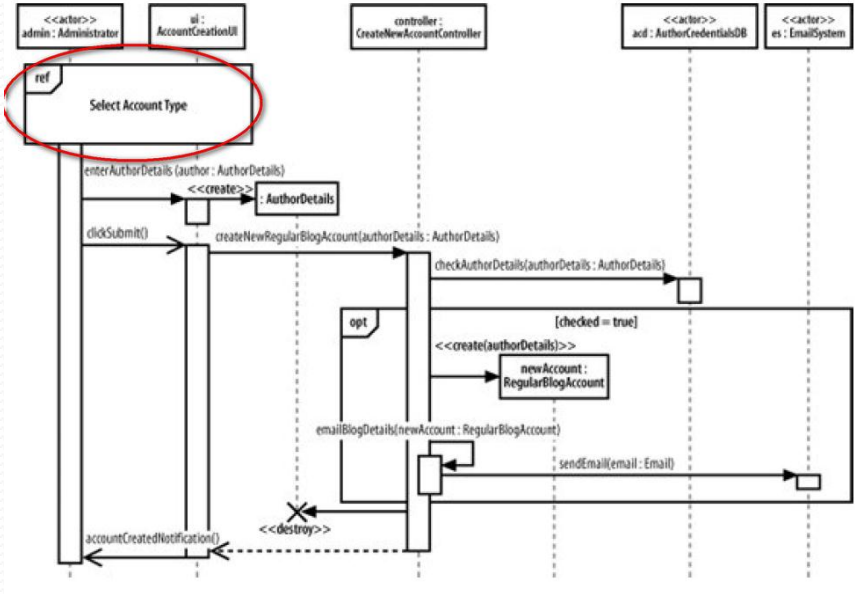


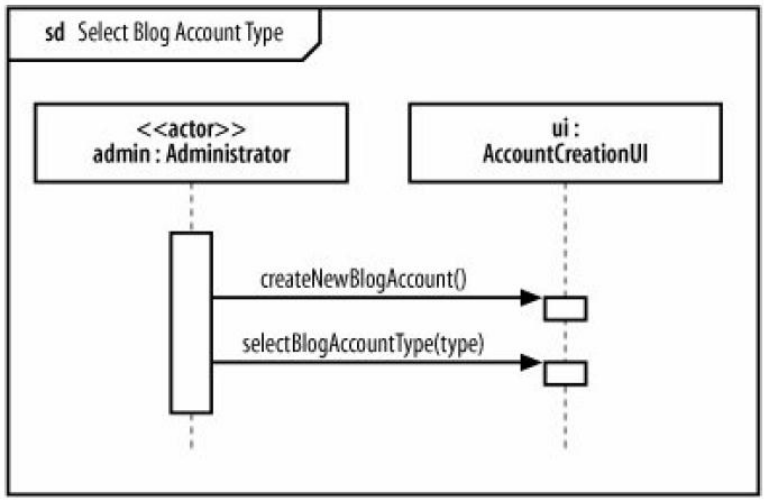
Uso de Interacción. Referencia

- Un uso de interacción es una **referencia** a una interacción o fragmento de interacción.
- Permite la reutilización de interacciones.
- Se representa mediante un marco con la palabra "**ref**" en la esquina superior izquierda y el nombre del diagrama de secuencia referenciado en el interior.

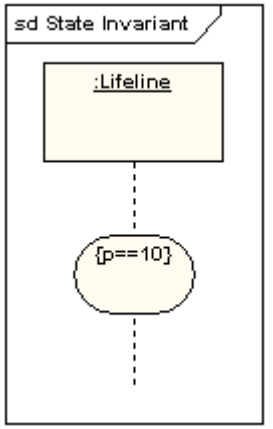


Uso de Interacción. Referencia



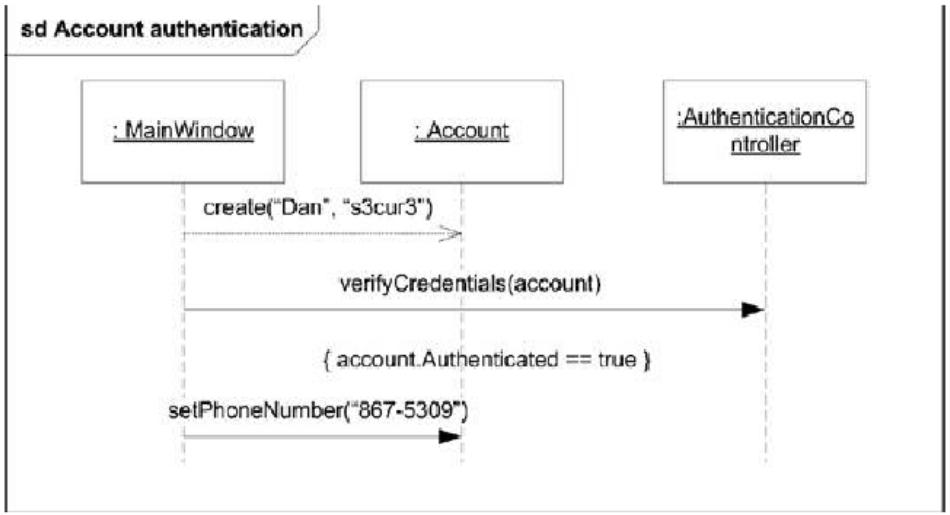


Invariantes de Estado



- Una **invariante** de estado es una restricción ubicada en una línea de vida que debe ser verdadera en el tiempo de ejecución. Esta se muestra como un rectángulo con los extremos en semi-circulos.

- Ej. La comprobación se hace una vez que los mensajes previos de la línea de vida han sido lanzados. En el ejemplo se muestran dos formas distintas de expresar la invariante.



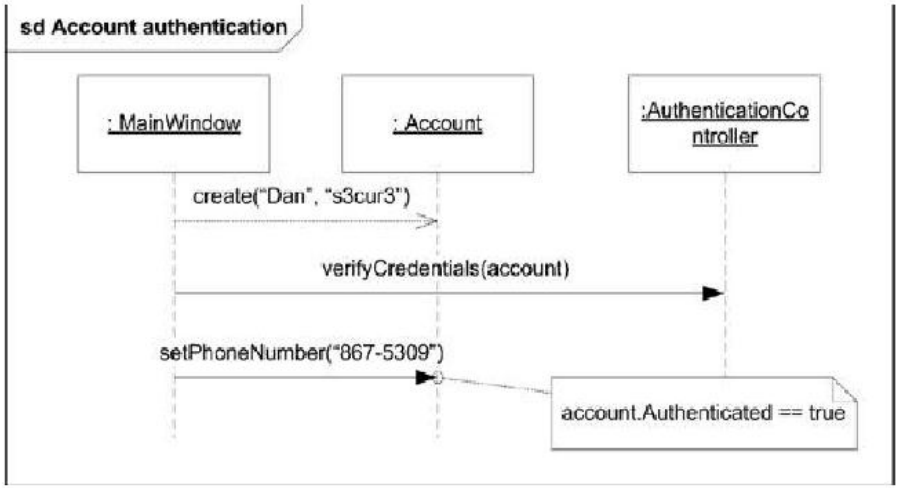


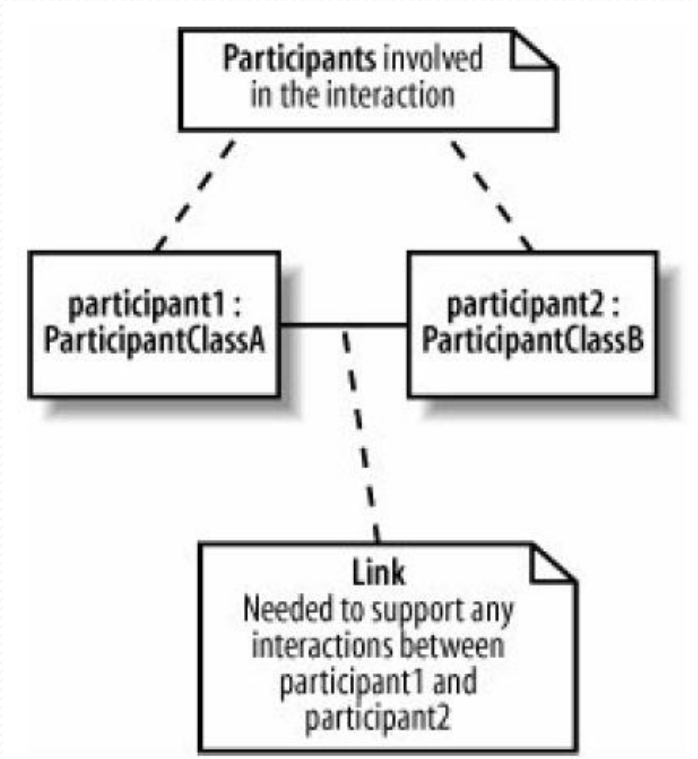
Diagrama de Comunicación

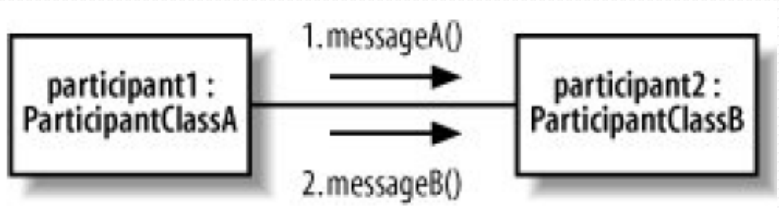
- Describen el comportamiento dinámico del sistema de información.
- Muestran la forma en cómo colaboran los objetos, es decir, con qué otros objetos tiene vínculos o intercambia mensajes un determinado objeto.
- Puede obtenerse a partir del diagrama de Secuencia (son isomorfos).
- Son útiles en la fase exploratoria para identificar objetos.
- La estructura estática viene dada por los enlaces; la dinámica por el envío de mensajes por los enlaces.

Diagrama de Comunicación

- Elementos:
  - **Objeto**: Se representa con un rectángulo dentro del que se incluye el nombre del objeto y, si se desea, el nombre de la clase, separando ambos por dos puntos.
  - ***Vínculo o enlace****: S*e representa como una línea continua que une ambos objetos y que puede tener uno o varios mensajes asociados en ambas direcciones. Un vínculo instancia una relación de asociación entre clases, también se puede indicar la navegabilidad del mismo mediante una flecha.
  - ***Mensaje:*** Se representa con una pequeña flecha colocada junto a la línea del vínculo al que está asociado. La dirección de la flecha va del objeto emisor del mensaje al receptor del mismo. Junto a ella, se coloca el nombre del mensaje y sus argumentos.

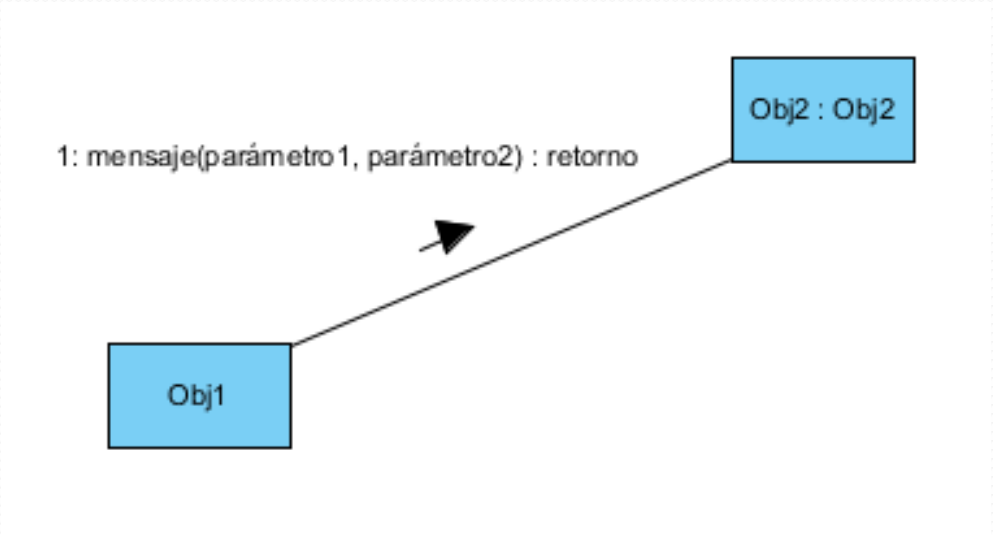
Diagrama de Comunicación





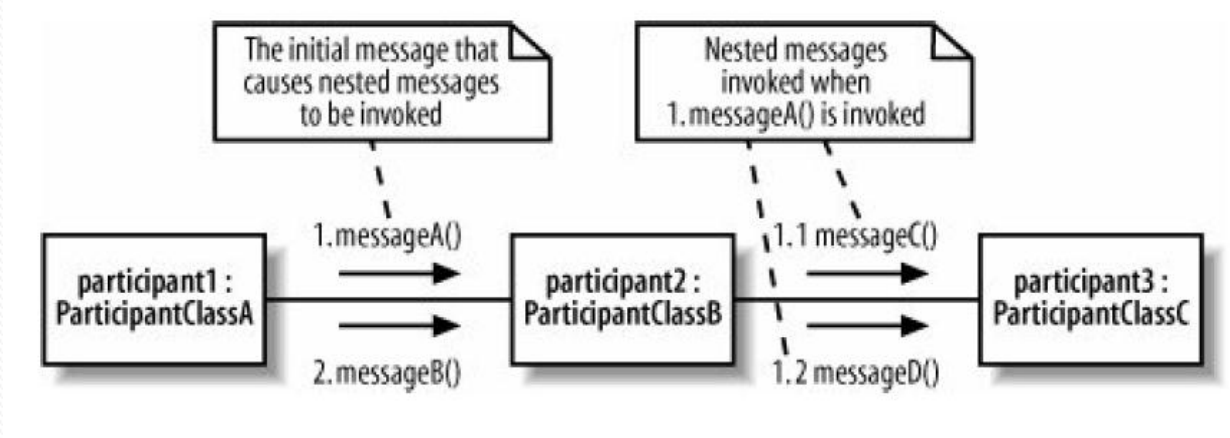
Mensajes

- Un mensaje desencadena una acción en el objeto destinatario.
- En los diagramas de comunicación **siempre se muestra el número de secuencia** del mensaje delante de su nombre, ya que no hay otra forma de conocer la secuencia de los mismos.
- Un mensaje puede tener definidos unos parámetros y un valor de retorno



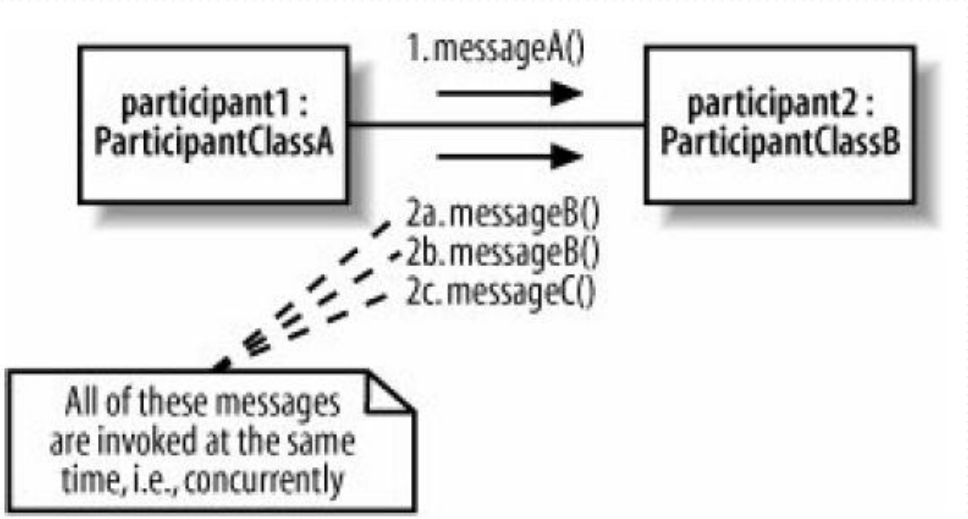
Mensajes anidados

- El anidamiento de mensajes se consigue a través de la jerarquía de números de secuencia.
- Ejemplo:
  - El mensajeC y mensajeD están anidados dentro del mensajeA y se ejecutan antes que el mensajeB



Mensajes concurrentes

- En los diagramas de secuencia teníamos el fragmento de tipo “par”.
- En los diagramas de comunicación se utilizan números de secuencia con letras para indicar que un mensajes se ejecuta al mismo tiempo que otro.

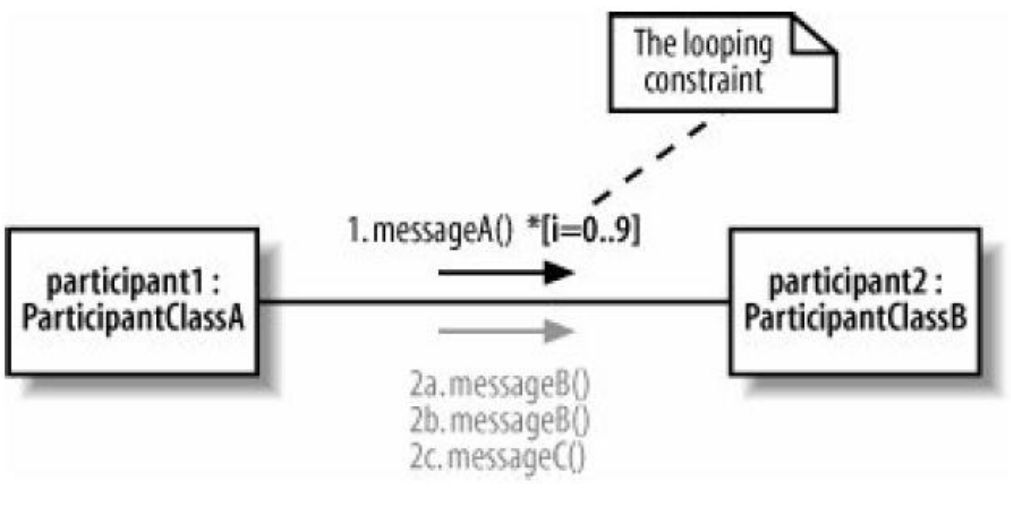


Mensajes Iterativos

- Para indicar que un mensaje se ejecuta un número de veces se utiliza el símbolo "*". La condición del bucle se indica entre corchetes seguidamente al asterisco.

*[i = 0 ...n]

- Ejemplo:
  - El mensajeA se enviará 10 veces antes de ejecutar los mensajes 2a, 2b y 2c



Mensajes Condicionados

- Los mensajes pueden llevar asociados una condición de guarda, que será una expresión booleana.
- Si la condición de guarda es verdadera el mensaje se enviará. Si la condición de guarda no se cumple el mensaje no se enviará.
- La condición Se representa entre corchetes detrás del mensaje

  Mensaje[guarda]
- Ejemplo:
  - EL mensaje 2a solo se enviará si se cumple la condición. Los mensajes 2b y 2c se enviarán en todo caso.

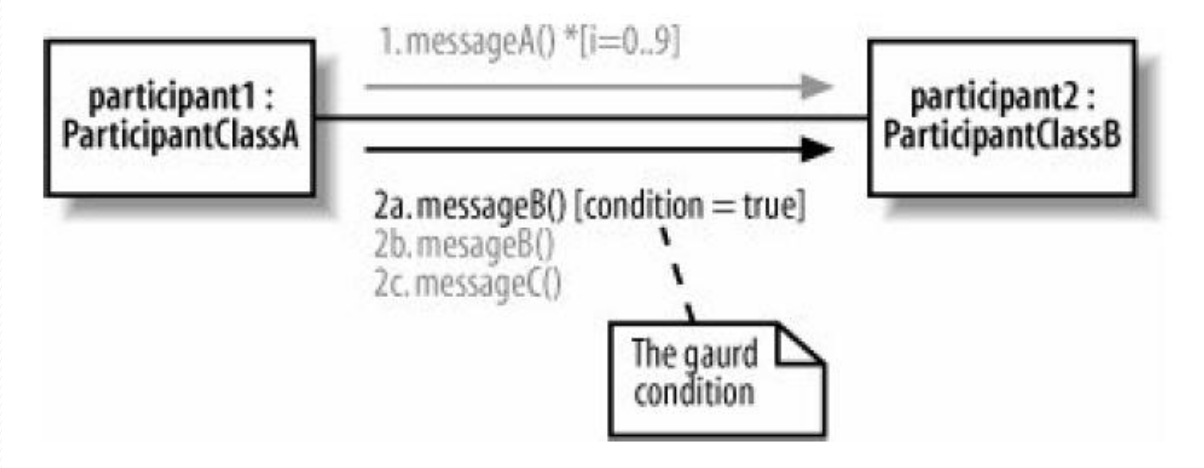


Diagrama de Comunicación

- Ejemplo: Caso de uso “prestar un ejemplar de una biblioteca”.

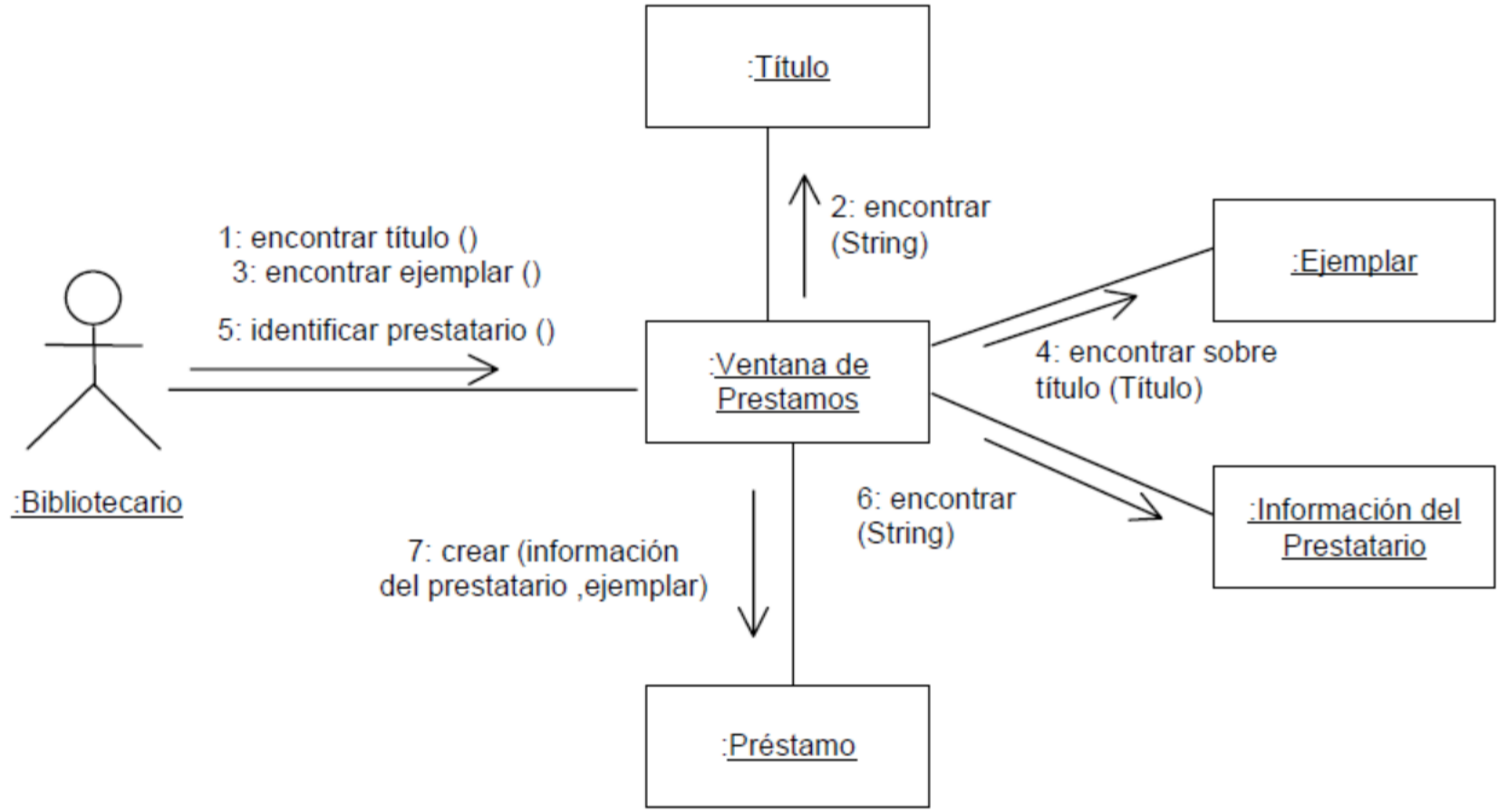
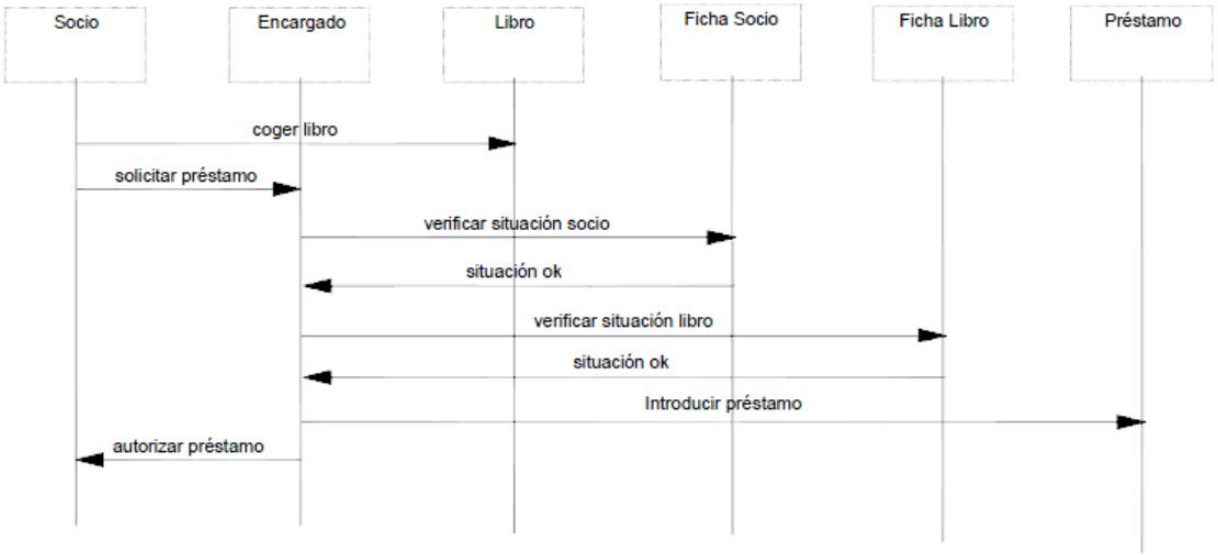


Diagrama de Secuencia **vs** Diagrama de Comunicación

- Equivalencia semántica
- **Diagramas de secuencia** muestran mejor el **orden** en que se ejecutan los mensajes
- **Diagramas de comunicación** muestran claramente los objetos con que interactúa un determinado objeto.

Ejemplo


Socio
Encargado
Libro
Ficha Socio
Ficha Libro
Préstamo
coger libro
solicitar préstamo
verificar situación socio
situación ok
verificar situación libro
situación ok
Introducir préstamo
autorizar préstamo

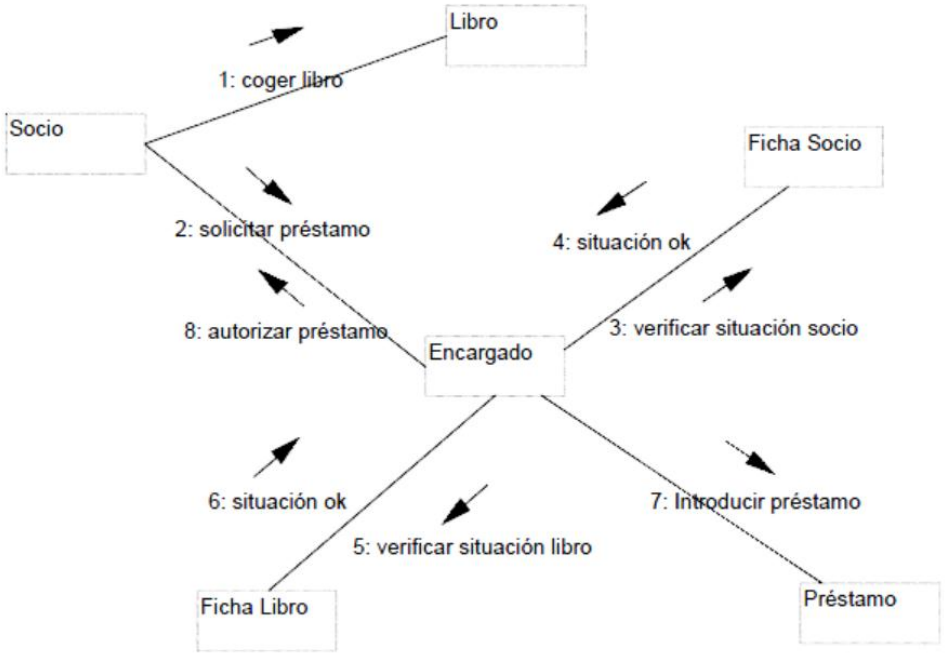
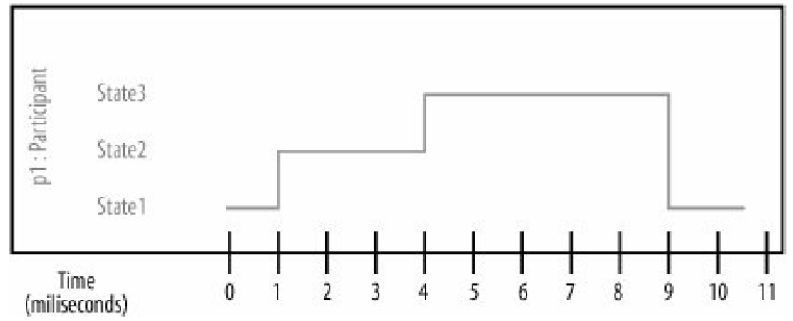

Libro
1: coger libro
Socio
Ficha Socio
2: solicitar préstamo
4: situación ok
8: autorizar préstamo
3: verificar situación socio
Encargado
6: situación ok
7: Introducir préstamo
5: verificar situación libro
Ficha Libro
Préstamo

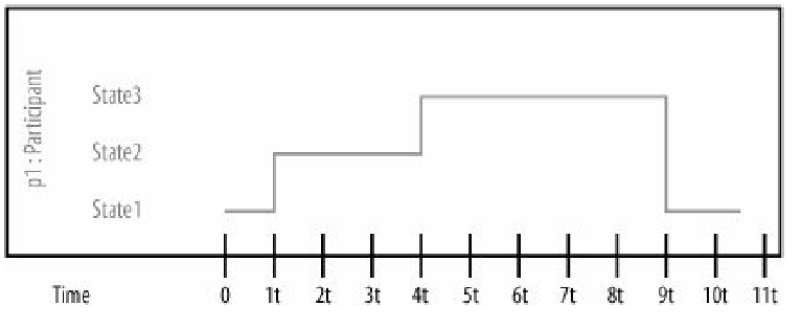
Diagrama de Tiempo

- Los diagramas de tiempos de UML se usan para mostrar el **cambio en el estado o valor** de uno o más elementos en el tiempo.
- Puede mostrar la interacción entre:
  - los eventos de tiempos
  - las restricciones de tiempos
  - la duración que los gobiernan.
- Elementos de un Diagrama de Tiempo
  - Tiene una o mas líneas de vida
    - El nombre del objeto se muestra en cada línea de vida
  - Eje vertical -> Estados de la línea de vida
  - Eje horizontal de tiempos
    - Marcas de tiempo
  - Eventos

Tiempo

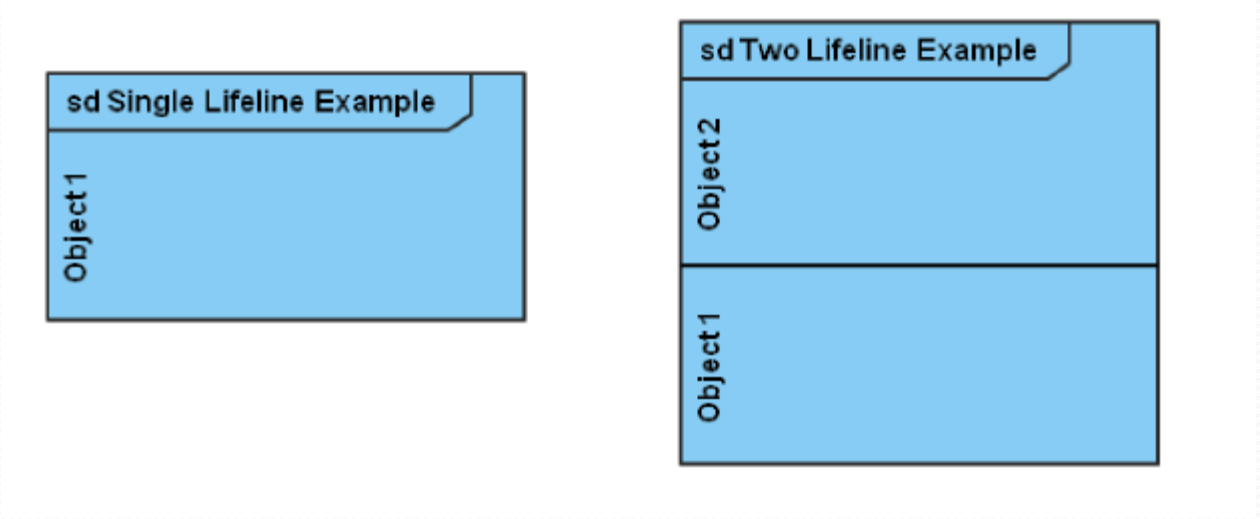
- Se representa en el eje horizontal
- Corre de izquierda a derecha
- Dos tipos de medidas:
  - Exactas
  - Relativas





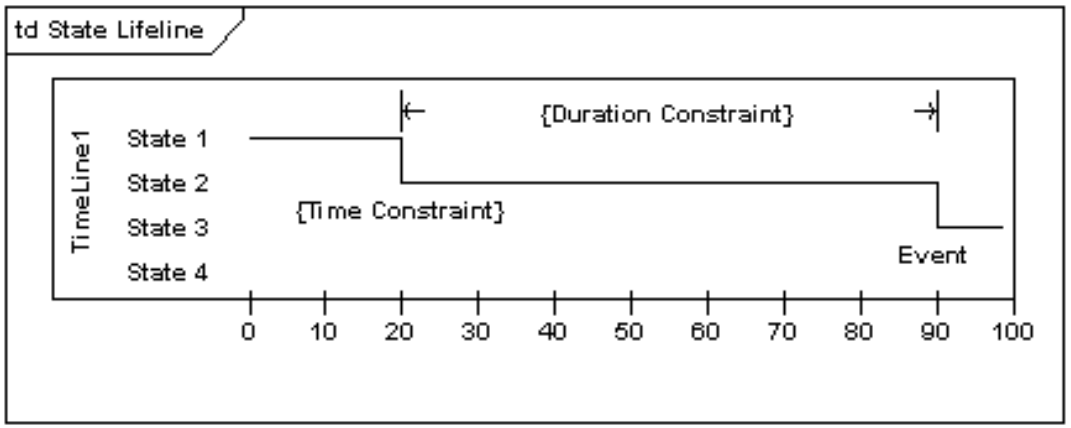
Línea de vida

- La líneas de vida se representan dentro de un marco rectangular
- Se leen de izquierda a derecha



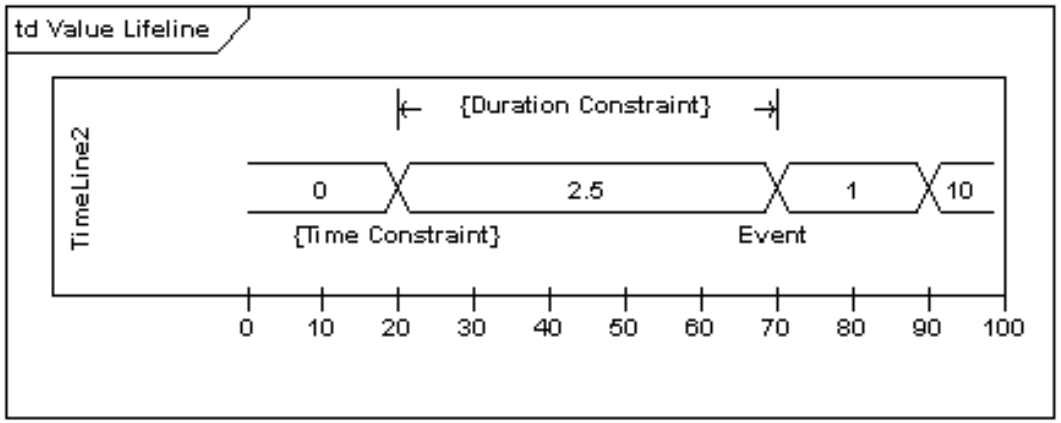
Línea de Vida de Estado

- Una línea de vida del estado muestra el cambio de estado de ítem en el tiempo
- El **eje-X** muestra el tiempo trascurrido en cualquier unidad que se elija
- El **eje-Y** se nombra con una lista de estados proporcionados

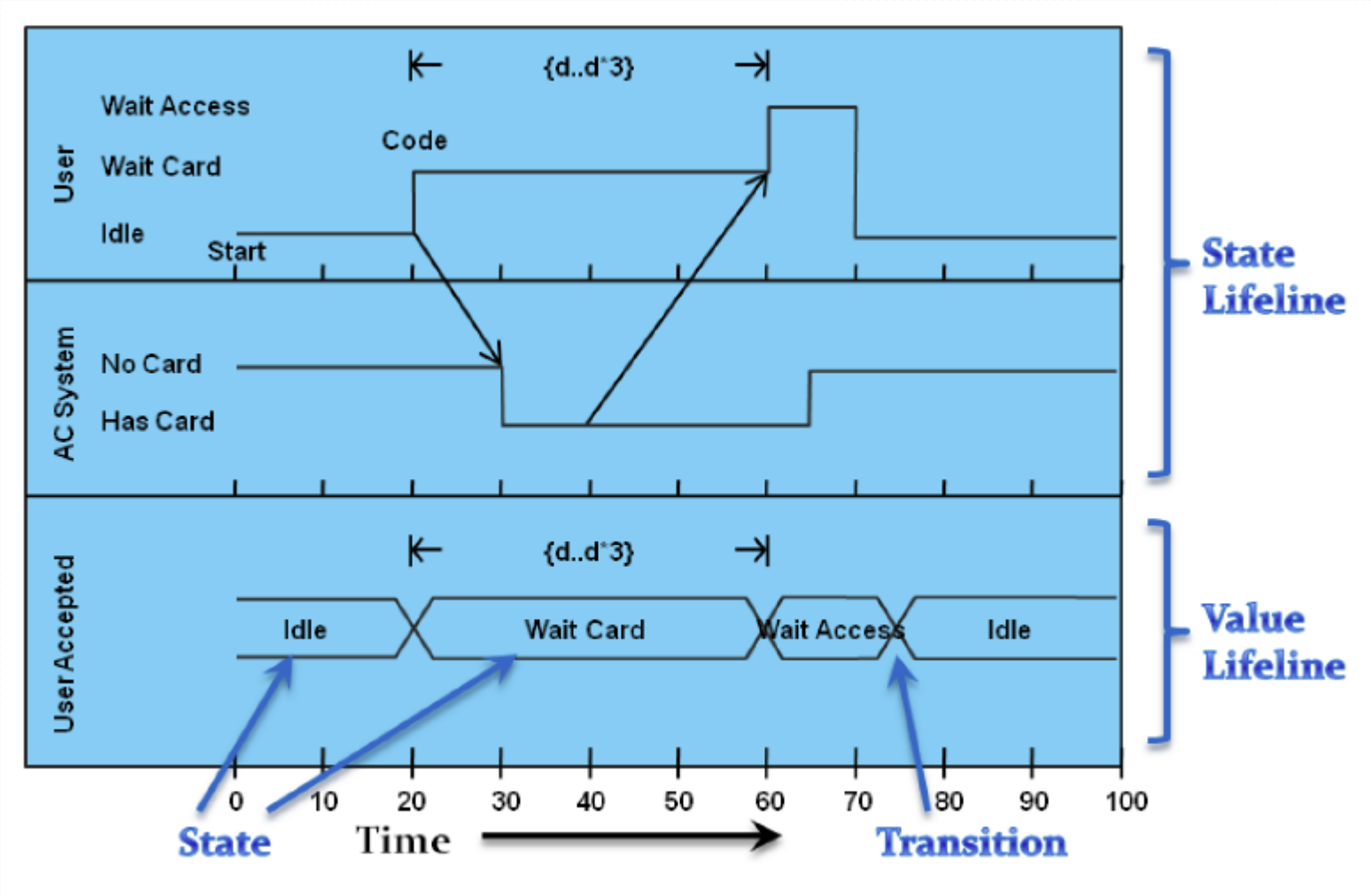


Línea de Vida del Valor

- Representación **compacta**
- Muestra el cambio del valor de un ítem en el tiempo
- El **eje-X** muestra el tiempo transcurrido en cualquier unidad que se elija, lo mismo que para la línea de vida del estado
- El valor se muestra entre el par de líneas horizontales que se cruzan en cada cambio del valor

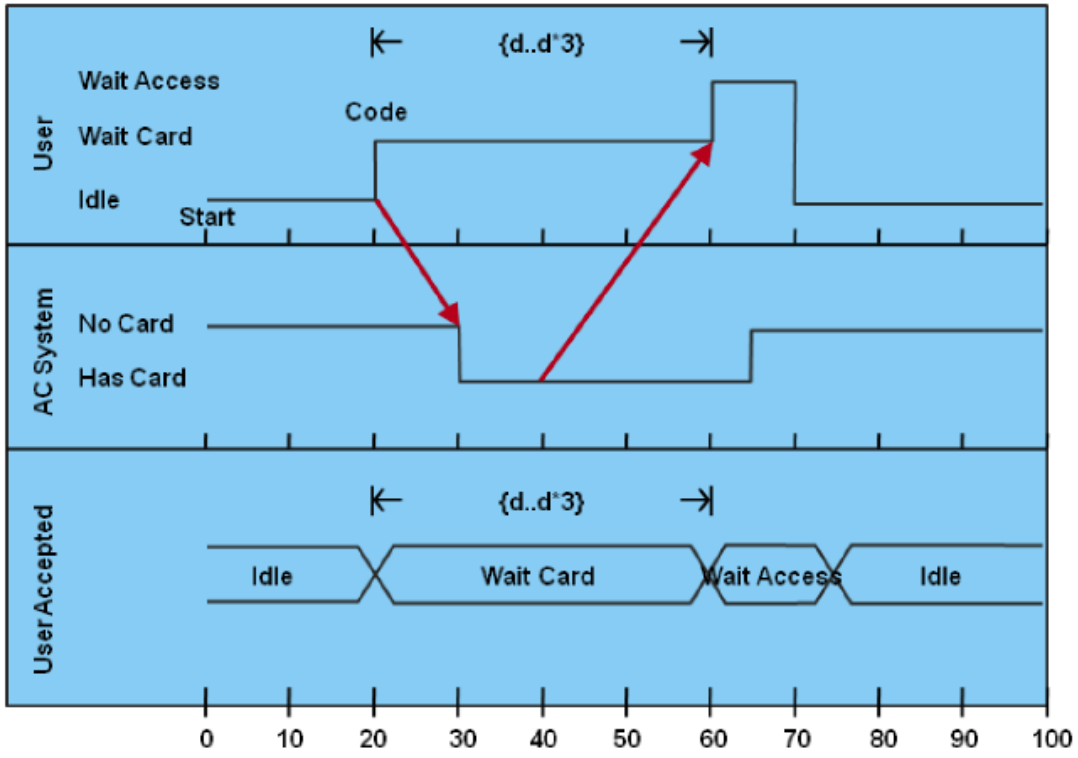


Representación conjunta



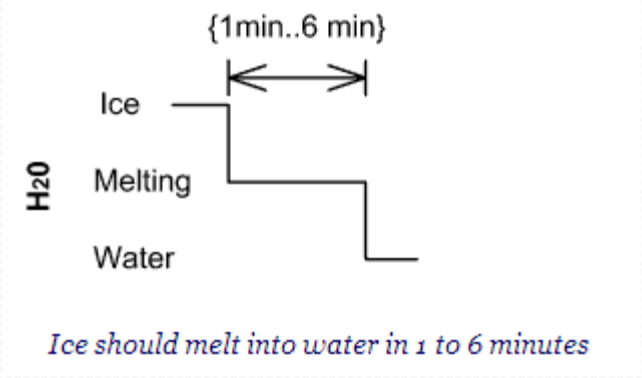
Eventos y Mensajes

- Los participantes cambian de estado en respuesta a eventos
- Se muestran como una flecha que sale de una línea de vida a otra



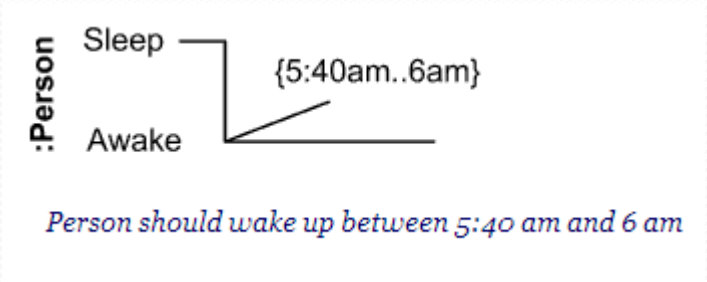
Restricciones de tiempo

- Las hay de dos tipos:
  - Restricciones de duración. Por ejemplo: la duración de un estado



Ice should melt into water in 1 to 6 minutes

  - Restricciones de tiempo. Indican restricciones en el comienzo o finalización de eventos o mensajes



Person should wake up between 5:40 am and 6 am

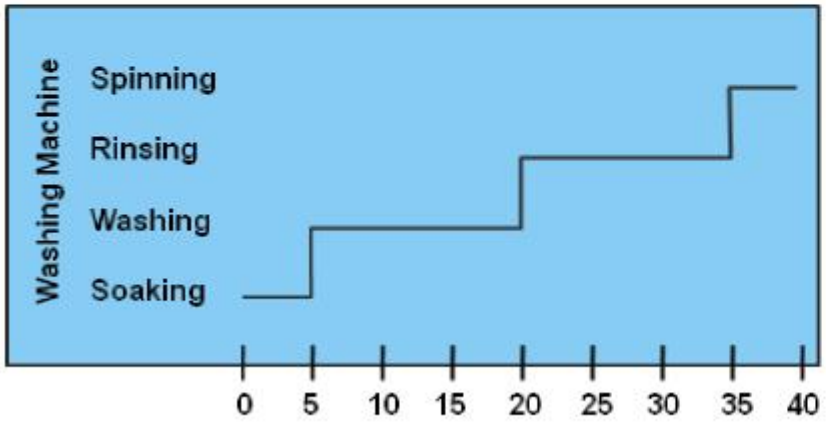
Ejemplos de restricciones de tiempo

Restricciones	Descripción
{t…t+5s}	La duración del evento o estado debe ser menor de 5 segundos.
{<5s}	La duración del evento o estado debe ser menor de 5 segundos. Es equivalente a {t….t+5s} pero menos formal.
{t}	La duración del evento o estado debe ser igual a t. t es un tiempo relativo que puede tomar cualquier valor.
{>5s,<10s}	La duración del evento o estado debe ser entre 5 y 10 segundos
{t…t*3}	La duración del evento o estado debe ser tres veces t (t es un valor relativo)

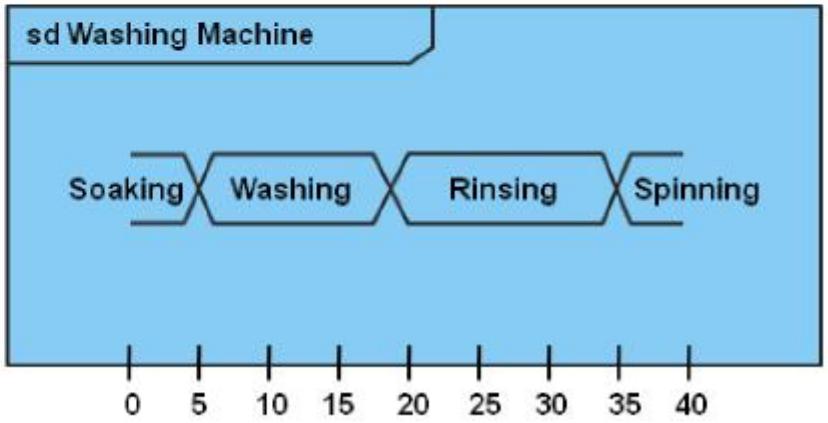
Ejemplo







Single State Lifeline



Single Value Lifeline

Diagrama de Tiempo: Resumen.

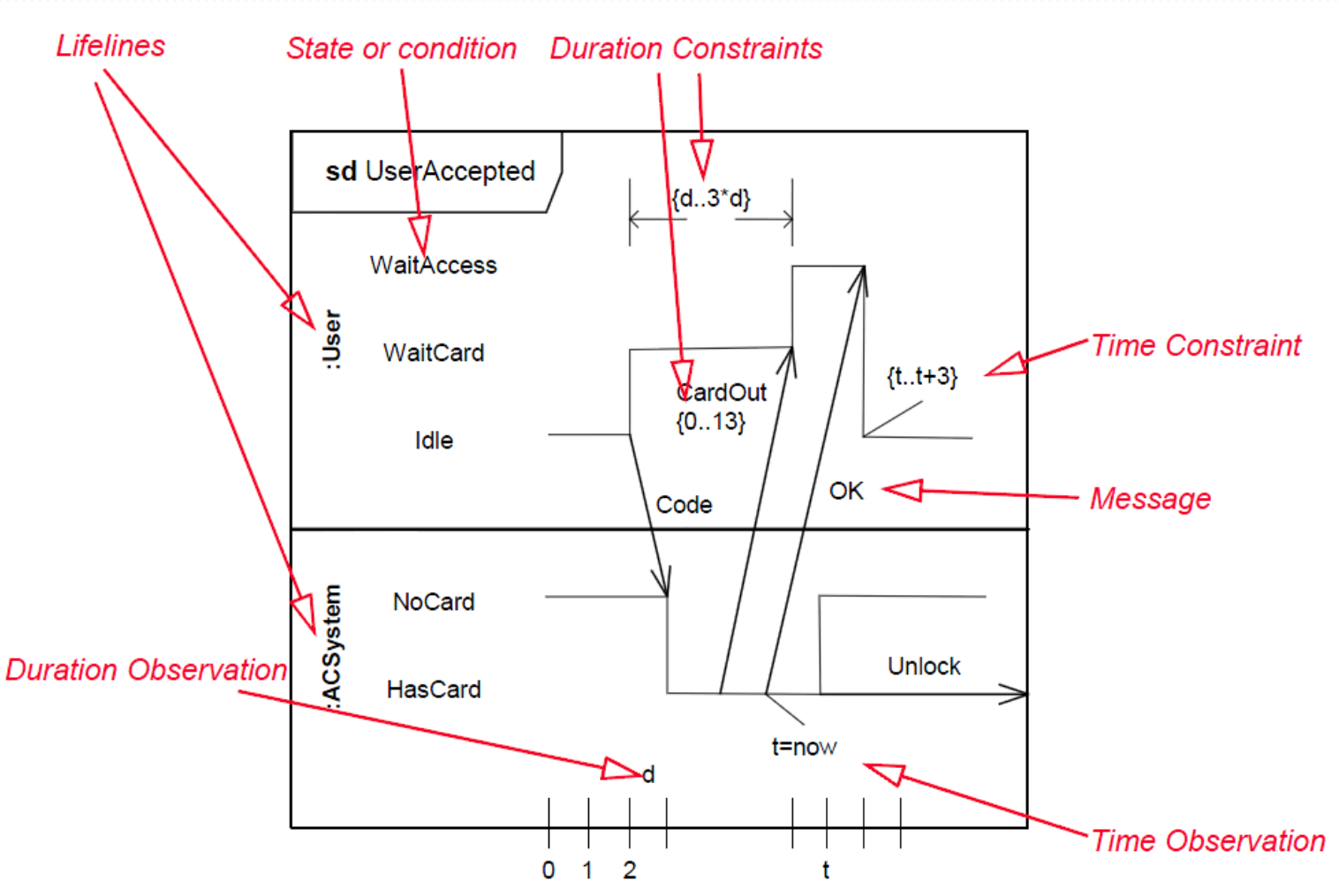


Diagrama de Global de Interacción

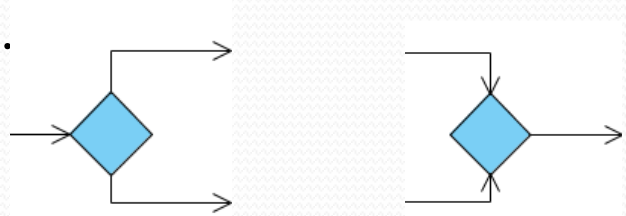
- Representa una **vista global** sobre el resto de diagramas de interacción (secuencia, comunicación y tiempo).
- Tiene la misma estructura que un diagrama de actividad, salvo que en vez actividades se representan interacciones.

Diagrama de Global de Interacción

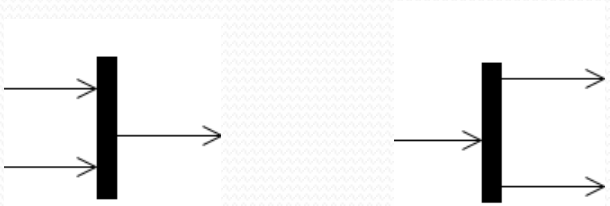
- Es una especialización de los diagramas de actividad:
  - Tienen un nodo de comienzo y un nodo de finalización.



  - Usa los pseudoestados nodos decisión y nodos fusión(“decision node” y “merge node”).

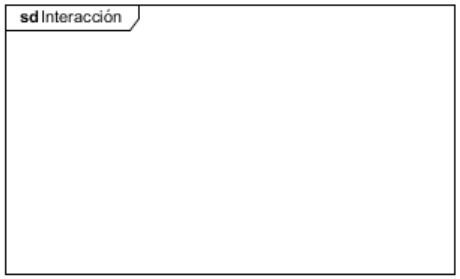


  - Usa los pseudoestados bifurcación y reunión (“fork node” y “join node”)



Interacción y Uso de Interacción

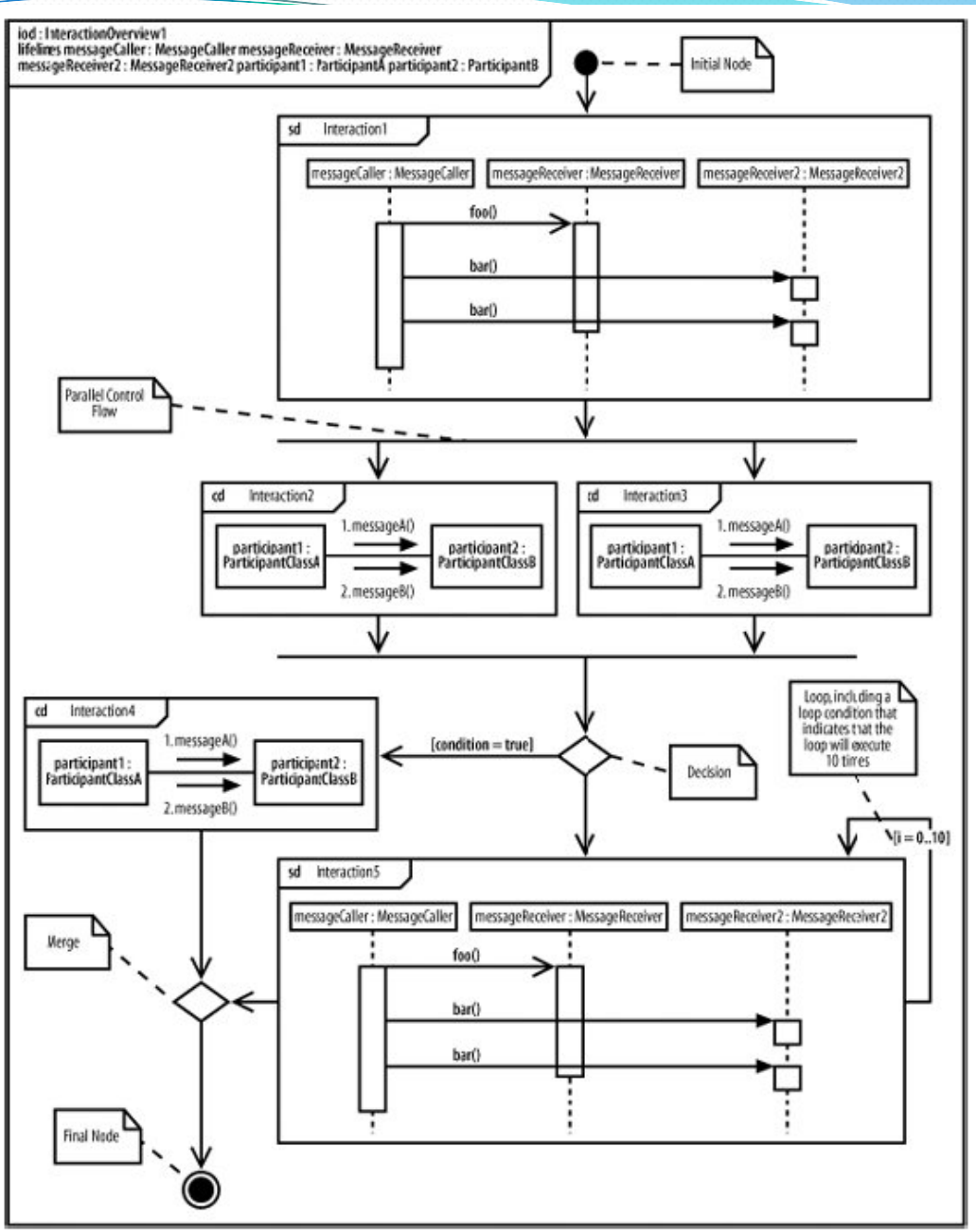
- Interacción:
  - Dentro se representan los distintos diagramas de interacción.
  - Pueden llevar el nombre en la parte superior izquierda.



- Uso de Interacción:
  - Indica una referencia a una interacción



Ejemplo


iod : InteractionOverview1
lifelines messageCaller : MessageCaller messageReceiver : MessageReceiver
messageReceiver2 : MessageReceiver2 participant1 : ParticipantA participant2 : ParticipantB
Initial Node
sd Interaction1
messageCaller : MessageCaller
messageReceiver : MessageReceiver
messageReceiver2 : MessageReceiver2
foo()
bar()
bar()
Parallel Control Flow
cd Interaction2
participant1 : ParticipantClassA
1.messageA()
2.messageB()
participant2 : ParticipantClassB
cd Interaction3
participant1 : ParticipantClassA
1.messageA()
2.messageB()
participant2 : ParticipantClassB
cd Interaction4
participant1 : ParticipantClassA
1.messageA()
2.messageB()
participant2 : ParticipantClassB
[condition = true]
Decision
Loop, including a loop condition that indicates that the loop will execute 10 times
[i = 0..10]
sd Interaction5
messageCaller : MessageCaller
messageReceiver : MessageReceiver
messageReceiver2 : MessageReceiver2
foo()
bar()
bar()
Merge
Final Node

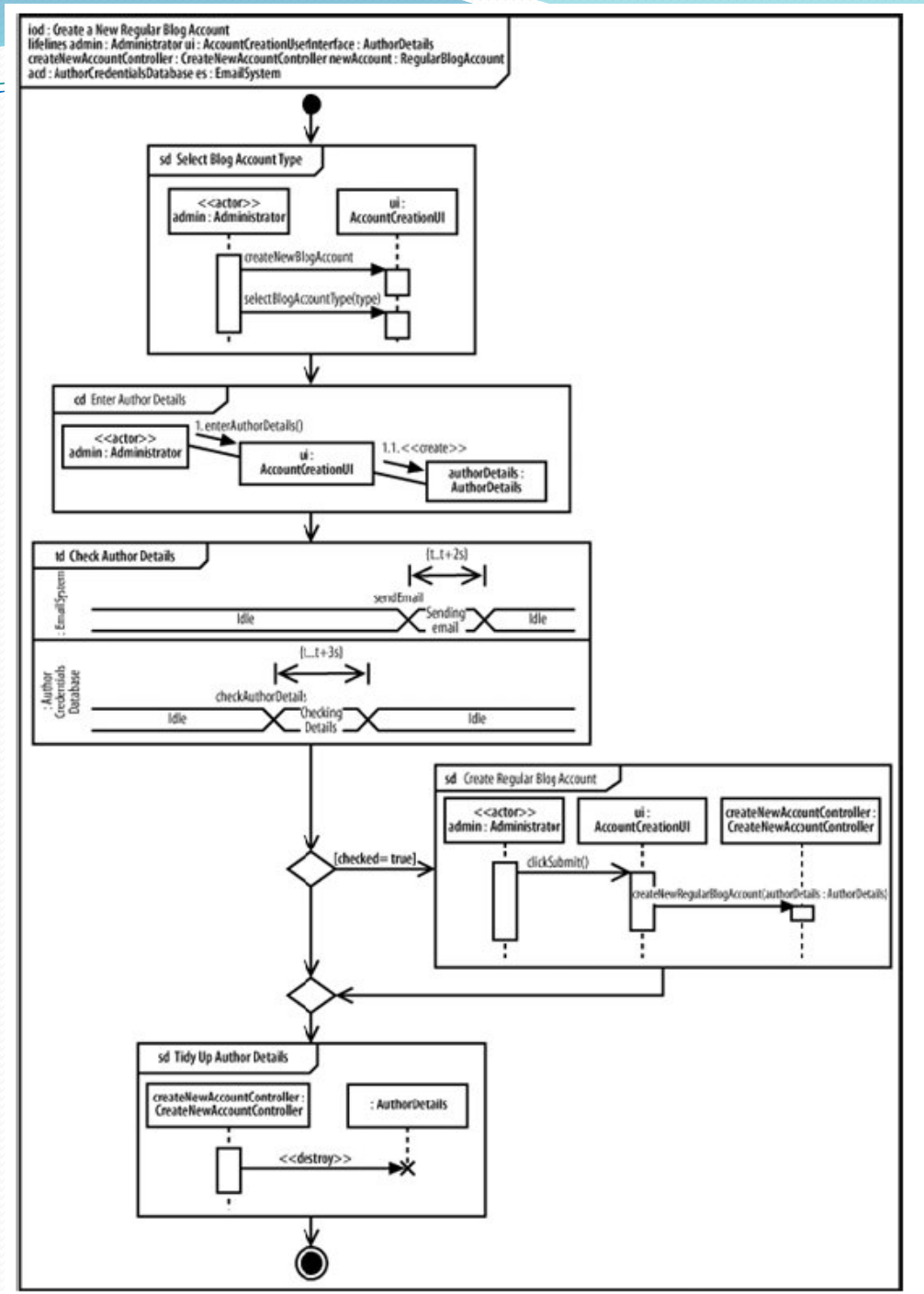
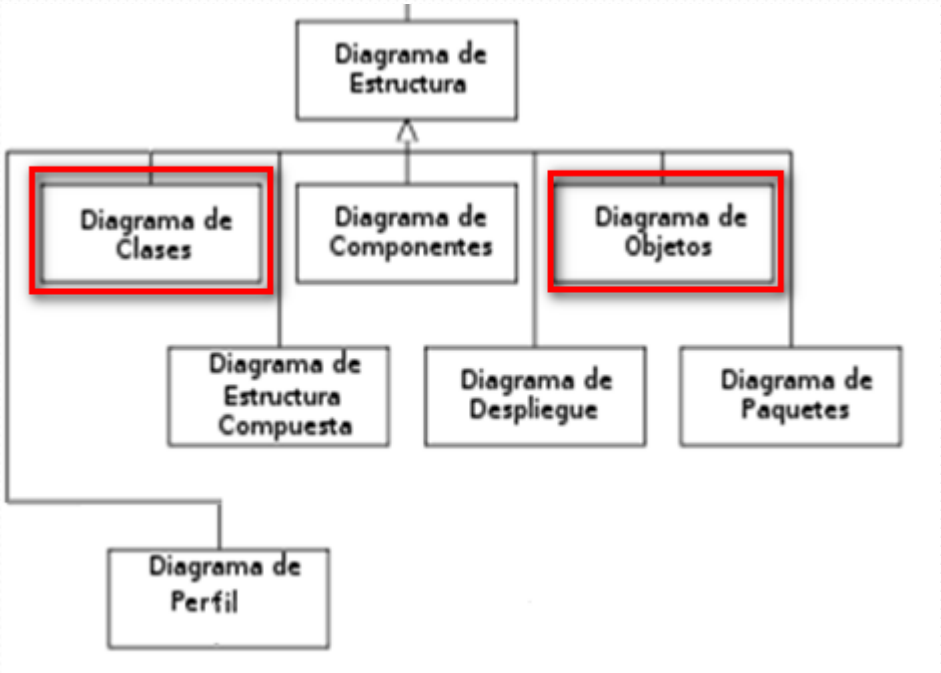

iod : Create a New Regular Blog Account
lifelines admin : Administrator ui : AccountCreationUserInterface : AuthorDetails
createNewAccountController : CreateNewAccountController newAccount : RegularBlogAccount
acd : AuthorCredentialsDatabase es : EmailSystem
sd Select Blog Account Type
<<actor>>
admin : Administrator
ui :
AccountCreationUI
createNewBlogAccount
selectBlogAccountType(type)
cd Enter Author Details
<<actor>>
admin : Administrator
1. enterAuthorDetails()
ui :
AccountCreationUI
1.1. <<create>>
authorDetails :
AuthorDetails
td Check Author Details
{t..t+2s}
: EmailSystem
sendEmail
Idle
Sending
email
Idle
{t...t+3s}
: Author
Credentials
Database
checkAuthorDetails
Idle
Checking
Details
Idle
sd Create Regular Blog Account
<<actor>>
admin : Administrator
ui :
AccountCreationUI
createNewAccountController :
CreateNewAccountController
clickSubmit()
createNewRegularBlogAccount(authorDetails : AuthorDetails)
[checked= true]
sd Tidy Up Author Details
createNewAccountController :
CreateNewAccountController
: AuthorDetails
<<destroy>>

Diagrama de clases y Diagrama de Objetos



Para qué utilizamos los diagramas de clases

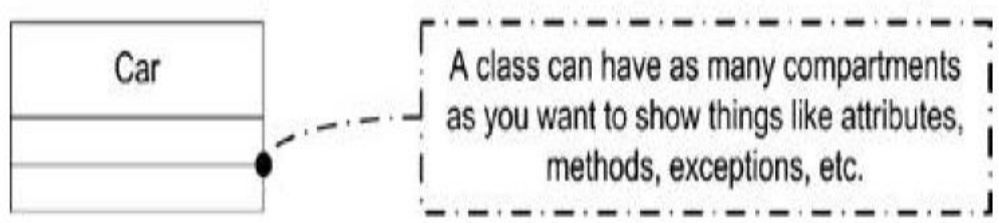
- Un diagrama de clases es un diagrama UML de estructura que muestra clases e interfaces con sus relaciones.
- Los diagramas de clases se usan para modelar:
  - **Vocabulario del sistema**
    - Las clases son el vocabulario y terminología de un área de conocimiento.
    - Se identifican clases para abstracciones relevantes del dominio del problema.
  - **Colaboraciones (parte estática)**
    - Se identifican clases e interfaces cuya interacción produce el comportamiento deseado.
  - **Esquema lógico de base de datos (clases persistentes)**

¿Qué diagrama de los utilizados en UML (Unified Modeling Language) representa la estructura y el comportamiento de cada uno de los objetos del sistema y sus relaciones con los demás objetos? (Analista programador Ayto de Madrid 2010)

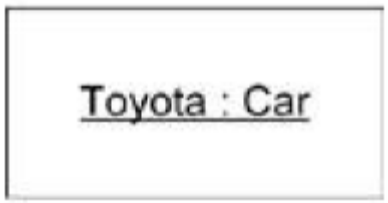
- Diagrama de usos
- **Diagrama de clases** ----------
- Diagrama de estados
- Diagrama de tiempos

Clases y objetos

- Una clase es un contenedor de datos (atributos) junto con las operaciones para manipular esos datos.
- **Clase = atributos + operaciones**
- Un **clase** es una **plantilla** de la cual se pueden crear **instancias** que serán los **objetos**.
- **Una clase es un tipo de datos y un objeto es una variable de ese tipo**.



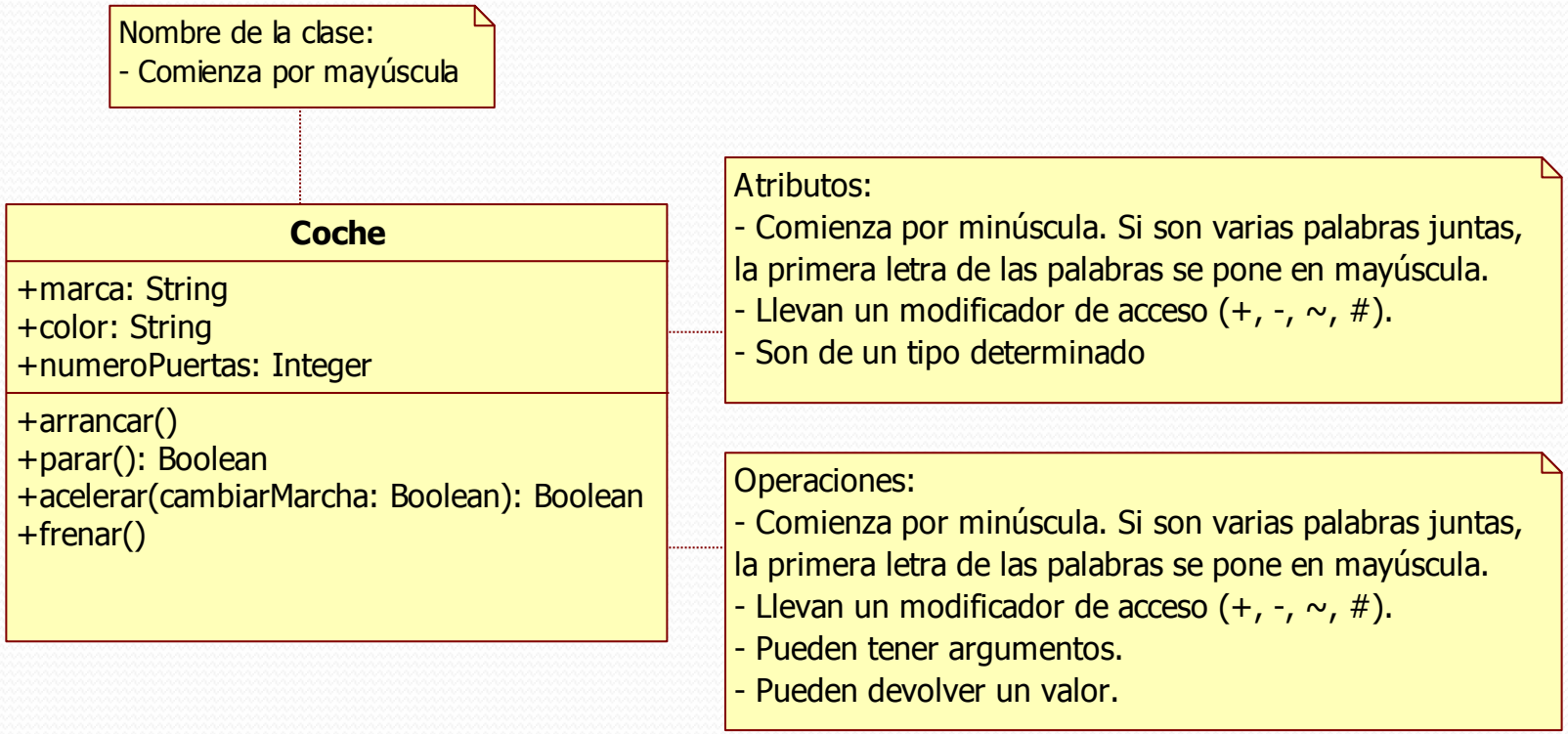
Clase



Objeto

Representación de una Clase

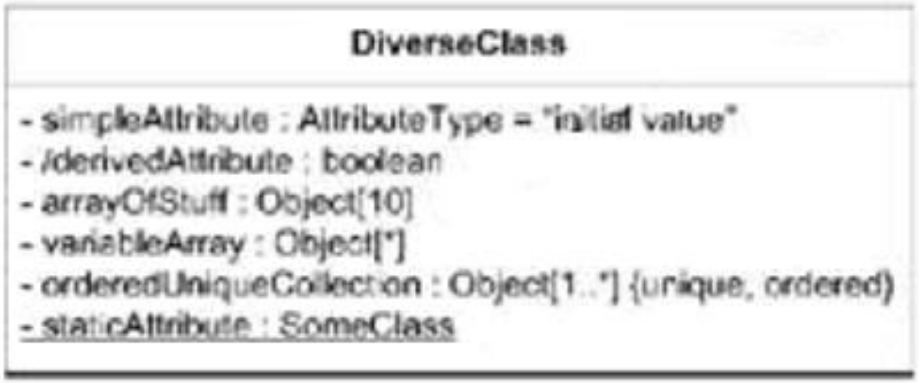
- Una clase se representa:
  - El nombre de la clase
  - Los atributos (Una clase puede tener varios o ningún atributo)
  - Las operaciones



Atributos de una clase

- Un **atributo** es una propiedad o **característica de una clase**.
- Todo objeto de la clase tiene un valor específico en cada atributo.
- **Los atributos representan el estado de un objeto**.
- Una clase puede tener varios o ningún atributo.
- Sintaxis:
  - Visibilidad (+, #, ~, -)
  - Atributo derivado (/)
  - Nombre
  - Tipo
  - Multiplicidad
  - Valor por defecto
  - Propiedades
  - Restricciones

```
visibility / name : type multiplicity = default
  {property strings and constraints }
```



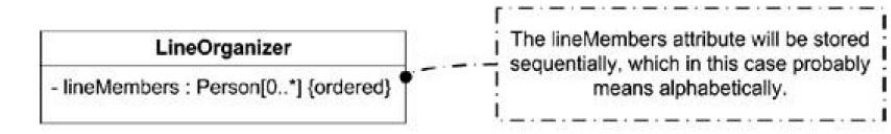
Atributos

- **Visibilidad** (nivel de encapsulamiento):
  - **Public** (+): Todo el mundo puede acceder al elemento.
    - Si es un dato miembro, todo el mundo puede ver el elemento, es decir, usarlo y asignarlo.
    - Si es un método todo el mundo puede invocarlo.
  - **Private** (-): Sólo se puede acceder al elemento desde la propia clase.
  - **Protected** (#): es una combinación de los accesos que proporcionan los modificadores public y private. Protected proporciona:
    - acceso público para las clases derivadas
    - acceso privado (prohibido) para el resto de clases
  - **Package** (~): se puede acceder al elemento desde cualquier clase del paquete donde se define la clase.
- **Atributo derivado** . Es aquel cuyo valor se puede calcular a partir de otros atributos de la clase. Se indica con el símbolo /.
- **Multiplicidad**. Especifica cuantas instancias de un tipo de atributo son creadas cuando se instancia la clase a la que pertenece dicho atributo.
  - Si no se especifica multiplicidad, la multiplicidad es 1.
  - La multiplicidad se expresa entre corchetes:

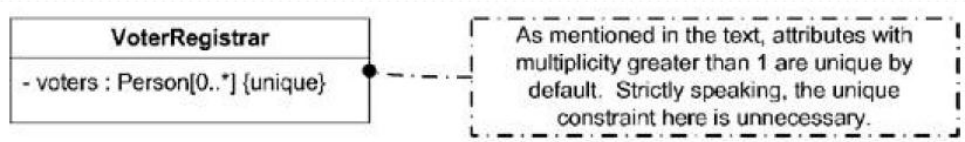
    Ej: wheels : Wheels[4] passengers: Person[1..5]

Atributos

- **Multiplicidad:**
  - **Propiedad ordered (ordenado)**: **Los elementos del atributo se añadirán de manera ordenada.** Se especifica que un atributo es ordenado poniendo la propiedad ordered entre llaves **{ordered}** detrás de los corchetes de multiplicidad.



  - **Propiedad unique (único):** Cada elemento del atributo será único. Se especifica que un atributo es único poniendo la propiedad unique entre llaves **{unique}** detrás de los corchetes de multiplicidad.

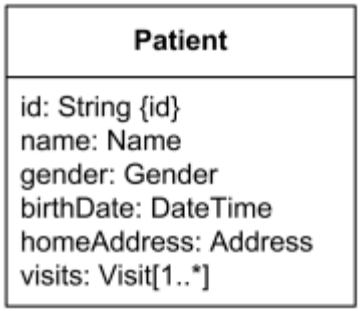


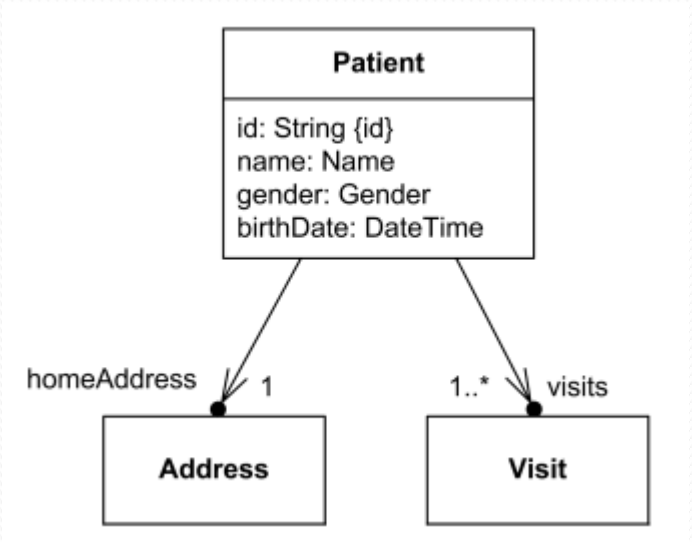
  - **Tipos de colecciones** en función del valor de sus propiedades ordered y unique

Orden	Unicidad	Tipo de colección asociado
Falso	Falso	Bag (bolsa)
Verdadero	Verdadero	Orderedset (conjunto ordenado)
Falso	Verdadero	Set (conjunto)
Verdadero	Falso	Sequence (secuencia, serie)

Notaciones para los atributos

- **Notación "en línea"**: los atributos se muestran dentro de la clase (notación clásica).
- **Notación por asociación**: los atributos no se muestran dentro de la clase. Esta notación produce un diagrama de clase más grande, pero muestra un mayor detalle para atributos complejos.





Propiedades

Modifier	Description
id	Property is part of the identifier for the class which owns the property.
readOnly	Property is read only (isReadOnly = true).
ordered	Property is ordered (isOrdered = true).
unique	Multi-valued property has no duplicate values (isUnique = true).
nonunique	Multi-valued property may have duplicate values (isUnique = false).
sequence (or seq)	Property is an ordered bag (isUnique = false and isOrdered = true).
union	Property is a derived union of its subsets.
redefines *property-name*	Property redefines an inherited property named ***property-name***.
subsets *property-name*	Property is a subset of the property named ***property-name***.
property-constraint	A constraint that applies to the property

Operaciones de una clase

- Las operaciones permiten manipular los datos.
- Sintaxis para las operaciones:

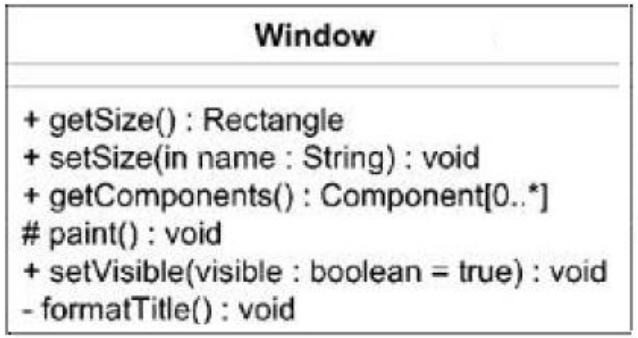
  visibilidad nombre (parámetros) : tipoDeRetorno {propiedades}
- Sintaxis para los parámetros:

  dirección nombre : tipo [multiplicidad] = valorPorDefecto {propiedades}

  - Dirección: es opcional, e indica cómo es usado un parámetro por un método. Los posibles valores son:
    - **in**: parámetro de entrada. Tiene asignado un valor. El método no modifica su valor, solo lo usa.
    - **out**: parámetro de salida. No tiene asignado un valor. El método modifica su valor.
    - **inout**: parámetro de entrada salida. El método lo usa y lo modifica.
    - **return**: el parámetro se devuelve como valor de retorno.
  - Propiedades (ordered, unique, readOnly)

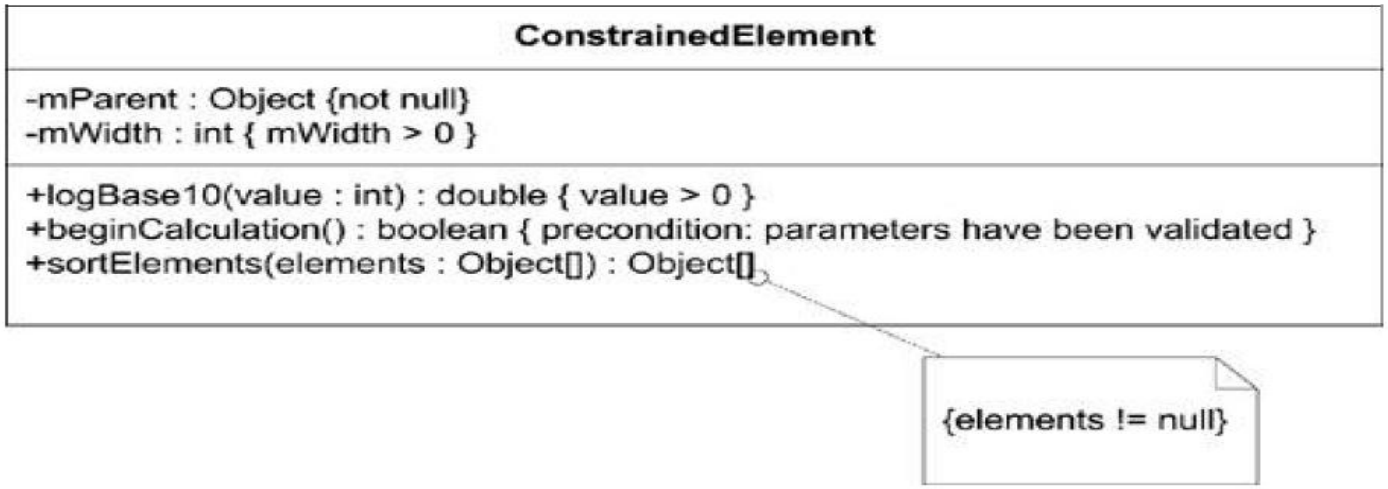
Operaciones de la clase Window





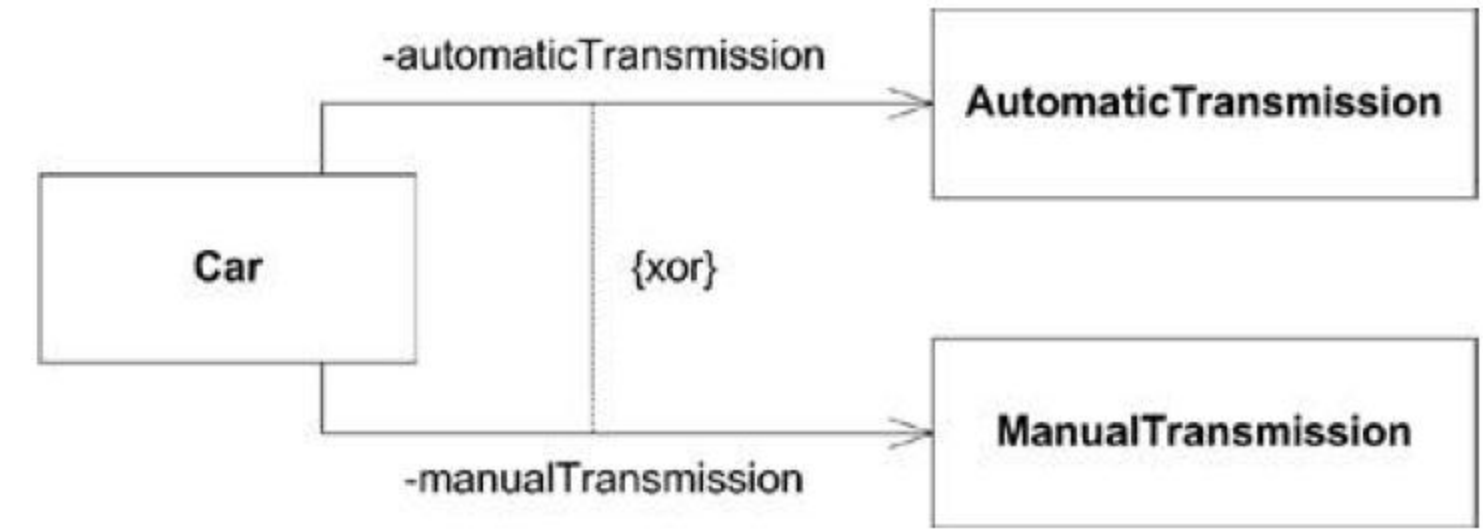
Restricciones

- Se pueden expresar
  - En lenguaje natural
  - En la gramática formal OCL
  - Mediante una nota
- Deben evaluar una expresión booleana.
- Se expresan entre llaves tras el elemento que restringen.



Restricciones de los atributos

- Ejemplo de restricción con notación “atributos por relación”:



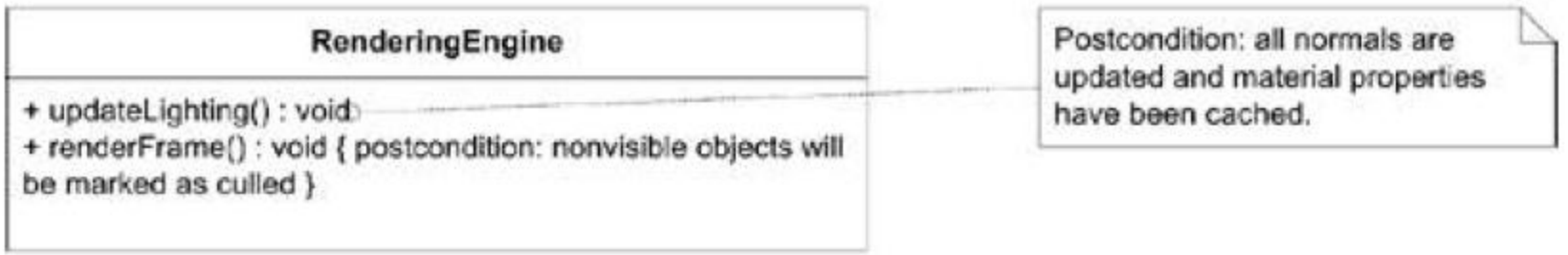
- Si hubiésemos utilizado notación “en línea”, la restricción la habríamos expresado en una nota.

Restricciones de las operaciones

- Una operación puede tener una serie de limitaciones o restricciones asociadas que ayudan a definir cómo la operación interactúa con el resto el sistema.
- Las limitaciones de una operación establecen un contrato que la implementación de la operación debe obedecer.
- **Tipos de restricciones:**
  - Precondiciones
  - Postcondiciones
  - Condiciones del cuerpo
  - Query

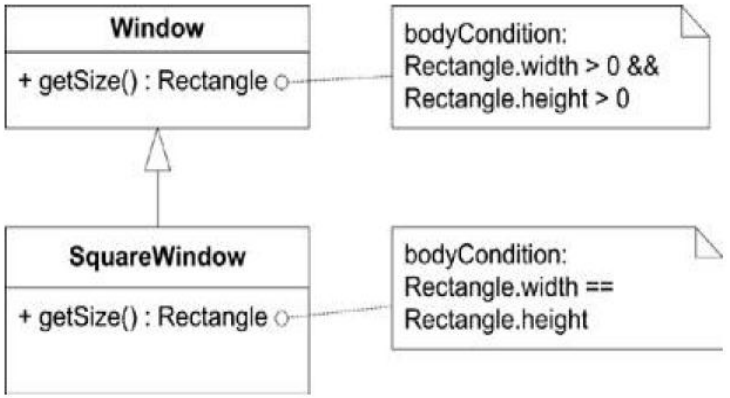
Restricciones de las operaciones

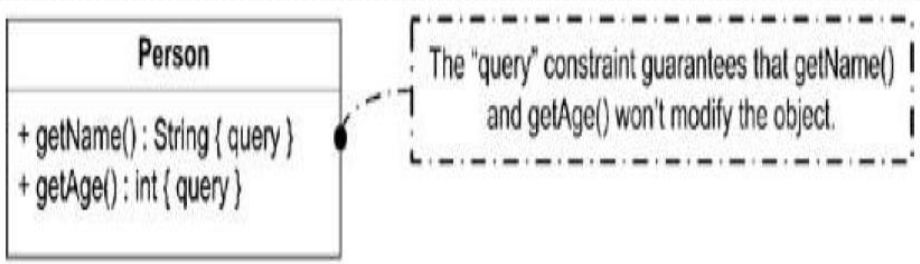
- **Precondiciones**:
  - Expresan el estado que ha de tener el sistema antes de poder invocar una operación.
  - En la práctica no se puede expresar el estado de todo el sistema. En su lugar, las condiciones previas o precondiciones suelen expresar los valores válidos para los parámetros, el estado de la clase que invoca la operación, o unos pocos atributos fundamentales del sistema.
- **Postcondiciones:**
  - Expresan garantías del estado en que quedará el sistema tras invocar una operación.



Restricciones de las operaciones

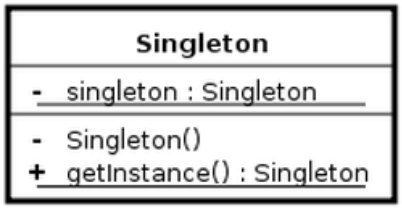
- **Condiciones del cuerpo:**
  - Se utilizan para restringir el valor de retorno de la operación.
- **Query**
  - Una operación puede ser declarada como {query} si su implementación no modifica es estado de la clase.





Elementos estáticos

- Son compartidos por todos los objetos de una clase: **todos los objetos de una clase comparten la misma copia de un atributo u operación**.
- No es necesario instanciar la clase para poder usarlos, ya que no dependen de la instancia, sino de la clase.
- Para indicar que un elemento es estático, **esta se subraya**.




```java
public class Singleton {
    private static Singleton INSTANCE = new Singleton();

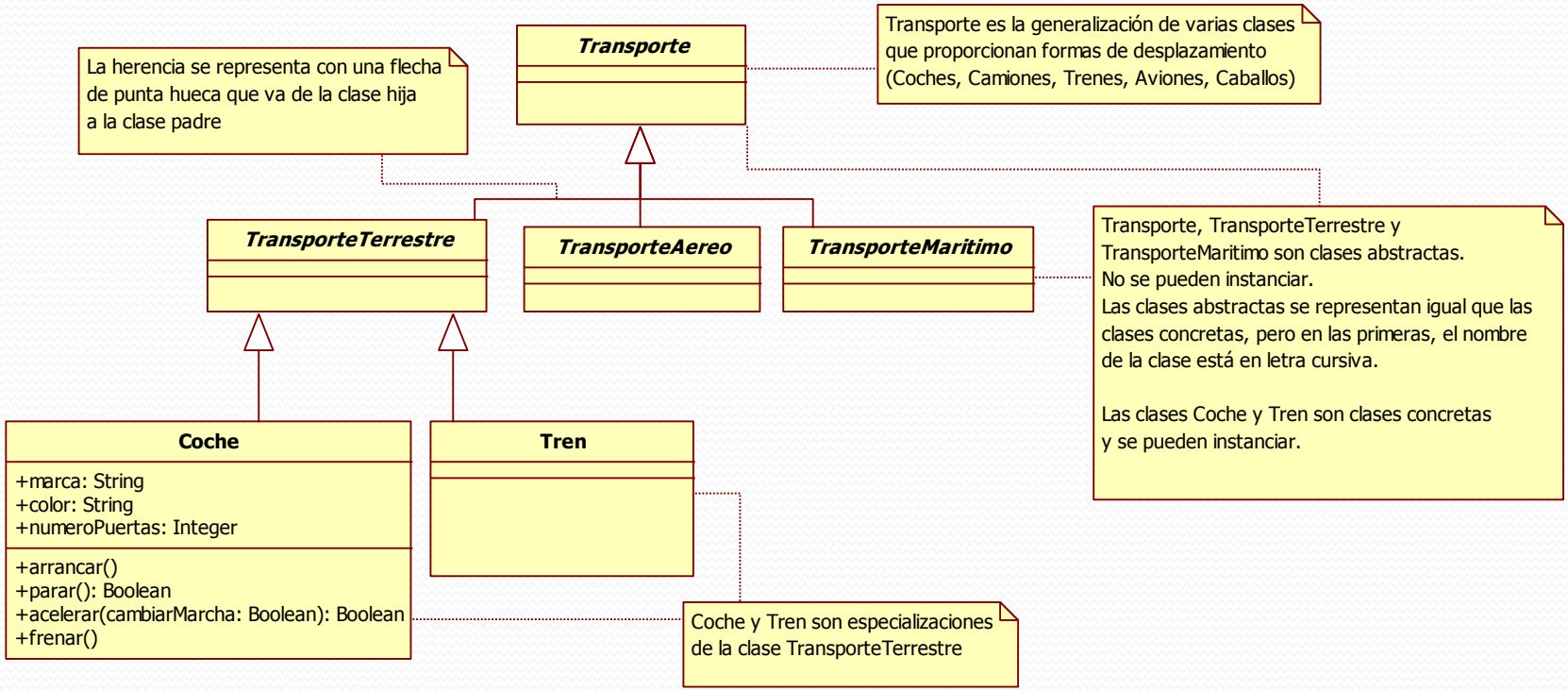
    // El constructor privado no permite que se genere un constructor por defecto
    // (con mismo modificador de acceso que la definicion de la clase)
    private Singleton() {}

    public static Singleton getInstance() {
        return INSTANCE;
    }
}
```

Generalización/Especialización

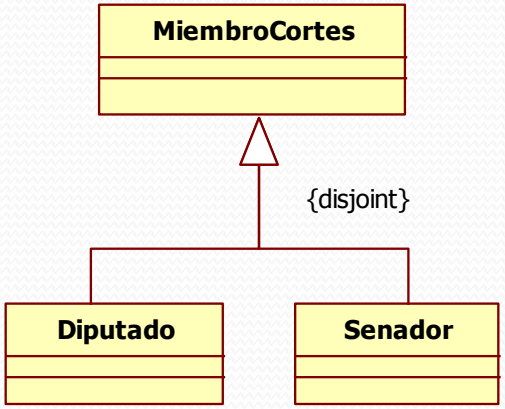
- **Generalización (clase padre)**:
  - Identifica y define los atributos y operaciones comunes en una colección de objetos.
  - **Se implementa a través de la herencia.**
- **Especialización (clase hija)**:
  - Algunas veces necesitamos especializar una clase, modificar la implementación de una parte de su comportamiento.
  - Para personalizar los miembros de una subclase, se deben sobrescribir los miembros de la superclase.
  - **La especialización es la herencia con la adición y modificación de métodos para resolver problemas específicos**.

Generalización/Especialización

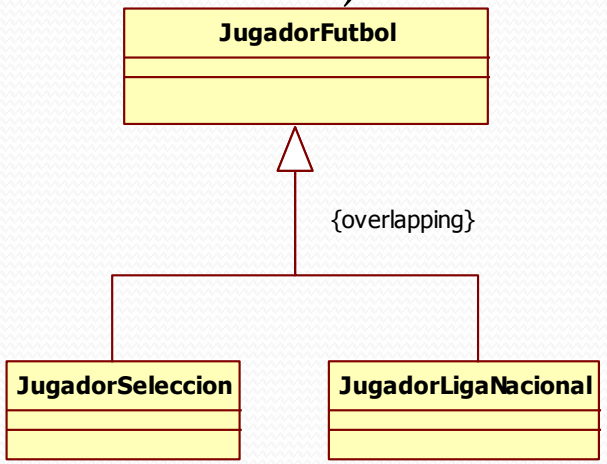


Restricciones de generalización

- **Disjoint (disjunto)**: una instancia del padre solo podrá ser de un tipo de hijo (xor). Solo puede tener un tipo en tiempo de ejecución.

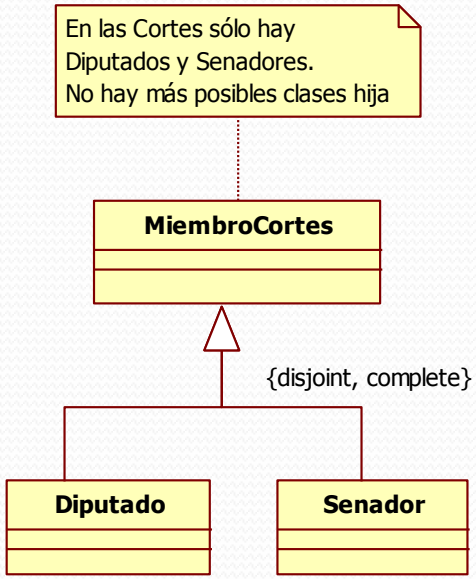


- **Overlapping (solapado)**: puede cambiar de tipo durante su vida, una instancia del padre puede ir cambiando de tipo entre los de sus hijos.

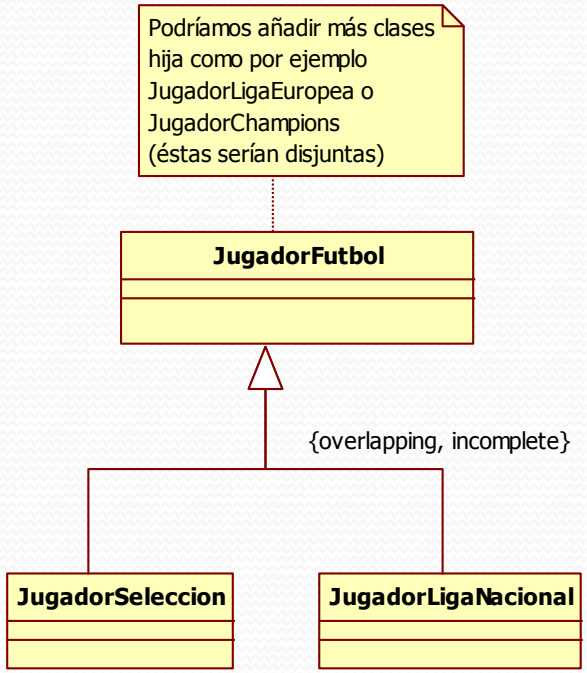


Restricciones de generalización

- **Complete (completo)**: La generalización ya no permite mas hijos.

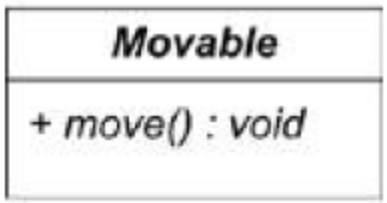


- **Incomplete (incompleto)**: Podemos incorporar mas hijos a la generalización.



Clases abstractas

- Las clases abstractas definen atributos y operaciones que deben ser implementadas.
- **No se pueden instanciar** ya que falta la implementación de una o varias operaciones.
- **Se utilizan como clase base de otras clases.**
- Son las subclases las que deben implementar las operaciones.
- Una clase es abstracta si tiene alguna operación abstracta.
- Para indicar que una clase es abstracta se pone el nombre de la clase en cursiva.



La clase Movable (movible) es abstracta y tiene un método move (mover). No se puede instanciar.
Las clases concretas que hereden de Movable, como por ejemplo Persona o Coche, deberán implementar la operación move.

Interfaces

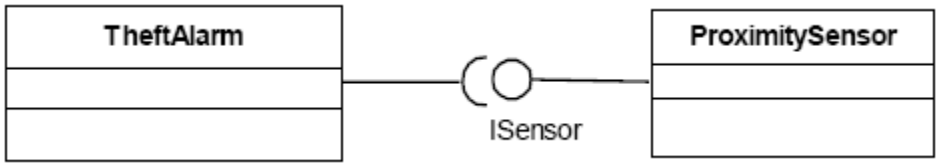
- Un interfaz es una **clase abstracta** pura, es decir, ninguna de las operaciones tiene implementación.
- Es un conjunto de operaciones que una clase presenta a otras (contrato).
- Un interfaz tiene
  - Atributos.
  - Operaciones
- Son un mecanismo para implementar herencia múltiple que evita los problemas derivados de esta.

- Relación interfaz-clase:
  - **Realización**: Es la relación que se produce entre una clase y un interfaz cuando una clase implementa ese interfaz. La clase es proveedora del interfaz.

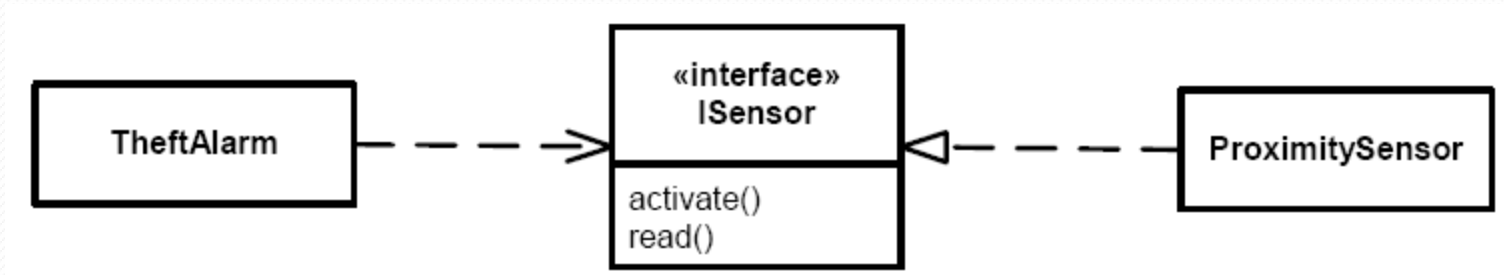
    Que un interfaz declare atributos no significa que necesariamente su realización tenga que implementar dicho atributo. Estos pueden aparecer solo como parte del interfaz para que sean vistos por observadores externos.
  - **Dependencia**: Es la relación entre una clase y un interfaz en la que la clase usa el interfaz. La clase requiere el interfaz.

Interfaces

- **Notación 1**: Círculo y semicírculo
  - Interfaz ofrecido o provisto: círculo o bola
  - Interfaz requerido: semicírculo o socket

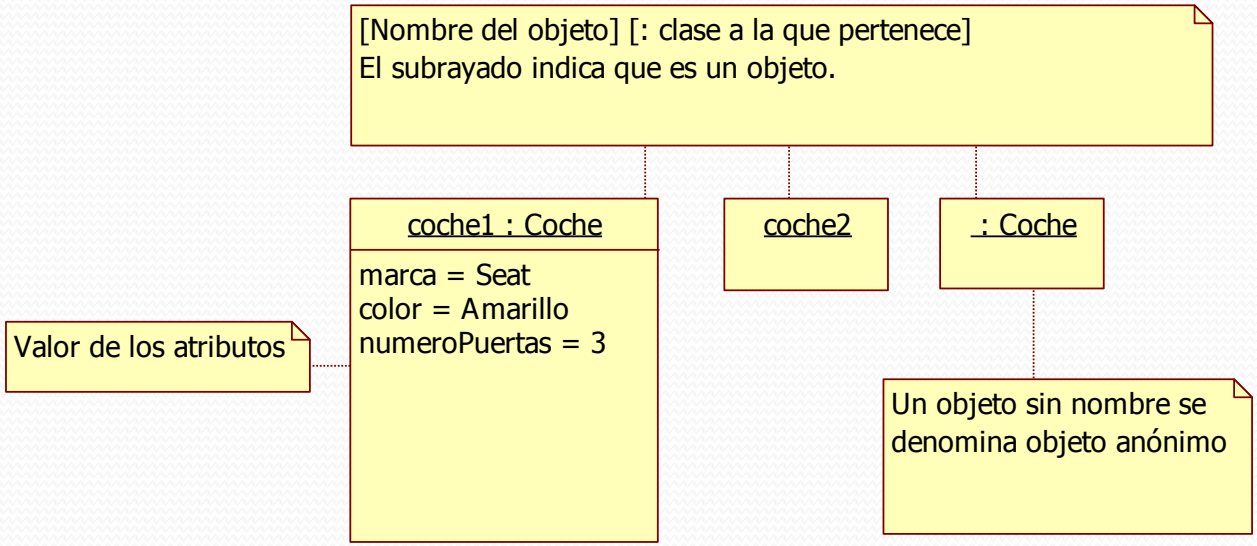


- **Notación 2**: Clase estereotipada y flechas
  - Interfaz ofrecido o provisto: flecha discontinua con punta hueca
  - Interfaz requerido: flecha de dependencia



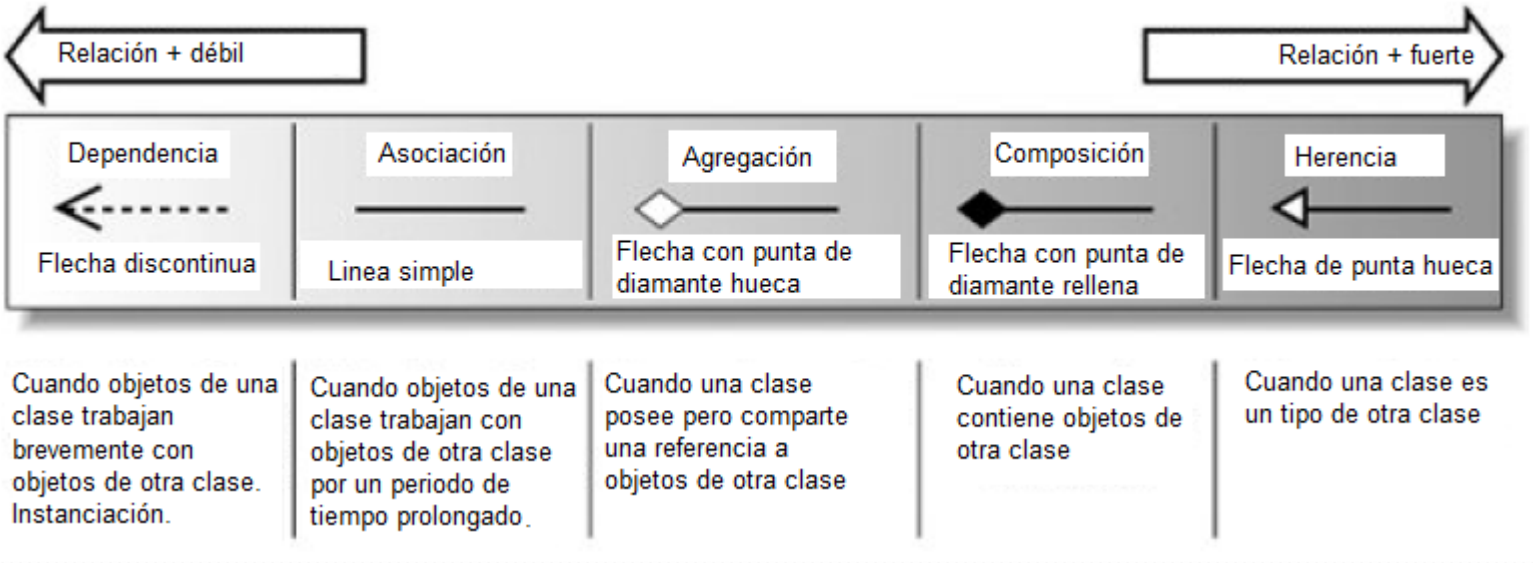
Objetos

- Un objeto es una instancia de una clase
  - Con la que se puede **interactuar**: se le pueden enviar mensajes y éste reaccionará ante ellos. Un mensaje es "algo" que le hacemos a un objeto. **Enviar un mensaje = Llamar a un método**
  - Que tiene **estado**: un objeto lo constituyen todos los datos (**atributos o variables de instancia**) que encapsula en un momento determinado cada uno de los cuales tiene un valor. Los atributos pueden ser constantes o cambiar de valor.
  - Que tiene **comportamiento**: el objeto puede reaccionar de manera diferente a un mensaje en función de su estado.
  - Que tiene **identidad**: al objeto se le hace referencia por un nombre (excepto en los objetos anónimos).



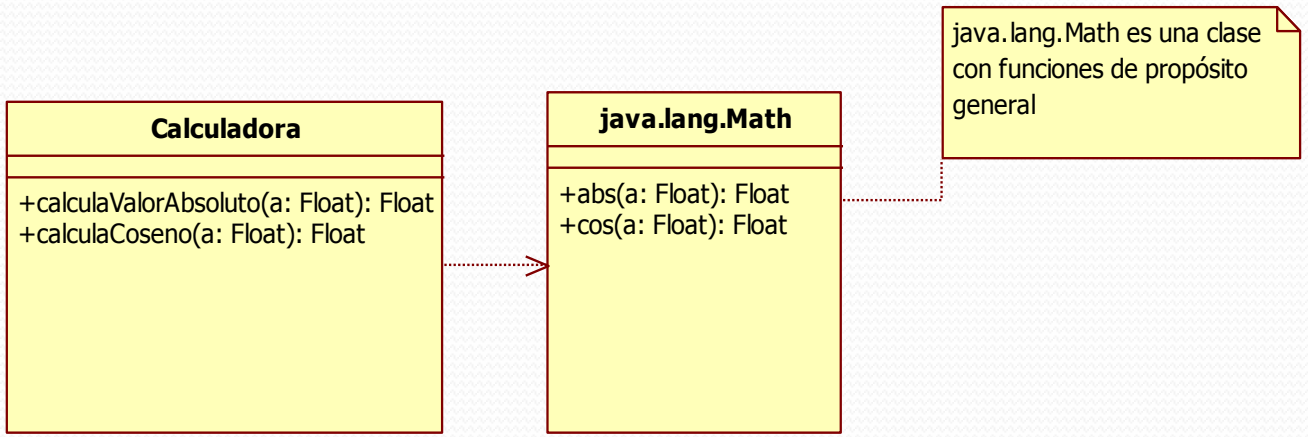
Relaciones entre clases

- Tipos de relaciones de menor a mayor acoplamiento:
  - Dependencia o instanciación
  - Asociación
  - Agregación
  - Composición
  - Herencia



Dependencia o Instanciación (uso)

- Es un tipo de relación en el que una clase utiliza a otra, es decir, **una clase es instanciada por otra**.
- La relación de dependencia se usa a menudo cuando hay clases que proveen un conjunto de funciones de propósito general. Ejemplo: clase java.lang.Math de Java.
- La dependencia se representa con una flecha discontinua.



Asociación

- La **asociación es una relación más fuerte que la dependencia**.
- Por lo general indica que una clase mantiene una **relación con otra clase durante un período prolongado de tiempo**.
- **Las líneas de vida** de dos objetos vinculados por una asociación probablemente **no están unidas** (la destrucción de uno no implica la destrucción del otro).
- En una asociación, una clase contiene una referencia a un objeto u objetos de otra clase en forma de atributos.
- Las asociaciones se suelen leer como "**... tiene un ...**". No confundir con composición y agregación (que se leen como "...está formado por...")

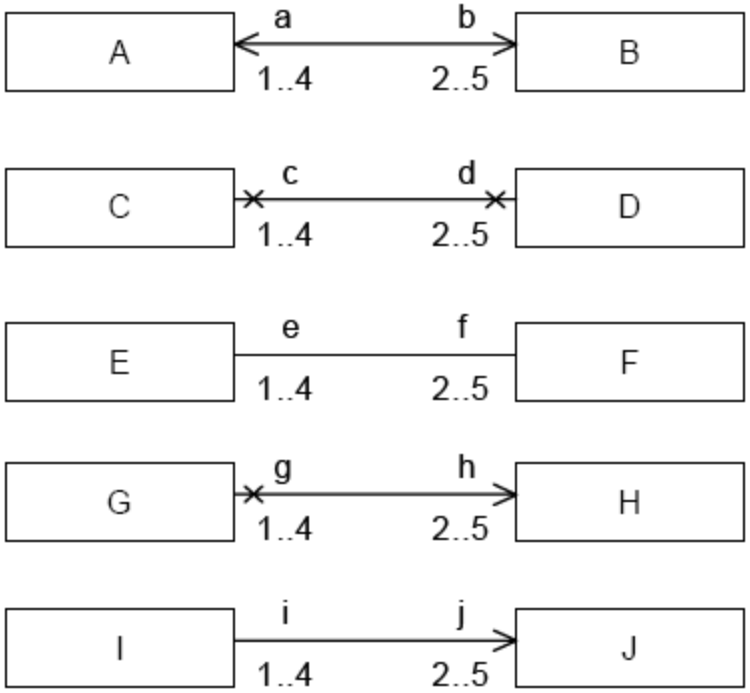


Una ventana tiene un cursor. El cursor no es parte de la ventana, sino del sistema. La ventana puede modificar el cursor . Lo hará durante toda su línea de vida.
La destrucción de la ventana no implica la del cursor.

Asociación

- Características de la asociación:
  - **Navegabilidad:** se aplica a menudo a una asociación para indicar qué clase contiene el atributo que soporta la relación.
  - **Nombre** de la asociación (opcional): frase para indicar el contexto de la asociación.
  - **Multiplicidad** de la asociación (opcional): indica la cantidad de objetos de una clase que pueden relacionarse con un objeto de una clase asociada

Asociación
Navegabilidad



La asociación es navegable en ambos sentidos. A tendrá un atributo de tipo B y B tendrá un atributo de tipo A.

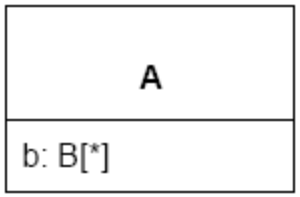
La asociación no es navegable.

No se especifica la navegabilidad.

La asociación es navegable en un solo sentido. G tendrá un atributo de tipo H.

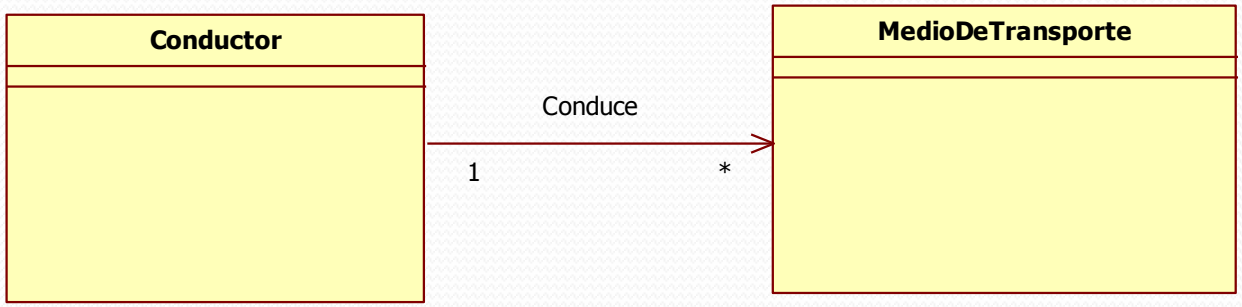
La asociación es navegable en uno de los sentidos y en el otro no se especifica la navegabilidad.
I tendrá un atributo de tipo y J puede tener o no un atributo de tipo I.

Clase A en la que se representa su asociación con B a través del atributo de tipo B.



Asociación
Nombre y multiplicidad

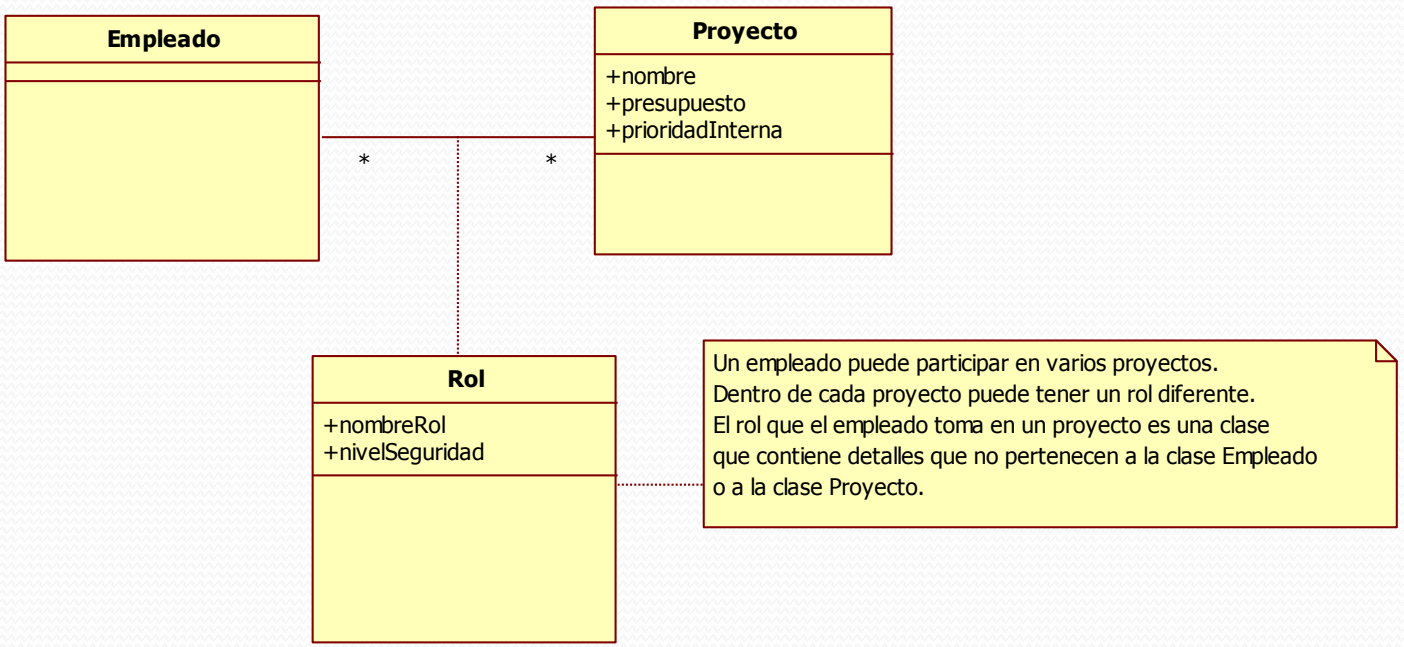
- **Nombre** de la asociación: frase para indicar el contexto de la asociación.
- La **multiplicidad** de la asociación: indica la cantidad de objetos de una clase que pueden relacionarse con un objeto de una clase asociada.




```
public class Conductor {
        private MedioDeTransporte
m_transportes[];
        // ...Otros atributos y métodos
}
public class MedioDeTransporte {
        // ...Atributos y métodos
}
```

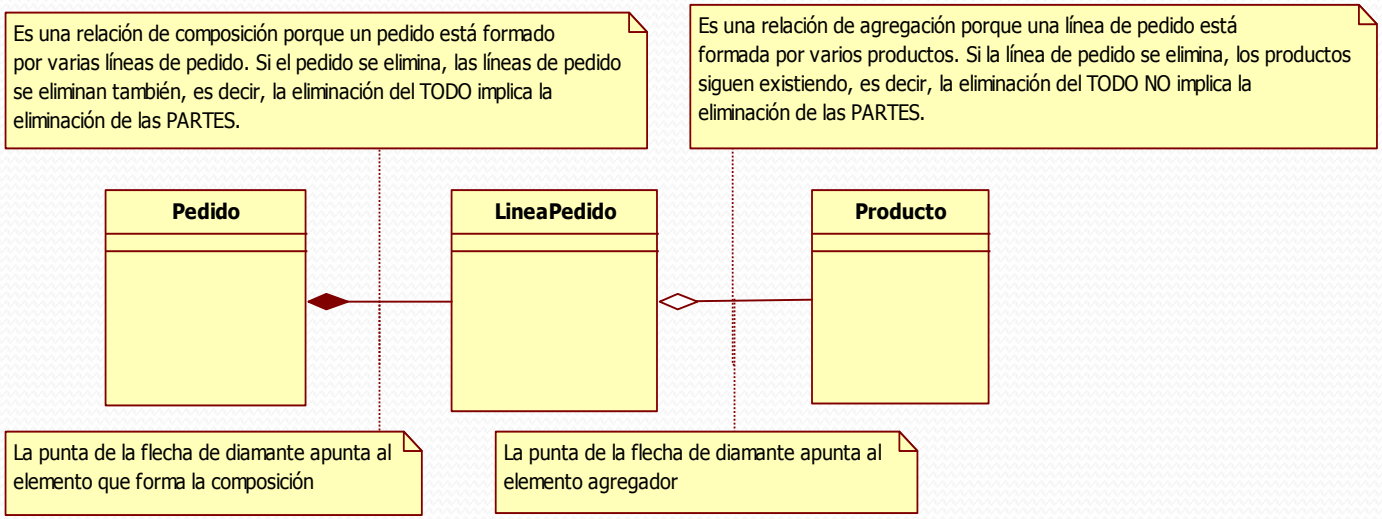
Clase de asociación

- **Una asociación**, al igual que una clase **puede tener atributos y operaciones**. Cuando este sea el caso tendremos una **clase de asociación**.
- Los atributos no pertenecen a ninguna de las clases asociadas, sino a la asociación.
- Surgen en las relaciones muchos a muchos.



Agregación y Composición

- Agregación:
  - **Es un tipo especial de asociación**.
  - Se produce **cuando una clase está formada por otras clases**. Se lee "**...es parte de...**" o "...está formado por..."
  - El tiempo de vida del objeto incluido es **independiente** del que lo incluye.
  - Se representa con una línea con punta de diamante hueca. La punta de diamante está en el lado del agregador.
- Composición:
  - **Es un tipo especial de asociación** (como agregación).
  - Se produce **cuando una clase está formada por otras clases** (como agregación).
  - El tiempo de vida del objeto incluido es **dependiente** del que lo incluye (distinto de agregación).
  - Se representa con una línea con punta de diamante rellena.



Ejemplo de diagrama de clases

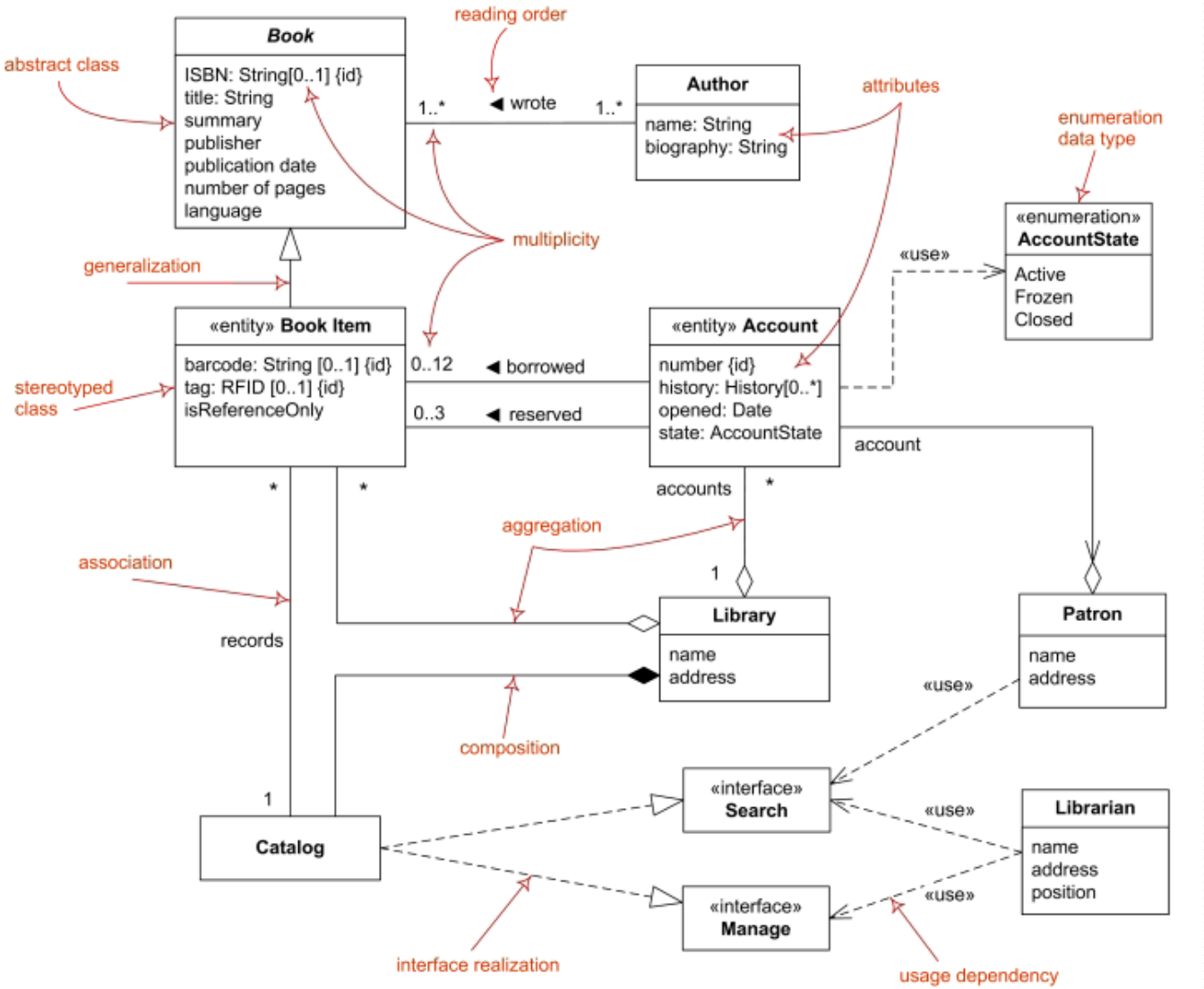
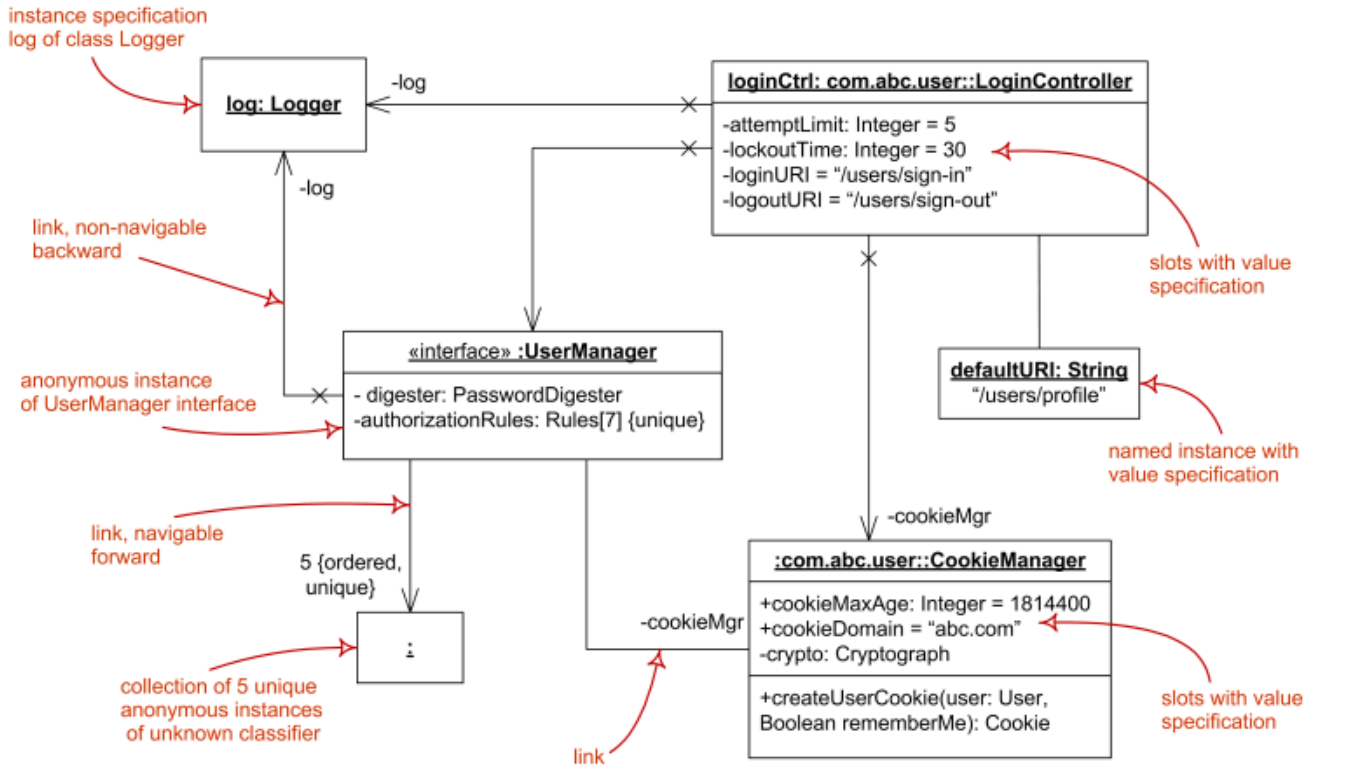


Diagrama de Objetos

- Contiene los objetos con sus valores. Muestra una instantánea del sistema en un determinado momento en el tiempo.



Object diagram overview - instance specifications, value specifications, slots, and links.

Diagramas

Node Type	*Notation*
Class	ClassName
Interface	InterfaceName <<interface>> InterfaceName
InstanceSpecification	Instancename : ClassName

Path Type	*Notation*
Aggregation	
Association	
Composition	
Dependency	
Generalization	
InterfaceRealization	

Diagrama de estructura compuesta

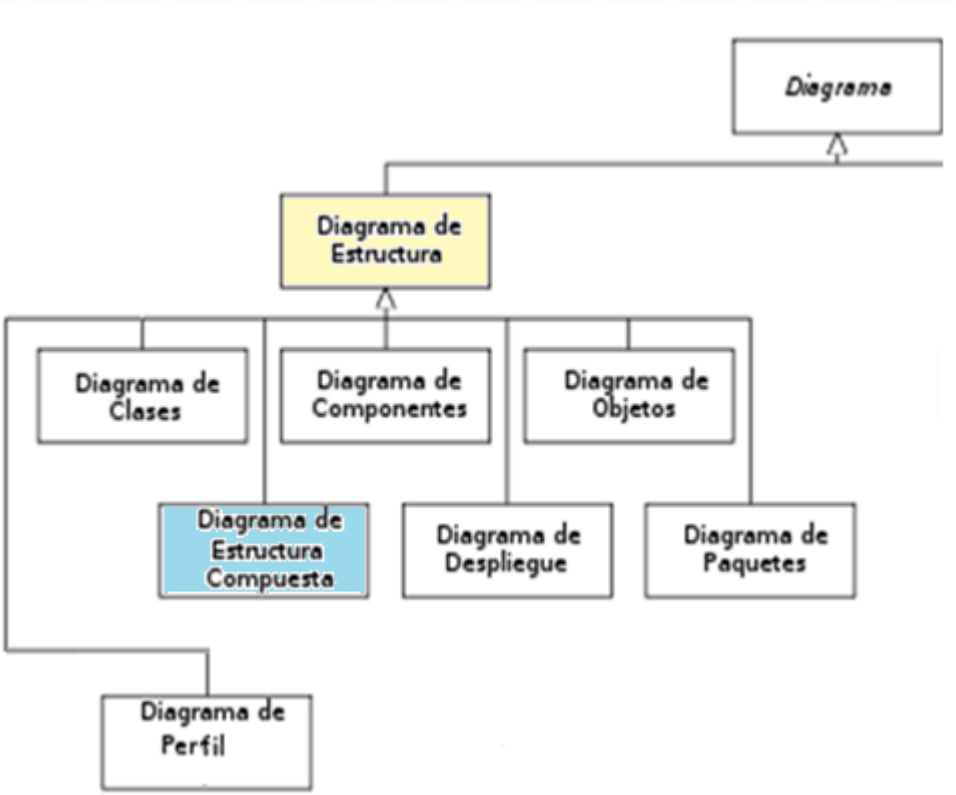


Diagrama de estructura compuesta

- Un diagrama de estructura compuesta muestra la **estructura interna de una clase** (visión de la estructura como una caja blanca), incluyendo sus puntos de interacción con otras partes del sistema (comunicación entre estructuras).
- Utilizaremos este tipo de diagramas cuando necesitemos mostrar una relación inusual entre elementos, algo que no podamos representar en un diagrama de clases.
- Son un complemento a los diagramas de clases.
- Elementos:
  - Parte o propiedad
  - Conector
  - Interfaces
  - Puerto
  - Colaboración

Partes y Conectores

- **Parte**:
  - es un elemento de **la estructura interna** de una clase.
  - **Representa una instancia o una colección de instancias** (**tiempo de ejecución**).
  - La vida de las partes depende de la de la clase contenedora.
  - Una parte se representa como un **rectángulo** si está asociada al contenedor por una **relación de composición**.
  - Una parte se representa como un **rectángulo de línea de puntos** si **no** está asociada al contenedor por una **relación de composición**.
- **Conector**:
  - une o comunica las partes de una estructura.
  - se representa con una línea.
  - conecta instancias concretas.

Partes y conectores

Diagrama de clases

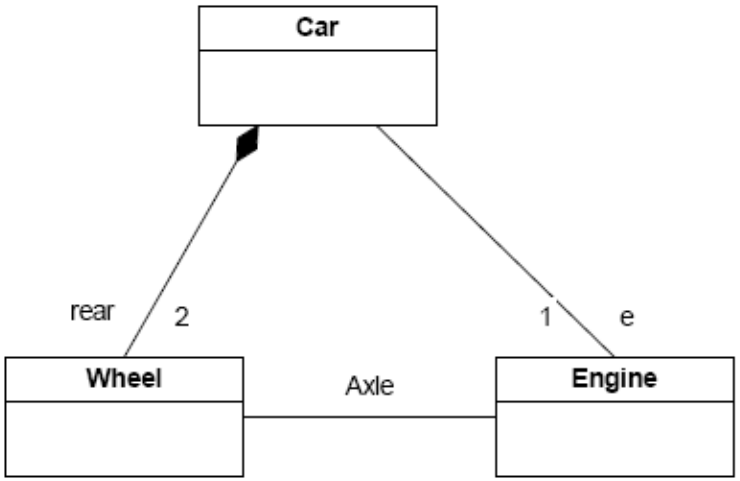
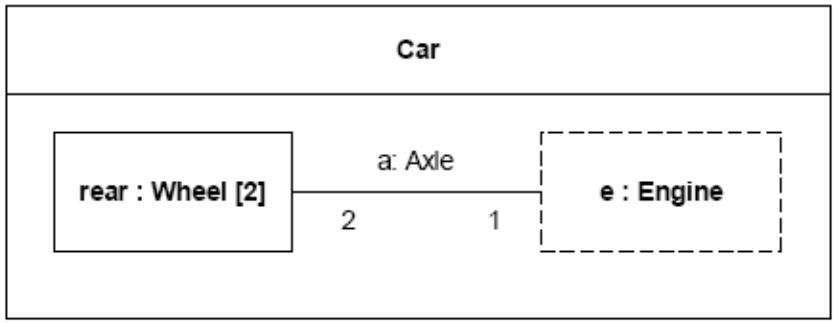


Diagrama de estructura de la clase Car

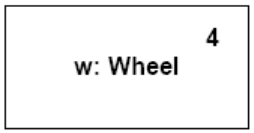


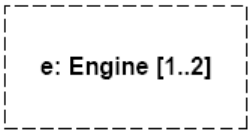
Rear: trasera Axle: eje

Multiplicidad

- Multiplicidad de las partes:

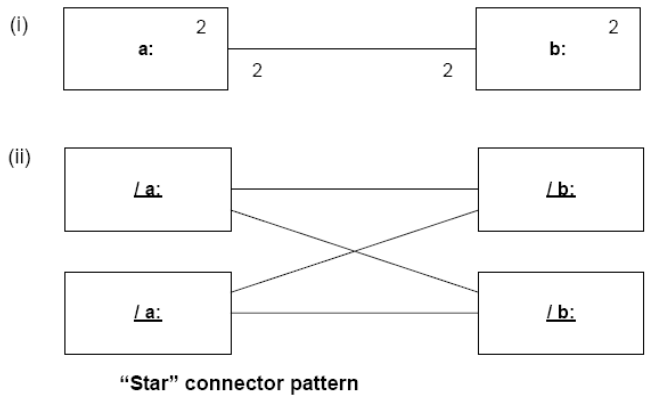
Indica 4 instancias de la clase Wheel





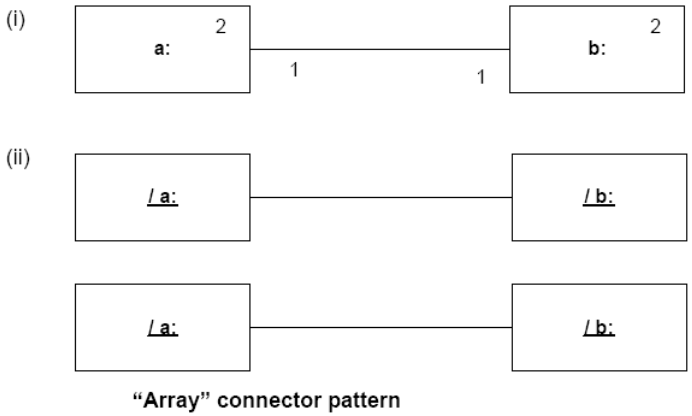
Indica una o dos instancias de la clase Engine

- Multiplicidad del conector:
  - Indica el número de instancias que se conectan a cada lado.



"Star" connector pattern

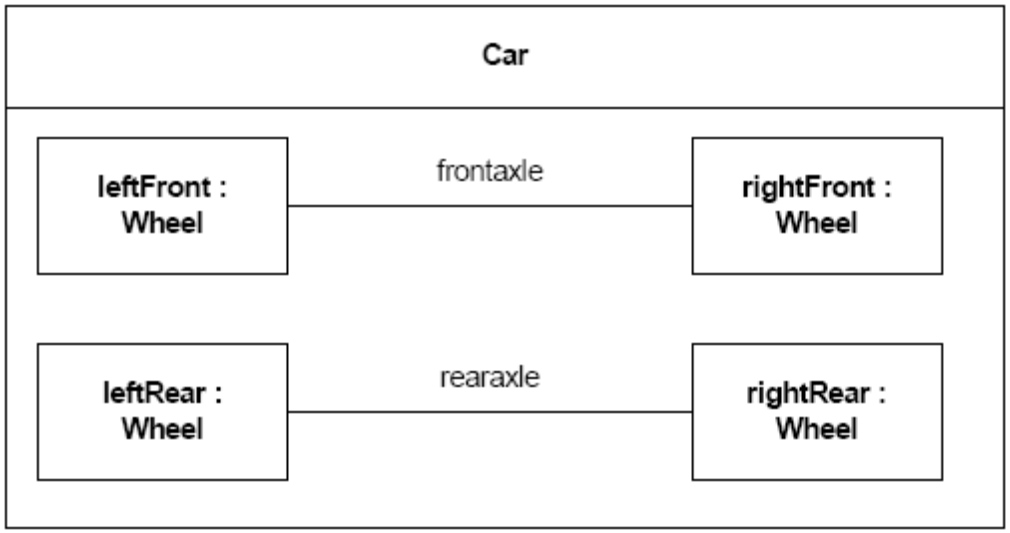
En el ejemplo siguiente el conector indica que cada elemento a se va a conectar con 2 elementos b y que cada elemento b se va a conectar con 2 elementos a.

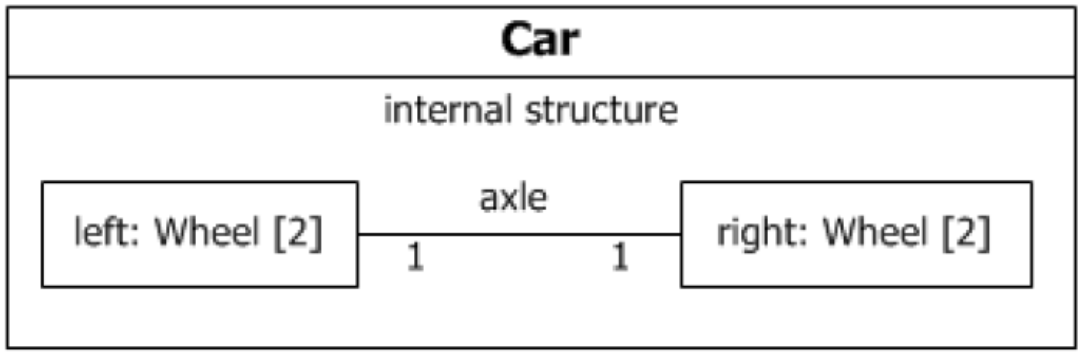


"Array" connector pattern

En el ejemplo siguiente el conector indica que cada elemento a se va a conectar con 1 elemento b y que cada elemento b se va a conectar con 1 elemento a.

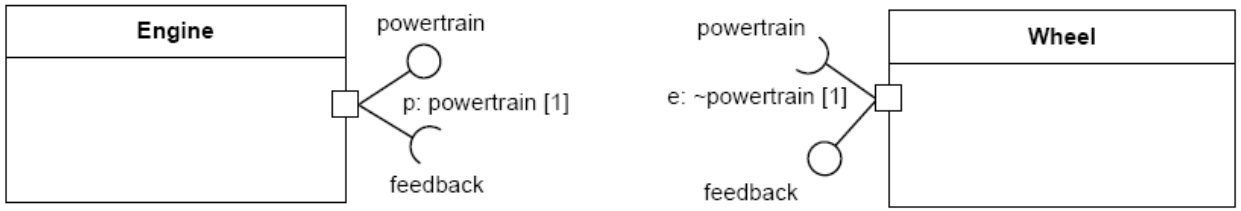
Estructuras equivalentes

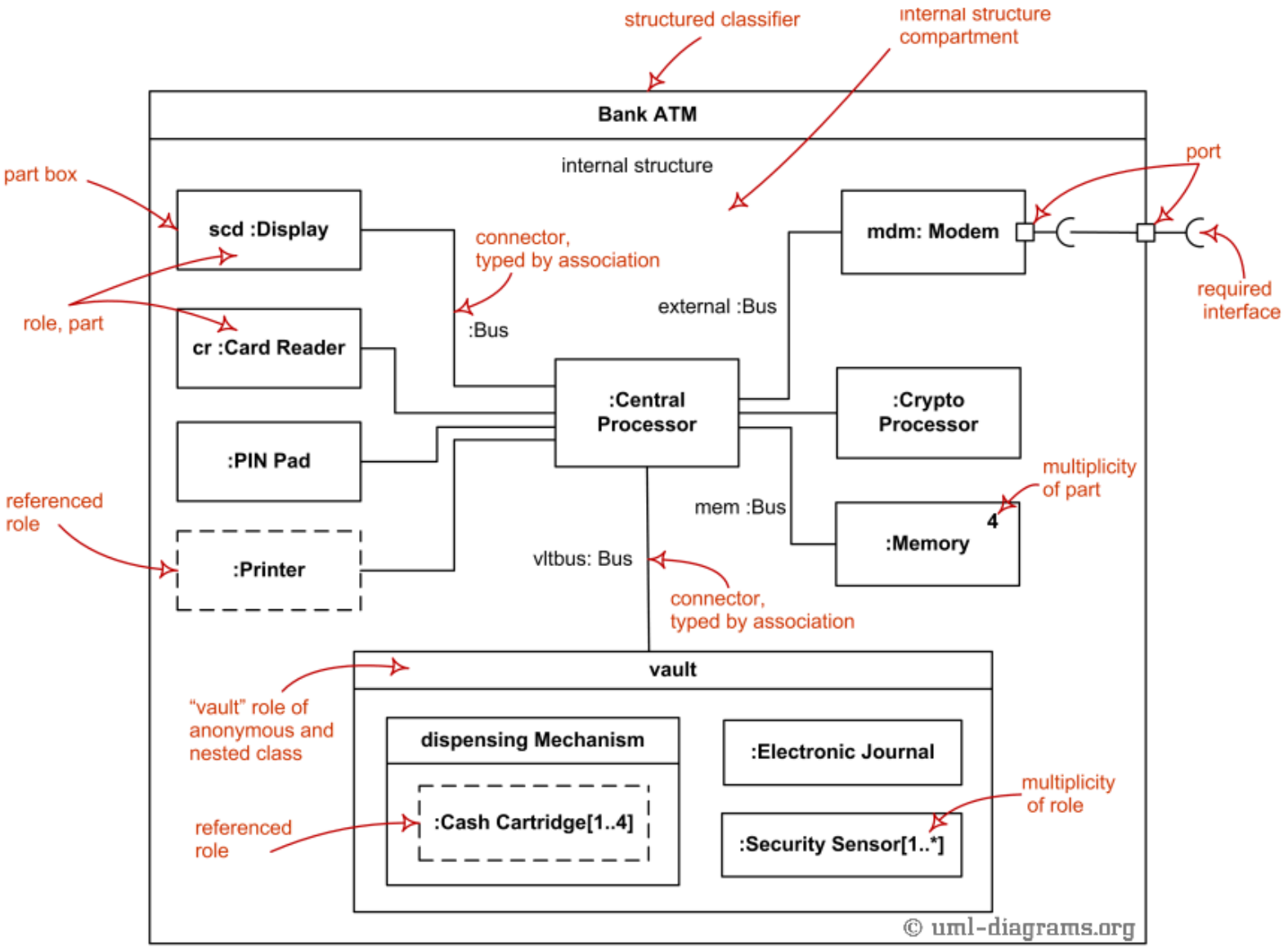




Puerto

- **Un puerto indica un punto de interacción entre la clase y su entorno.**
- Un puerto puede especificar los servicios que una clase provee, así como también los servicios que esta requiere de su entorno.
- Los mensajes que recibe un puerto, los pasa a las partes.
- **Un puerto tiene un conjunto de interfaces que definen las operaciones que son permitidas con el medio externo**.
- Es común que las clases tengan interfaces asociados con puertos. **Un puerto se puede utilizar para agrupar interfaces relacionados.**
- Un puerto se muestra como un rectángulo con nombre en el borde de la clase.
- Los puertos pueden:
  - Importar interfaces (interfaz requerida): se representa con un círculo
  - Exportar interfaces (interfaz proporcionada): se representa con un semicírculo





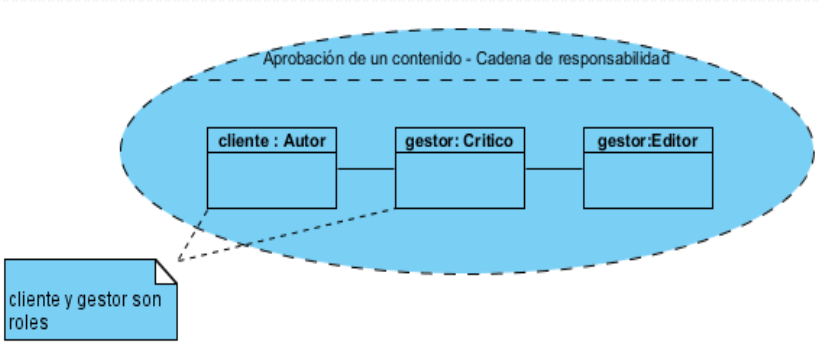
Composite structure diagram overview shows elements of internal structure of structured classifier - roles, parts, connectors.

Colaboración

- Una colaboración define un conjunto de **roles** que cooperan entre si para ilustrar una funcionalidad especifica.
- Un rol es el papel que una instancia desempeña.
- Una colaboración debería solo mostrar los roles y los atributos requeridos para lograr sus tareas o funciones definidas.
- **Aislar los roles permite clasificar el comportamiento y poder reusarlo**. Un ejemplo de esto son los **patrones de diseño**.
- **Un elemento colaboración a menudo implementa un patrón** (solución a un problema de diseño que se repite).
- Por tanto, el objetivo principal de las colaboraciones es explicar como un sistema de instancias se comunica para realizar una tarea o conjunto de tareas sin tener que incorporar necesariamente detalles que son irrelevantes para la explicación, siendo muy útiles a la hora de capturar patrones de diseño estándar.

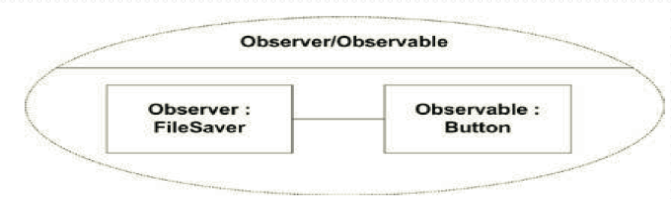
Colaboración

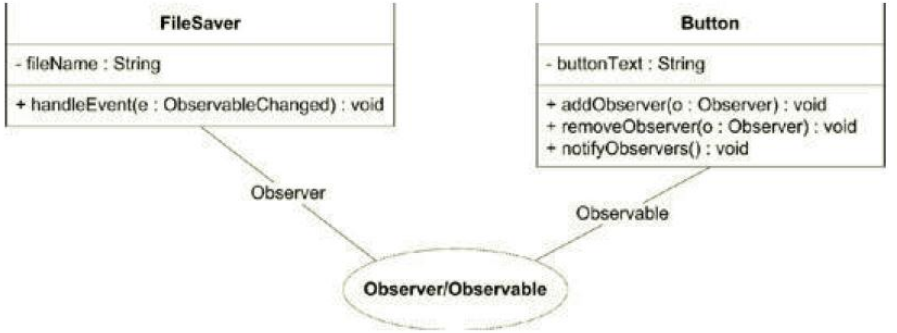
- Notación 1: Un elemento colaboración se muestra como un elipse (punteada, según la especificación). La elipse tiene dos apartados:
  - Nombre de la colaboración
  - Estructura interna de la colaboración compuesta por roles y conectores.



•Design Pattern Chain of responsibility (Patrón de diseño Cadena de responsabilidad): permite establecer la línea que deben llevar los mensajes para que los objetos realicen la tarea indicada.
•El conector entre roles indica qué comunicaciones deben existir entre las instancias participantes.

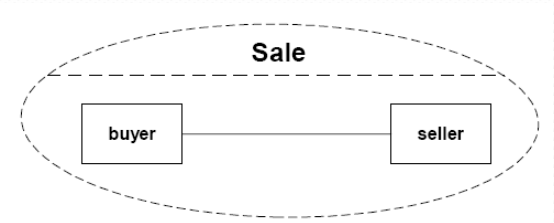
- Notación 2: se muestran los detalles de la colaboración fuera de la elipse.
  - En la colaboración Observer/Observable, colaboran dos roles (Observable y Observer) para producir el comportamiento deseado.



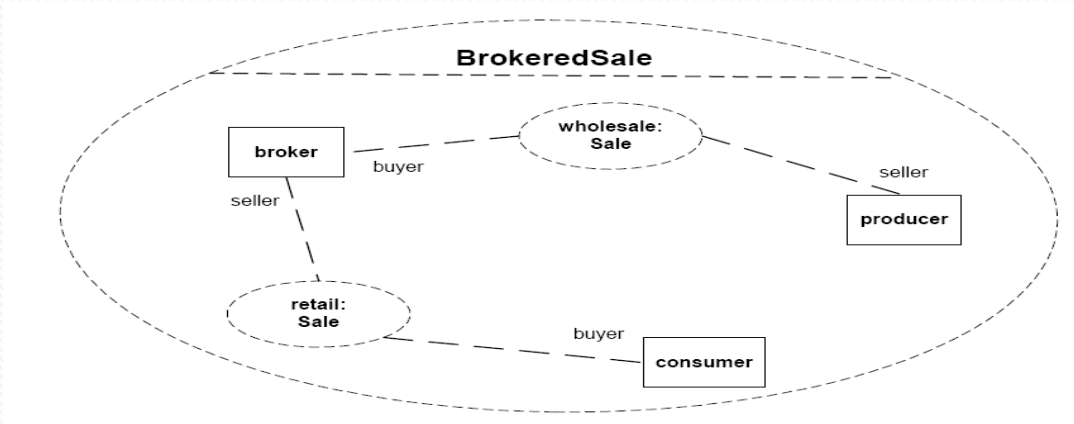


Uso de la colaboración

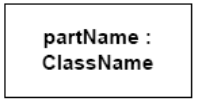
- Representa el uso particular (ocurrencia) de una colaboración descrita por un patrón en un contexto determinado.
  - Ejemplo de colaboración: una venta (Sale) es una colaboración entre dos roles, un vendedor (seller) y un comprador (buyer).



  - En el siguiente ejemplo se muestra la colaboración Brokered Sale (venta negociada) que usa 2 instancias de la colaboración Sale (venta). En la instancia **wholesale (venta al por mayor)** colaboran los objetos producer (productor), que asume el rol de vendedor (seller) y el objeto broker (comercial) que asume el rol de comprador (buyer). En la instancia **retail (venta al por menor),** colaboran los objetos broker, que en esta colaboración asume el rol de vendedor y consumer (consumidor) que asume el rol de comprador.



Diagramas

Node Type	Notation
Part	
Port	portName: ClassifierName 
Collaboration	
CollaborationUse	

Path Type	Notation
Connector	
Role binding	

Diagrama de componentes

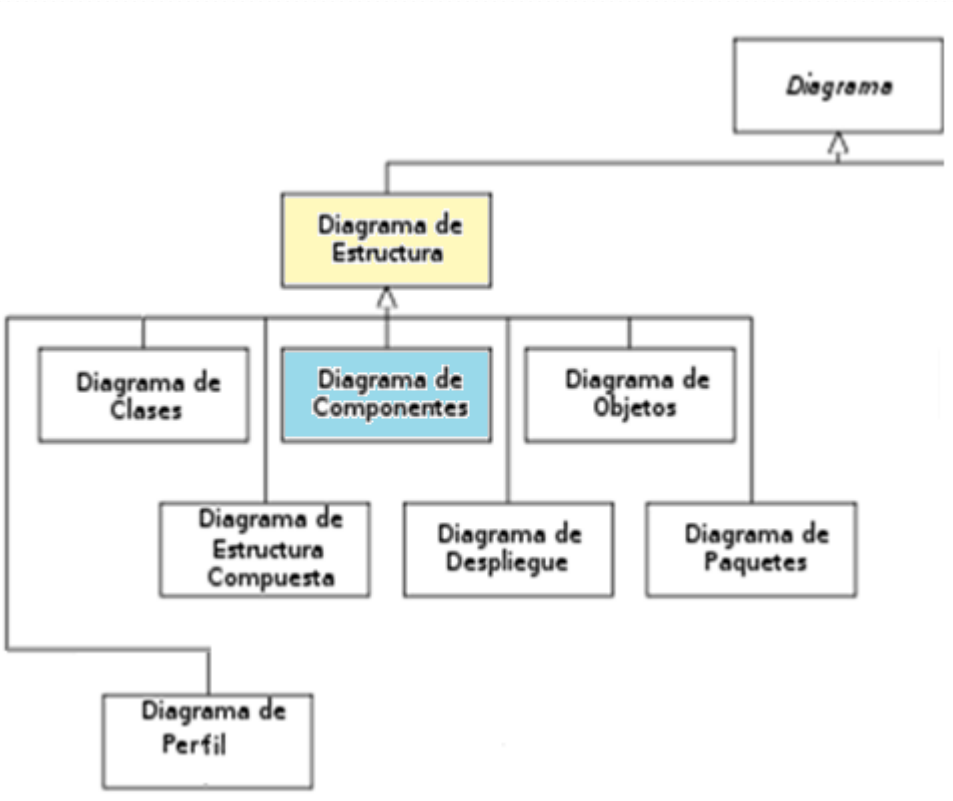


Diagrama de componentes

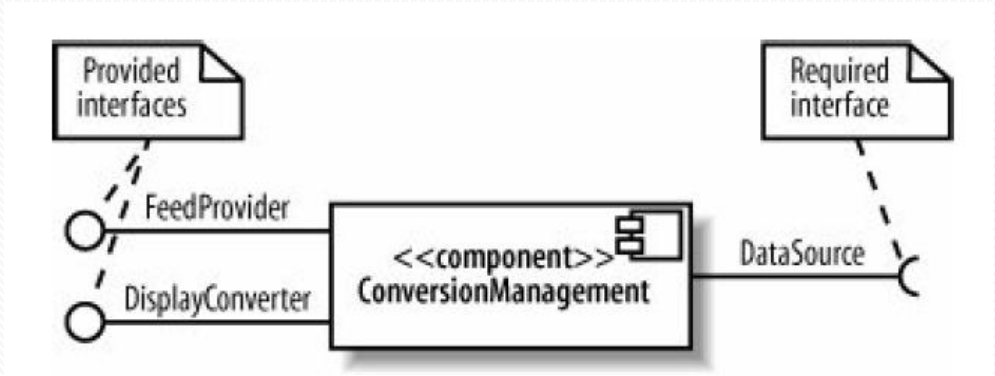
- Es un diagrama que muestra los componentes del sistema y sus dependencias.
- **Los componentes aportan reusabilidad**. Ejemplo: componentes de terceras partes (**third-party**) que usamos en nuestro software.
- Podemos **crear componentes para aquellos servicios que vayamos a usar con frecuencia**. Ejemplo: **librería** de acceso a base de datos.

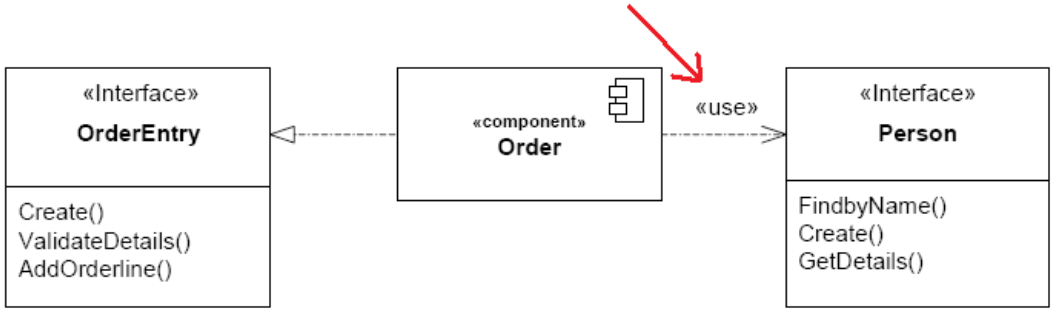


UML 1.4 UML 2.x

Interfaz de un componente

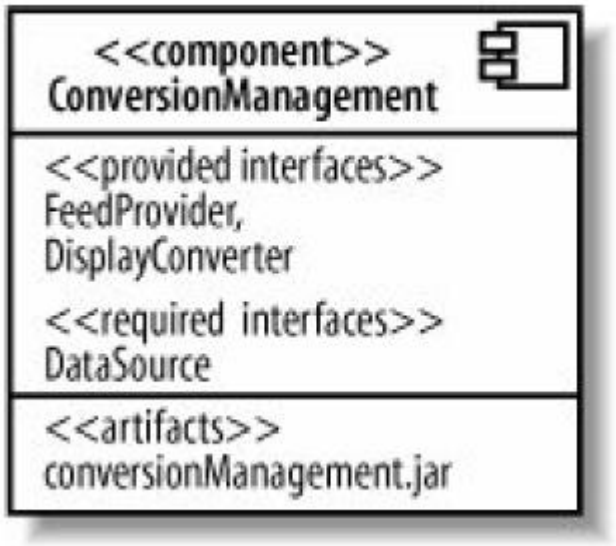
- **Un componente puede accederse a través de su interfaz**, una colección de operaciones.
- La relación entre un componente y su interfaz se llama **realización**.
- Hay dos tipos de interfaces:
  - Si un componente usa los servicios de otro, utiliza una **interfaz de importación (interfaz requerida)**.
  - Si un componente realiza una interfaz, proporciona una **interfaz de exportación (interfaz ofrecida o proporcionada)**.





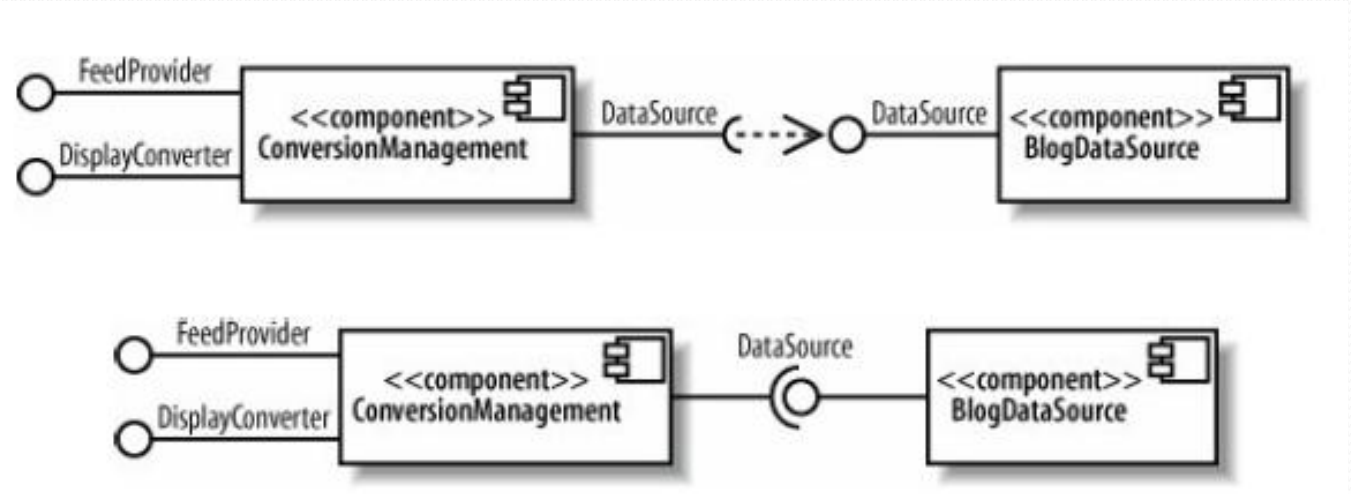
Otra notación para interfaces
Lista de interfaces

- Se listan en una sección del componente los interfaces ofrecidos y requeridos.
- Adicionalmente se pueden listar en otra sección los **artefactos** o ficheros físicos del componente.
- Esta representación también se conoce como de **caja blanca**.

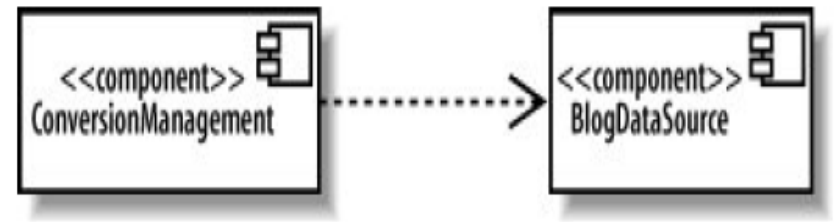


Conexión entre componentes

- Si un componente requiere un interfaz, necesita que otro componente lo implemente o realice.

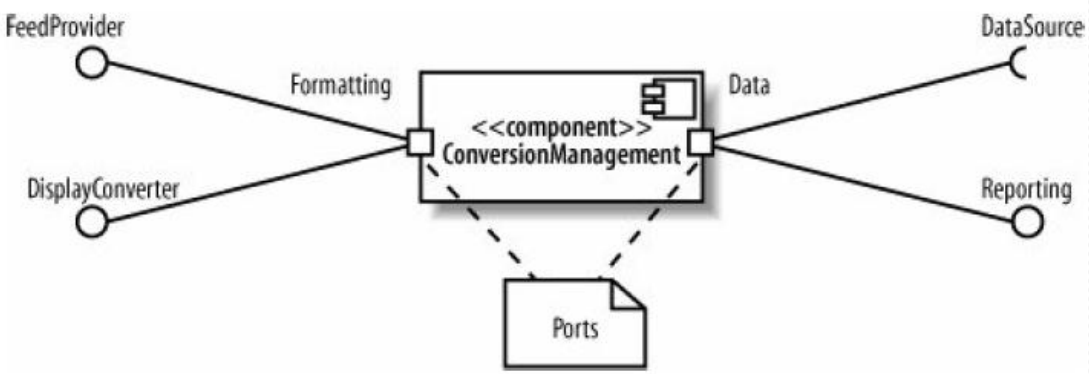


- Otra notación: Se omite el interfaz. No se recomienda.

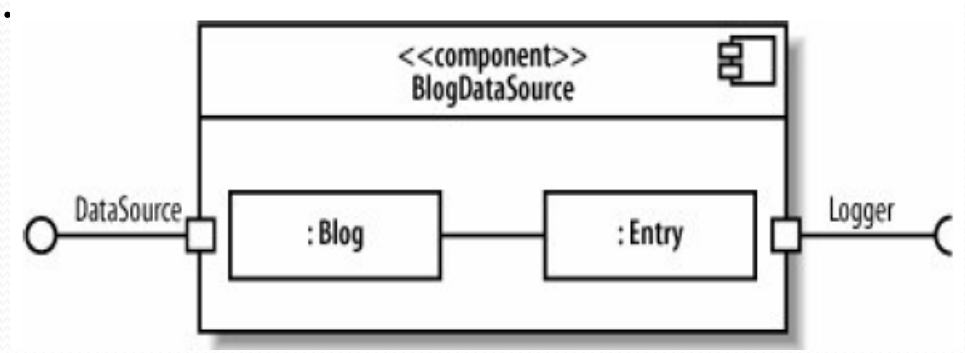


Puertos y estructura interna

- Al igual que en los diagramas de estructura, los componentes pueden tener puertos y estructura interna.

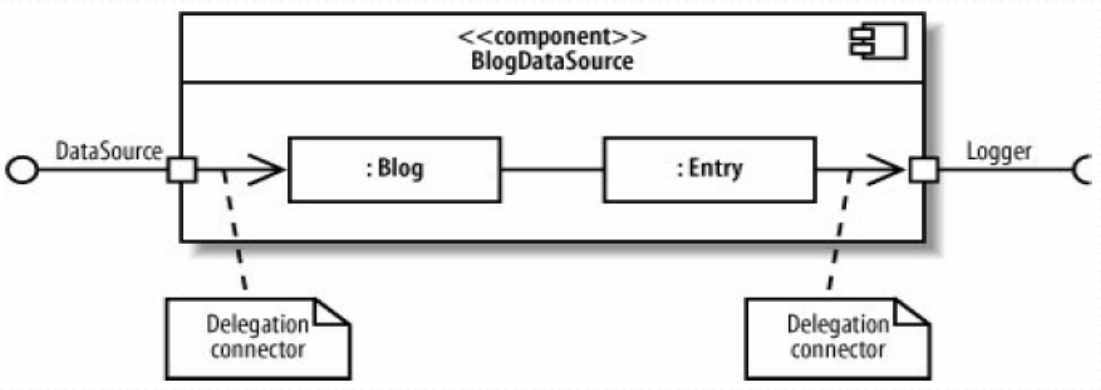


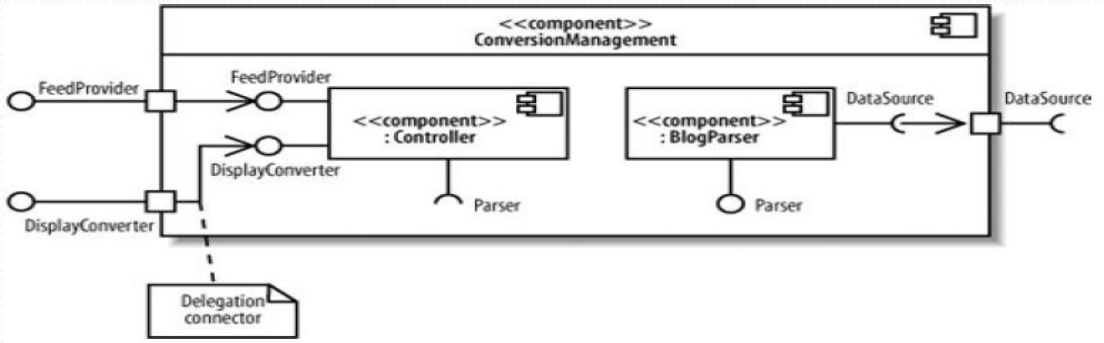
- Podemos mostrar la estructura interna del componente de manera que se vean las partes que lo componen (diagrama de estructura del componente).



Conectores de delegación

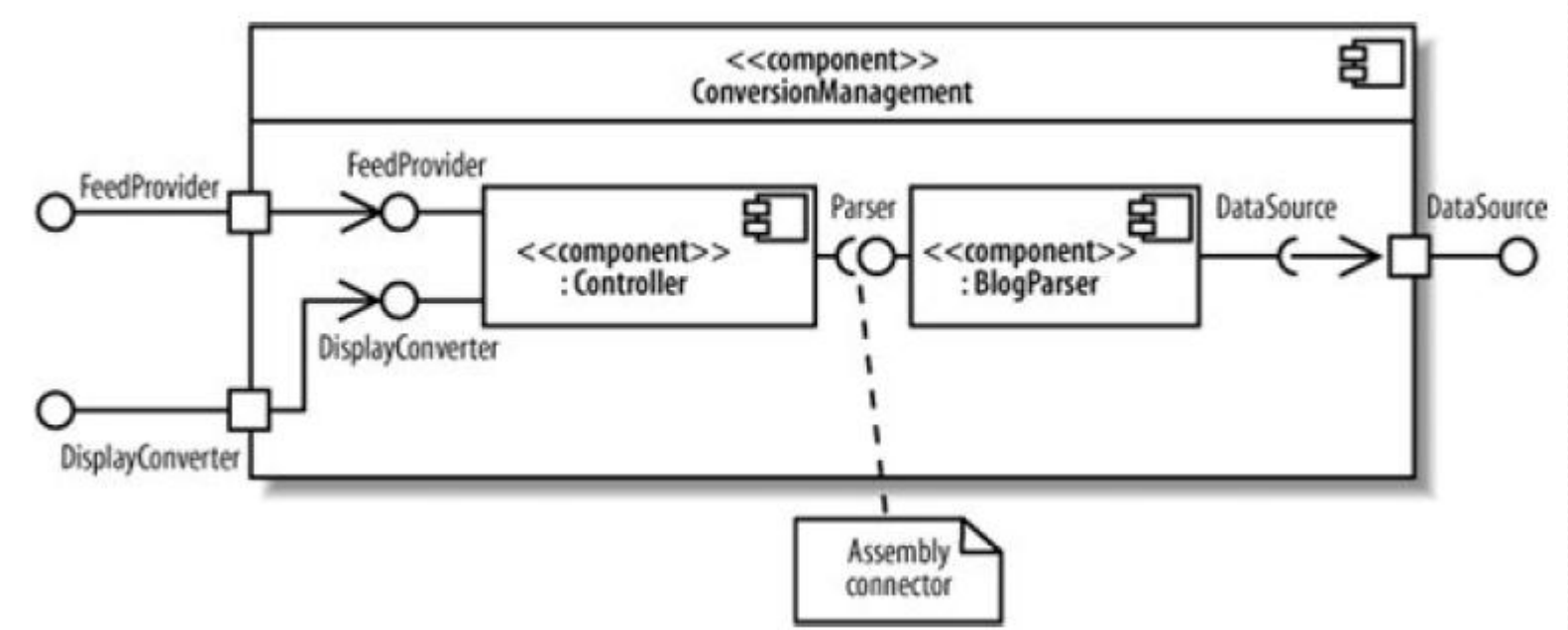
- **Los conectores de delegación muestran qué clases del componente proporcionan o requieren un interfaz.**
- Los conectores de delegación conectan el puerto que está unido al interfaz con la clase interna.
  - Si la clase implementa el interfaz: el conector va del puerto a la clase.
  - Si la clase requiere el interfaz: el conector va de la clase al puerto.





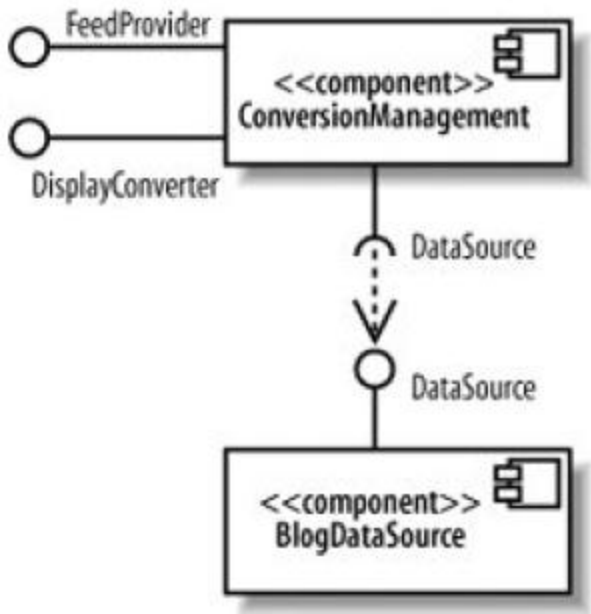
Conectores de ensamblado

- Muestran que un componente requiere un interfaz provisto por otro componente.

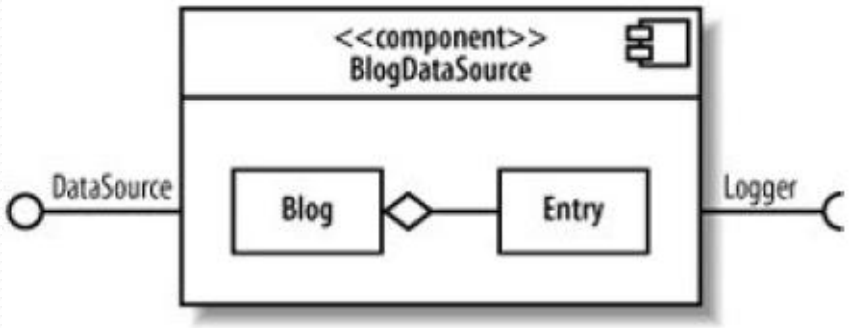


Vistas de un componente

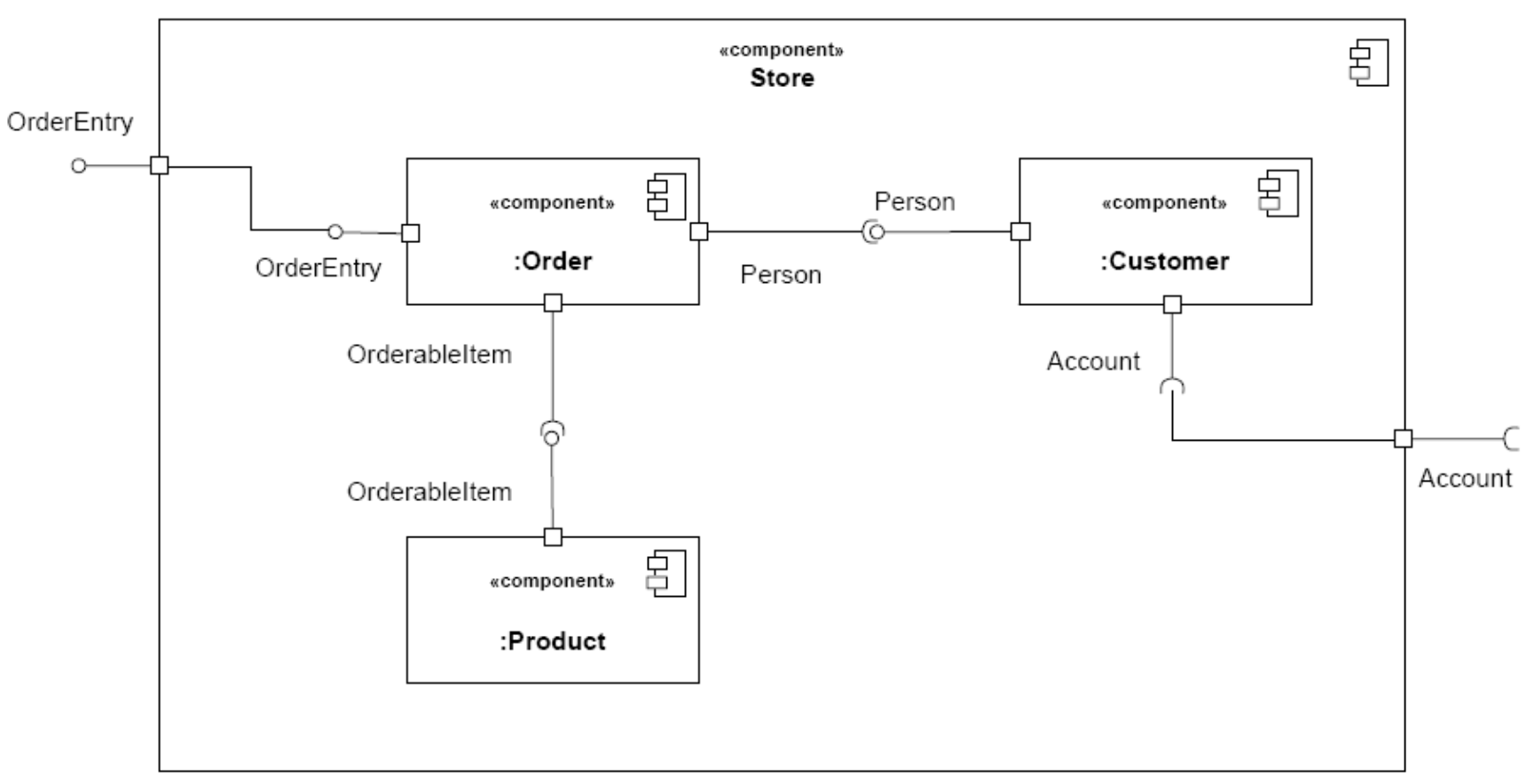
- **Vista de caja negra**:
  - Muestra más de un componente
  - Mas adecuada para ver los componentes clave de un sistema y sus relaciones



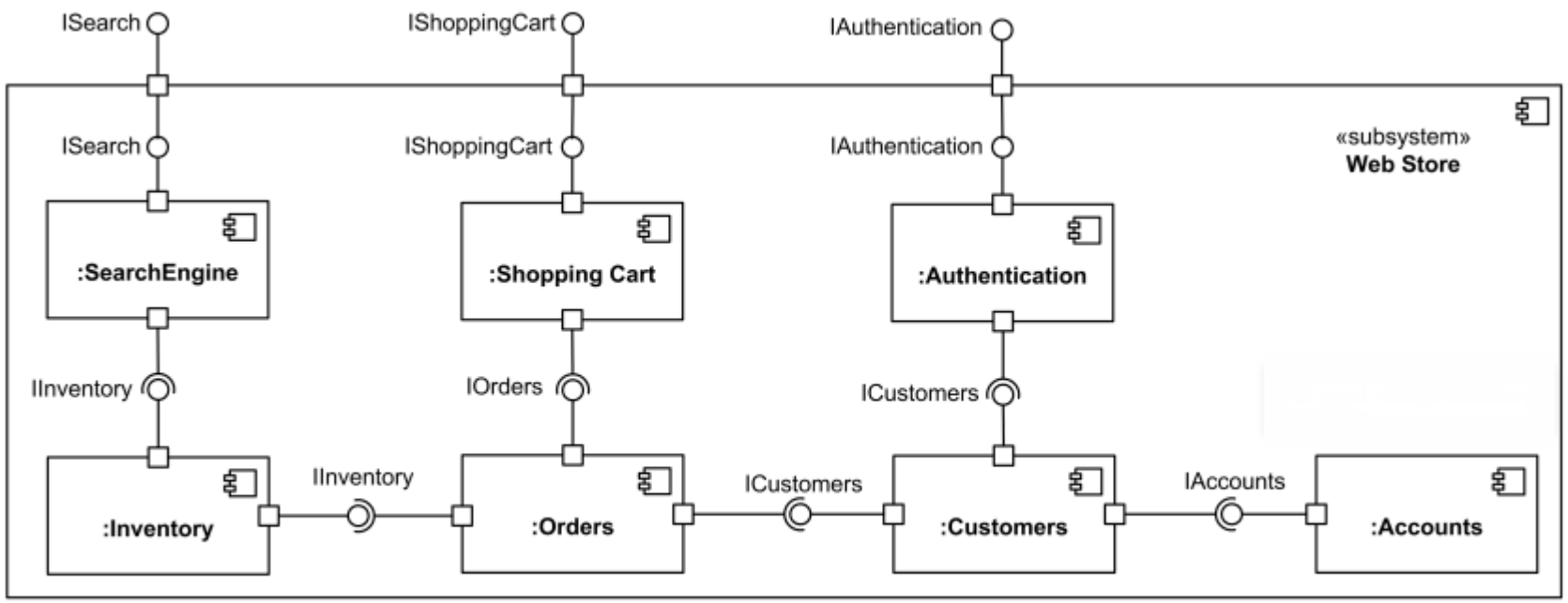
- **Vista de caja blanca:**
  - Muestra la estructura interna de un solo componente .
  - Realización interna del comportamiento externo.



Vistas de caja blanca de un componente que contiene otros componentes



Ejemplo



Online shopping UML component diagram example.

Agrupación de componentes

- Los componentes se pueden agrupar en paquetes de la misma forma que se organizan las clases.
- Además se pueden especificar entre ellos relaciones de dependencia.

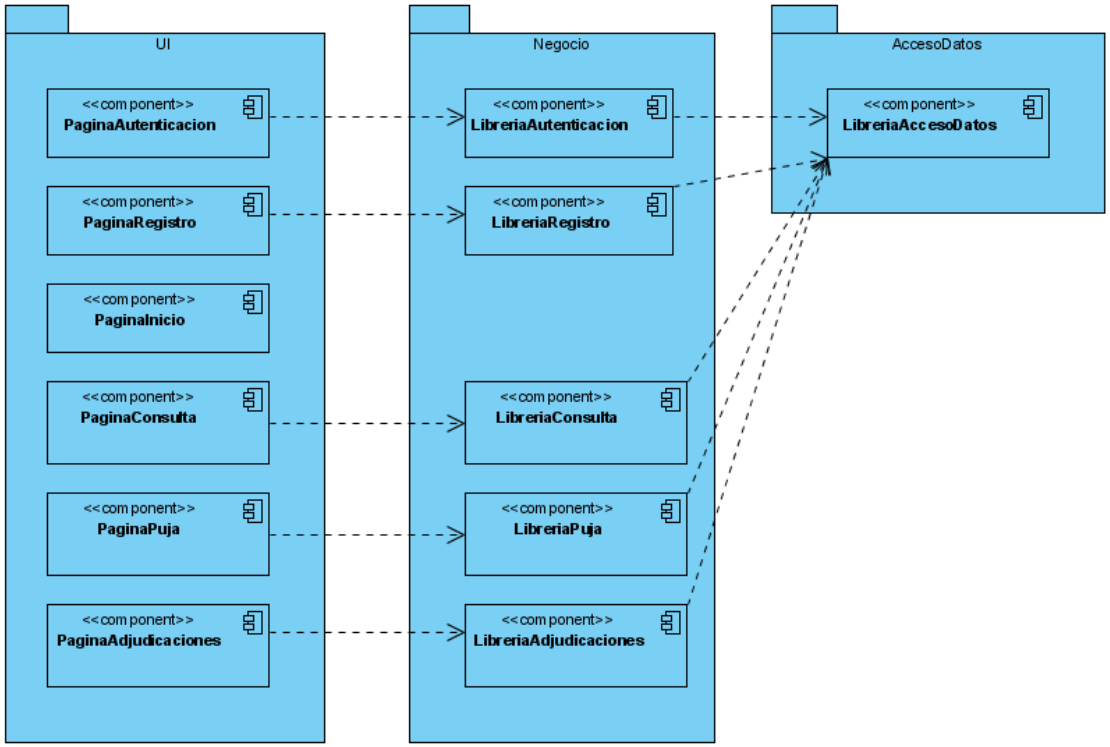


Diagrama de paquetes

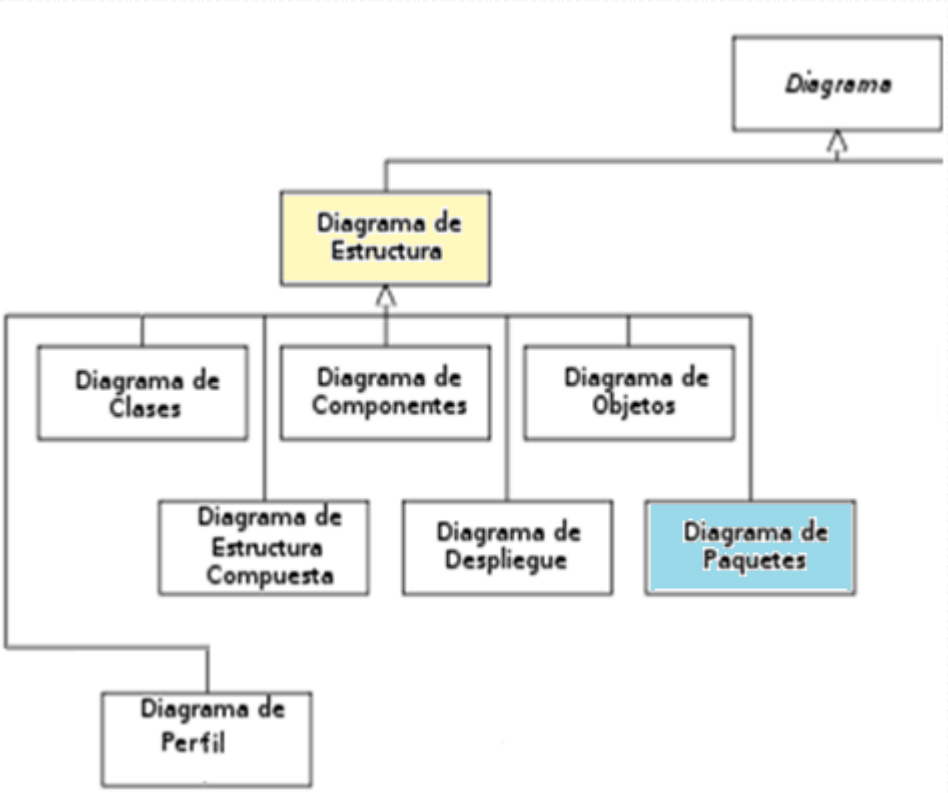


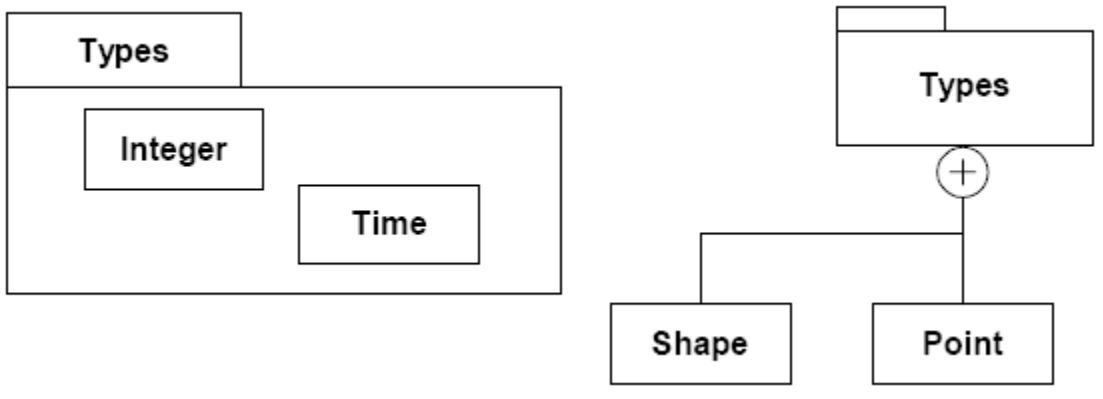
Diagrama de paquetes

- Un **paquete es un mecanismo de propósito general para organizar un** modelo de manera jerárquica.
- Ofrece un mecanismo para agrupar elementos de modelado:
  - Diagramas de casos de uso
  - Diagramas de clase
  - Diagramas de componentes
- **Un paquete es un contenedor**.
- Son un mecanismo importante para gestionar la complejidad del modelado.
- Utilidades principales:
  - **Organizar** los elementos en los modelos para comprenderlos más fácilmente.

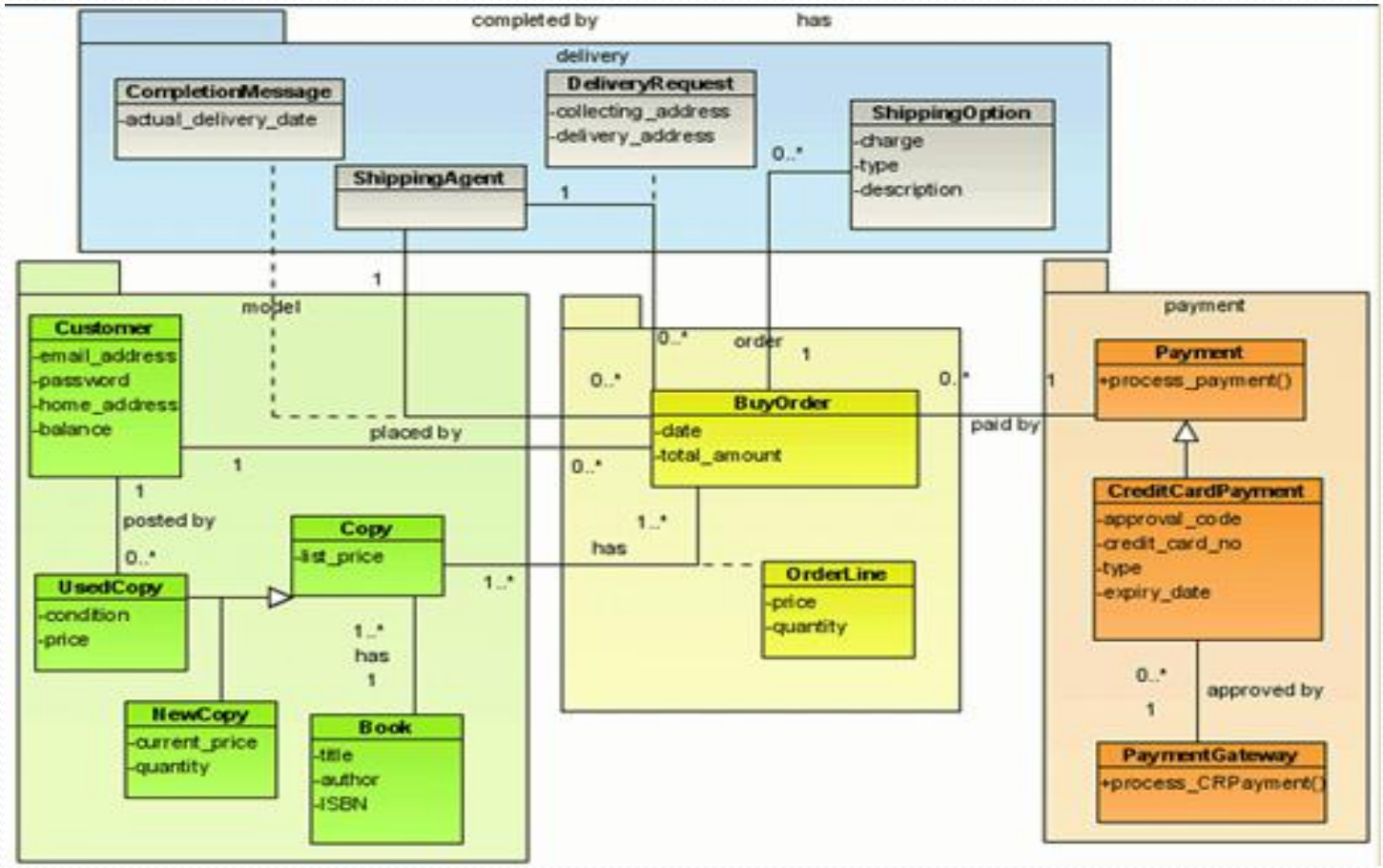
    Proveen un mecanismo sencillo para visualizar las dependencias entre las partes del sistema y a menudo son usados para buscar problemas o determinar el orden de compilación.
  - **Controlar el acceso** a sus contenidos para controlar las líneas de separación de la arquitectura del sistema (visibilidad).

Contenido de un paquete

- Un paquete puede contener clases, casos de uso…
- Hay dos maneras de representar el contenido de un paquete: con los elementos dentro o con los elementos fuera.

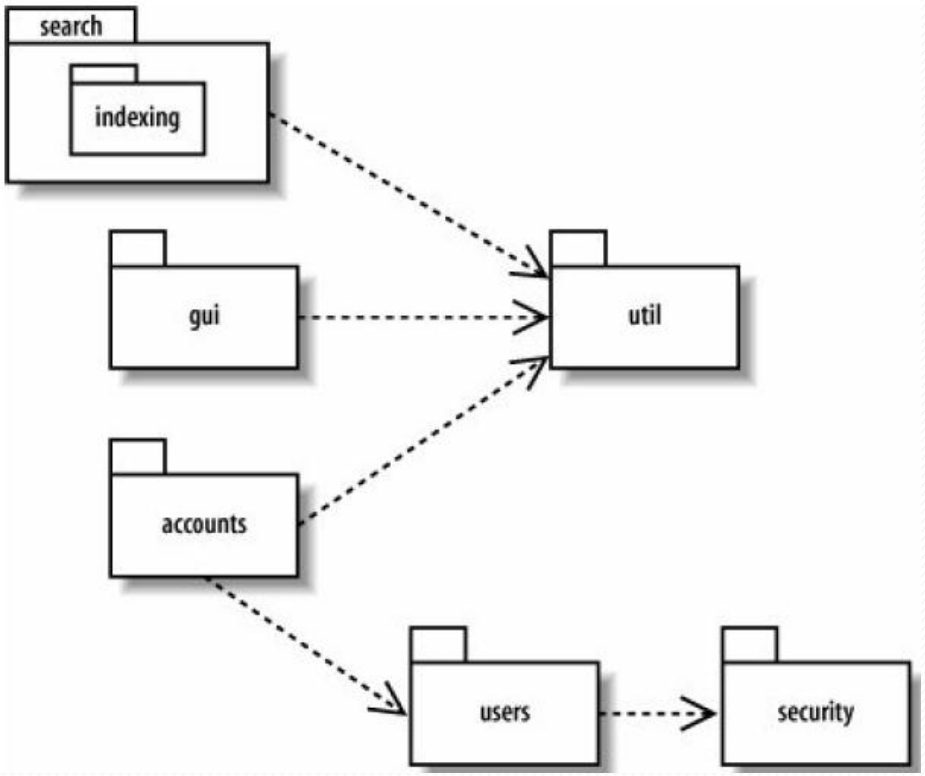


Ejemplo de empaquetado de un diagrama de clases por funcionalidad



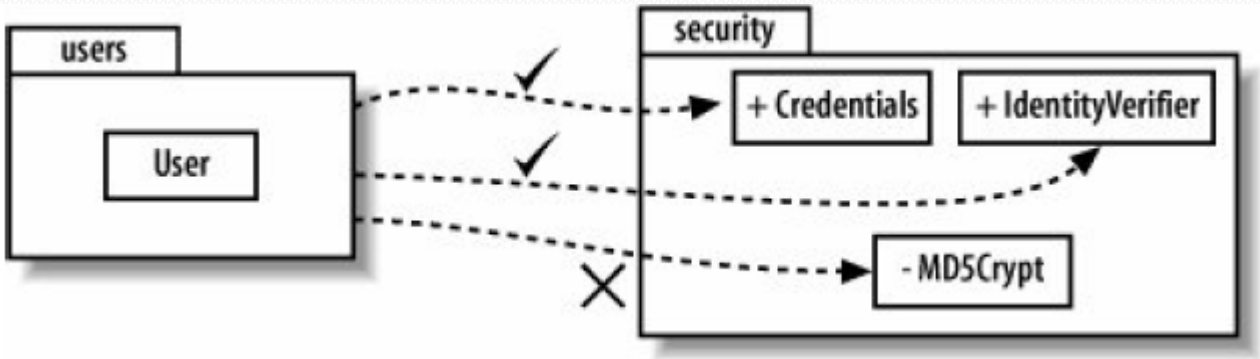
Dependencia entre paquetes

- Cuando un paquete necesita una clase de otro paquete se da una relación de dependencia entre paquetes.



Visibilidad

- Los elementos de un paquete pueden tener visibilidad:
  - **Pública** (+): son accesibles desde fuera del paquete
  - **Privada** (-): no son accesibles desde fuera del paquete, solo desde el paquete.



- Los paquetes se pueden utilizar, aparte de para organizar elementos, como control de acceso: se pueden declarar elementos privados en el paquete para prevenir que el elemento sea usado por otros paquetes.

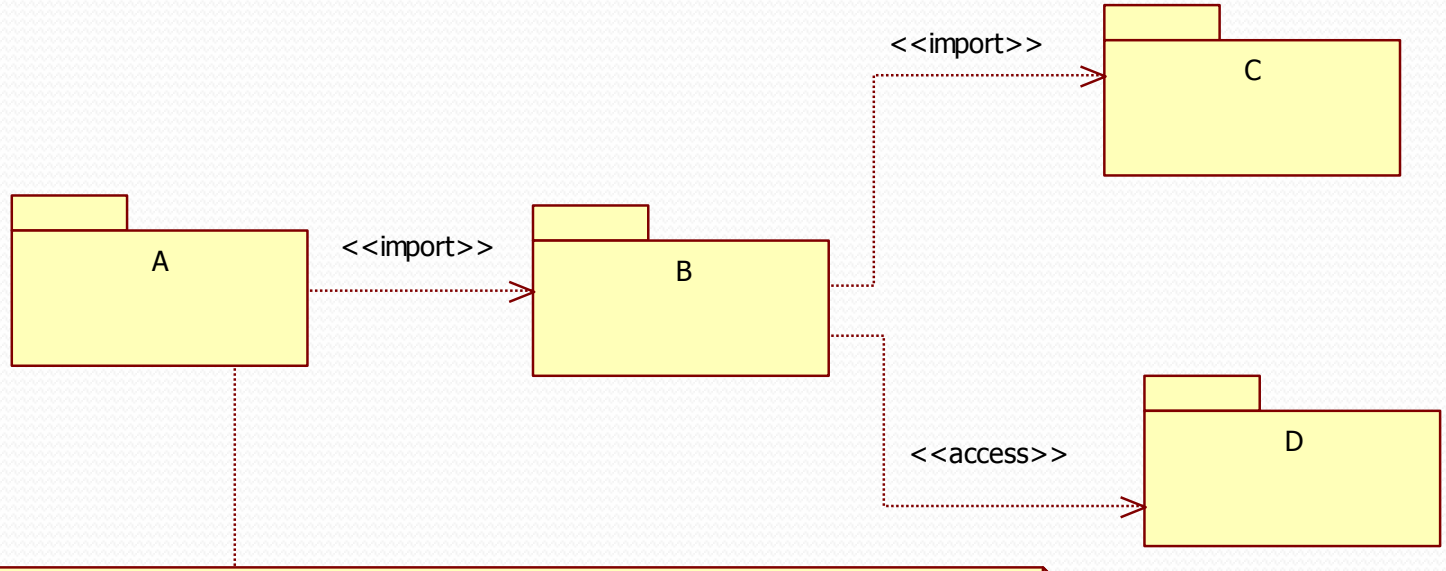
Dependencia entre paquetes
Importación

- **Importación**: cuando un paquete importa otro, este puede usar los elementos del paquete importado sin tener que especificar el identificador de ámbito (namespace) del elemento. Cuando se importa un paquete, solo los elementos públicos están disponibles.

  La importación, como otros elementos, tiene visibilidad:

  - **Pública** (por defecto): los elementos importados tienen visibilidad pública en el paquete que los importa. Se estereotipa <<**import**>>.
  - **Privada**: los elementos importados tienen visibilidad privada en el paquete que los importa. Se estereotipa <<**access**>>.

Dependencia entre paquetes
Importación

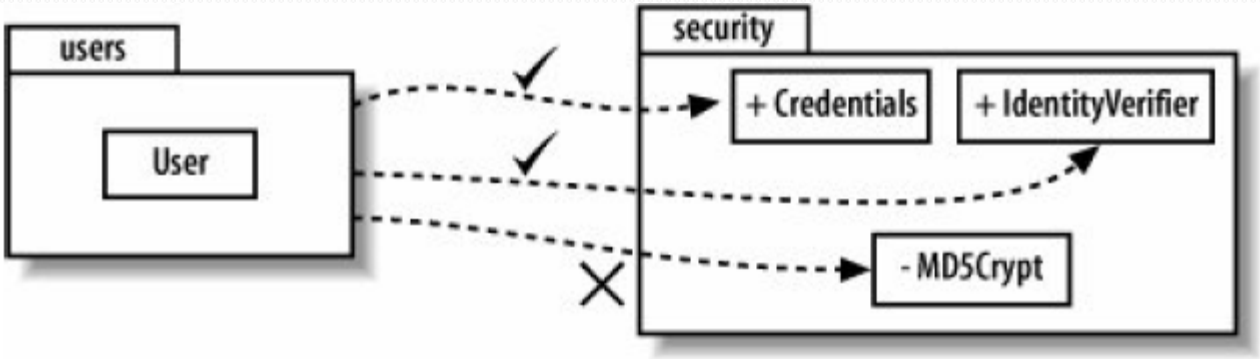


B importa C y accede a D --> puede ver los elementos públicos de C y D

A importa B
--> puede ver los elementos públicos de B
--> puede ver los métodos públicos de C, ya que B hace importación pública de C
--> no puede ver los elementos públicos de D, ya que B hace importación privada de D

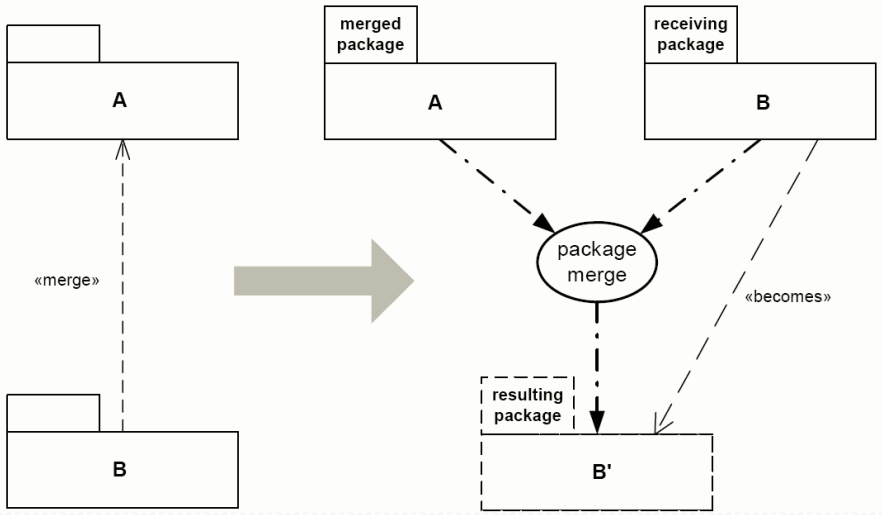
Dependencia entre paquetes
Exportación

- Exportación: **modelado implícitamente a través de la visibilidad pública en los elementos de un paquete**.
  - No se exporta explícitamente a algún paquete.
  - Se pone público, para que cualquier otro paquete pueda importarlo. La parte pública de un paquete son sus exportaciones.



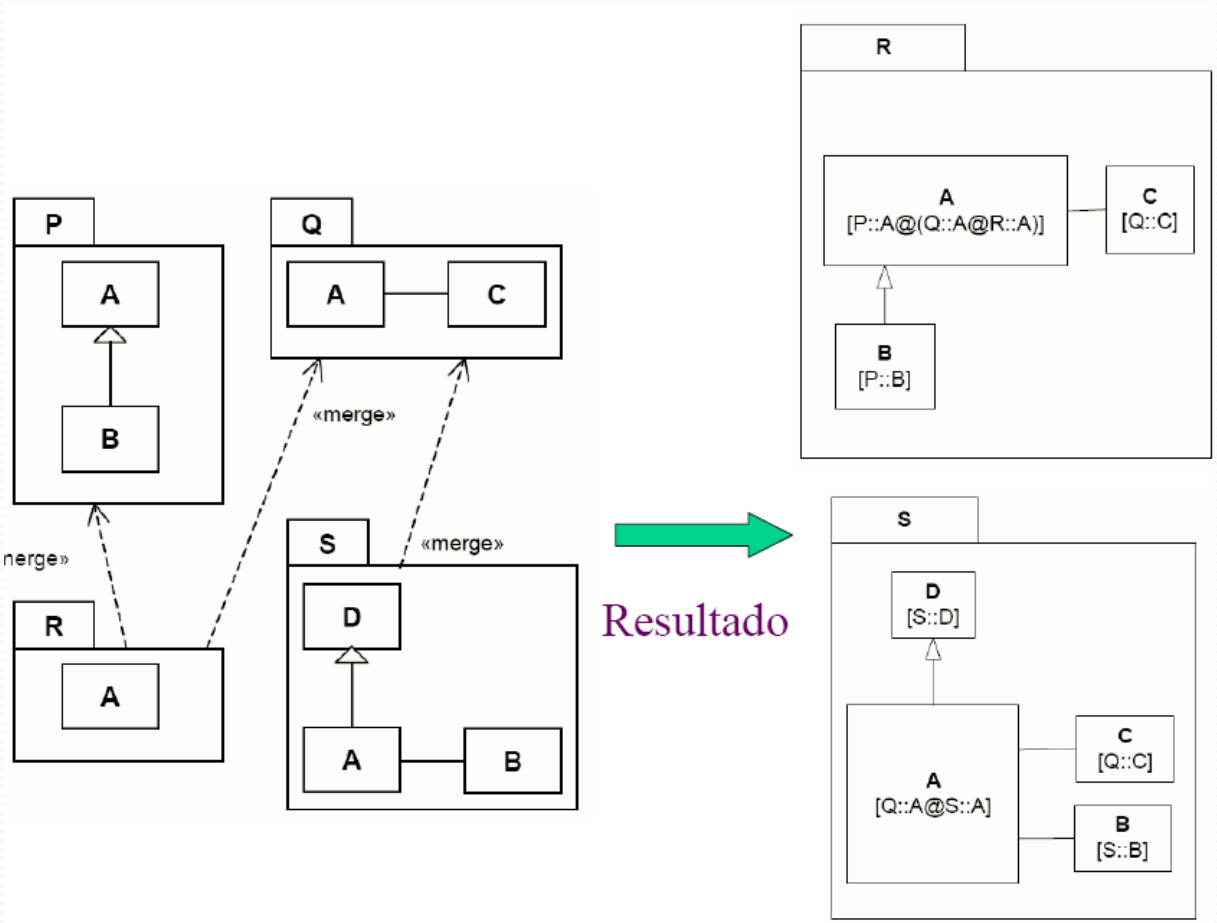
Relación de dependencia merge
Fusión de paquetes

- Una relación de **fusión (merge) entre dos paquetes** especifica que los contenidos de ambos paquetes son combinados.
- El contenido del paquete origen (receptor) se extiende con el contenido del paquete destino.
- Este mecanismo debería ser usado cuando elementos definidos en paquetes diferentes tienen el mismo nombre y representan el mismo concepto.
- Conceptualmente es muy similar a la generalización en el sentido en que el elemento origen añade las características del elemento destino a las suyas propias originando un elemento con las características de ambos.



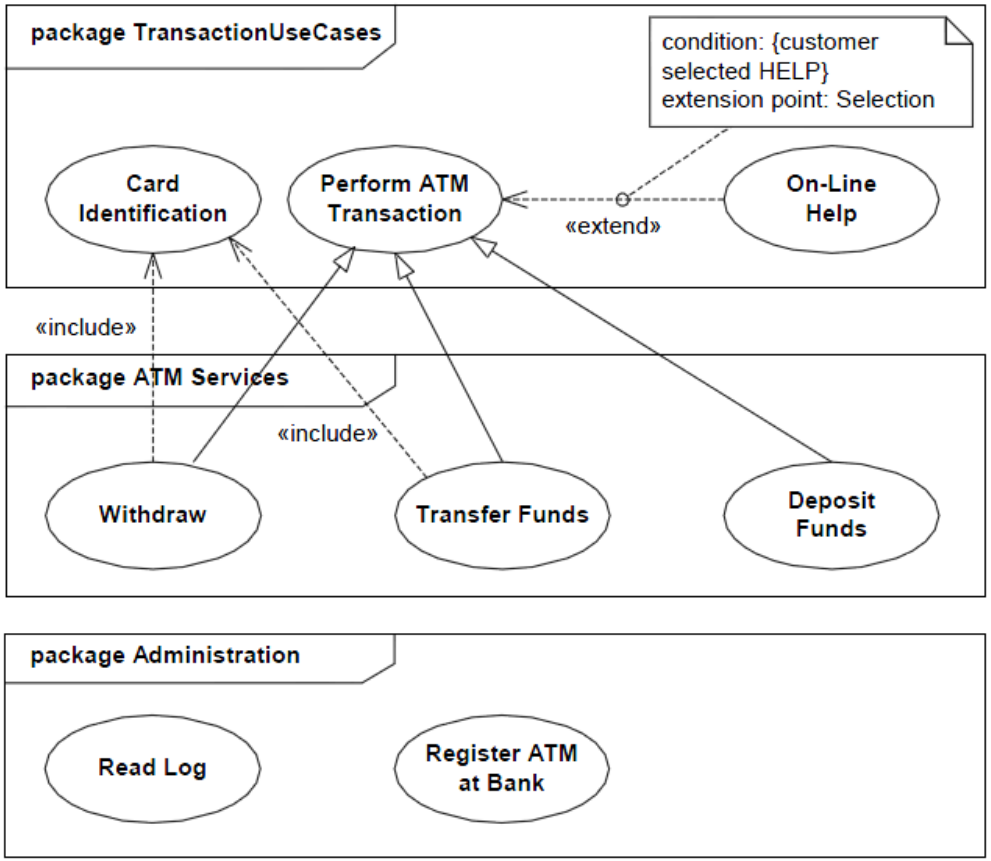
- Paquete origen B se mezcla con paquete destino A generándose B’

Ejemplo de relación <<merge>> extraída de la especificación UML 2



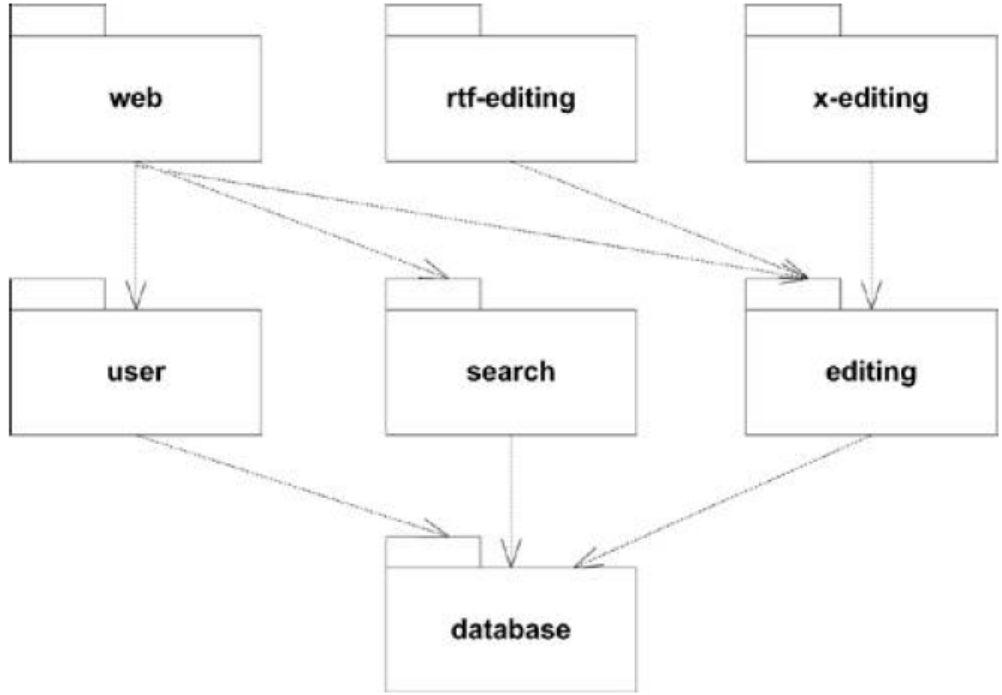
Organización de casos de uso

- Los paquetes se pueden usar para organizar casos de uso dando una visión de alto nivel de cómo los actores interaccionan con el sistema.
- Organizan el comportamiento funcional del sistema durante el análisis.



Descripción textual en diagramas de paquetes de alto nivel

- Si un diagrama de paquetes de alto nivel no aporta mucho significado por si mismo, por lo que será necesario acompañarlo de una descripción textual.



Descripción textual en diagramas de paquetes de alto nivel

- **web**
  - Requiere habilidades especiales: HTML, CSS, Structs, tecnologías de presentación. Depende de la mayoría de los otros paquetes.
- **database**
  - Requiere habilidades de manejo y modelado de base de datos. Pocas dependencias.
- **user**
  - Será construido por un equipo externo.
- **search**
  - Sistema auto contenido. Requiere habilidades con tecnologías y técnicas de motor de búsqueda.
- **editing**
  - Comprende las utilidades de edición básicas para la distribución de la primera release.
- **rtf-editing**
  - Comprende las utilidades de edición para la distribución de la segunda release.
- **x-editing**
  - Comprende las utilidades de edición para un cliente particular.

Consejos para agrupar elementos en paquetes

- Un paquete está **bien estructurado si:**
  - **Es cohesivo**.
    - Proporciona un límite bien definido alrededor de un grupo de elementos relacionados.
    - Conviene agrupar elementos que proporcionen un mismo servicio.
  - **Está poco acoplado**.
    - Exportando sólo los elementos que otros paquetes necesitan ver realmente.
    - Importando sólo aquellos elementos necesarios y suficientes para que los elementos del paquete hagan su trabajo
  - No está profundamente anidado.
  - Posee un conjunto equilibrado de elementos.

- Se recomienda agrupar en paquetes los elementos que:
  - Tienen un objetivo común o relaciones conceptuales fuertes
  - Pertenecen a un mismo árbol de herencia
  - Pertenecen al mismo caso de uso
- O formando paquetes más grandes de tipo arquitectónico:
  - Paquete “Modelo del diseño”.
  - Paquete “Clases e interfaces del modelo”.
  - Paquete “Interfaces de usuario”.
  - Paquete “Servicios base de datos”.

Diagrama de paquetes Resumen

- Presentan cómo se organizan los elementos de modelado en paquetes y las dependencias entre ellos, incluyendo importaciones y fusiones de paquetes.
- **Contenido de un diagrama de paquetes:**
  - Paquetes
  - Dependencias entre paquetes
    - Import
    - Access
    - Merge

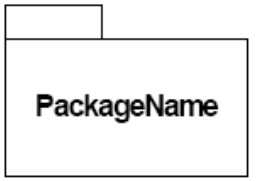
Package	
Package Merge	«merge»
PackageImport (public)	«import»
PackageImport (private)	«access»

Diagrama de despliegue

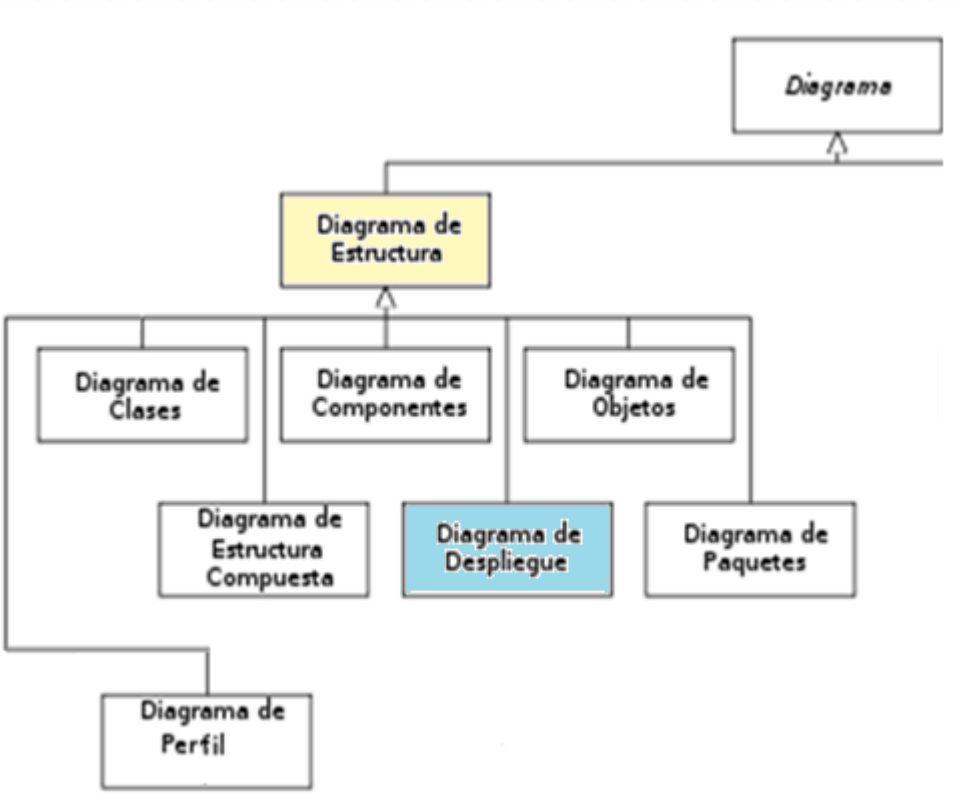


Diagrama de despliegue

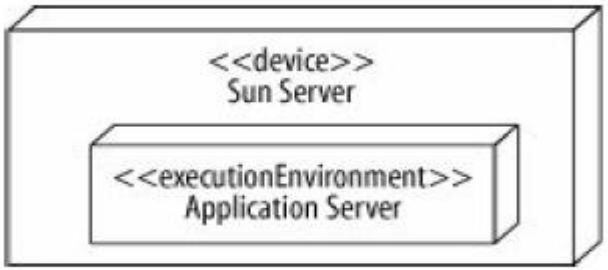
- Muestra la **vista física** del sistema.
- En el **diagrama de despliegue** se indica la situación física de los componentes lógicos desarrollados y como se comunican.
- Se sitúa el software en el hardware que lo contiene.
- También se le conoce como **diagrama de distribución**.

Qué diagrama nos permite mostrar la disposición de las particiones físicas del sistema de información y la asignación de los componentes software a estas particiones (TIC A2 2009):

- Diagrama de componentes
- Diagrama de descomposición
- Diagrama de despliegue ----------
- Diagrama de estructura

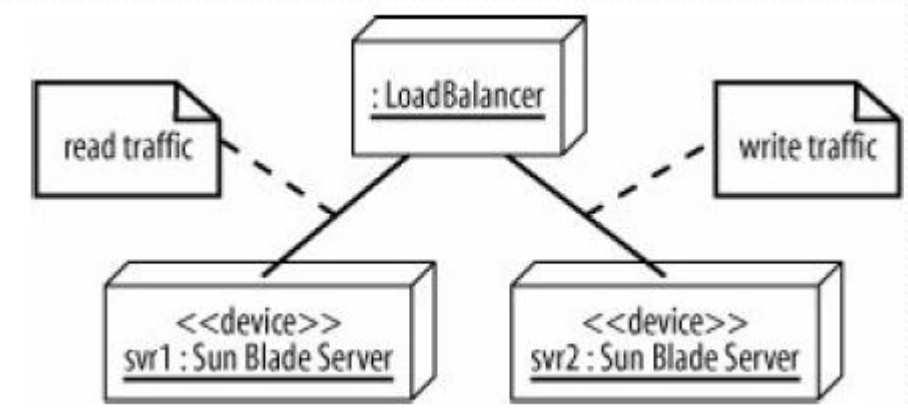
Nodos

- **Un nodo puede ser un recurso hardware, pero también software** en el que hospedar software o ficheros relacionados.
- Por lo general, un **nodo software** estará representado por **software de terceros que da servicio a nuestro software**.
- Ejemplo de nodos HW: servidor, PC de sobremesa, discos
- Ejemplo de nodos SW: sistema operativo, contenedor J2EE, web server, servidor de aplicaciones.
- **Los nodos HW se estereotipan como <<device>>** (dispositivo)
- **Los nodos SW se estereotipan como <<executionEnvironment>>** (entorno de ejecución)



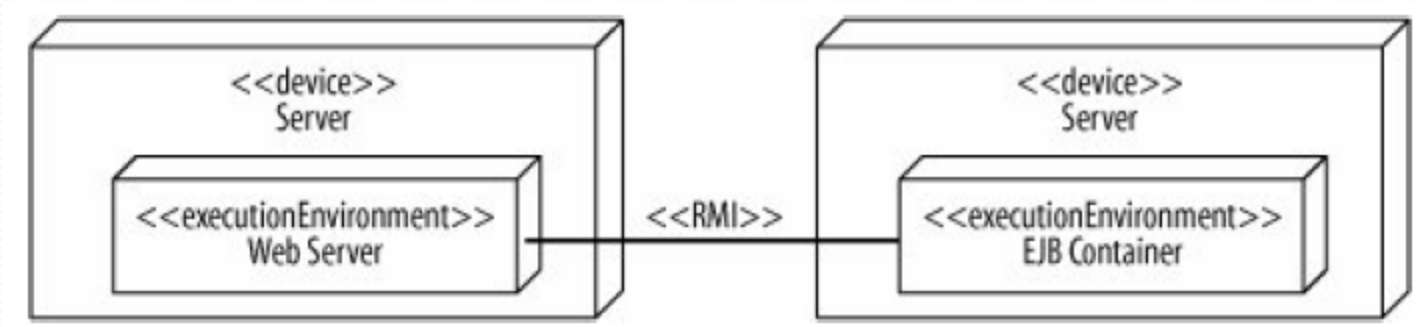
Instancias de nodos

- Cuando nuestro diagrama incluye varios nodos del mismo tipo, pero queremos poner de relieve el hecho de que son instancias diferentes, podemos mostrar instancias de los nodos usando la notación nombre:tipo



Comunicación entre nodos

- Para realizar su trabajo los nodos se comunican entre si.
- La comunicación se expresa con una **línea que une los nodos**.
- **Se le puede asignar un estereotipo a la comunicación indicando el protocolo**. El estereotipo debe ser del nivel más alto posible. Por ejemplo, si dudamos entre poner como estereotipo <<TCP/IP>> y <<RMI>>, como regla elegiremos el de más alto nivel, en este caso <<RMI>>.



Artefactos

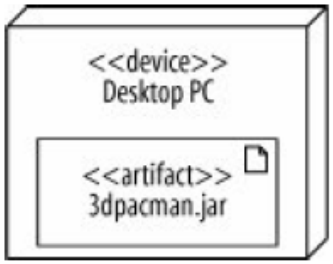
- **Una pieza física de información que representan el software que se ubicará en los nodos**.
- Se representa como un rectángulo. Si no tiene el icono documento en la esquina superior derecha, será necesario estereotiparlo como <<artifact>>.
- Un artefacto (UML 2.x) es un componente (UML 1.x).
- Notación:



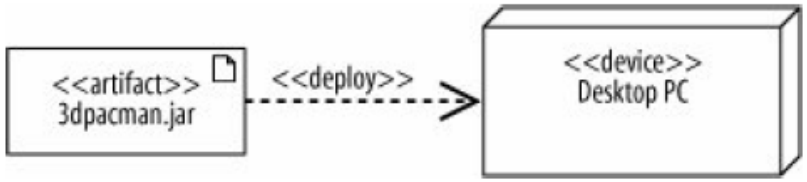
- Artefactos más comunes:
  - Ejecutables: .exe, .jar
  - Librerías: dlls, .jar de soporte
  - Ficheros fuente: .java, .cpp
  - Ficheros de configuración: .xml, .properties, .txt, scripts
  - BBDD: scripst, tablas

Despliegue de un artefacto
Notación

- Dentro del nodo



- Fuera del nodo: se dibuja una flecha de dependencia del artefacto al nodo estereotipada <<deploy>>

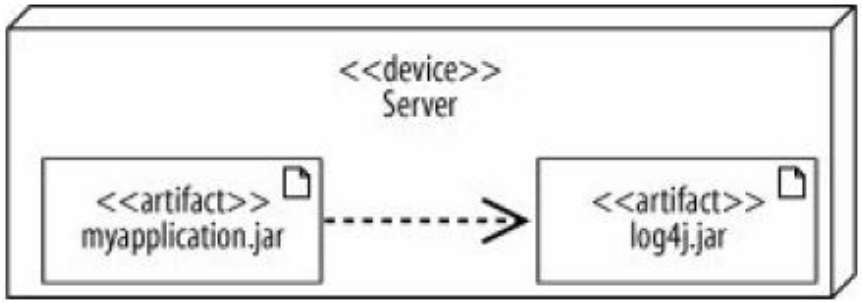


- Listando los artefactos dentro del nodo. Si el diagrama es muy grande puede ahorrarnos espacio, pero impide mostrar las dependencias entre artefactos.



Dependencias entre artefactos

- Se utiliza la flecha de dependencia



- **Una pieza física de información que es usada o generada por un proceso de desarrollo de software, incluyendo ficheros binarios ejecutables y scripts, se especifica en UML (unified modeling language) 1.4.2 como? (Analista programador Ayto de Madrid 2010):**
  - Component
  - Deployment
  - Artifact ----------
  - Element

Relación entre artefactos y componentes

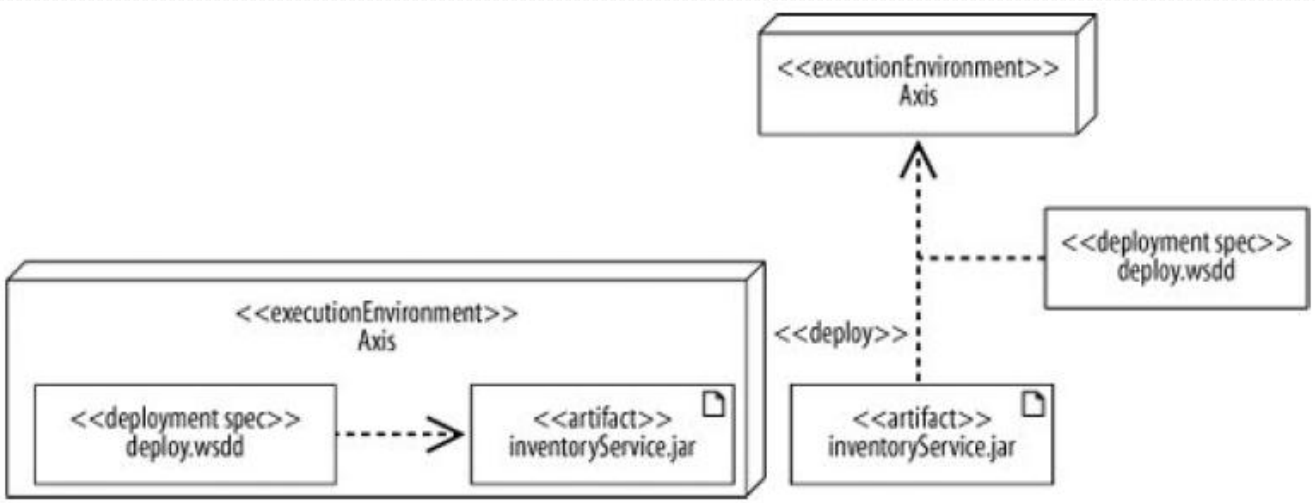
- En versiones de UML anteriores a la 2.0 los componentes se podían ubicar en los nodos.
- A partir de la versión 2.0 **en los nodos se ubican artefactos (no componentes).**
- Para relacionar artefactos y componentes se utiliza la relación de dependencia estereotipada con <<manifest>> (un artefacto manifiesta, revela un componente).



- Para no hacer muy confuso el diagrama de despliegue, la relación entre el artefacto y el componente se mantiene fuera del diagrama.
- La relación se puede mostrar en el diagrama de componentes (sección <<artifacts>> del componente).
- En el diagrama de despliegue solo se muestra pues el artefacto.

Especificación de despliegue

- Una especificación de despliegue es un artefacto especial que especifica cómo otro artefacto se va a desplegar en un nodo.
- Se representa como un artefacto estereotipado <<deployment spec>>



Especificación de despliegue

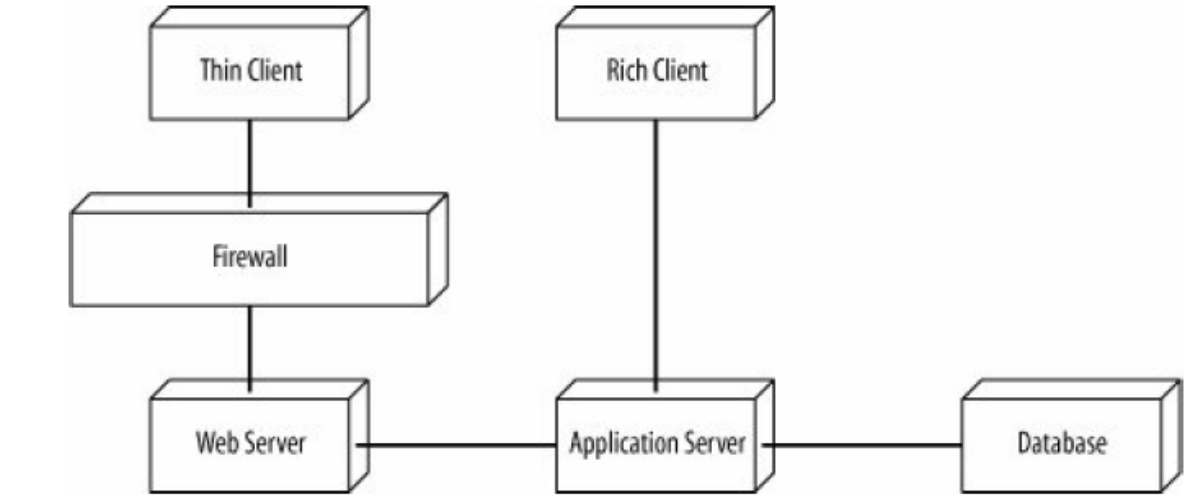
- Se puede instanciar la especificación de despliegue para mostrar los valores de sus parámetros.



- En una especificación de despliegue no es necesario listar todas las propiedades del despliegue, solo aquellas que se consideren importantes.

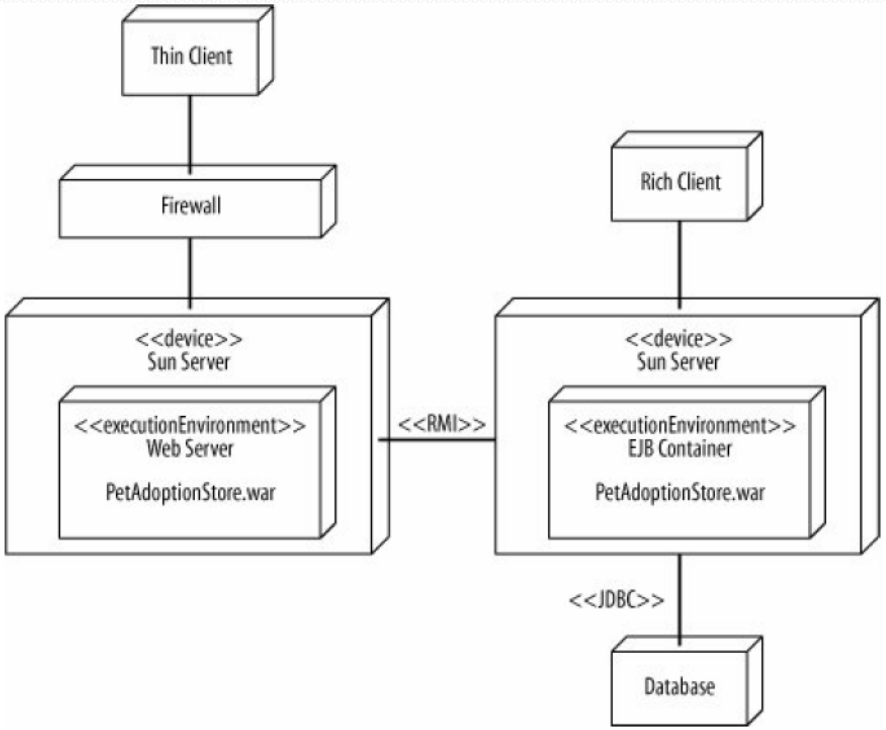
Cuando usar un diagrama de despliegue

- Los diagramas de despliegue son útiles en todas las fases del procesos de desarrollo.
- Por ejemplo, estamos desarrollando una aplicación web , pero aun no hemos decidido que HW usar y tampoco sabemos los nombres de los artefactos, pero queremos comunicar características importantes para el sistema como:
  - La arquitectura incluye un servidor web
  - Los clientes accederán a la aplicación a través de un browser (thin client) o un GUI rico (fat client)
  - El servidor web está protegido por un firewall.
- Incluso en esta etapa tan temprana del desarrollo se puede usar un diagrama de despliegue para modelar estas características.
- Los nombres de los nodos no han de ser precisos y no es necesario especificar los protocolos de comunicación.



Cuando usar un diagrama de despliegue

- Los diagramas de despliegue son también útiles en las últimas fases del desarrollo, en las que se pueden refinar los diagramas de despliegue iniciales, añadiendo detalles sobre tecnologías, comunicaciones, protocolos y artefactos a usar.



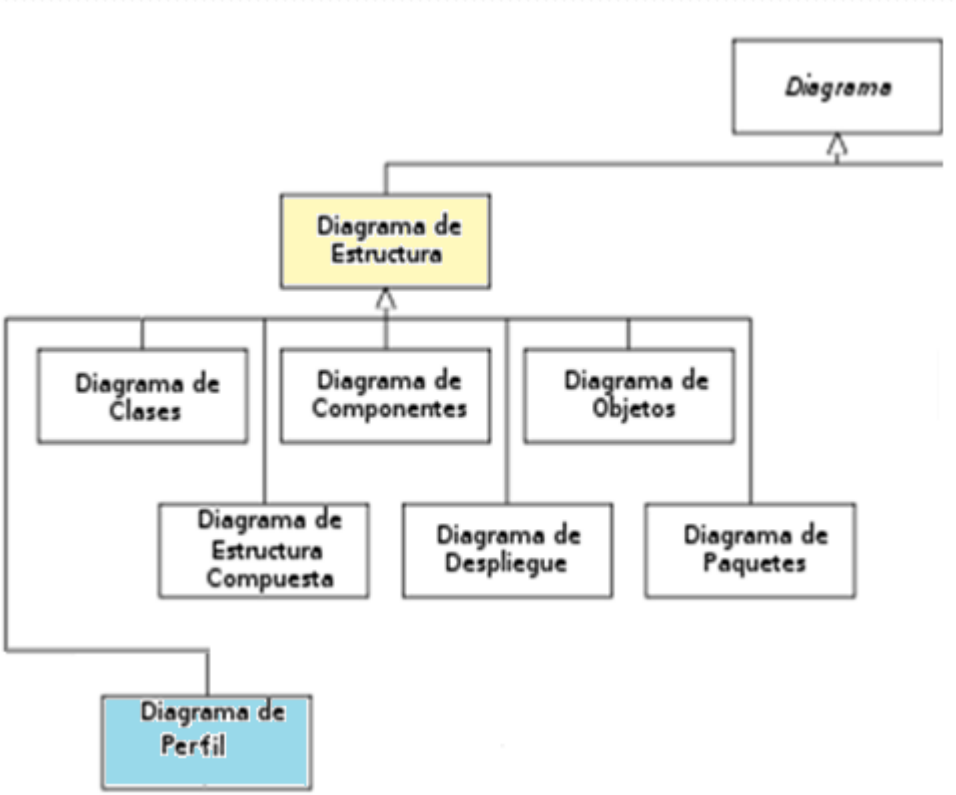
Diagramas

Node Type	Notation
Artifact	«artifact» **ArtifactName**
Node	**NodeName**
Node with deployed Artifacts	**Node** «artifact» **ArtifactName**

Path Type	Notation
Association	
Dependency	
Deployment	«deploy»
Manifestation	«manifest»

Node Type	Notation
Deployment specification	«deployment spec» **Name**
Deployment specification - with properties	«deployment spec» **Name** execution: execKind transaction : Boolean
Deployment specification - with property values	«deployment spec» **Name** execution: thread transaction : true
Artifact with annotated deployment properties	«artifact» **ArtifactName** {execution=thread, transaction =true}

Diagrama de perfil

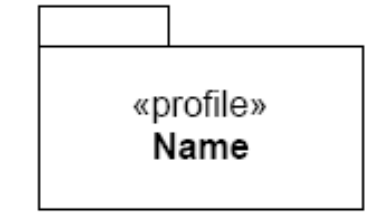


¿Qué es un perfil?

- Los Perfiles UML son el mecanismo que proporciona el propio UML para **extender su sintaxis y su semántica** para expresar los conceptos específicos de un determinado dominio de aplicación.
- Cuando modelamos un perfil estamos trabajando con el **meta-modelo** de UML, personalizando reglas UML para un determinado contexto.
- Los perfiles permiten adaptar el **metamodelo** a diferentes:
  - **plataformas** (como J2EE o .NET)
  - **dominios** (como modelado de procesos de negocio o de tiempo real).
- Los perfiles (profiles) fueron introducidos en UML 2.0 aunque no es hasta la versión 2.3 cuando aparece el diagrama de perfil.

Mecanismos de extensión

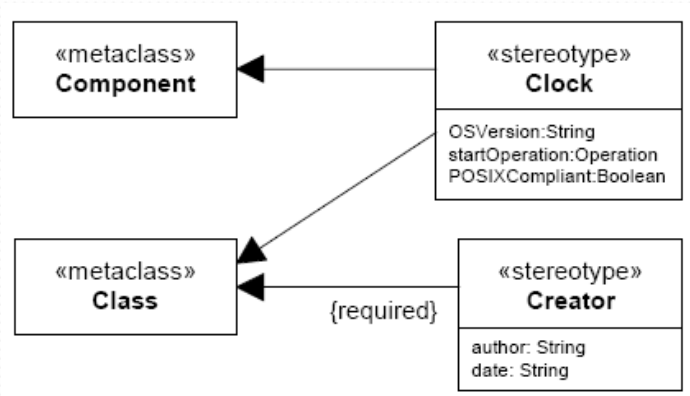
- Un **Perfil** se define en un **paquete UML, estereotipado <<profile>>**, que extiende a un metamodelo o a otro Perfil. El paquete contendrá las extensiones del meta-modelo.



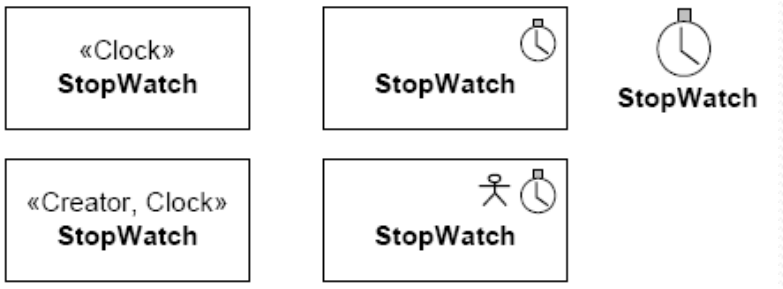
- Hay tres mecanismos que se utilizan para definir Perfiles:.
  - **Estereotipos** (stereotypes)
  - **Valores etiquetados** (tagged values)
  - **Restricciones** (constraints)

Mecanismo de extensión: Estereotipos

- Los **estereotipos** son **meta-clases específicas** que se obtienen extendiendo una meta-clase existente en el modelo de referencia.
- La notación para el estereotipo es la misma que para una clase pero usando la clave <<**stereotype**>>. Algunos estereotipos incluyen una representación gráfica (icono)



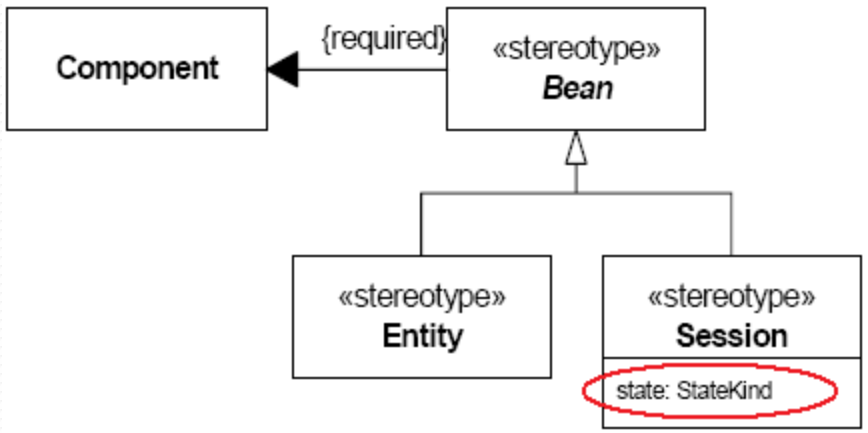
Opciones de presentación para una clase extendida (**instancias** del estereotipo Clock)



Si se aplican varios estereotipos al elemento, se separarán por comas.

Mecanismo de extensión: Valores etiquetados

- **Valor etiquetado**: es un meta-atributo adicional que se asocia a una meta-clase del metamodelo extendido por un Perfil.
- Todo valor etiquetado ha de contar con un **nombre** y un **tipo**, y se asocia un determinado estereotipo.
- **Los valores etiquetados son las propiedades de los estereotipos.**

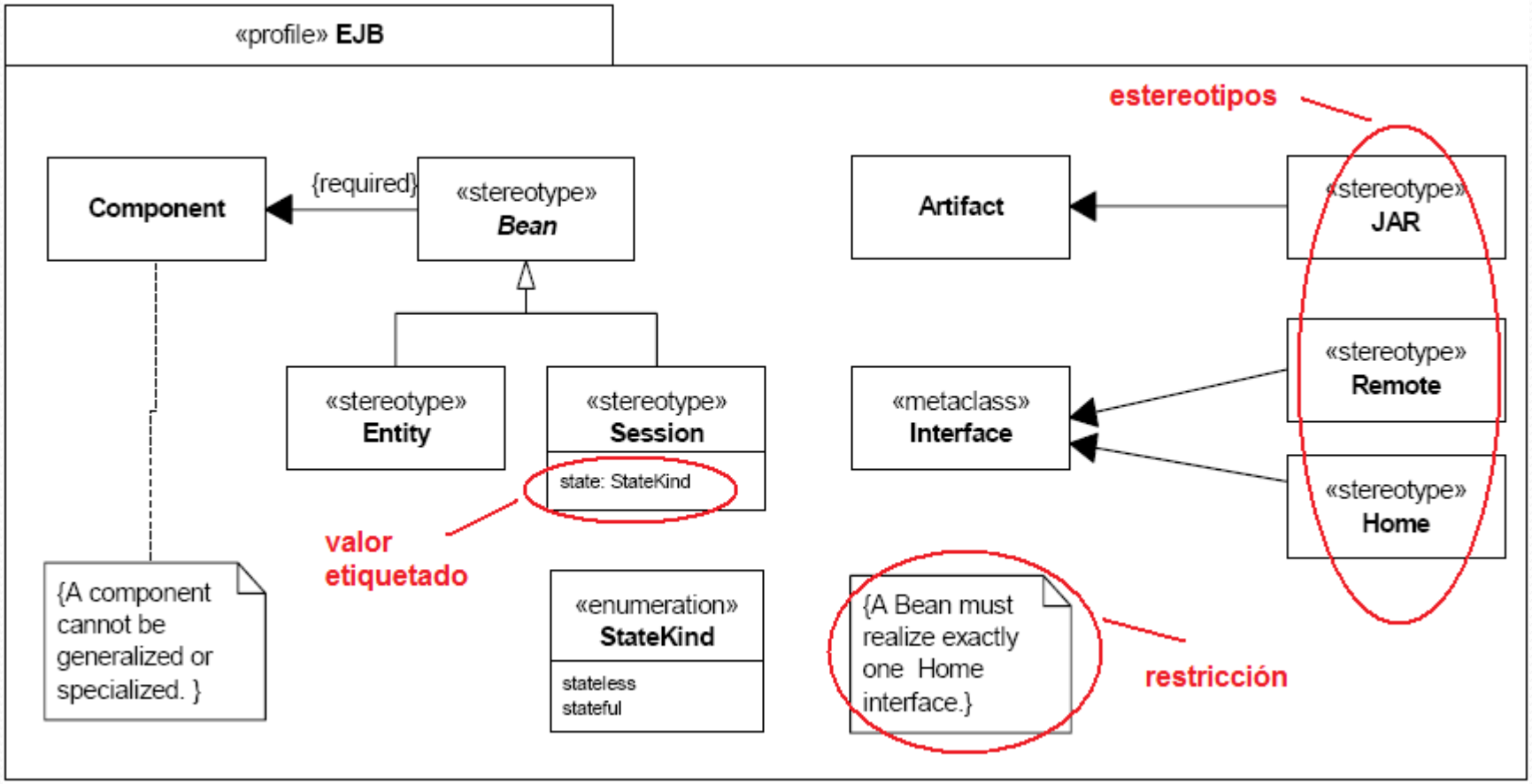


El estereotipo **Bean** es abstracto (cursiva), lo que implica que no se podrá instanciar. Se instanciarán los estereotipos concretos como **Entity** y **Session**. La extensión {**required**} se aplica a todos los estereotipos concretos que heredan de **Bean**.

Mecanismos de extensión: Restricciones

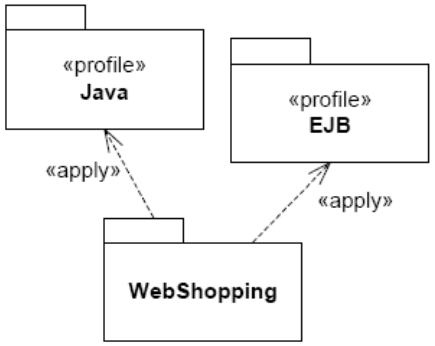
- **Las restricciones pueden expresarse tanto en lenguaje natural como en OCL** (**O**bject **C**onstraint **L**anguage – Lenguaje de Restricciones de Objetos).
- OCL es un lenguaje para consultar y restringir los elementos de un modelo, definido por OMG y es parte integrante de UML.
- OCL permite escribir expresiones sobre modelos, como por ejemplo invariantes, pre- y post-condiciones, reglas de derivación para los atributos y las asociaciones, el cuerpo de las operaciones de consulta, etc.
- El término "Constraint" que aparece en el nombre OCL perdura de la primera versión de OCL, que sólo permitía definir restricciones. Sin embargo, la versión 2.0 de OCL proporciona un lenguaje de consultas mucho más general, con una expresividad similar a la de SQL.

Ejemplo
Perfil EJB

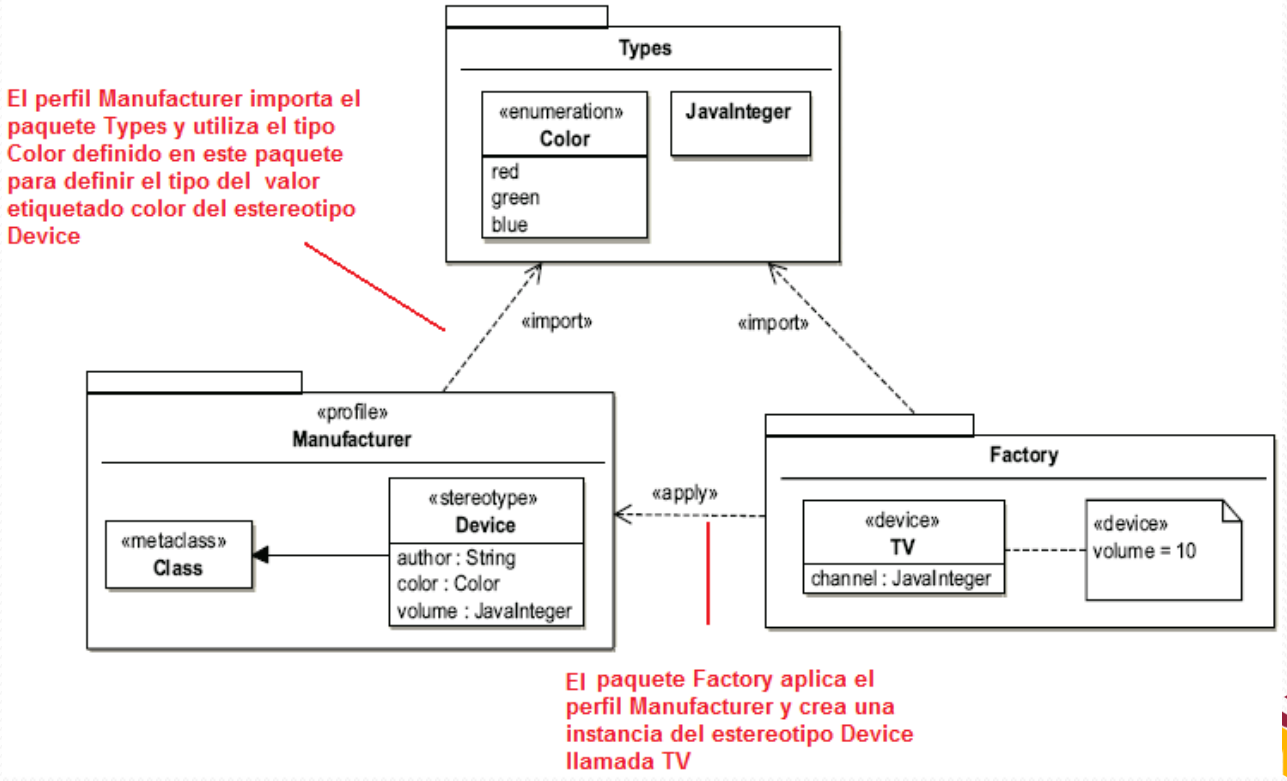


Aplicación de un perfil predefinido

- Aplicar un perfil significa que está permitido, pero no necesariamente requerido utilizar los estereotipos definidos como parte de ese perfil.
- Se indica que un perfil se aplica en un paquete usando una flecha discontinua estereotipada <<**apply**>> que va del paquete al perfil que este desea utilizar.



Aplicación de los perfiles Java y EJB al paquete WebShopping



Diagramas

Node Type	Notation
Stereotype	«stereotype» **Name**
Metaclass	«metaclass» **Name**
Profile	«profile» **Name**

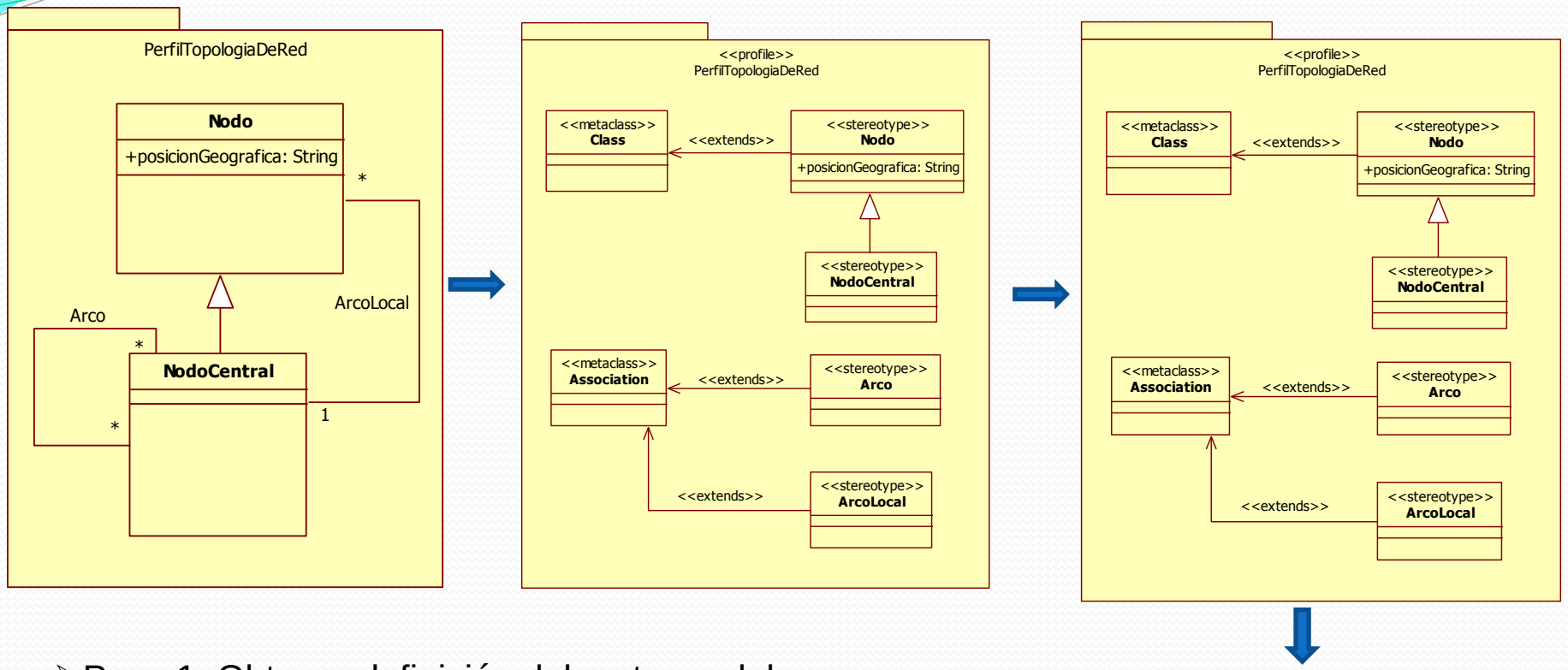
Node Type	Notation
Extension	
ProfileApplication	«apply»

Pasos a seguir para definir un perfil UML

- Paso 1: Obtener definición del meta-modelo.
  - **Es preciso disponer de la correspondiente definición del meta-modelo** de la plataforma o dominio de aplicación **a modelar con un Perfil**.
  - **Si no existiese**, entonces **definiríamos dicho meta-modelo utilizando los mecanismos del propio UML** (clases, relaciones de herencia, asociaciones, etc.), **de la forma usual como lo haríamos si nuestro objetivo no fuese definir un perfil UML**. Deberemos incluir la definición de las entidades propias del dominio, las relaciones entre ellas, así como las restricciones que limitan el uso de estas entidades y de sus relaciones.
- Paso 2: Definir los estereotipos.
  - Dentro de un paquete estereotipado «**profile**» incluiremos **un estereotipo por cada uno de los elementos del meta-modelo** que deseamos incluir en el Perfil.
  - **Estos estereotipos tendrán el mismo nombre que los elementos del meta-modelo**, estableciéndose de esta forma una relación entre el meta-modelo y el Perfil.
  - En principio cualquier elemento que hubiésemos necesitado para definir el meta-modelo puede ser etiquetado posteriormente con un estereotipo.
- Paso 3: Definir los valores etiquetados.
  - Definir como valores etiquetados de los elementos del Perfil los atributos que aparezcan en el meta-modelo. Incluir la definición de sus tipos, y sus posibles valores iniciales.
- Paso 4: Definir las restricciones.
  - Definir las restricciones que forman parte del Perfil, a partir de las restricciones del dominio.
  - Por ejemplo:
    - Las multiplicidades de las asociaciones que aparecen en el meta-modelo del dominio.
    - Las propias reglas de negocio de la aplicación deben traducirse en la definición las correspondientes restricciones.

- **¿Cuál de los siguientes mecanismos no se utiliza para definir un perfil (profile) en UML 2.0? (TIC A2 2010)**
  - Estereotipos
  - Valores etiquetados
  - Artefactos ----------
  - Restricciones

Pasos a seguir para definir un perfil UML.



- Paso 1: Obtener definición del meta-modelo.
- Paso 2: Definir los estereotipos.
- Paso 3: Definir los valores etiquetados.
- Paso 4: Definir las restricciones.

context UML::InfrastructureLibrary::Core::Constructs::**Class**
inv : -- Los nodos deben estar conectados exactamente a un Nodo Central
self.isStereotyped("Nodo") implies
self.connection->**select**(isStereotyped("ArcoLocal"))->**size** = 1 **and**
self.connection->**select**(isStereotyped("Arco"))->**isEmpty**

context UML::InfrastructureLibrary::Core::Constructs::**Association**
inv : -- La asociación ArcoLocal se ha de hacer entre parejas (Nodo, NodoCentral) con la misma
-- posicionGeografica
self.isStereotyped("ArcoLocal") **implies**
self.connection->**select**(participant.isStereotyped("Nodo") or
participant.isStereotyped("NodoCentral")) ->
forAll(n1, n2 | n1.posicionGeografica = n2.posicionGeografica)
inv : -- La asociación ArcoLocal implica que exista un participante de la relación que sea un NodoCentral
-- y que la multiplicidad de ese lado de la relación sea 1
self.isStereotyped("ArcoLocal") **implies**
self.connection-> **exists**(participant.isStereotyped("NodoCentral") **and** multiplicity.min=1 and
multiplicity.max=1)
inv : -- La asociación Arco implica que los participantes de la relación pueden ser NodoCentral pero no
-- Nodo y que los elementos NodoCentral han de tener distinta posicionGeografica
self.isStereotyped("Arco") **implies**
self.connection->**select**(participant.isStereotyped("Nodo"))->**isEmpty and**
self.connection->**select**(participant.isStereotyped("NodoCentral")) ->
forAll(n1, n2 | n1.posicionGeografica <> n2.posicionGeografica)

Perfiles UML

- Perfiles UML **estandarizados** por la OMG:
  - Platform Independent Model (PIM) & Platform Specific Model (PSM) for Software Radio Components (also referred to as UML Profile for Software Radio)
  - UML Profile for CORBA® and CORBA® Component Model (CCM) [This specification supersedes the separate profiles for CORBA® and the CORBA® Component Model.]
  - UML Profile for Enterprise Application Integration (EAI)
  - UML Profile for Enterprise Distributed Object Computing (EDOC)
  - UML Profile for Modeling QoS and Fault Tolerance Characteristics and Mechanisms
  - UML Profile for Schedulability, Performance and Time
  - UML Profile for System on a Chip (SoC)
  - UML Profile for Systems Engineering (SysML)
  - UML Testing Profile
- Otros muchos Perfiles UML **no estandarizados**:
  - "UML/EJB Mapping Specification", que ha sido definido por JCP (Java Community Process).
  - Perfiles UML para determinados lenguajes de programación, como pueden ser Java o C#
  - Etc.

Referencias

- Enlaces de interés.
  - http://uml.org
  - http://www.omg.org
  - http://www.uml-diagrams.org
- Bibliografía.
  - Learning Uml 2.0 - K Hamilton, R Miles - O'reilly – 2006.
  - Oreilly, Uml 2.0 In A Nutshell (2008).
  - UML Distilled (3ed 2004) - Addison Wesley
  - Wrox.Model.Driven.Development.with.Executable.UML .Jul.2009



¡Mucha suerte en los exámenes!

José Sampedro Martínez
jsampedro@minetur.es